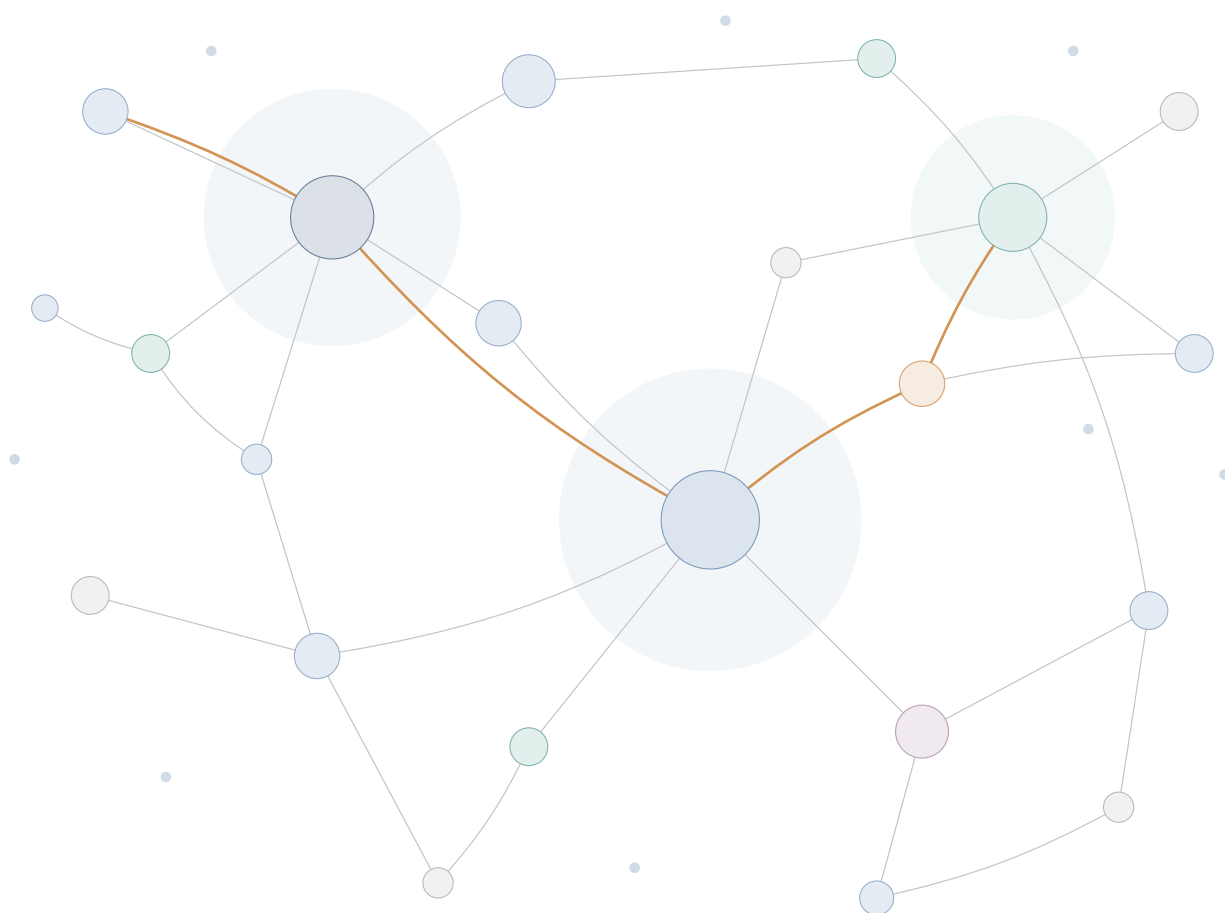


アーカイブズ学を学ぶ人のためのテキスト

Records in Contexts 入門

アーカイブズ記述の新しい国際標準を理解する

第 1.5.0 版 2026 年 9 月



久保山哲二

学習院大学大学院人文科学研究科アーカイブズ学専攻/計算機センター

本書について

本書は、国際文書館評議会 (ICA: International Council on Archives) が策定したアーカイブズ記述の新しい国際標準 Records in Contexts(RiC) を、アーカイブズ学を学ぶ人に向けて解説するテキストである。以下の一次資料の内容に基づいて執筆した。

- *Records in Contexts—Foundations of Archival Description* (RiC-FAD) Version 1.0, ICA EGAD, 2023 年 11 月
- *Records in Contexts—Conceptual Model* (RiC-CM) Version 1.0, ICA EGAD, 2023 年 11 月
- *Records in Contexts—Ontology* (RiC-O) Version 1.1, ICA EGAD, 2025 年 5 月 (OWL ソースファイル、公式ドキュメント、公式サンプルデータを含む)
- *Records in Contexts—Application Guidelines* (RiC-AG) Version 0.1(草案), ICA EGAD, 2025 年 10 月 (本文、公式クロスワーク、ダイアグラム類を含む)

一次資料はいずれも英語で書かれ、しかもその中心である RiC-CM と RiC-O は「モデルの仕様書」という性質上、初学者への配慮が十分とは言えない。定義は抽象的で、具体的な作業手順はほとんど示されない。「どう使えばよいのか」という問いに答える第 4 部 RiC-AG は、2025 年 10 月によりやく最初の草案 (0.1) が公開されたところであり、実装戦略を論じる部分 (§3) の冒頭で「段階的な手順書を提供することは不可能」と自ら明言する、事例と考え方の集成である。本書は、仕様書と実務の間になお残る距離を埋めることを目的に、4 部すべての内容を踏まえて、原文を忠実に紹介しつつ、背景となる考え方、他分野の技術との関係、実践への道筋を補って解説する。

■**本書の構成** 第 1 章で RiC 標準の全体像と一次資料の構成を示す。第 2 章で従来の標準 (ISAD(G) など) と、参照 (記述同士をつなぐ仕組み) がどう扱われてきたかの経緯を確認し、第 3 章で RiC の新しさを論じる。第 4 章は RiC-CM(概念モデル)、第 5 章は RiC-O(オントロジー) の解説である。第 6 章では推論という、RiC-O の利点が最も分かりやすく現れる仕組みを具体例で示す。第 7 章は実践の手順、第 8 章は必要となるコンピュータ科学の素養を扱う。巻末付録に実体・属性・関係の一覧、よく使う RiC-O 語彙、確認問題のヒント、用語集、一次資料と参考リソースの案内を収めた。

■**読み進め方** 前から順に読むように書いてあるので、通読するならそれでよい。ただし全員が全章を同じ密度で読む必要はない。図 1 は、第 8 章 8.1 節で示す 4 つの役割ごとの目安である。第 5 章と第 6 章は Turtle という記法が続いて身構えるかもしれないが、目録担当に必要なのは**読めること**であって書けることではない。第 7 章は節ごとにほぼ独立しているので、関心のある節から読める。付録の実体・属性・関係の一覧は、通読するものではなく、記述を設計するときに開く資料である。

目次

本書について	ii
第 1 章 RiC とは何か — 標準の全体像と一次資料	1
1.1 RiC の輪郭	1
1.2 標準の構成 — 4 つの部分	1
1.3 一次資料の入手先と構成	5
1.4 開発の経緯	6
1.5 この標準は誰のためのものか	7
第 2 章 ISAD(G) の世界から RiC へ	8
2.1 ICA の 4 つの記述標準	8
2.2 従来の記述を支えていた前提	9
2.3 記述と通信技術	11
2.4 参照と識別子 — 記述をつなぐ仕組み	13
2.5 ISO 23081 との関係	18
2.6 隣接標準との関係 — GLAM の隣人たち	18
2.7 移行はどう構想されているか	20
第 3 章 RiC がもたらした変化	21
3.1 変化 1: 検索手段のモデルから、世界のモデルへ	21
3.2 変化 2: 多段階記述から多次元記述へ	24
3.3 変化 3: 出所概念の拡張	25
3.4 変化 4: 「記述単位」の解体	28
3.5 変化 5: 記録と具現化 (Instantiation) の分離	29
3.6 記述の自由度と、そのトレードオフ	30
第 4 章 RiC-CM を読む — 実体・属性・関係	35
4.1 準備: 「実体」という言葉	35
4.2 実体の階層	37
4.3 属性	43
4.4 関係	47
4.5 記述を記述する	52

第 5 章	RiC-O — オントロジーという実装	56
5.1	RDF 入門 — 3 語の文でできた世界	56
5.2	オントロジーとは何か	64
5.3	なぜ 1 つのデータに混ぜて書けるのか — RDF・OWL・SKOS の層構造	73
5.4	RiC-O の全体像	74
5.5	RiC-CM から RiC-O へ — 何がどう対応するか	76
5.6	実例を読む	79
第 6 章	推論 — グラフだからできること	82
6.1	推論とは何か	82
6.2	場面 1: 階層をまたぐ検索 — 推移性	83
6.3	場面 2: どちら向きに書いても見つかる — 逆プロパティ	86
6.4	場面 3: 詳しく書いた人の損にならない — 下位プロパティのロールアップ	86
6.5	場面 4: 近道の自動生成 — プロパティ連鎖	87
6.6	場面 5: 組織の変遷をたどる — アーカイブズらしい総合問題	88
6.7	場面 6: 自分の規則は足せるか — 拡張性と、OWL では書けない規則	90
6.8	処理系の中身 — 書いた記述を誰が実行するか	92
6.9	推論は魔法ではない — 注意点	96
第 7 章	実践の手順	98
7.1	記述を始める前に用意するもの	98
7.2	3 つのシナリオ	99
7.3	シナリオ A の標準的な工程	100
7.4	通し例: 目録と典拠レコードの連携は、RiC でどうなるか	102
7.5	ドメイン知識をどこに置くか — 3 つの置き場所	112
7.6	共有の基盤は誰が用意するのか — 識別子と拡張のインフラ問題	117
7.7	日本語データの実務 — 和暦・字体・多言語	121
7.8	個人情報とアクセス制限 — どこまで公開するか	123
7.9	保存期間・評価選別・廃棄 — 記録管理の工程を記述する	123
7.10	シナリオ B: 新規記述のワークフロー	131
7.11	道具箱	132
7.12	最初の一步の例 — 演習として	134
第 8 章	どこまで技術を学ぶ必要があるか	137
8.1	役割別の必要水準	137
8.2	要素技術の地図	138
8.3	リレーショナルデータベースの経験はどう活かせるか	139
8.4	形を締める — SHACL を 1 つ書いてみる	141
8.5	学習ロードマップ	143
付録 A	付録	145

A.1	RiC-CM 1.0 実体一覧	145
A.2	RiC-CM 1.0 属性一覧 — 原文との対照	146
A.3	関係の 13 の型 (RiC-CM §5.2)	147
A.4	RiC-CM 1.0 関係一覧 — 原文との対照	148
A.5	よく使う RiC-O 語彙の抜粋	151
A.6	確認問題のヒント	151
A.7	用語集 — 原語との対照、分野で意味がずれる言葉	152
A.8	一次資料と参考リソース	155
索引		157

第 1 章

RiC とは何か — 標準の全体像と一次資料

1.1 RiC の輪郭

Records in Contexts(RiC) は、国際文書館評議会 (ICA) の記述専門家グループ EGAD(Expert Group on Archival Description) が策定した、アーカイブズ記述のための国際標準である。その名の通り「文脈の中の記録 (records *in contexts*)」を記述することを目的とする。従来の ICA 標準である ISAD(G)、ISAAR(CPF)、ISDF、ISDIAH の 4 つを統合して置き換える、単一の包括的標準である (RiC-CM §1.2)。

RiC の最大の特徴は、記述の**成果物** (検索手段=ファインディングエイド) の様式を定めるのではなく、記述の**対象となる世界そのもの**—記録、それを作り使った人々、人々の活動、そしてそれらの間の関係—をモデル化する点にある。従来の階層的な「フォンド単位の記述」も表現できるが、それにとどまらず、記録をめぐる関係のネットワーク (グラフ) 全体を記述できる。この転換が何を意味するかは第 3 章で詳しく論じる。

1.2 標準の構成 — 4 つの部分

RiC は単一の文書ではなく、性格の異なる 4 つの部分から構成される (RiC-CM §1.1)。この構成を頭に入れておくことが、一次資料を読むときの最初の助けになる。

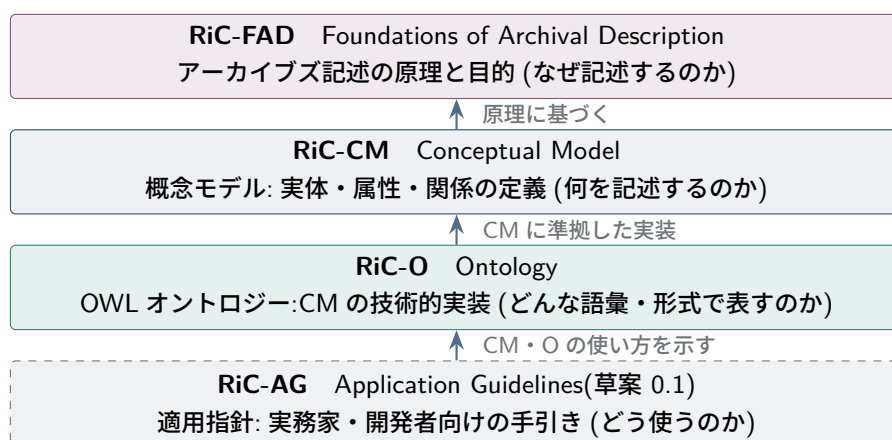


図 1.1 RiC 標準の 4 部構成。上から下へ、抽象 (原理) から具体 (実務) の順に並ぶ。

1.2.1 RiC-FAD — 原理

RiC-FAD(Foundations of Archival Description) は、わずか 8 ページほどの短い文書で、アーカイブズ記述の基礎にある原理と目的を述べる。記録管理の歴史 (メソポタミアの粘土トークンにまで遡る)、出所原則 (Principle of Provenance) の成立とその拡張、アーキビストの役割、記述の 3 つの目的 (記録の管理・保存・利用) を扱う。RiC-CM が「なぜこうモデル化するのか」を正当化する理論的な基礎であり、CM 本文でも、出所原則の詳しい議論はこちらを見よと案内されている (RiC-CM §1.5.1 と §1.5.3 の脚注)。もともと RiC-CM 0.1 の一部だったものが独立の文書になった経緯があり、CM と併せて読むことを前提としている。

1.2.2 RiC-CM — 概念モデル

RiC-CM(Conceptual Model) が標準の中心文書である (約 130 ページ)。実体関連モデル (ERM: Entity-Relationship Model) の枠組みを用いて、記述の対象を「実体 (entity)」「属性 (attribute)」「関係 (relation)」として定義する。RiC-CM 1.0 は 19 の実体、42 の属性、85 の関係 (多くは逆向きの関係と対で定義される) を定める。

重要なのは、RiC-CM が「高水準の (high-level) 概念モデル」だと自己規定していることである (RiC-CM §1.2)。RiC-CM は次のいずれでもないと言明される。

- 記述内容の書き方 (データ内容規則) を定める標準ではない
- システムの実装仕様ではない
- 記録の物理的管理のモデルではない
- データ交換フォーマットではない

表 1.1 RiC-CM が扱わないものと、それを担うものの例。中央の列は本書による例示である (RiC-CM §1.2 は、物理的管理と保存については「既存の標準」と書くだけで、個別の名を挙げていない)。

RiC-CM はこれではない (§1.2)	それを担うものの例	本書の関連箇所
記述内容の書き方 (データ内容規則)	米国の DACS、カナダの RAD、各機関の記述規則	第 7 章 7.1 節
システムの実装仕様	各機関の目録システム・記録管理システムの設計。RiC-O はその手がかりになる	第 5 章、第 7 章 7.1 節
記録の物理的管理のモデル	物理的管理と保存を扱う既存の標準。デジタル保存なら PREMIS	第 2 章 2.6 節
データ交換フォーマット	EAD、EAC-CPF、RDF の各構文 (Turtle、RDF/XML など)	第 2 章 2.3 節、 第 5 章 5.1 節

つまり、EAD(Encoded Archival Description。検索手段をコンピュータで交換できる形式に符号化するための標準) と、各国の目録規則 (たとえば DACS や RAD) とも役割が異なる。表 1.1 は、4 つ

の「ではない」を、それぞれを担う側の標準や仕組みと並べたものである。§1.2 は、RiC-CM がこれらの開発の参考にはなりうることも添えている。RiC-CM は「世界をどんな概念で切り分けるか」だけを定め、その表現方法や運用は実装側に委ねる。ここが、「自由度の高さ」(第3章 3.6 節)と「ガイドラインの不足」の両方の源泉になる。記述をこの標準から直接始めようとするとなんが足りないかは、第7章 7.1 節で具体的に見る。

1.2.3 RiC-O — オントロジー

RiC-O(Ontology) は、RiC-CM を Web オントロジー言語 OWL 2(ウェブ技術の標準化団体 W3C が定める形式言語) で形式化した「特定の一実装 (a specific implementation)」である。RiC-CM が紙の上の概念図だとすれば、RiC-O は機械が読める語彙集であり、アーカイブズ記述を Linked Open Data(LOD。ウェブ上で相互にリンクされた公開データ。第5章) として公開し、SPARQL(そのデータのための検索言語。第6章) で検索し、論理推論を行うための道具立てを提供する。RiC-O 1.1 は 107 のクラス、75 のデータタイププロパティ、480 のオブジェクトプロパティを定義しており、高水準の RiC-CM よりはるかに詳細である。CM に存在しない構成要素 (Proxy クラスなど) も多く含む。第5章で詳しく扱う。

なお「CM の実装は RiC-O だけとは限らない」ことにも注意したい。RiC-O の公式ドキュメントは、RiC-CM の別の技術的実装が今後作られうると明記し、あわせて、従来の ICA 標準の技術的実装である EAD や EAC-CPF の XML スキーマも将来は RiC-CM に近づけるべきだと述べている。CM と O は「概念とその一実装」という関係である。

1.2.4 RiC-AG — 適用指針 (草案)

RiC-AG(Application Guidelines) は、実務家・開発者・管理者向けに RiC の理解と実装を助ける第4部である。最初の草案 0.1 が 2025 年 10 月 29 日、バルセロナの ICA 大会で公開された。EGAD はこれを「生きた文書 (living document)」と位置づけ、コミュニティからのフィードバックを募りながら育てていくとしている。

内容は、AG 自身の説明によれば大きく 2 つに分かれる。

- RiC 全体を主題とする部分。RiC の効用 (利用者・実務家・IT 管理者・管理者それぞれにとって)、実装戦略、他標準との併用を扱う。
- 実践の指針。既存の ISAD(G) 型目録を RiC に写す実例 (英国国立公文書館の目録を素材にした Getting Started)、FAQ 群 (記録か記録集合か、具現化、順序、活動の記述、語彙の使い方、拡張の方法など)、先住民コミュニティに関するケーススタディ、セルバンテスと『ドン・キホーテ』出版を素材にした拡張例が並ぶ。

このほかに、既存 4 標準から RiC-CM への公式クロスウォーク (対応表) と、EAD 2002 から RiC-O 1.1 へのクロスウォークが表計算形式で付いている。表 1.2 は AG の各部を番号順に並べ、AG 自身が勧める読者の目安を添えたものである。AG は、初学者には §2 から入り、続いて §6 の各 FAQ の導入に目を通すよう勧めている。

注意すべきはその性格である。RiC-AG は実装戦略を扱う §3(Implementation strategies) の冒頭で

表 1.2 RiC-AG 草案 0.1 の構成。右端の列は、AG の「How to use the Application Guidelines」が示す読者の目安である。

§	原題	内容	勧められている読者
1	What are the benefits of using RiC?	利用者・実務家・IT 管理者・管理者 それぞれにとっての効用	管理職
2	Getting started with RiC	英国国立公文書館のオンライン目録 Discovery の一部を RiC-CM に写す例	初学者はここから
3	Implementation strategies	実装戦略。段階的な手順書は示さず、検討 しうる行動を挙げる	管理職
4	Combining RiC with other standards	RiC が置き換える ICA 標準、記録管理、 EAD/EAC-CPF、デジタル保存など	管理職 (§1・§3 の次に)
5	Mappings	4 標準→ RiC-CM、EAD 2002 → RiC-O 1.1 のクロスワーク (xlsx) と、その注記	手元に置いて参照する
6a–6g	FAQ	一般的な問い、記録か記録集合か、 具現化、デジタル化・評価選別・ 保存期間満了時の措置などの活動、 順序、語彙、拡張	実践で使う人 (初学者は各 FAQ の 導入から)
7	Case study	先住民コミュニティと少数者集団	実践で使う人、 中級・上級者、 理論に関心のある人
8	Extended example	セルバンテスと『ドン・キホーテ』の出版	実践で使う人、 中級・上級者、 理論に関心のある人 (§7 の次に)

「利用場面は多様すぎるため、段階的な手順書 (step-by-step guide) を提供することは不可能」と明言し、代わりに「検討しうる行動のリスト」を示す方針をとった。他の部でもこの姿勢は変わらず、規則の代わりに事例とダイアグラムが並ぶ。つまり RiC-AG が出ても、RiC は「これに従えばよい」型の標準にはならなかった。判断の支援はするが、判断そのものは実装者に残る—この設計思想は第3章・第7章で改めて論じる。

コラム: 仕様書とガイドライン — 期待できることの違い

RiC-CM を初めて読んだ人の多くが「結局、何をどう書けばいいのか分からない」という感想を持つ。これは文書の性格から来る自然な反応である。RiC-CM は語彙の定義集であり、そこに書かれているのは「世界の切り分け方」である。ISAD(G) は 26 の記述要素と適用規則をコンパクトに示していたので、記入すべき項目の一覧としてそのまま使えた。RiC-CM にはそれに当たる様式がない。2025 年に公開された RiC-AG 草案はこの落差を埋めるための文書で、事例を通じて判断の仕方を示すという形をとる (AG 自身、記録か記録集合かの判断について「決定的な正解・不正解はない」と書く)。手順を求めるなら AG と本書第7章、語彙の意味を確かめるなら CM、と役割を分けて読むのが实际的である。

1.3 一次資料の入手先と構成

RiC の一次資料はすべて無償で公開されている。RiC-FAD・RiC-CM・RiC-O は Creative Commons Attribution 4.0 ライセンスで利用できる (RiC-AG 草案には、ライセンスの明示が見当たらない)。GitHub リポジトリが開発と公開の場を兼ねているため、仕様書本体に加えて、版の履歴、コミュニティからのコメント (Issues)、公式サンプルデータまでが同じ場所に揃う「生きた一次資料」になっている。

表 1.3 RiC 一次資料の所在 (2026 年 7 月現在)

文書	最新版	主な入手先・内容
RiC-FAD	1.0(2023 年 11 月)	GitHub ICA-EGAD/RiC-FAD。PDF 本体と旧版 (RiC-IAD 0.2 ほか)
RiC-CM	1.0(2023 年 11 月)	GitHub ICA-EGAD/RiC-CM および ICA ウェブサイト。PDF 本体、旧版 (0.1、0.2、1.0-beta)、公開コメントへの回答記録
RiC-O	1.1(2025 年 5 月)	GitHub ICA-EGAD/RiC-O。OWL ソース (RiC-O_1-1.rdf)、HTML 版ドキュメント、クラス・プロパティの CSV 一覧、公式サンプル (仏国立公文書館ほか)、ダイアグラム集
RiC-AG	0.1 草案 (2025 年 10 月)	GitHub ICA-EGAD/RiC-AG およびウェブ版。本文 (Markdown/HTML)、公式クロスワーク (xlsx)、ダイアグラム (draw.io 形式)
関連	—	RiC-O 解説サイト (ica-egad.github.io/RiC-O)、プロジェクト一覧 (RiC-ResourceList)、利用者メーリングリスト (Records in Contexts users, Google Group)

RiC-O のリポジトリ構成は一次資料としてとりわけ有用なので、少し立ち入って紹介する。

- `ontology/current-version/RiC-O_1-1.rdf` — オントロジー本体 (RDF/XML 形式の OWL ファイル)。すべてのクラスとプロパティに英語・フランス語・スペイン語のラベルが付き (1.1 ではクラスに独語ラベルも加わった)、定義コメントは英語で与えられる。RiC-CM のどの構成要素に対応するかの注記 (`rico:RiCCMCorrespondingComponent`) も、対応があるものには付いている。事実上、これ自体が詳細版のリファレンスマニュアルである。なお RDF/XML は、RDF を XML の構文で書く形式の 1 つである。RDF そのものは第 2 章 2.3 節から扱う。
- `ontology/current-version/CSV_lists_of_components/` — クラス・プロパティの一覧表 (CSV)。オントロジーエディタを使わずに全体を見渡すのに便利で、RDF や OWL に不慣れでも扱える。
- `examples/` — 実データの例。実在の EAD・EAC-CPF(典拠レコードの符号化標準。第 2 章 2.1 節)・METS(デジタル資料の構造と各種メタデータを 1 つの XML にまとめる標準) のファイル

と、その RiC-O 版 RDF が対で置かれている。スイスの docuteam 社によるデジタル保存パッケージの例 (PREMIS との併用)、フランス国立公文書館の目録・典拠レコードの変換例、ストラスクライド大学文書館の例が含まれる。「実際のトリプルがどう書かれるか」を知る最良の教材である。ただし EGAD 自身が付属の説明で「最新でも代表的でもない」と断っている点には注意がいる。

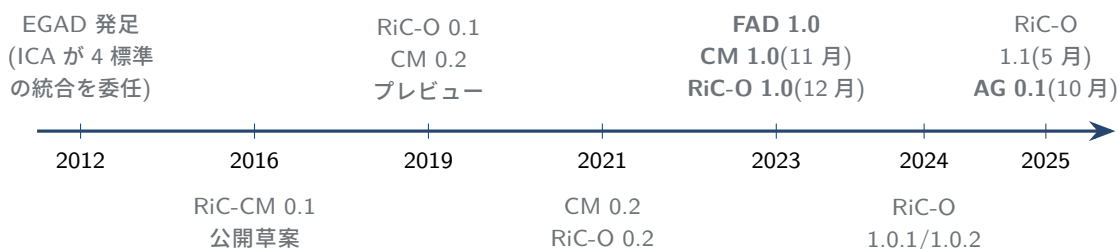
- diagrams/ — 概念図 (draw.io 形式の XML と、その JPEG 画像)。

1.4 開発の経緯

開発の経緯をたどると、どの版が何に対応しているか (たとえば RiC-O 0.2 は RiC-CM 0.2 対応、RiC-O 1.1 は RiC-CM 1.0 対応) が見えてくる。ウェブ上の解説記事やツールがどの版を前提にしているかは、この対応関係を手がかりに判別できる。表 1.4 に RiC-O の版と RiC-CM の版の対応をまとめた。オブジェクトプロパティの数が版ごとに違うので、解説記事に挙がっている数からも版の見当がつく。

表 1.4 RiC-O の版と、対応する RiC-CM の版 (RiC-O 1.1 の履歴ノートと、RiC-O・RiC-FAD リポジトリの README による。— は資料に数の記載がないもの)

RiC-O	公開	対応する RiC-CM	オブジェクトプロパティ数
0.1	2019 年 12 月	CM 0.2 プレビューと同時に公開	—
0.2	2021 年 2 月	0.2(2021 年 7 月公開) に準拠	423
1.0	2023 年 12 月	1.0(2023 年 11 月公開) に準拠	400
1.0.1、1.0.2	2024 年 5 月、9 月	1.0(RiC-O 1.0 の修正版)	—
1.1	2025 年 5 月	1.0 に準拠	480



経緯の要点は次のとおりである (RiC-CM §1.7、RiC-O 履歴ノートによる)。

- 2012 年、ICA が EGAD を設置し、既存 4 標準を「調停し、統合し、その上に築く」単一標準の開発を委任した。メンバーは 15 か国から延べ 31 名。
- 2016 年に最初の草案 RiC-CM 0.1 が公開され、19 か国から 62 件 (数百ページ分) のコメントが寄せられた。このフィードバックを受けて構成が全面的に再編された。
- 2019 年 12 月に RiC-O 0.1 が公開。RiC-O はフランス国立公文書館の実証実験 (PIAAF) などですぐから実地に試されてきた。

- 2023 年 11 月に FAD 1.0 と CM 1.0、12 月に RiC-O 1.0 が公開された。RiC-O のリポジトリは 1.0 を「最初の安定した完全版」と呼んでいる。
- その後も RiC-O は改訂が続き、2025 年 5 月の 1.1 が最新である (CM 1.0 に準拠)。クラス数 107、オブジェクトプロパティ 480 で、0.2 の頃 (オブジェクトプロパティ 423) より語彙は増えている。
- 2025 年 10 月、バルセロナの ICA 大会で RiC-AG の最初の草案 0.1 が公開され、4 部がすべて (草案を含め) 出揃った。

EGAD 自身が代表性の限界を認めている。メンバーは「世界のアーカイブズ共同体をおおむね代表している」としつつ、「行政と統治の長く優れた歴史を持ち、それに伴う記録の作成・利用・管理の伝統を持つ多くの地域が代表されていない。アジアの多くと東欧がそうである」と述べ、「アフリカと南米は代表されてきたが、その代表性はより広く、より包摂的であるべきだ」と続ける (RiC-CM §1.7)。RiC が日本のアーカイブズ実務に合うかどうかは、これから日本のコミュニティが検証しフィードバックしていくべき課題として残されている。

1.5 この標準は誰のためのものか

RiC-CM §1.3 は想定読者を列挙している。順に、アーカイブズの専門家 (第一の読者)、レコードマネジャー、隣接する文化遺産コミュニティ (図書館・博物館)、システム開発者、そして研究者である。初学者にとって示唆的なのは、システム開発者に向けた次の一節である。「RiC-CM は高水準とはいえ詳細で複雑である。しかしシステムの開発者は、複雑さを覆い隠すユーザインタフェースを設計・実装することで、データ作成と維持の知的・技術的・経済的負担を軽減できる」(RiC-CM §1.3)。つまり EGAD は、実務者が RiC の複雑さと生のまま格闘するのではなく、道具がそれを媒介することを期待している (§1.6.2 も、RiC-CM に準拠したソフトウェアが開発されることへの期待を述べる)。RiC を学ぶことは、生の RDF を手書きする技能を身につけることではなく、道具が扱う概念の意味を理解することである。この視点は第 8 章で再論する。

確認問題 (ヒントは付録 A.6)

- 問 1.1** RiC の 4 つの部分 (FAD・CM・O・AG) の役割を、それぞれ一文で説明せよ。
- 問 1.2** RiC-CM のバージョンが 1.0 のまま、RiC-O だけが 1.1 に進んでいる。概念モデルとオントロジーが別々に改版されるのはなぜか、両者の役割の違いから説明せよ。
- 問 1.3** 自分の目的 (概念の理解/実装/実務の指針) を 1 つ選び、4 部のどれをどの順で読むのが近道か、本章の内容から組み立てよ。

第 2 章

ISAD(G) の世界から RiC へ

RiC を理解する早道は、それが置き換えようとする従来標準を正確に理解することである。本章では ICA の既存 4 標準を確認し、それらがどんな世界観に立っていたか、どこに限界があったかを、RiC-CM 自身の分析 (§1.5–1.6) に沿って整理する。続いて、記述どうしをつなぐ**参照**と**識別子**がどう扱われてきたかを、本書の観点からたどる (2.4 節)。そのあと ISO 23081 や図書館・博物館などの隣接する標準との関係を見て (2.5 節、2.6 節)、最後に RiC-CM が描く移行の見通しを確かめる。

2.1 ICA の 4 つの記述標準

表 2.1 ICA の既存 4 標準

標準	初版/2 版	記述の対象
ISAD(G)	1994/2000	記録 (フォンドとその構成部分)。26 の記述要素と多段階記述の規則
ISAAR(CPF)	1996/2004	記録の作成者など (団体・個人・家族) の典拠レコード
ISDF	2007	機能 (組織の活動・業務)
ISDIAH	2008	所蔵機関

ISAD(G)(General International Standard Archival Description) は 1994 年に初版が出た、最も広く普及した標準である。フォンドを頂点とする多段階記述 (multilevel description) を定式化し、日本を含む世界の目録実務とシステム (および符号化標準 EAD) の土台になってきた。

ここで用語を 1 つ整理しておく。本書にはこの先「検索手段」と「目録」の両方が登場する。両者は、同じものを広さの違う言葉で呼び分けたものである。**検索手段** (finding aid) は、利用者が記録にたどり着くための道具の**総称**で、英語圏の標準文書はこの語を使う (訳語はアーカイブズ学で定着したもの)。目録、ガイド、索引などはその種類であり、日本の実務で圧倒的になじみ深い検索手段が**目録**である。だから ISAD(G) は「検索手段 (の中身となる記述) の標準」であり、その成果物は日本ではふつう目録として現れる。本書では、標準や符号化 (EAD) の話では原語に合わせて検索手段と書き、具体物としてのなじみを優先する場面では目録と書く。

残る 3 標準は、記述の主要な構成部分を**分離**する構想の産物である。ISAAR(CPF) は「作成者の記述」を記録の記述から切り離して独立に維持することを可能にし (その符号化標準が EAC-CPF)、ISDF は「機能」を、ISDIAH は「所蔵機関」を独立の記述対象とした。分離の狙いは、同じ作成者情報を複

数のフォンドの記述で使い回せるようにし、記述の経済性と正確さを高め、さらに新しい切り口からのアクセスを可能にすることだった。

しかし RiC-CM §1.6.1 は、この 4 標準体制の問題を率直に指摘する。4 標準は 14 年にわたり**ばらばら**に開発され、「分離された構成部分をどう関係づけて全体の記述を形づくるか」についての一貫した構想を欠いていた。その結果、4 標準は「首尾一貫した単一の記述モデル」を構成していない。普及の実態にも差があり、ISAD(G) は広く使われ、ISAAR(CPF) はある程度使われ、ISDF と ISDIAH はほとんど使われていない。ICA が EGAD に委任したのは、この寄せ集めを、最初から一貫した単一の多実体モデルとして設計し直すことだった。

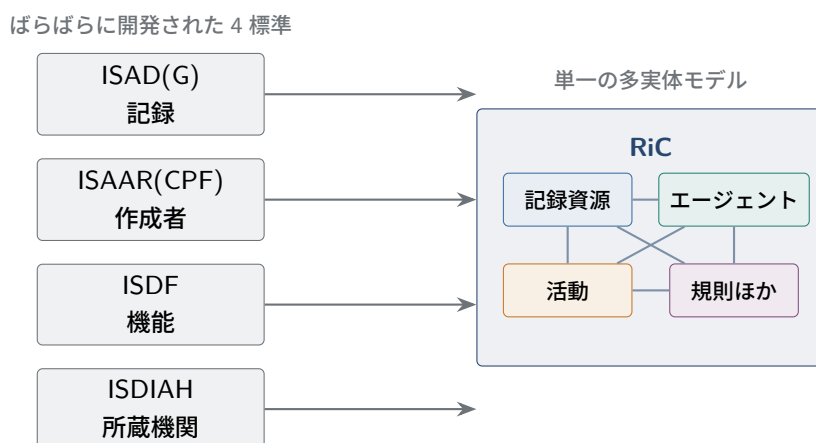


図 2.1 4 標準から RiC へ。分離されていた記述対象が、関係で結ばれた実体として 1 つのモデルに統合される。

2.2 従来の記述を支えていた前提

RiC-CM §1.5–1.6 の分析によれば、ISAD(G) 的な記述は次の前提の上に成り立っていた。

■(1) 伝統的な出所原則 記述は出所原則 (Principle of Provenance) の伝統的理解、すなわちフォンド尊重 (respect des fonds: ある個人・団体が生活や業務の中で作成・蓄積・使用した記録はひとまとまりに保ち、他と混ぜない) と原秩序尊重 (respect for original order: 蓄積・使用の文脈で形成された記録間の相互関係を保存する) に基づく。

この 2 つは役割が違う。フォンド尊重が定めるのは記録群の**外側の境界** (どこまでを 1 つのまとまりとして扱うか) であり、原秩序尊重が定めるのは**その内側の並び** (まとまりの中をどう保つか) である。境界は記述者が引くのではなく、蓄積した主体の存在によってあらかじめ与えられる、という点が重要で、だからこそ第 3 章で見る RiC の「1 つの記録が複数の記録集合に属せる」という設計が大きな変更になる。

なお、この 2 つはもともと一体だったわけではない。19 世紀フランスの respect des fonds は、フォンドの内部の秩序 (respect de l'ordre intérieur) までは必ずしも要求していなかった。フォンドの一体性に加えて原秩序 (Registraturprinzip) の維持も定めたのは 1881 年のプロイセン国立文書館の規程であり、2 つを整理と記述の規則として示して広く影響を与えたのが 1898 年の**ダッチマニュアル**^{*1}であ

^{*1} オランダの 3 人のアーキビスト Muller・Feith・Fruin による Handleiding voor het ordenen en beschrijven van

る。今日ふつう「出所原則には2つの側面がある」と説明されるのは、この結合の結果である。RiC もこの2原則自体は「方法論として今も健全」と再確認している (RiC-CM §1.5.3)。問題は、伝統的理解が出所を「ファンドを蓄積した単一の主体」に切り詰めてきたことにある (第3章)。

■(2) 単一ファンドの自己完結的・階層的記述 記述はファンドの記述から始まり、その構成部分、さらにその構成部分へと降りていく。1つのファンドの記述は内向きに閉じ、外の世界との関係をほとんど持たない。

■(3) 入力と出力の同一視 記述システムへの入力 (データ) と利用者への提示 (出力) が同じものだと暗黙に想定されてきた。ISAD(G) は建前上これを否定している (I.6 節) が、要素の線形な並びは紙の検索手段をなぞっており、実装も「印刷物に似た画面」を作りがちだった。

■(4) アクセスポイントによる代用 記録の記述の中に、人物・団体・場所・主題などは「アクセスポイント」という形で名前だけ埋め込まれ、それ自体は記述されない。ISAAR(CPF) 以降はこの限界への対応だったが、体系的な統合には至らなかった。

4つの前提のそれぞれに RiC-CM がどう応えるかを、表 2.2 にまとめた。いずれも第3章以降で詳しく見る。

表 2.2 ISAD(G) 的な記述の4つの前提と、RiC-CM の対応

従来の前提	RiC-CM の対応	本書で扱う箇所
(1) 伝統的な出所原則	2つの側面を方法論として健全と認めたうえで、出所を、他の記録・活動・人々との関係の網として捉え直す (§1.5.3)	第3章 3.3 節
(2) 単一ファンドの自己完結的・階層的記述	多段階記述を多次元記述に置き換える。従来の階層はその中にそのまま収まる (§1.6.2.2)	第3章 3.2 節
(3) 入力と出力の同一視	入力の概念構造だけを定め、出力とユーザインタフェースは縛らない (§1.6.2)	第3章 3.1 節
(4) アクセスポイントによる代用	人物・団体・活動・場所なども独立の実体として記述する (§1.6.2)	第3章 3.1 節、 第4章 4.2 節

コラム: RiC から見た ISAD(G) の評価

RiC-CM は ISAD(G) を高く評価している。「ISAD(G) に従った記述に本質的に誤ったところは何もない」と明言し (§1.6.2)、ファンド単位の階層的記述が当面は支配的であり続けるだろうとも述べている (§1.6.1)。理由も具体的で、伝統的出所原則に対応していること、コミュニティに深く理解されていること、既存のシステムと方法が揃っていること、そして複雑で労働集約的な課題への比較的経済的な解であることが挙げられている。RiC が着目するのは ISAD(G) の表現力の上限である。現在の通信技術で可能になったこと、利用者が期待するようになったことに応えるに

は、より広い枠組みが要る—これが RiC の出発点である。

2.3 記述と通信技術

RiC-CM §1.5.2 は、アーカイブズ記述が利用可能な通信技術に依存して形を変えてきたことを強調する。この節は本書の後半 (第 5 章・第 8 章) への伏線になるので、丁寧に追っておく。

20 世紀末の 20 年間に、2 つの技術がアーカイブズ記述を紙からコンピュータへ移行させた。1 つはマークアップ技術で、文書の中に「タグ」を埋め込んで構造を示す。その代表が XML、アーカイブズでの応用が EAD である。もう 1 つはリレーショナルデータベース (表の形でデータを管理する方式。操作言語が SQL) である。RiC-CM の観察が面白いのは、「どちらの技術も支配的になりきれなかった」事実を、フォンド記述そのものの性質から説明する点である。フォンド単位の記述は、文書のような部分 (来歴の散文など) と表のような部分 (統制された要素値など) が混在する「どっちつかず (betwixt and between)」の情報であり、木構造の文書として扱う XML にも、表の集まりとして扱う RDB にも、部分的にしか適合しない。だから実装は両者を使い分け、あるいは慎重に組み合わせてきた。

XML は本書の各所に顔を出すので、文法の基礎をここでまとめておく。

コラム:XML の文法 — これだけ知れば読める

XML(Extensible Markup Language) は、文書に構造の印を書き込むための W3C 標準である。印の付け方だけを定める規格で、どんなタグを使うかは応用ごとに決める。EAD はアーカイブズ記述のためのタグの決め方、RDF/XML は RDF のためのタグの決め方である。

要素: 構造の単位を**要素** (element) と呼び、開始タグ `<unittitle>` と終了タグ `</unittitle>` で内容をはさむ。内容が空なら `<dao/>` と 1 つで書ける。

属性: 要素に付ける補助的な値を**属性** (attribute) と呼び、開始タグの中に **名前="値"** の形で書く。値は必ず引用符で囲む。

入れ子: 要素の中に要素を入れられる。開始と終了は交差してはならず、`<a><b></a></b>` は誤りである。この規則があるので、XML 文書の構造は必ず木になる。最も外側の要素はちょうど 1 つ (ルート要素) である。

```
<c level="series" id="ser02">
  <did>
    <unitid>P-83/2</unitid>
    <unittitle>防疫統計系列</unittitle>
    <unitdate normal="1948/1994">昭和23年-平成6年</unitdate>
  </did>
  <c level="file" id="fil07">
    <did><unittitle>伝染病発生報告綴</unittitle></did>
  </c>
</c>
```

この断片は EAD の一部で、シリーズ要素 `<c>` の中にファイル要素 `<c>` が入れ子になっている。記述の階層が、タグの入れ子でそのまま表される。この断片を木として描くと図 2.2 のようになる。

空白と改行: タグの外側の字下げや改行は、読みやすさのために入れるのが普通で、多くの応用では意味を持たない。ただし XML の規格上は要素の内容の一部として扱われ、読み込んだ側のプログラムにそのまま渡されるため、「無視してよい空白かどうか」は応用ごとの取り決めになる。この点は Turtle(第5章)と違って割り切れていない。

整形式と妥当: タグの入れ子が正しく、引用符が閉じているなど、文法規則を満たす文書を**整形式** (well-formed) という。さらに「どのタグをどこに置いてよいか」を定めたスキーマ (EAD なら DTD や XML Schema) に適合する文書を**妥当** (valid) という。整形式でなければそもそも読み込めず、妥当でなければ応用の要求を満たさない。

その他の約束: 文書の先頭には、文字コードなどを宣言する行を置くのが普通である (省略もできる)。

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- ここはコメント。データとしては読めない -->
```

コメントは <!-- から --> までである。< と & は構文に使う文字なので、内容に書きたいときは < と & に置き換える。要素名と属性名は大文字小文字を区別する。

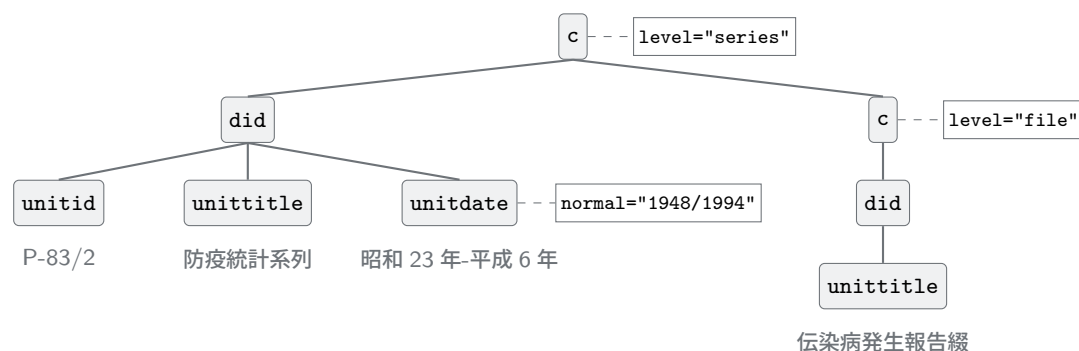


図 2.2 EAD 断片の要素の木。入れ子がそのまま親子になる。灰色の箱が要素、破線でつないだ白い箱が属性、箱の下の小さな文字が要素の内容である (id 属性は省いた)。

XML と RDB の 2 つに対して、RiC-CM は第 3 の技術としてグラフ技術を挙げる。W3C の RDF (Resource Description Framework) に代表されるグラフ技術は、データをノード (実体) とアーク (関係) の網として表現し、関係をたどる問い合わせを可能にする。

ここでの**グラフ** (graph) は、棒グラフや折れ線グラフのことではない。数学でいうグラフ、すなわち**点と、点をつなぐ線からなる構造**を指す。点を**ノード** (node) または頂点、線を**アーク** (arc) または辺と呼ぶ。路線図が駅 (点) と線路 (線) からなり、家系図が人 (点) と血縁 (線) からなるのと同じ構造である。アーカイブズ記述に当てはめれば、記録・人・活動が点、それらの間の関係が線になる。図 2.3 の右側がその形である。木 (階層) はグラフの特殊な場合で、すべての点がつながり循環がなく、根には親がなく、それ以外の各点には親が 1 つある。だから階層記述はグラフの中にそのまま収まる。

ここで一点、正確を期しておく。「XML は木しか表現できない」のではない。XML の**構文**が直接与える構造が木 (タグの入れ子) なのであって、要素に ID を振って参照し合う規約を載せれば、グラフも表現できる。現に RDF の XML 形式 (第 5 章) も、グラフ記述言語 GraphML も XML である。ただしその場合、グラフの構造は構文ではなく**規約の側**にあり、入れ子という構文の後押しを受けられるの

は木の部分だけである。EAD で階層だけが入れ子で自然に書け、それ以外の関係が ID 参照の記載にとどまるのは (第 7 章 7.4 節)、この非対称の現れである。これに対し RDF は、**データモデルそのものがグラフ**であり、どの関係も同じ資格のトリプル (triple) として書ける。トリプルとは「主語・述語・目的語」の 3 語でできた文のことで、1 本のアークを 1 文で表す (第 5 章 5.1 節)。

「現実の世界は、空間と時間の中に位置づけられた人と物の、動的に相互関連する広大なネットワークとして理解できる」のだから、グラフのほうが現実 に即した表現形式だ、というのが RiC の技術選択の論理である。RiC-CM はグラフとしての記述の**概念的な基礎**を与え、RiC-O がそれを RDF グラフとして実装する。

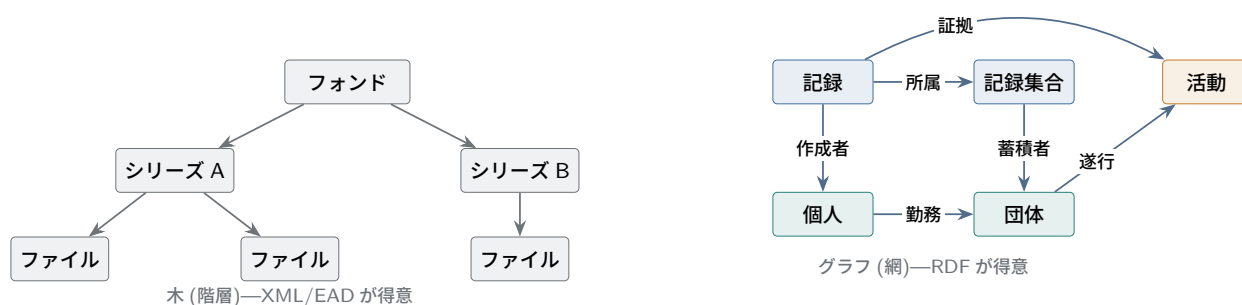


図 2.3 木からグラフへ。階層は「親子」1 種類の関係しか表せないが、グラフでは関係の種類も向きも自由に増やせる。なお木はグラフの特殊な場合なので、階層記述はグラフの中にそのまま収まる。

2.4 参照と識別子 — 記述をつなぐ仕組み

RiC-O のデータが目指す姿は Linked Data、すなわち「つながったデータ」である。この *Linked* の中身、つまり**参照** (ある記述の中から別のものを名指すこと) は、本書全体を通じて使う考え方である。ところが参照は、実務では整理番号や典拠 ID という地味な姿で現れるため、「管理の都合」程度に受け取られがちで、その役割の大きさに見合う扱いを受けてこなかった。本節では、参照が何のためにあるのか、参照には何が必要なのか、参照にはなぜ向きがあるのか、という基本から積み上げ、最後に参照手段の歴史を追う。ここを押さえておくと、第 5 章 (RDF) と第 7 章 (変換の実際) の意味がよく見えるようになる。

2.4.1 参照は何のためにあるか

記述とは、世界を切り分けて書いた断片の集まりである。フォンドの記述、その中のシリーズの記述、作成者の記述—これらの断片は、ばらばらに置かれていては役に立たない。断片同士をつなぐ仕組みが参照である。

参照の第一の効用は、**同じことを二度書かなくてよくなる**ことである。ある県庁が作成したフォンドが 30 件あるとする。参照がなければ、県庁の沿革を 30 件の記述それぞれに書き写すことになり、組織改編が起きれば 30 箇所を直して回ることになる (そして必ず直し漏れる)。参照があれば、県庁の記述は 1 つだけ作り、30 件の記述はそれを**指す**。事実は 1 箇所にだけ書かれ、更新も 1 箇所で済む。データベース設計で**正規化** (normalization) と呼ばれる原則が守ろうとしているのも、これと同じことである。

コラム: 正規化とは何をするのか

正規化は、1枚の表に混ざっている複数の対象を、対象ごとの表に分ける作業である。目録を1枚の表で管理している場面を考える。作成者は第7章の通し例と同じ某県衛生部で、1994年に保健福祉部へ改組されたので、ここではその年を廃止年の列に書いている。

資料 ID	資料名	作成者名	設置年	廃止年	上位機関
A-1	衛生行政文書	某県衛生部	1948	1994	某県庁
A-2	伝染病統計	某県衛生部	1948	1994	某県庁
A-3	検疫記録	某県衛生部	1948	1994	某県庁

作成者に関する4列(作成者名・設置年・廃止年・上位機関)が、資料の件数だけ繰り返されている。これが引き起こす不都合は3つある。第一に、設置年の誤りが見つかったとき、直す場所が資料の件数だけある(**更新時の不都合**)。1箇所でも直し漏れれば、同じ組織に2つの設置年が記録されることになる。第二に、まだ1件も資料を受け入れていない組織を登録できない(**挿入時の不都合**)。作成者の情報を書く場所が資料の行しかないからである。第三に、ある組織の資料をすべて廃棄して行を消すと、その組織自体の情報も消える(**削除時の不都合**)。

原因ははっきりしている。「資料」と「作成者」という**別々の対象**についての事実が、1枚の表に同居していることである。そこで表を分け、資料の側からはIDで参照する。

資料表

資料 ID	資料名	作成者 ID
A-1	衛生行政文書	P-01
A-2	伝染病統計	P-01
A-3	検疫記録	P-01

作成者表

作成者 ID	名称	設置年	廃止年	上位機関
P-01	某県衛生部	1948	1994	某県庁

設置年は表の中に1度しか現れないので、直す場所は1箇所である。まだ資料のない組織も作成者表に登録できる。資料をすべて消しても組織の情報は残る。3つの不都合が同時に解消した。

ただし、これで扱えるのは「1件の資料に作成者は1人」という場合だけである。共同作成のように多対多になると、作成者IDの列に複数を書き込むわけにいかない。そこで**関係そのものを表にし**、資料表からは作成者IDの列を外す(次の表は一部の行だけを示す)。

資料－作成者表

資料 ID	作成者 ID	役割
A-1	P-01	起案
A-1	P-02	決裁
A-3	P-01	起案

この第3の表を**結合表**と呼ぶ。行1つが「どの資料と、どの作成者が、どういう関係にあるか」を表す。役割の列に注目してほしい。これは資料の属性でも作成者の属性でもなく、**両者を結ぶ関係の属性**である。関係を独立の行に立てたからこそ書ける。

以上の3段階は、本書がこれから追う道筋とそのまま重なる。作成者を別の表に分けることが ISAAR(CPF) の典拠レコードに当たり (本章冒頭)、結合表を立てることが RiC の関係の実体化に当たる (第4章 4.4.4 項、第5章 5.5.3 項)。RDB を設計した経験があれば、RiC-CM の実体・属性・関係はこの延長として読める (第8章 8.3 節)。

第二の効用は、より根本的である。そもそも**検索手段それ自体が、現物への参照の仕掛け**なのである。記述の価値は、それを読んだ利用者が (あるいはシステムが) **現物にたどり着ける**ことにあり、そのたどり着く手段がレファレンスコード (請求記号) である。つまりアーカイブズ記述は、記述→現物、記述→記述 (上位・下位・関連)、記述→典拠という参照の網としてはじめて機能する。参照は記述の存在意義そのものを担っている。

2.4.2 参照には識別子が必要である

参照が成り立つためには、指す先を**間違いなく特定できる名前**が要る。これが**識別子 (identifier)**である。「山田花子」のような名前の文字列は識別子になれない。同名の別人がいるし (一意でない)、表記がゆれるし (同定できない)、改姓・改称で変わってしまう (永続しない)。使い物になる識別子の条件は3つある。条件1と条件2は、上の欠点のうち1つ目と3つ目の裏返しである。条件3は条件2を守るためのものである。2つ目の欠点 (表記のゆれ) は、名前とは別に識別子を持つこと自体で解消する。

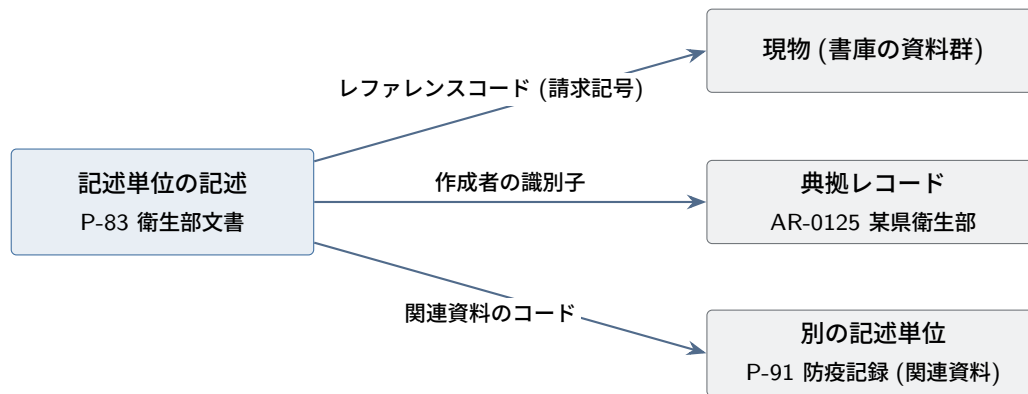
1. **一意である**: 通用する範囲の中で、1つの識別子は1つのものだけを指す。
2. **永続する**: 一度与えた識別子は変えない。変えれば、それを指していたすべての参照が壊れる。
3. **意味に依存しない**: 識別子に意味 (組織名や分類) を埋め込むと、実態が変わったとき識別子ごと変えたくなり、条件2と衝突する。「衛生部-1948-005」という番号は、衛生部が改組された時点で現在の組織名と食い違う。だからといって付け直せば条件2に反する。意味を埋め込まないのは、この板挟みを避けるためである。

ここまで来ると、ISAD(G) の設計の見え方が変わるはずである。ISAD(G) は26の記述要素のうち、国際交換に不可欠な必須要素を6つだけ挙げるが、その**筆頭**がレファレンスコード (3.1.1) である。タイトルよりも日付よりも先に、識別子が来る。しかも国コード (ISO 3166)+ 機関コード + 個別番号という構造まで指定して、機関の外でも一意性が保たれるように設計されている。この並びは次のように読める。**識別子を持つこと—参照の受け手になれること—が、記述単位が記述単位であるための条件**なのであり、整理番号はその条件を満たす手立てである。多段階記述は上位と下位の記述が互いを指せてはじめて成立するし、他機関との交換も、利用者による出典表示も、すべて識別子を前提にしている。

2.4.3 参照には向きがある

参照は「AがBを指す」という**非対称**の行為である。指す側と指される側があり、記載は指す側に置かれる。図2.4は、1つの記述単位から出る3種類の参照を示す。

記載をどちら側に置くかについて、文書の世界でよくとられるのは「**多数の側が、1つの側を指す**」形である。30件のフォンド記述がそれぞれ1つの典拠レコードを指し、典拠レコードの側には30件のフォンドを列挙しない。指す側に置けば、記載は各記述に1行で済み、新しいフォンドが増えても典拠



3 本とも「記述の側が指す」向き。逆向きの一覧（この典拠を指す記述はどれか）は、
文書の世界では、別に索引を作らない限り得られない

図 2.4 1 つの記述から出る 3 種類の参照。検索手段は現物・典拠・他の記述への参照の網として機能する。参照の記載は指す側に置かれ、向きを持つ。

側を触らなくてよいからである。ISAAR(CPF) や EAC-CPF には典拠の側から記録を列挙する要素もあるが、それは別の文書の中の別の記載であり、両者を一致させる仕組みはない（第 7 章 7.4 節）。

しかしこの形には、避けがたい代償がある。**逆向きにたどれない**のである。「この典拠レコードを指しているフォンドをすべて挙げよ」という、利用者にとってごく自然な問い（この人の記録はほかにどこにあるか）に、文書の集まりは直接答えられない。全文書を総なめするか、あらかじめ逆引きの索引を別に作って維持するしかない。紙の世界の相互参照（「→～も見よ」）や、データベースの索引は、この代償を人手と手間で払う仕組みである。そして、参照を双方向に、しかも自動でたどれるようにすることこそ、前節で見たグラフ技術と、第 6 章で見る推論（逆プロパティ）が解く問題である。参照の向きの定石と限界をここで押さえておくと、後の章の議論がすべてこの延長線上にあることが分かる。

2.4.4 参照手段の歴史 — 名前の文字列から IRI へ

以上の道具立てで、アーカイブズ記述の歴史を「参照手段の強化の歴史」として読み直してみよう。

■**第 1 期: 紙の目録 — 人・団体への参照は文字列、解決は人間** 紙の検索手段にも、識別子による参照はすでにあった。記述から現物への参照（請求記号）がそれである。文字列のままだったのは、**人物・団体・主題への言及**である。同一人物の別表記を束ね、別人の同名を区別する仕組みとして**名称統制**（典拠形の統一）が図書館の世界で発達し、「→～を見よ」という相互参照も整備されたが、これらはすべて**人間の読者**に向けた仕組みである。名指された相手に実際にたどり着く作業—参照の**解決**（resolution）と呼ぼう—is、人間が目録室で行うものだった。

■**第 2 期: ISAD(G) とアクセスポイント — 統制された文字列** ISAD(G) の記述では、人物・団体・場所・主題は、記述要素の中の名前や、それに基づく**アクセスポイント**として記録の記述に埋め込まれる（RiC-CM §1.6.2）。検索の入口としては前進だが、参照の実体は依然として文字列であり、名指された人物・団体それ自体は記述されない。同名異人・改称・組織改編に構造的に弱い。誤解のないように言えば、ISAD(G) が識別子を軽んじていたわけではない。2.4.2 項で見たとおりレファレンスコードは必須要素の筆頭であり、指す相手が**記録**であれば識別子で参照できた（関連する記述単位を指す要素 3.5.3

にはレファレンスコードを書ける)。文字列にとどまったのは、指す相手が**人・団体・主題**の場合である。「記録へは識別子で、行為者へは名前で」という非対称がこの段階の実像であり、その後半分を解いたのが次の ISAAR(CPF) である。

■第3期:ISAAR(CPF) と典拠レコード — 文字列からレコード ID へ ISAAR(CPF) は作成者の記述を独立の**典拠レコード**に切り出し、典拠レコードそのものに識別子を与えた(5.4.1)。記録の記述と典拠レコードは、この**識別子**を介して結べるようになった (ISAAR(CPF) 自身は第6節で、典拠レコードの側に関連資料の識別子を書く要素を定めている)。参照が「名前の文字列」から「レコード ID」へ格上げされた、記述史上の画期である。ただし ID の通用範囲は機関 (せいぜい国の典拠システム) にとどまり、ID から相手のレコードを機械的に取得する手続きは、各システムの内部事情に任された。

■第4期:EAD と EAC-CPF — 参照の機械可読化 符号化標準は参照を機械が読める属性にした。EAD の `authfilenumber`、EAC-CPF の関係要素が持つリンク属性などである。しかし3つの限界が残った。第一に、ID や URL が将来も有効である保証は運用次第である。第二に、参照の**意味** (なぜ指すのか) は要素名による粗い分類にとどまる。第三に、参照は各文書の中の記載なので、双方向の整合は保証されない。この第三の点は第7章7.4節で図とともに詳しく扱う。

■第5期:RiC と LOD — 世界規模の名前と、参照でできたデータモデル RiC-O が乗る RDF は、この系譜の到達点に当たる。第一に、名前が **IRI**(第5章) になり、名前を区別する範囲が機関内から世界規模へ広がる。同じ IRI を別のものに再利用しない管理は、発行者が担う。第二に、参照の意味がプロパティという標準化された語彙で与えられ、「作成した」のか「蓄積した」のか「所蔵している」のかは機械可読に区別される。第三に、これが最も深い変化だが、RDF では**データモデルそのものが参照でできている**。トリプルの目的語に IRI を書くこと、それ自体が参照であり、参照の集まりがそのままグラフである。参照はここで記述の本体そのものになる。

表2.3に5つの期を並べた。人物・団体を名指す形が文字列から ID、IRI へと移り、それにつれて一意に名指せる範囲と、相手へのたどり着き方が変わってきたことが読み取れる。図2.5は、そのうち名指しの形の変化を3段階に絞って描いたものである。

表 2.3 人物・団体への参照手段の5期 (本節の記述をまとめたもの)

期	人物・団体を名指す形	一意に名指せる範囲	名指した相手へのたどり着き方 (解決)
1 紙の目録	名前の文字列。名称統制と「→～を見よ」で補う	なし	人間が目録室で行う
2 ISAD(G)	記述要素の中の名前と、それに基づくアクセスポイント	なし	人間が行う
3 ISAAR(CPF)	典拠レコードの ID	機関の中 (せいぜい国の典拠システム)	各システムの内部事情に任される
4 EAD、EAC-CPF	機械が読める属性 (<code>authfilenumber</code> など)	同上	機械で読めるが、ID や URL が有効であり続けるかは運用次第
5 RiC、LOD	IRI	世界規模	ウェブの標準の仕組み



図 2.5 人物・団体への参照の形の変遷。名指しの一意性が通用する範囲と、名指された相手にたどり着く手段(解決)が段階的に強化されてきた。

この節の見取り図は、以後の議論で繰り返し使う。矢印と参照の向きの読み方は第5章 5.1.2 項、文書間参照・外部キー・グラフの辺という参照アーキテクチャの比較は第7章図 7.2、IRI をだれがどう発行し維持するかというインフラの問題は第7章 7.6 節で扱う。

2.5 ISO 23081 との関係

アーカイブズ標準の外にも近縁者がいる。ISO 23081(記録管理のメタデータ)は、記録を作成の時点から証拠として管理するためのメタデータ枠組みであり、RiC-CM とのあいだに「目的の実質的な共通性」がある(RiC-CM §1.6.3)。RiC-CM の多実体モデル自体、ISO 23081 の枠組み(記録・エージェント・業務・マナデート(権限)などの実体を関係づける)から強い影響を受けたと明記されている。実務にとっての意味は、レコードマネジメントの段階で作られたメタデータをアーカイブズの記述に再利用する流れが、今後いっそう重要になるということである。両者の実体モデルにはなお差異が残るが、収斂させていくことが目標として掲げられている。

2.6 隣接標準との関係 — GLAM の隣人たち

文化資源を記述する仕事は、隣接する分野と分け合っている。美術館・図書館・文書館・博物館は、文化資源を収集・保存し記述するという共通の仕事を持つことから、英語の頭文字を取って **GLAM**(Galleries, Libraries, Archives, Museums) と総称される。それぞれのコミュニティが、記述の標準を並行して発展させてきた。RiC の位置は、これらの隣人と並べたときによく見える—そして実務では、これらと**組み合わせ**て使うことになる。以下では RiC の側の用語を先取りして使う箇所がある。とくに**具現化**(instantiation)は、記録の内容そのものと区別して、それが紙やファイルという担体の上に記録された現れを指す(第3章 3.5 節)。IIIF と PREMIS との接点はここにある。

■**CIDOC CRM — 思想上の最近傍** 博物館の CIDOC CRM は、文化財の情報を「モノ」の属性の束ではなく**出来事の連鎖**(誰がいつ制作し、誰が取得し、どこへ移動したか)として記述するオントロジーで、ISO 21127 として標準化されている。関係を第一級に据え、グラフを前提とし、来歴を出来事で追う—RiC が 2010 年代に選んだ道を、博物館界は一足早く歩いていた。違いは焦点である。CRM は文化財一般の汎用出来事モデルで抽象度が高く、RiC は記録と出所というアーカイブズ学の関心に特化する。両者の意味的マッピングは研究として行われており(RiC-CM から CIDOC CRM への対応付け)、博物館所蔵の文書資料や、文書館所蔵の美術資料のような境界事例で将来役に立つ。

表 2.4 GLAM 領域の隣接標準と RiC の接点

標準	コミュニティ	記述の中心単位	RiC との関係
CIDOC CRM (ISO 21127)	博物館	出来事 (制作・取得・移動…)	関係中心・グラフ前提という思想が最も近い。意味的マッピングの研究がある
RDA / IFLA LRM、 BIBFRAME	図書館	著作—表現形—体現形—個別資料 (LRM)、 Work—Instance—Item (BIBFRAME)	内容と担体の分離という発想を共有。名称典拠の蓄積と識別子を借りられる
IIIF	画像・AV 配信	マニフェスト (閲覧単位)	メタデータ標準ではなく配信 API。具現化から接続し、閲覧体験を分業
PREMIS	長期保存	保存イベント・フォーマット	知的記述 (RiC) と保存管理 (PREMIS) の分業。具現化が接点
ISO 23081	レコードマネジメント	記録と業務文脈	前節参照。RiC-CM の多実体モデルに強い影響を与えた

■**図書館 — 内容と担体の分離、そして典拠** 図書館の概念モデル IFLA LRM(目録規則 RDA の土台)は、著作 (内容)—表現形 (内容の実現)—体現形 (版)—個別資料 (現物) という階層で書誌世界を切り分ける。米国議会図書館の BIBFRAME 2.0 は別のモデルであり、Work・Instance・Item の 3 層を中心に RDF で記述する。「同じ内容の複製が多数ある」世界だからこそ発達したこの内容/担体の分離は、RiC の記録資源/具現化の区別 (第 3 章) にも通じる。ただし、各モデルは目的と定義が異なり、層どうしがそのまま対応するわけではない。実務上より重要なのは典拠である。人物・団体・主題の名称統制は図書館が最も厚い蓄積を持ち、VIAF や各国の典拠 (日本なら Web NDL Authorities) は IRI で参照できる。RiC の Agent 実体をこれらに `owl:sameAs`(2 つの IRI が同じものを指すと宣言する OWL のプロパティ) で結べば、図書館界のインフラをそのまま借りられる (第 7 章 7.6 節)。ただし、リンク先が人物・団体そのものを指す IRI であることを確かめる。典拠レコードや紹介ページの IRI とは同一視できない。

■**IIIF — 閲覧との分業** IIIF(International Image Interoperability Framework) はメタデータ標準ではなく、画像・視聴覚資料の**配信と表示**の API 標準である (画像 API と、閲覧単位をまとめるマニフェスト=JSON-LD)。RiC との接点は具現化にある。デジタル化画像を RiC 側では具現化の実体として記述し、そこから IIIF マニフェストへつなげば、「何であるか」(RiC のグラフ) と「どう見せるか」(IIIF ビューア) を分業できる。

IIIF との接続を、本書で最初の RDF の断片で示す。ここで使っている Turtle という記法は、本書がこの先ずっと用いるものだが、文法は第 5 章 5.1 節のコラムでまとめて説明する。それまでは、次のことだけ頭に入れておけばよい。「主語 述語 目的語」の 3 語で 1 つの文を作り、ピリオドで文が終わる。セミコロンは主語が同じ次の文へ続くことを、述語の位置の `a` は「～というクラスの一員である」を表す。

```
ex:r1 rico:hasOrHadDigitalInstantiation ex:r1scan .
ex:r1scan a rico:Instantiation ;
rico:identifier "https://archives.example.jp/iiif/r1/manifest" .
```

1行目は「記録 ex:r1 はデジタルの具現化 ex:r1scan を持つ (持っていた)」、2～3行目は「ex:r1scan は具現化であり、その識別子は右の文字列である」と読む。ここでは簡単のため、IIIF マニフェストの URL を具現化の識別子として書いた。RiC-O 1.1 は、マニフェストそのものを、文書形式種別 IIIFManifest で分類した記録として記述する手立ても用意している (第5章 5.4 節)。

■PREMIS — 保存との分業 PREMIS はデジタル長期保存のメタデータ辞書で (OWL 版もある)、フォーマット、固定性検査 (チェックサムを取り直してファイルが変化していないことを確かめる作業)、マイグレーションといった**保存イベント**を記録する。RiC は知的・文脈的な記述を、PREMIS は保存管理を受け持ち、同じ具現化を接点に併用する、というのが実際の整理である。

まとめれば、RiC は孤立した発明ではなく、GLAM 各界で同時進行してきた「属性の束からグラフへ」という転換の、アーカイブズ版である。RiC を学ぶことは、この共通の動きへの入口でもある。

2.7 移行はどう構想されているか

RiC-CM §1.6.4 は移行 (transition) について現実的な見通しを述べる。要点は2つある。第一に、RiC-CM は ISAD(G) に成文化された既存の記述実務を**包含**する (accommodate) ので、既存の記述は捨てずに RiC の枠組みの中に置き直せる。第二に、移行は漸進的であり、資源のある機関が先行して実装し、経験を共有して「道を舗装する」ことが期待されている。すべての機関が一斉に移行することは想定されていない。

少なくともこれから学ぶ側にとって言えることは明確である。ISAD(G) と EAD の知識は無駄にならない。それどころか、既存データからの変換 (第7章) を設計するには、変換元である ISAD(G)/EAD の構造を正確に理解していることが前提になる。RiC の学習は従来標準の学習を置き換えるのではなく、その上に積み上がる。

確認問題 (ヒントは付録 A.6)

問 2.1 ISAD(G) が国際交換に不可欠とする 6 要素の筆頭はレファレンスコードである。その理由を「参照」と「識別子」という言葉を使って説明せよ。

問 2.2 第7章 7.4 節の EAD/EAC-CPF の例 (または手元の目録) から、文字列による参照と識別子による参照を1つずつ探し出せ。

問 2.3 EAC-CPF の resourceRelation による参照は、EAD の authfilenumber による参照と向きが逆である (例はいずれも第7章 7.4 節)。それぞれ、どの文書がどの相手を指しているか述べて。

問 2.4 図書館の典拠 (VIAF や Web NDL Authorities) がアーカイブズにとっても重要なのはなぜか、本章 2.6 節の観点から説明せよ。

第3章

RiC がもたらした変化

本章では、RiC が従来標準に対して持ち込んだ変化を5つに絞って説明し、最後に「自由度が上がった代わりに何が大変になったのか」というトレードオフを正面から論じる。

3.1 変化 1: 検索手段のモデルから、世界のモデルへ

最も根本的な変化は一文で言える。「既存の ICA 標準は**記述**を、すなわち検索手段 (finding aid) をモデル化していた。RiC-CM は**実体そのもの**を、特定の成果物を予定せずにモデル化する」(RiC-CM §1.2)。

ISAD(G) の 26 要素は、突き詰めれば「よい検索手段が備えるべき項目」の一覧だった。だから要素の並び順があり、記述の単位 (unit of description) があり、階層があった。ISAD(G) 自身は序論 I.6 で、記述の入力と出力を同じものと見なさないとは断っていた。しかし要素を 1 列に並べた構成がその同一視を暗に前提しており、ISAD(G) の実装の多くもそれに従った (RiC-CM §1.6.2)。結果として ISAD(G) は、紙の検索手段という**出力**の形を写し取ったものになった。RiC-CM はこの発想を逆転させる。まず世界のほう (記録・人・活動・規則・日付・場所) を実体と関係のネットワークとして記述する。標準が定めるのは、このシステムへの**入力**の形である。検索手段・展示・データ公開などの**出力**は、そのネットワークから必要に応じて生成すればよい。出力の形式は標準では縛らない (図 3.1)。「革新と実験の妨げにならないように、出力とユーザインタフェースは無制約のままにしておく」(RiC-CM §1.6.2)。

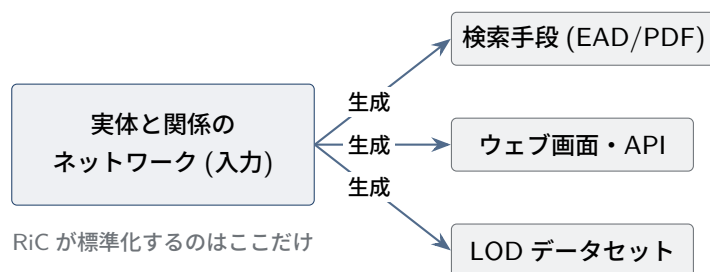


図 3.1 入力と出力の分離。RiC は入力 (記述データ) の概念構造だけを定め、出力は実装に委ねる。

RiC-CM 自身、リレーショナルデータベースの世界では実体と関係を分けて入力し、それを機関や利用者の求めに応じてさまざまな出力に描き分けられると述べている (§1.6.2)。この分離は現代のシステム設計では常識的な考え方であり、リレーショナルデータベースの世界で「データと表示の分離」とし

て実践されてきたものと同じ方向を向いている。アーカイブズ記述標準がようやくそこに追いついたと見ることもできる。

コラム:「世界のモデル」という構想 — 知識表現の歴史との距離

「世界そのものをモデル化する」という言い方に、既視感を覚える読者がいるかもしれない。1980年代を中心とする第二次人工知能ブームは、文字どおり「世界の知識を形式的に書き下し、機械に推論させる」構想(エキスパートシステム、フレーム、意味ネットワーク)を追求し、そして挫折した。挫折の主因ははっきりしている。知識を手作業で形式化する費用が膨大で割に合わないこと(知識獲得のボトルネック)、書いた知識の保守が続かないこと、そして常識的な推論が形式論理では手に負えないことである。人類の常識を書き尽くそうとした Cyc プロジェクトの苦闘は、その象徴としてよく引かれる。1990年代末に構想され、2001年の雑誌記事で広く知られるようになったセマンティックウェブ(Semantic Web)も、当初の壮大な絵のとおりには実現しなかった。ウェブ上の情報を人間だけでなく機械にも意味の分かる形で書き、リンクをたどって推論させようという構想で、本書が扱う RDF・OWL・SPARQL はいずれもその系譜の技術である。RiC が同じ轍を踏まないと言えるかどうかは、検討に値する。

RiC の一次資料が、この歴史を明示的に論じている箇所はない。策定文書に「AI ブームの教訓を踏まえて」という記述は見当たらない。しかし設計の実質を見ると、RiC は挫折した路線とは別の、意図的に切り詰められた道を選んでいく。次の4点である(表3.1に対比をまとめた)。

第一に、対象領域の限定。RiC がモデル化する「世界」とは、記録・作成者・活動という、**記録管理の職業が責任を持つ範囲**だけである。RiC-CM は冒頭近くの §1.2 で自分が扱わないものを明示し、主題となりうる万般の事物は Thing という受け皿で**名指すだけ**にとどめ、その記述は各分野のコミュニティに委ねる。百科事典を作る気は最初からない。

第二に、推論の弱さは意図的である。第6章(特に6.8.4項)で見るとおり、RiC-O の推論は分類の階層・逆関係・推移性・経路の近道という、構造的な種類のものが中心である。常識推論はしないし、目指してもいない。セマンティックウェブの分野でよく引かれる「少しの意味論でも大きな効果がある」(a little semantics goes a long way) という標語の考え方を、領域を限って実行したものと読める。

この「常識推論をしない」という選択には、技術的な裏側がある。常識推論の難所の1つは**否定の扱い**である。「この記録には作成者が書かれていない。ゆえに作成者はいない」と結論するには、手元のデータがその点について完全だという前提が要る。この前提を**閉世界仮説**(closed world assumption、CWA)と呼ぶ。リレーショナルデータベースの問い合わせや、Prolog の「失敗による否定」(証明できないことを偽と見なす扱い)は、この考え方と関係が深い。ただし閉世界仮説と非単調推論は同じものではない。非単調推論とは、新しい情報を得たときに以前の結論を取り消すことを許す推論である。「ふつう鳥は飛ぶ」と仮定していた個体がペンギンだとわかれば、その結論を取り消す。常識推論の研究では、このような例外や仮定を扱うさまざまな形式が考えられてきた。

ところが閉世界仮説は前提が強い。データが完全でない領域では、書かれていないだけのことを「偽」と断じてしまう。アーカイブズ記述はその典型である。作成者が不明の記録も、まだ調査が及んでいない記録も、日常的に存在する。そこで OWL は逆の側、**開世界仮説**(open world

assumption) を採る。「書かれていないことを、それだけで偽とはしない」という立場である (第 5 章のコラム「RDB の世界地図で読むオントロジー」、第 6 章 6.9 節)。機関を越えてデータを寄せ集めるウェブでは、どのデータセットも部分的でしかありえないので、この選択は必然でもある。また OWL の推論は単調である。データを足しても、それまでに導いた結論が取り消されることはない。常識推論をあきらめる代わりに、データを寄せ集めても結論が覆らない推論だけを残したのである。RiC-O の推論が地味に見えるのは、この割り切りの結果である。実務では、「作成者が記録されていない記録の一覧」のような欠如の問い合わせを、論理の否定ではなくデータの検査として書き分けることになる (第 6 章 6.9 節、第 8 章 8.4 節)。

第三に、知識獲得のボトルネックに対する答えを持っている。RiC が形式化しようとする知識は、新たに書き起こすものではなく、アーキビストが 1 世紀以上にわたり検索手段という形で**すでに生産してきた専門知識**である。RiC はその再形式化であって、百科事典的な新規記入の要求ではない。知識を書き、維持し続ける動機を持つ職業集団が、標準の成立以前から存在している。これが第二次ブームの多くのプロジェクトとの決定的な違いである。

第四に、判断を機械に移さない。記録か記録集合か、同じ記録か新しい記録か。RiC はこうした判断の**表現形式**を与えるだけで、判断そのものは一貫して人間に残す (RiC-AG の「決定的な正解・不正解はない」という態度)。機械に世界を理解させる構想ではなく、専門家の理解を機械と共有可能な形式に写す構想である。

つまり本節の見出しの「世界のモデル」は、「世界知識の網羅」ではなく「記述対象を (検索手段という出力越しにではなく) 直接モデル化する」という意味に切り詰めて読むべきであり、その限定こそが RiC の生存戦略である。ただし、教訓のすべてを免れているわけではない。形式化と保守の費用、共有基盤の不在 (§3.6、第 7 章 7.6 節) は、知識獲得ボトルネックの領域固有版にほかならない。RiC が第二次ブームの二の舞を避けられるかどうかは、語彙の設計ではなく、この費用を誰がどう負担し続けるかで決まる。

表 3.1 第二次 AI ブームの知識表現と RiC の違い (コラムの 4 点の対比)

観点	第二次ブームの知識表現	RiC
対象領域	常識を含む世界の知識全般	記録と、その作成者・活動など、記録管理の職業が責任を持つ範囲。ほかの事物は Thing で名指すだけ
推論	例外や否定を含む常識推論を目指した	分類の階層・逆関係・推移性・経路の近道が中心。開世界仮説に立ち、単調である
知識の供給源	専門家から新たに聞き取って形式化する (知識獲得のボトルネック)	アーキビストが検索手段の形で蓄積してきた知識を形式化し直す
判断の所在	機械に推論させる	判断は人間に残し、その表現形式だけを与える

3.2 変化 2: 多段階記述から多次元記述へ

ISAD(G) の多段階記述 (multilevel description) は、フォンドを頂点とする 1 本の自己完結した階層だった。RiC-CM はこれを多次元記述 (multidimensional description) に置き換える (§1.6.2.2)。記述はグラフ (ネットワーク) の形をとり、その中に従来型の階層もそのまま収まるが、階層に閉じ込められない関係も表現できる。

原文が挙げる例が分かりやすい。ある活動が複数の組織によって順番に遂行され、1 つのシリーズがその活動を文書化している場合を考える。フォンドを頂点とする 1 本の階層では、このシリーズはどれか 1 つの組織のフォンドの下にしか置けない。多次元記述では、シリーズを活動に関係づけ、同時に歴代の各組織のフォンドの中に位置づけることができる (図 3.2)。1 つの記録・記録集合が複数の記録集合に同時に所属できる (RiC-E03 のスコープノート^{*1}) ことは、研究者が作る主題別のコレクション、バーチャル展示、機関横断のプロジェクトなど、「二次的な集合」の記述にも道を開く。

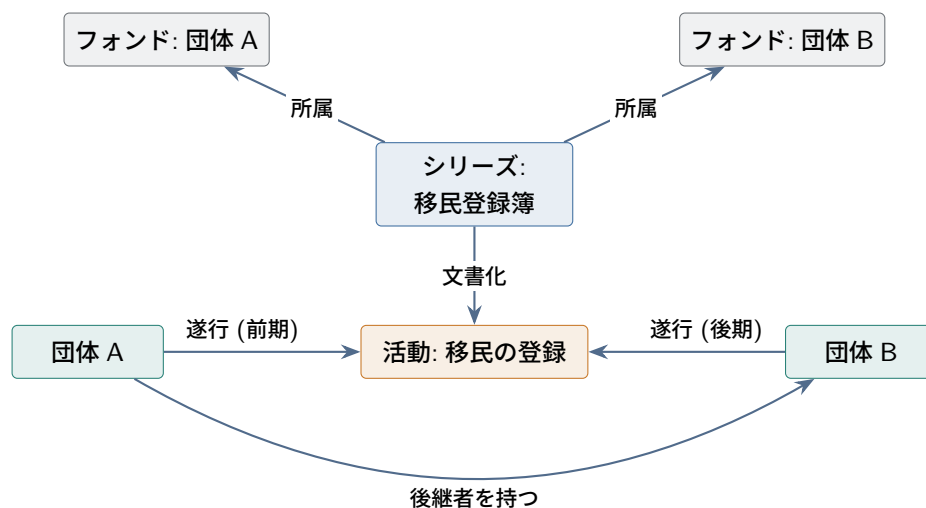


図 3.2 多次元記述の例。活動を引き継いだ 2 つの組織 (団体 A の後継者が団体 B) のフォンド双方に、同じシリーズが位置づけられる。木構造ではこの状況を正確に表せない。

注意すべきは、RiC が階層記述を否定していないことである。記録集合は記録集合をメンバーに持てる (シリーズ→サブシリーズ→ファイルという入れ子)。RiC-CM §1.6.2.2 はさらに、記録集合の記述と、そこに含まれる記録の記述の関係を精密化し、「記述の継承 (inheritance of description)」を機械処理可能にすることを目指すとする。集合レベルに書かれた情報のうち、メンバー全員が共有する属性・関係 (たとえば「全記録が同一活動を文書化する」) は各メンバーの記述として正当に継承できるが、集合それ自体の記述と、要約的な情報 (たとえば年代範囲) は、メンバー個々の属性ではなく「文脈」としてのみ継承される。図 3.3 に、図 3.2 のシリーズでこの 2 種類の違いを示す。活動への関係は各記録にもそのまま成り立つが、年代範囲はどの記録の作成日でもない。この区別は第 4 章の属性論 (§4.3) で再

^{*1} RiC-CM は実体・属性・関係のそれぞれに ID を与えている。実体は RiC-E01 以降、属性は RiC-A01 以降、関係は RiC-R001 以降で、本書は E17・R026 のように接頭辞を省くこともある。そして 1 つ 1 つを、決まった欄の組で記述する。実体なら ID・名称・定義 (Definition)・スコープノート (Scope Notes)・例 (Examples)・注釈 (Comments) の 6 欄である (§2.1.1)。スコープノートは、定義だけでは伝わらない適用範囲や判断の目安を述べる欄で、本書が「スコープノートは～と言う」と書くときはこの欄を指している。

登場する。

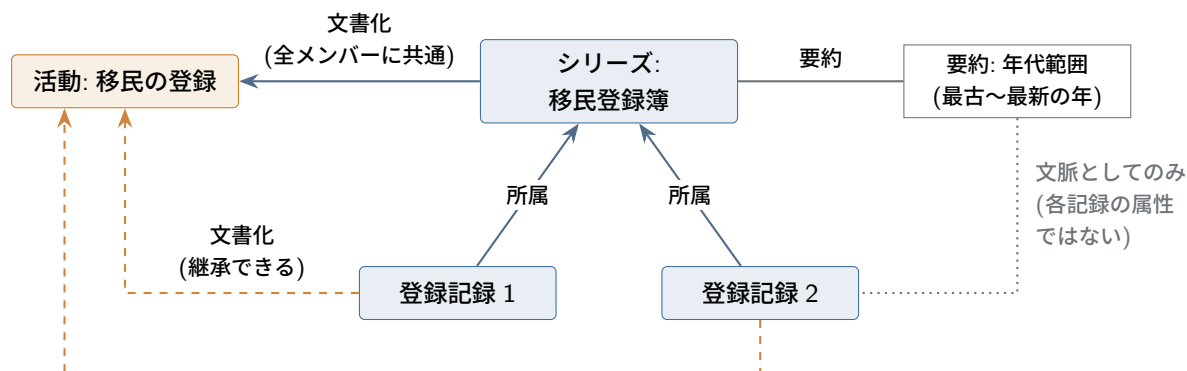


図 3.3 記述の継承。集合の全メンバーが共有する関係 (左) は各メンバーの関係として継承できるが、要約的な情報 (右) は文脈としてしか継承されない (RiC-CM §1.6.2.2)。破線は継承によって各記録に成り立つ関係、点線は文脈としてだけの継承。

3.3 変化 3: 出所概念の拡張

「Records in Contexts」という標準名の複数形 contexts に、この変化が凝縮されている。伝統的な出所原則は、フォンドを蓄積した単一の個人・団体を特権化してきた。これに対して近年のアーカイブズ理論は 2 方向から批判を加えてきた (RiC-CM §1.5.3、RiC-FAD §6)。

知的批判はこうである。記録の起源と歴史には、蓄積者だけでなく、作成や利用に直接関わった他の人々・団体、そして記録に関わる諸活動が含まれる。記録は複数の主体によって共同で、あるいは継起的に作られることがあり、デジタル環境の共同編集はこの複雑さをさらに増している。倫理的批判はこうである。蓄積者だけを特権化する記述は、記録の作成・利用に参加した他の人々、そして**記録の主題とされた人々** (本書で例を挙げれば、植民地行政の記録に記載された先住民や、収容所の記録に記載された被収容者) を、記述から見えなくしてしまう。

RiC の立場は、知的批判と倫理的批判の両方を受け入れるというものである。伝統的な出所原則の方法論的な健全さを再確認しつつ、出所を「記録が他の記録・活動・人・団体との動的な関係の網の中で生まれ、存在し続けること」として拡張的に理解する。多実体モデルと関係のネットワークは、この拡張された出所理解を記述として実現するための仕組みである。誰を記述の中心に置くかをモデルが強制しないので、同じ記録の網に対して複数の視点 (perspective) からの記述と複数のアクセス経路が共存できる。

コラム: 出所と伝来 — 何が違うのか

出所は、隣にある**伝来**という概念とまぎれやすい。整理しておく。

出所 (provenance) が指すのは**生成の文脈**である。RiC-FAD §5 の言い方では「ある個人・集団が生活や業務の中で作成・蓄積・使用した」という関係で、記録がどの主体の、どんな営みの産物かを問う。米国アーキビスト協会の用語辞典^aも、第一義を「ものの起源または源」(the origin or source of something) とし、その注記で「コレクション内の資料を作成または受領した個人・家族・組織」を指すと説明している。

伝来が指すのは、記録が作成者の手を離れた**あと**の変遷である。ISAD(G) の記述要素 3.2.3 がこれにあたり、国立公文書館による日本語訳の見出しが「伝来」である。目的は訳文で「真正性、完全性、及び解釈に際して重要な意味をもつ、記述単位の履歴についての情報を提供すること」と述べられている。

英語の名称には注意が要る。ISAD(G) 3.2.3 の英語見出しは archival history だが、米国の DACS で同じ位置にある要素 (5.1) は custodial history と呼ばれる。英語では 2 語あり、日本語ではどちらも伝来と訳される。

2 つの標準で範囲は少し違う。ISAD(G) は「所有権、責任、管理の変遷」に加えて「編成の履歴、現在の検索手段の作成、他の目的による記録の再利用、ソフトウェアのマイグレーション」まで挙げる^bので、所蔵機関自身の整理作業も伝来に入る。DACS 5.1 は「作成者の手を離れてから所蔵機関が受け入れるまで」と期間で切り、機関の作業には触れない。始点の扱いも近い。ISAD(G) も、任意の規則としてではあるが、「もし必要ならば、作成者から記述単位を直接入手した場合は、伝来を記録せず、収集による入手先としての情報を記録する」(3.2.4 へ回す)としている。

出所と伝来を分けているのは 2 つの軸である。1 つは**時間**で、出所は記録が生まれる場面を指し、伝来はそれ以後に記録がたどった経路を指す。もう 1 つは**問いの向き**で、出所は「この記録は何の証拠か」に答え、伝来は「この記録は本物か、途中で何が起きたか」に答える。

ややこしいのは、出所という語自体に広い語義があることである。同じ用語辞典は第二義として「資料またはコレクションの**起源・保管・所有**に関する情報」を挙げており、この意味では伝来も出所に含まれる。実際に両者が衝突した例も報告されている。ある写真群を寄贈者 (=保管の連鎖の一点) の出所として整理した結果、本来の撮影者という出所が見えなくなった、という事例である。用語辞典が「出所原則の適用が、起源にもとづく出所を覆い隠した」と要約するとおりで、広い語義で使うときは、どちらの意味かを文脈で示す必要がある。

RiC はこの区別を**関係の型**として持ち込んだ。出所関係のうち主体に向かうもの (R026 出所を持つとその下位) の値域はエージェントで、定義は「作成・集積・受信・送信する主体」に限られる。保管はここに入らず、管理関係 (R038 管理者である、R039 所持者である) として別の型に置かれる。散文で書くときは History 属性 (RiC-A21) が両方を受け止め、その仕様書きは「起源、責任、所有、保管、統制、配列、記述、管理」の履歴を含みうると列挙する。つまり RiC は、散文としては ISAD(G) の伝来を 1 つの属性に畳み、構造化して書くなら型の違う関係に振り分ける、という設計になっている。ここまでの整理を表 3.2 にまとめる。出所関係の下位の関係の階層は、第 4 章の図 4.10 に示す。

隣接分野では意味がずれる。美術館や古書の世界で provenance といえば、作品が作られてから現在の所蔵先に至るまでの**所有の履歴**を指すのがふつうで、真正性の証明と、略奪美術品の返還請求の根拠に使われる。アーカイブズという伝来にはほぼ重なり、出所とは重心が違う。

重心が違うのは、問いの出どころが違うからである。アーカイブズの出所は、配列の問題への答えとして立てられた。フランス革命は、王室の文書庫、パリ高等法院、周辺の修道院、旧政府の各省、亡命貴族と、出所の異なる文書を国立古文書館という 1 つの保管所に集めた。初代・2 代の館長 Camus と Daunou は、これを 1 つのまとまりとみなして 5 つの主題別部門に分け、その中を場所や年代や王の治世で並べた。結果として、多くの文書は出所さえたどれなくなる。1841 年 4

月 24 日の内務省通達 (起草 Natalis de Wailly) が respect des fonds を打ち出したのは、この状態に対してである^c。美術の provenance のほうは、これより古い慣行に根がある。前の所有者を書き留める記載は、ゲッティ研究所の Provenance Index が採録する欧州の売立目録で 1650 年、家財目録で 1520 年にさかのぼる。市場を経由して個々に移動する作品では、所有の連鎖が真贋と権原の手がかりになるからである。20 世紀末に加わったのは、調べた結果を公開する義務のほうで、ナチス略奪美術品の返還を扱う 1998 年のワシントン会議原則がその節目にあたる^d。

一方、データの世界の provenance(W3C の PROV 標準) は「あるデータやものを生み出すのに関与した実体・活動・人についての情報」と定義され、こちらはアーカイブズの出所に近い。RiC-CM も、記録をそれを生んだ活動に結ぶ R033(文書化している) を「機能的な出所」を書くための関係として出所の型に入れている。RiC-O はこれと主体への出所とをまとめて rico: hasOrganicOrFunctionalProvenance(値域は主体または活動) の下に置いており、PROV の発想と重なる位置にある。同じ語が分野をまたぐときは、まず「起源の話か、所有の連鎖の話か」を確かめるとよい。

^a 米国アーキビスト協会 (Society of American Archivists, SAA) の Dictionary of Archives Terminology. 用語辞典は出発点として有用だが、そのまま信用してよいわけではない。同じ辞典の digital signature の項は、注記で「メッセージダイジェストを公開鍵 (public key) で符号化したもの」が電子署名であり、受信者は「対応する私有鍵 (private key)」で送信者の同一性を確かめられると述べている。これは誤りである。公開鍵暗号では 2 つの鍵の役割が逆で、署名は署名者の私有鍵で作成、受信者は署名者の公開鍵で検証する (NIST SP 800-63-4「私有鍵でデータに署名し、公開鍵で署名を検証する非対称鍵の操作」)。そもそも受信者が送信者の私有鍵を持てるなら、署名は成り立たない。筆者は 2025 年 9 月にこの誤りを指摘するメールを SAA に送ったが、2026 年 7 月の時点で本文は変わっていない。定義は複数の典拠に当たり、可能ならその分野の規格で確かめたい。なお private key には「秘密鍵」という訳も広く行われているが、本書は「私有鍵」を採用。NIST の用語では secret key が対称鍵暗号で当事者間に共有される鍵 (共通鍵) を指し、非対称鍵暗号の private key とは別物だからである (NIST SP 800-175B Rev. 1 は secret key を「対称鍵アルゴリズムで使う単一の鍵。対称鍵ともいう」と定義し、「公開鍵 (非対称鍵) アルゴリズムで使う private key と対比せよ」と注記する)。

^b 「現在の検索手段」の英語原文は production of contemporary finding aids である。この contemporary は「その時々で作られた」の意で、訳文の「現在の」は所蔵機関がいま作る目録だけを指すのではない。

^c Michel Duchein, "Theoretical Principles and Practical Problems of Respect des fonds in Archival Science," Archivaria 16 (1983): 64–82(原載 La Gazette des Archives 97 (1977): 71–96)。この通達を「フォンドという觀念の出生証明書」と呼ぶ。同論文は、それ以前の扱い方を、ポンペイの発掘で美術品を出土の文脈から切り離して収集品として扱ったのと同じだ、とも評している。

^d 1998 年 6 月の AAMD(Association of Art Museum Directors) 報告書、同年 12 月のワシントン会議原則 (44 か国が署名)、2000 年の AAM(当時の名称は American Association of Museums、現在の American Alliance of Museums) の推奨手続きという流れ。メトロポリタン美術館は自館の説明で provenance を ownership history と同格に置いている。

コラム: 原秩序への問い

RiC-FAD §6 は原秩序尊重にも率直な疑問を向けている。「原 (original)」とはいつの状態か。蓄積の途中、記録の秩序は動的で流動的であり、何度も並べ替えられる。「秩序 (order)」とは物理的配列か知的関係か。合意はない。秩序がまったく判別できないまま記録群が移管されてくることもあり、そのときはアーキビストが記録そのものを手がかりに秩序を再構成することになる。RiC は秩序を「関係」として、つまり物理配列から切り離された知的な情報として記述する枠組みを持っており、この議論とよく対応している。RiC-O で順序の表現を担うのは、第 5 章 5.5.4 項で扱う Proxy と順序プロパティ群である。

表 3.2 出所と伝来の比較 (RiC-FAD §5、ISAD(G) 3.2.3、DACS 5.1、RiC-CM に基づく)

	出所	伝来
英語の名称	provenance	archival history(ISAD(G) 3.2.3) custodial history(DACS 5.1)
指すもの	生成の文脈。記録を作成・蓄積・使用した主体とその営み	作成者の手を離れたあとの所有・責任・保管の変遷。ISAD(G) は編成の履歴・検索手段の作成・ソフトウェアのマイグレーションも含める
時間	記録が生まれる場面	それ以後
答える問い	この記録は何の証拠か	この記録は本物か、途中で何が起きたか
RiC で関係として書く	出所関係。R026 出所を持つとその下位 (R027 作成者を持つ など)。活動へは R033 文書化している	管理関係。R037 所有者である、R038 管理者である、R039 所持者である
RiC で散文として書く 隣接分野で近いもの	History 属性 (RiC-A21) が両方を受け止める W3C の PROV	美術館・古書の provenance(所有の履歴)

3.4 変化 4: 「記述単位」の解体

ISAD(G) の「記述単位 (unit of description)」は、フォンドも、シリーズも、1 通の書簡も、同じ 26 要素で記述できる便利な抽象だった。RiC-CM はこれを 3 つの異なる実体に解体する (§1.6.2.1、§2.2.2)。

記録 (Record) 生活・業務の中で形成され、担体に記録された、ひとまとまりの情報内容。

記録集合 (Record Set) 1 つ以上の記録を、共有する属性や関係に基づいてある主体がまとめたもの。

フォンド、シリーズ、ファイル、コレクションはすべて記録集合の種別 (Record Set Type) である。

記録部分 (Record Part) 記録の知的完結性に寄与する、独立の情報内容を持つ構成部分 (電子メールの添付ファイル、証書に付された国璽など)。

名前について先に注意しておく。Record Set という名前からは、記録集合を「大きな記録」、つまり記録の一種と考えたくなる。しかし記録と記録集合の間に上下 (一方が他方の一種だという) 関係はなく、両者は記録資源の下に並ぶ別種の実体である。記録が記録集合に「属する」ことは、実体の分類とは別に、関係として記述される。この区別は第 4 章のコラムで詳しく扱う。

両者を分ける理由は、記録と記録集合が**別の活動の産物**だという点にある。個々の記録を作る行為と、記録をまとめて集合を作る行為 (分類、編成、蓄積) は、しばしば別の主体により、別の時点で、別の目的で行われる。フォンドの蓄積者と、フォンド内の個々の記録の作成者は一般に一致しない (フォンド内の記録は「混合出所」であるのが普通である)。同一視すれば記述は曖昧になる。記録集合それ自体の属性 (いつ誰が何のためにまとめたか) と、メンバーの要約 (年代範囲、言語の分布) を区別することで、記述の意味が明確になる。

もう 1 つ重要なのは、記録集合が**知的な集まり**であって物理的な集まりではないと明言されたことである (RiC-CM §2.2.2、RiC-E03 のスコープノート)。メンバーが物理的に同じ場所にある必要はない。また「何を記録と数え、何を部分と数えるか」は文脈と判断に依存することも明記されている (RiC-E02 スコープノート)。メール添付の写真は記録か記録部分か。どちらの指定も RiC は支持し、記録と記録部分の記述に使う属性・関係を**すべて共有**するように設計することで、判断の違いが致命傷にならないようにしている。この態度 (規範ではなく判断の支援) は RiC 全体を貫いている。RiC-AG はこの種の判断に FAQ を 1 本割いており、指針も实际的である。同一性がそれ自身に由来するなら記録、メンバーに由来するなら記録集合という原則に加え、「必要な属性・関係が一方にしかなければ、それが答えの手がかりになる」こと、そして**同じ対象への判断が時間とともに変わってよいこと** (内部では何年分もの大きな表が個々の輸入を記録する記録集合と見られるが、そこから半期分を抜き出して公表した表は、全体で 1 つの記録と見るのが自然になる、など) を、英国の輸入統計データやノルウェーの教会簿の実例で示している。

3.5 変化 5: 記録と具現化 (Instantiation) の分離

RiC-CM が導入したもう 1 つの区別は、記録 (伝えられる情報内容・メッセージ) と、その具現化 (instantiation: 担体上への記録) の分離である (§2.2.2)。同じ議事録が Word ファイルとしても、PDF としても、紙の綴りとしても存在するとき、RiC ではこれを「1 つの記録 (Record) と 3 つの具現化 (Instantiation)」として記述できる。図 3.4 は、RiC-CM 自身が挙げる地図の例である。

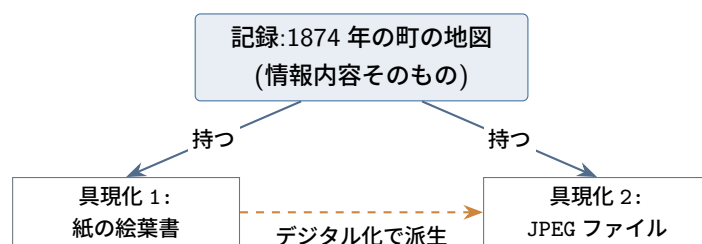


図 3.4 記録と具現化。RiC-E06 のスコープノートに基づく例。絵葉書をデジタル化した JPEG は「同じ記録の新しい具現化」とも「新しい記録」とも見なせる。どちらと見なすかは記述する主体の判断である。

規則は次のとおりである。記録と記録部分は、少なくとも 1 つの具現化を持たなければ存在しない (ただしその具現化が記述の時点で現存する必要はない。だから失われた原本も記述できる)。記録集合は具現化を持ちうる (合冊製本された台帳など) が、必須ではない。1 つの記録が同時に、あるいは時を隔てて複数の具現化を持ちうる。具現化から別の具現化が派生しうる (複写、デジタル化、フォーマット移行)。表 3.3 に、記録資源の 3 つの下位実体と具現化を並べて比べる。同一性がどこから来るかと、具現化が必須かどうかの 2 行が、3 つを分ける目安になる。

そして RiC はここでも判断を記述者に委ねる。派生した具現化を「同じ記録の具現化」と見るか「新しい記録」と見るかは、視点と利用の文脈に依存する (RiC-CM §2.2.2)。RiC-AG の FAQ は見分け方の目安を与えている。記述したい事柄が**具体物そのものの性質** (紙に印刷されている、スキャンの解像度がいくつである) なら具現化について、**実現のされ方から独立な事柄** (女王の手になる、この行政過程の産物である) なら記録資源についての記述である。また、機械的な複製は元の記録の新しい具現化にと

表 3.3 記録資源の3つの下位実体と具現化 (RiC-CM §2.2.2 と E03～E06 に基づく)

	記録集合 (E03)	記録 (E04)	記録部分 (E05)	具現化 (E06)
定義の要点	共有する属性・関係に基づき、主体がまとめた1つ以上の記録	生活・業務の中で主体が形成し、担体に記録したひとまとまりの情報内容	記録の知的完結性に寄与する、独立の情報内容を持つ構成部分	情報を担体に記録したもの
同一性の由来	メンバーから	それ自身から	属する記録から	—
具現化	持ちうるが必須でない	少なくとも1つ必須 (現存は不要)	少なくとも1つ必須 (現存は不要)	—
RiC-CM の例	Cotton Manuscripts Collection	Sketch Map of the Qatar Peninsula	3rd Great Seal of King Charles I	1521年の書簡の1950年の白黒複写
RiC-O のクラス	<code>rico:RecordSet</code>	<code>rico:Record</code>	<code>rico:RecordPart</code>	<code>rico:Instantiation</code>

どまるが、注釈が付くなど独自の歴史を持ち始めた時点で新しい記録と見なしうる、という時間依存の指針も示される。担体の物質性を重んじる好古家 (antiquarian) にとって、デジタル画像は別の記録かもしれない。伝達される情報を重視する利用者にとっては同じ記録である (RiC-E06 のスコープノート)。原文は、記録と具現化の関係が「二つ以上の等しく決定的な選択肢」を生むと明言している (§2.2.2)。この区別は PREMIS(デジタル保存メタデータ) の Intellectual Entity と Representation の区別に対応することも注記されており (RiC-CM 脚注 15)、デジタルアーカイブの実務と接続しやすい設計になっている。

読み分けが要る点:FRBR/IFLA LRM との違い

図書館情報学を学んだ人は、Work-Expression-Manifestation-Item という4層 (FRBR、現 IFLA LRM) を思い出すだろう。RiC の Record/Instantiation は2層である。RiC は FRBR/IFLA LRM を下敷きにせず、別に設計されている。RiC-O が独自のオントロジーを作った理由として挙げるのは、2013年当時「アーカイブズ共同体の固有の必要に応える汎用の領域オントロジーが存在しなかった」ことである。加えて、領域オントロジーとして「現段階では」他分野のモデル (IFLA-LRM、CIDOC CRM、PREMIS、PROV-O) から借りない方針をとっている。借りないほうが機関にとって理解・実装・保守が容易だとし、他のモデルとの対応づけは「可能な範囲で」将来の改訂で行うとしている。その背景にある事情は RiC-CM の側が述べている。文化遺産の各共同体は「自らが管理責任を持つ対象の性質について、根本的に異なる理解を持つ」 (§1.3)。RiC と FRBR/IFLA LRM は、対応させるよりも別々の道具として使い分けるのが確実である。

3.6 記述の自由度と、そのトレードオフ

ここまでの変化を「記述者の自由」という観点でまとめ直すと、RiC が何を可能にし、その代償として何を要求するかが見えてくる。

3.6.1 何が自由になったか

1. **構造の自由**。記述は1本の階層に縛られない。多重所属、機関横断の関係、時間をまたぐ関係(旧機関と新機関、原本と写し)を直接表現できる。
2. **視点の自由**。誰の視点から記録を文脈づけるかをモデルが強制しない。蓄積者中心の記述も、記録に写された人々を中心とする記述も、同じ枠組みで共存できる。
3. **粒度の自由**。よく知らないシリーズは1つのノードと2~3の関係だけで粗く記述し、重要な記録は具現化・イベント・規則まで細かく記述する、という濃淡が同じデータセットの中で許される。RiC-Oは上位の(粗い)クラス・プロパティと下位の(細かい)それらを階層化して提供し、どの水準で書いてもよいようにしている(RiC-O設計原則3「柔軟な枠組み」)。
4. **表現方法の自由**。同じ事実に対して、簡便な書き方(第5章でいう簡便法)と精密な書き方(精密法)の両方が用意されている(第5章5.5節)。簡便な書き方とは、値を文字列や数値そのままを書くことである。この「そのままの値」をリテラル(literal)と呼ぶ。「山田花子」「1868」のように、それ自体を読めば用が足り、そこから先へたどる相手を持たない値のことである。精密な書き方とは、同じ値を独立した記述対象として立て、そこを指すことである。RiC-Oでは、たとえば作成日を文字列で書くこと(rico:creationDate)も、Date実体として関係づけること(rico:hasCreationDate)もできる。どちらを選ぶかは、第4章4.1.2項で見た設計A(作成者を名前の文字列にとどめる)と設計B(作成者を実体として立てる)の対比と同じ問題である。リテラルにした値は読めるだけだが、実体として立てれば、そこから先の関係をたどれる。
5. **成果物の自由**。入力と出力の分離により、同じデータから検索手段もLODも展示も生成できる。

3.6.2 代償として何が大変になったか

自由の代償は、およそ次の6点に整理できる。

■(1) **判断の負担が増える** ISAD(G)は「埋めるべき26要素」を与えてくれた。RiCは、RiC-CMの19の実体・42の属性・85の関係と、RiC-Oの107のクラス・555のプロパティ(オブジェクトプロパティ480、データタイププロパティ75)という語彙を与えるが、**どれをどこまで使うか**は記述者(機関)が決めなければならない。記録か記録部分か、同じ記録の具現化か新しい記録か、日付をリテラルで書くか実体化するか。RiCが「どちらでもよい」と言うこうした場面のすべてで、機関としての方針決定が必要になる。決定を積み重ねて文書化したものが、その機関の適用プロファイルになる(第7章)。

■(2) **一貫性の維持が難しくなる** 様式が固定されていれば、一貫性は様式が保証してくれた。自由な語彙では、同じ事実を人によって違う書き方で記述する余地が常にある。RiC-Oが同じ情報に複数の表現方法を認めている(移行を容易にするための意図的な設計である)ことは、裏面では「データセット内の不統一」というリスクである。検索・推論はデータの一貫性に依存するから、不統一は機能低下として跳ね返る。

■(3) **識別子の管理という新しい仕事生まれる** 機関の外から指され、長く参照されるノードには、安定した識別子(IRI)が要る。作成者・活動・場所・規則を独立の実体にするということは、それらすべ

てに一意で永続的な ID を発行し、維持し続けるということである。従来「文字列としてのアクセスポイント」で済んでいたものが、ID 付きの資源の管理業務に変わる。これは組織的・長期的なコミットメントであり、技術というより運営の問題である。

■(4) 道具立てが未成熟である ISAD(G)/EAD には目録システム、編集ツール、交換の実績が揃っている。RiC 対応を謳うシステムはまだ少ない。2025 年 10 月に適用指針 RiC-AG の草案がようやく公開されたが、それ自身「段階的な手順書の提供は不可能」と述べる事例集であり(第7章)、「これに従えばよい」という様式は依然として存在しない。先行実装機関の経験と AG の事例を参照しながら、各機関が方針を作っている段階である。

■(5) 学習コストが上がる 多実体モデル、グラフ、オントロジーという考え方そのものに慣れる必要がある。ただし、必要な技術的素養は役割によって大きく異なる(第8章)。全員が OWL を書けるようになる必要はない。

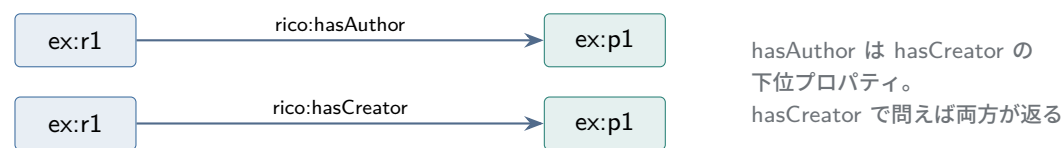
■(6) 表現の統一には、機関や共同体での取り決めが要る 最後の点は、(2)の一貫性の問題をさらに一段深めたものである。RiC 自体は、同じ現実に対する表現を一つには定めない。適用プロファイルで書き方を限定することはできるが、対象をどう解釈するかという判断まですべて固定できるわけではない。何を実体に切り出すか、どの粒度で切るか、どの関係を張るか、直接プロパティで書くか実体化するか。モデルの表現力が増すほど、これらの選択の組み合わせは掛け算で増える。記述できることが増えるほど、同じことの書き方も増える。表現の自由度と表現の曖昧さは同じものの両面であり、設計方針で選択肢を絞っても、判断を要する場面は残る。正規形(これが標準的な書き方だ、という一意な形)を RiC は定めておらず、定めようとすれば自由度を手放すことになる。

この点は現場に重くのしかかる。「どうにでも書ける」は「どう書いてよいか分からない」と実質的に同じであり、書き方が機関ごと・担当者ごとに発散すれば、機関を越えた検索や交換という RiC の眼目そのものが損なわれる。様式の固定こそが標準の効用だったと考える立場からは、RiC は標準の名に値するのかという疑問さえ成り立つ。

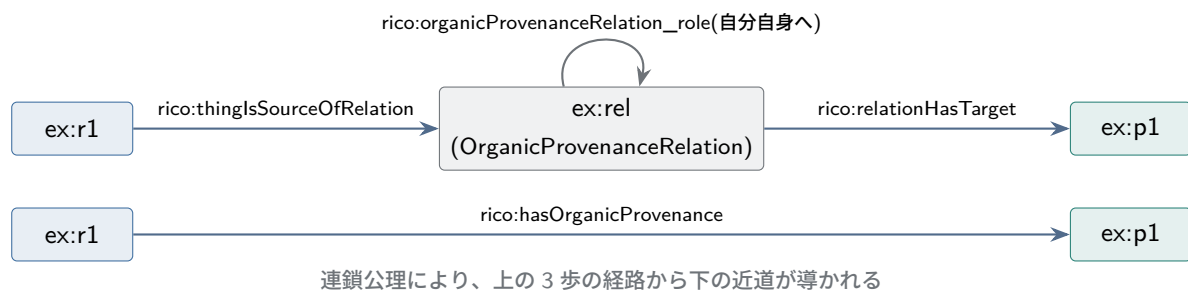
RiC の側に用意されている緩和策は、部分的なものにとどまる。第一に、RiC-O の推論(第6章)は、**あらかじめ预期された表現の揺れ**については検索時の同値性を回復する。長い経路と近道はプロパティ連鎖(遠回りの経路をたどれたら近道のプロパティも成り立つと語彙の側で宣言しておく仕組み)で、粗い記述と細かい記述はプロパティ階層(細かいプロパティを粗いプロパティの下位として宣言しておく仕組み)で、同じ問い合わせに答えるように設計されている^{*2}。しかし第二に、**预期されていない揺れ**は機械的には吸収されない。記録と記録部分のどちらに切るか、新しい記録か同じ記録の新しい具現化か、実体をどこまで立てるか、といった揺れである。これらは意味の水準の判断差であり、推論は救済しない。図 3.5 に、吸収される揺れと吸収されない揺れの例を並べる。上の 2 つは、推論を通せば同じ 1 つの問い合わせで両方の書き方が返る。下の 1 つは、`rico:Record` を型として問うと片方しか返らない。残るのは運用による収束、すなわちコミュニティで適用プロファイルを共有し、変換ツールや記述システムが事実上の正規形を作り、公式の事例集が判断の相場を形成する、という道である。

^{*2} プロパティ連鎖には 1 つ注意がある。連鎖の途中にある役割の段(`rico:organicProvenanceRelation_role` など)は、RiC-O では OWL 2 RL の規則推論が扱わない形の公理で定義されている。そのため規則推論で近道を導くには、関係ノードを自分自身に結ぶ 1 行をデータに書いておく必要がある。RiC-O 自身のスコープノートがそう求めている(第6章 6.5 節)。

(a) 粒度の揺れ: 推論で吸収される (上: 機関 X、下: 機関 Y)



(b) 経路の揺れ: 推論で吸収される (上: 機関 X、下: 機関 Y)



(c) 記録が記録部分かの揺れ: 推論では吸収されない (上: 機関 X、下: 機関 Y)

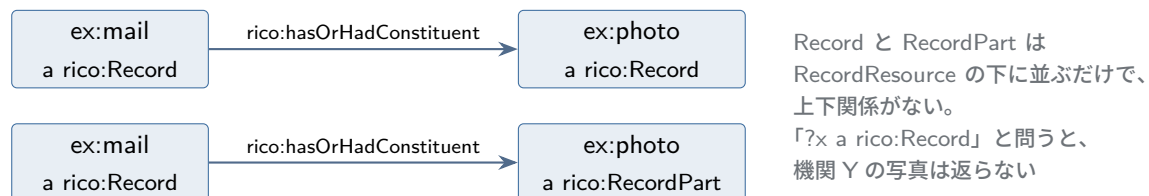


図 3.5 表現の揺れには、推論で吸収されるもの ((a)(b)) とされないもの ((c)) がある。ex:r1 は調査旅行日誌、ex:p1 は山田花子、ex:mail は電子メール、ex:photo はその添付写真。

実際、2025 年に公開された RiC-AG 草案はこの分析を裏書きしている。AG は「段階的な手順書は不可能」と明言した上で、記録か記録集合かの判断に「決定的な正解・不正解はない」と書き、正規形を定める代わりに、判断が分かれる状況を事例で示し、判断材料 (3.4 節で見た目安) を提供する方針をとった。つまり EGAD 自身、一意性の回復ではなく判断の質の底上げを選んだのである。

RiC は「様式の標準」であることをやめて「語彙と意味の標準」になったのだから、書き方の収束はもはや標準文書の内側では起こらず、実装コミュニティの側で起こるしかない。この移行が成功するかどうかは現時点で未知数であり、本書はこれを RiC の未解決の構造的リスクとして明記しておく。なお、収束を支えるはずの**共有基盤そのもの** (拡張のレジストリ、機関を越えた識別子の典拠) が未整備であるという、もう 1 つの構造的な問題を第 7 章 7.6 節で論じる。

表 3.4 に、5 つの自由と負担の対応をまとめる。最後の 2 行は、5 つに共通する前提 (実体を独立させたこと、表現力を広げたこと) から生じる負担である。

一言でまとめれば、RiC は「様式を埋める仕事」を「モデルを設計する仕事」に変えた。この変化は、目録作成を単純作業から知的設計へ引き上げるものだと肯定的に評価することも、現場に過大な設計負担を転嫁するものだと批判的に評価することもできる。EGAD の想定は、負担の多くをシステムと共有ガイドラインが吸収する、というものである (RiC-CM §1.3、§1.6.2)。RiC-AG 草案と公式クロスワークの公開はその第一歩だが、想定が満たされるかどうかは、今後のツール開発と AG の成熟にか

表 3.4 自由度とトレードオフの対応

得られた自由	引き換えに生じた負担
構造の自由 (多重所属・横断関係)	「どの関係を張るか」の選択と根拠の明示。関係の数だけ保守対象が増える
視点の自由	記述方針 (誰を・何を記述するか) の機関としての明文化
粒度の自由	粒度の判断基準の設定。粗い記述と細かい記述の混在の管理
表現方法の自由	機関内での書き方の統一 (適用プロファイル)。不統一だと検索・推論が劣化
成果物の自由	出力を作る実装力が別途必要 (標準は出力を助けてくれない)
実体の独立 (作成者・活動・場所など)	IRI の発行・維持という恒久的な管理業務
表現力の拡大 (全般)	正規形の不在。同じ現実の表現が一意に定まらず、収束は実装コミュニティの運用に委ねられる

かっている。

確認問題 (ヒントは付録 A.6)

- 問 3.1** 本章の 5 つの変化から、自分 (または想定する機関) にとって最も価値が大きいものと、最も負担が大きいものを 1 つずつ選び、理由を述べよ。
- 問 3.2** 「表現の一意性がない」とはどういうことか。同じ事実の 2 通りの書き方の例を作り、それが検索にどう影響するかを述べよ。
- 問 3.3** 「検索手段のモデルから世界のモデルへ」という転換に対して、第二次 AI ブームの挫折を知る立場からはどんな懸念が可能か。またそれに対する RiC 側の限定は何か。

第 4 章

RiC-CM を読む — 実体・属性・関係

本章は RiC-CM 1.0 の本体 (第 2～6 章) の読解である。まず前提となる実体関連モデルの考え方を説明し、次に実体・属性・関係を順に見ていく。

4.1 準備: 「実体」という言葉

RiC-CM を読む前に、entity(実体)という言葉について立ち止まっておきたい。この語は分野によって意味の力点が異なるので、RiC-CM がどの用法を採っているかをはっきりさせておくと、以降の読解がなめらかになる。

RiC-CM が採用するのは、情報システム設計の伝統である実体関連モデル (ERM: Entity-Relationship Model) の用法である (RiC-CM §1.4)。1976 年に P. Chen が提案した ERM では、関心の対象となる「もの」を実体 (entity)、そのものの特徴を属性 (attribute)、もの同士のつながりを関係 (relation/relationship) と呼ぶ。哲学の存在論で言う実体 (substance) のような重い意味はなく、「データベースで 1 件のレコードとして管理したい対象」くらいの実務的な概念である。図書館の貸出システムなら「利用者」「資料」「職員」が実体、「氏名」「請求記号」が属性、「借りる」「所蔵する」が関係になる。

ただし正確には、ERM は「実体型 (entity type)」と「実体インスタンス (entity instance)」を区別する。「個人 (Person)」という型と、「ネルソン・マンデラ」という個々の実例である。この実例は原典から借りた。RiC-CM は E08 個人の Examples にこの人物を挙げている。RiC-CM が定義する 19 の実体はすべて実体型であり、各定義に添えられた Examples が実例に当たる。日常語の「実体」はこの 2 つを区別しないので、RiC-CM の実体一覧を「19 個のものの一覧」と読むのではなく、「もの 19 種類の分類体系」と読むのが正しい。

さらに紛らわしいことに、この後の章で登場する RDF/OWL の世界では、ほぼ同じ概念に別の名前が付いている。実体型に相当するのがクラス (class)、実体インスタンスに相当するのが個体 (individual) ないし資源 (resource) である。RDB では実体型がテーブルに、インスタンスが行に対応する。本書では対応関係を都度示すが、まず「同じ発想に分野ごとの方言がある」と理解しておけばよい。

表 4.1 「実体」をめぐる用語の対応 (概念は完全一致ではないが発想は共通)

	型 (種類)	インスタンス (実例)	つながり
ERM(RiC-CM)	実体 entity	実体の実例	関係 relation
RDF/OWL(RiC-O)	クラス class	個体 individual/資源 resource	プロパティ property
RDB	テーブル table	行 row	外部キー・結合
オブジェクト指向	クラス class	オブジェクト object	参照 reference

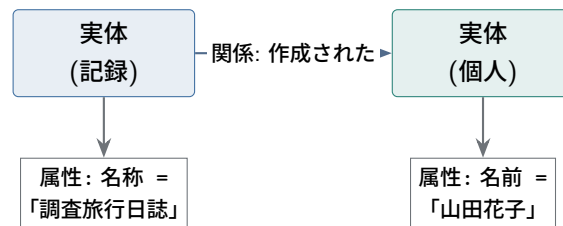
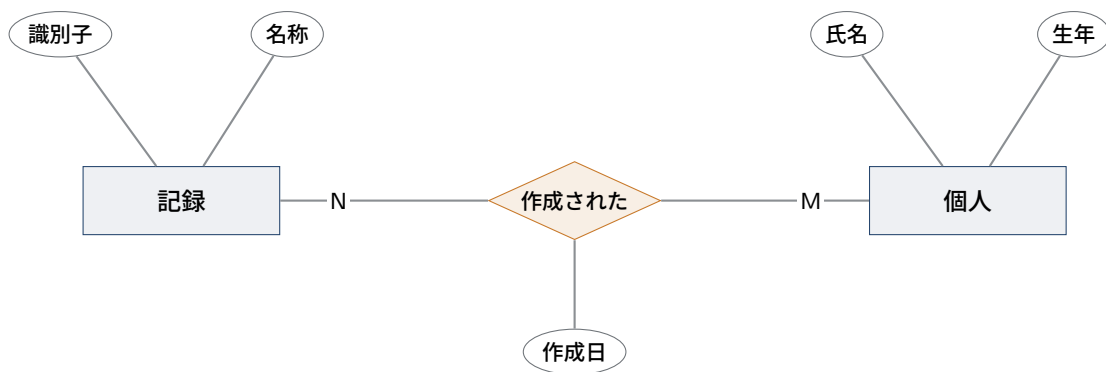


図 4.1 ERM の基本要素。箱が実体、箱同士を結ぶのが関係、箱にぶら下がる値が属性である。

4.1.1 ER 図の読み方

ERM には図の記法が伴う。1976 年の Chen の原論文に由来する古典的な記法では、実体を長方形、関係をひし形、属性を楕円で描き、線でつなぐ (図 4.2)。実務では長方形の中に属性を列挙する簡略な記法 (いわゆる IE 記法、あるいは UML のクラス図) を使うことも多いが、読み方の原理は同じである。RiC-CM は導入部の図 1 でこの記法を使っているが、実体・属性・関係の定義そのものは散文と表で与えており、本書の図も RiC-CM の内容を読みやすく図示したものである。



長方形=実体、ひし形=関係、楕円=属性。N と M は多重度で、「1 つの記録が複数の個人に作成されうる。1 人の個人が複数の記録を作成しうる」を表す

図 4.2 Chen 記法による ER 図の例。RiC-CM の図 1 も同じ記法で記録・作成・個人を描いている (原典の図 1 では日付は記録の属性である)。関係にも属性 (作成日) を付けられる点に注目してほしい。RiC-CM が関係の属性 (§4.4) を定義するのは、この伝統を引いている。

図 4.2 で読み取るべき要素は 4 つある。第一に、長方形が実体型である。第二に、ひし形が関係で、2 つの実体型を結ぶ。第三に、楕円が属性で、実体だけでなく **関係にも** 付けられる。「いつ作成したか」は

記録の属性でも個人の属性でもなく、作成という関係の属性である。第四に、線に添えた記号が**多重度** (cardinality) で、片方の 1 件がもう片方の何件と結びつくうるかを示す。1 対 1、1 対多、多対多の 3 種類があり、この図は多対多である。

RiC-CM も関係ごとに多重度を明記している (§5.4 の Cardinality 欄。たとえば R026 「出所を持つ」は M to M(多対多)、R002 「部分を持つ」は 1 to M(1 対多))。関係に属性を付けられるという ERM の性質も、RiC-CM がそのまま受け継いでいる (§4.4)。

4.1.2 同じ対象に対してモデルは 1 つに決まらない

ERM を学ぶうえで最も大切な点は、**同じ現実に対して正しいモデルが複数ありうる**ことである。何を実体として切り出し、何を属性にとどめるかは、記述の目的によって変わる。図 4.3 に、同じ事実「調査旅行日誌は山田花子が作成した」に対する 2 つの設計を並べた。

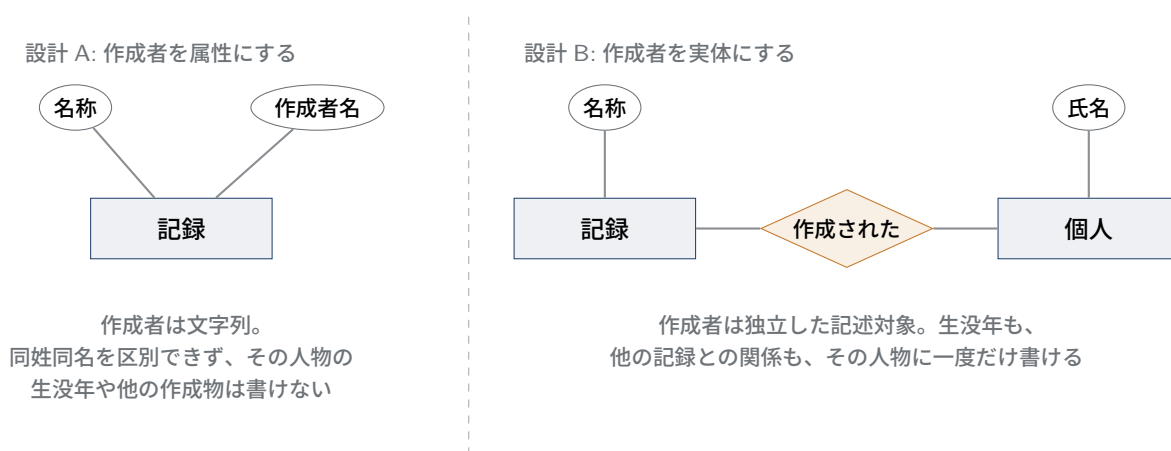


図 4.3 同じ事実に対する 2 つのモデル化。どちらも誤りではなく、記述の目的が選択を決める。

設計 A は簡便で、記録 1 件だけを見るぶんには十分である。設計 B は手間がかかるが、作成者を独立の記述対象に昇格させたことで、同姓同名の区別、生没年や経歴の記述、「この人物が作成した記録の一覧」という検索が可能になる。どちらが正しいかは一概に言えず、選ぶのは記述の目的である。

この選択は本書の主題そのものに直結する。ISAAR(CPF) が作成者の記述を典拠レコードとして独立させたのは、設計 A から設計 B への移行だった (第 2 章)。RiC-CM が 19 の実体を立てたのも同じ方向の徹底であり、逆に「概念的に統制値をとる属性は、実装では実体として扱うほうが便利なが多い」という RiC-CM 自身の注記 (§4.3) は、この 2 つの設計のあいだを行き来してよいという表明である。第 3 章 3.6 節で述べた「表現の自由度と表現の曖昧さは同じものの両面」という論点も、ここに根がある。同じ現実に対して RiC 準拠の表現が一つに定まらないのは、ERM という枠組みが最初からそういう性質のものだからである。

4.2 実体の階層

RiC-CM 1.0 は 19 の実体を定義する。実体は平らに並んでいるのではなく、「A は B の一種である (A is a kind of B)」という分類の関係で階層をなす。頂点はすべてを包み込む「もの (Thing)」である。

このうち、記録資源 (とそれに密接する具現化)、エージェント、活動の 4 つが中核実体 (core entities)

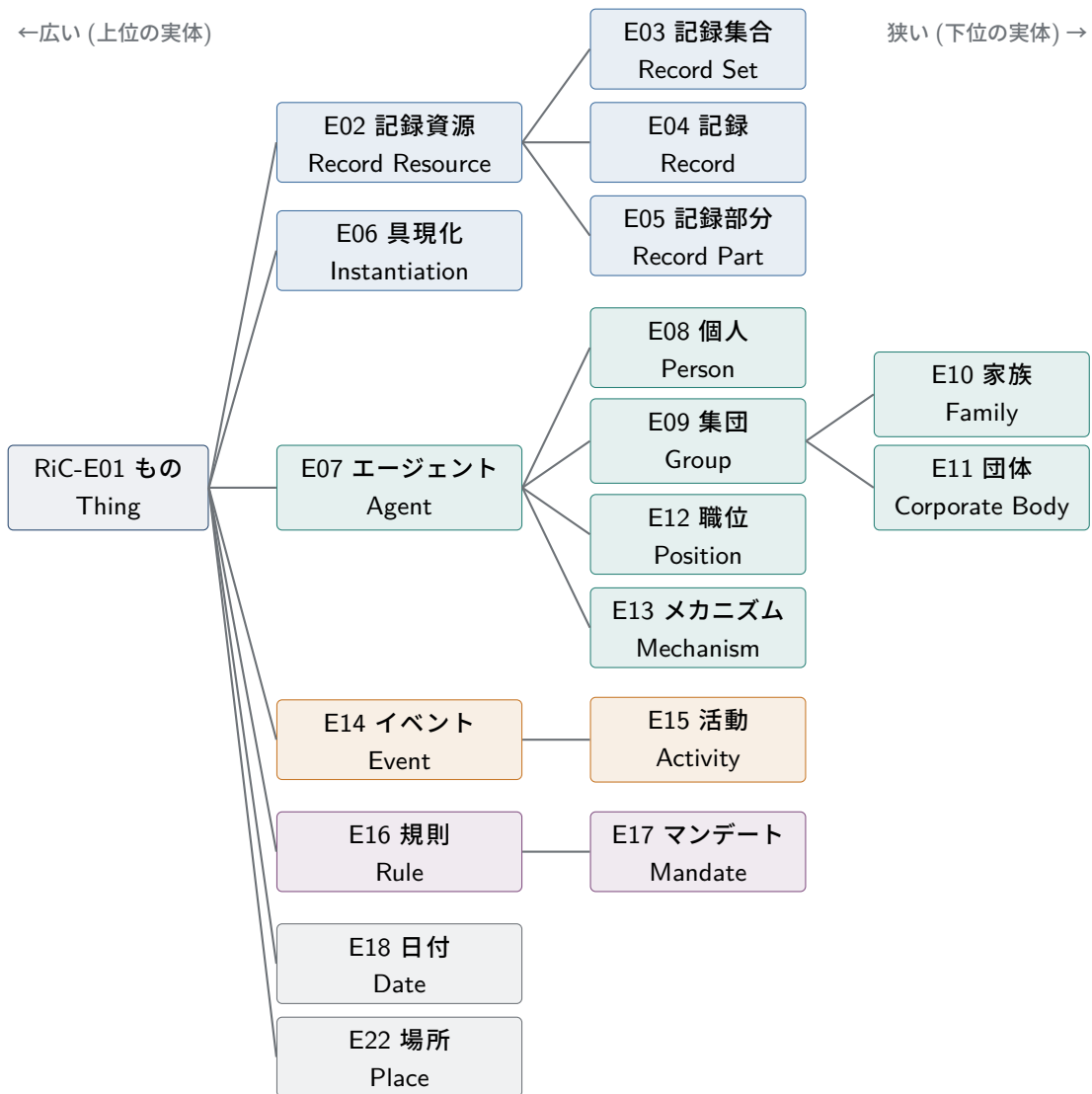


図 4.4 RiC-CM 1.0 の実体階層 (RiC-CM §2.1 の表に基づく)。線はすべて「～の一種である (is a kind of)」を表す。記録集合が記録を「含む」という関係はこの図には現れないことに注意 (§4.4)。E19–E21 が欠番なのは、0.2 版にあった日付の下位実体 (単一日付・日付範囲・日付集合) が 1.0 で廃止されたためである。

とされる (RiC-CM §2.1)。原文によれば、これらの実体は「活動の目的の達成に役立てるために記録された情報を用いながら、世界の中で行為するエージェント」を表し、「記録された情報は活動の遂行の証拠である」。これが RiC の世界観の要約であり、中核実体はその登場人物である。残りの実体 (イベント、規則、日付、場所) は、中核実体を十全に記述するための脇役と位置づけられる。

コラム: RiC-CM の全体像の図を読むときの注意

RiC-CM §2.1 には「RiC-CM v1.0: a global overview」と題された 1 枚の図 (原典 図 3) がある。実体を入れ子の箱で並べ、そのあいだに関係の矢印を引いたもので、RiC の解説でよく引用される。よくできた図だが、そのまま受け取ると誤解しやすい点が次の 4 つある。

- 入れ子は「含む」ではない。箱の入れ子が表しているのは「～の一種である (is a kind of)」

だけである。記録集合 (E03) と記録 (E04) が記録資源 (E02) の箱の中に並んでいるのは、どちらも記録資源の一種だという意味しかない。「記録集合が記録を含む」という関係 (R024) はこの図には描かれていない。図 4.4 を階層専用の図として分けたのは、この 2 つを混ぜないためである。

- **もの (E01) が図に現れない。**すべての実体の根であり、多くの関係の定義域・値域になっている実体が、全体像の図から落ちている。「もの」を相手にとる関係は本書では図 4.9 に分けて描いた。
- **描かれている関係は 7 対だけ。**R019・R025・R026・R033・R060・R063・R080 とその逆である。85 ある関係のうちの見本であって、関係の地図ではない。どの実体からどの実体へ矢印が伸びうるかを知りたいければ、§5.3 の階層チャートか、定義域の順に並んだ §5.6 の一覧表 (日付関係の一部が載っていない。§4.4 の注意書き) を見ることになる。
- **中核実体の位置が揃っていない。**中核実体は記録資源・具現化・エージェント・活動の 4 つだが、活動だけは第 2 階層にない。イベント (E14) の下位実体だからである。原典もこの点を図の直前で明示している。「4 つの中核実体のうち、活動を除くものは階層の第 2 階層にある」。図の見た目では活動が一段奥まって見えるが、重要度が低いわけではない。

以下、実体を順に見る。定義はすべて RiC-CM §2.2 からの要約である。

4.2.1 もの (Thing, E01)

「人間の経験の領域内にある、あらゆる観念・物質的な物・出来事」。すべての実体の根であり、RiC-CM が明示的に定義しない対象もすべて Thing である。原文の例示 (RiC-E01 Examples) は意図的に幅広い。「奴隷制」という概念、「印象派」という芸術運動、シェイクスピア劇の登場人物パックのような架空の存在、「フランス革命」という出来事、そしてフラットアイアンビルのような建築物までが並ぶ。定義が「観念 (idea)」から始まることに表れているとおり、物理的な形を持つかどうかは問われない。記録は何についてのものでもありうるから、主題を指す先としてこの包括的な実体が必要になる。OWL の根クラス owl:Thing との対応も注記されている。

4.2.2 記録資源 (Record Resource, E02) とその下位実体

「エージェントが生活または業務活動の中で生み出したりは取得し、保持した情報」。下位に記録集合 (E03)、記録 (E04)、記録部分 (E05) を持つ。ここで「生み出す (produce)」は「作成する (create)」より広い。記録資源を生み出すことは最初の作成を指す場合もあれば、既存の情報を組み合わせ・並べ替え・選択・再フォーマットして再利用することを指す場合もあるとスコープノートは述べている。3 者の区別と意義は第 3 章 3.4 節で述べた。定義上の要点は、記録の同一性は記録自体に由来するのに対し、記録集合の同一性はメンバーに依存して派生し、記録部分の同一性は属する記録に依存して派生する、という非対称性である。

例示が紙の文書に偏っていないことにも注意したい。RiC-CM が挙げる記録の公式例には、国王の印章が付された 17 世紀の任命状と並んで、「添付ファイル 2 点を持つ、デジタル署名付きの電子メール」がある。その添付ファイルは記録部分の例として現れる (RiC-E04, E05)。さらにスコープノー

ト (RiC-E02) は、ボーンデジタルの情報オブジェクトをどう数えるかという問いを正面から論じている。文書ファイルに埋め込まれた画像のビットストリーム (ファイルの中でそれ自体ひとまとまりをなすビット列) は記録か記録部分か。ZIP に圧縮されてひとつのファイルになった複数のビットストリームは、記録か記録集合か。定型的な文書形式 (documentary form) を持たないデジタル情報では境界の判定が難しくなる、という認識が、この実体設計の前提にある。

「集合」という名前の読み方 — Record と Record Set は分類上の兄弟である

Record Set という名前からは、2つの読み違いが生じやすい。「記録集合とは、記録をたくさん集めた**だけ**のものだ」という読みと、「記録は、記録集合の (1件だけの) 特別な場合だ」という読みである。順にほどいておく。

第一に、実体階層 (図 4.4) の線が表すのは「～の一種である」という**分類**だけである。記録も記録集合も「記録資源の一種」だが、記録が記録集合の一種であることも、その逆もない。両者は分類上の兄弟であり、別の種類のものである。一方、記録が記録集合に「含まれる・属する」ことは、この分類の線とは別に、通常**の関係** (RiC-R024、RiC-O では `rico:includesOrIncluded` とその逆 `rico:isOrWasIncludedIn`) として記述される。本書の作例では、その下位にあって直接のメンバーだけを指す `rico:directlyIncludes` を使う。第6章 6.2 節の推論で、間接的なメンバーまでたどれるようにするためである。「一種である」(分類) と「含まれる」(関係) を混同しないことが、階層図を読む第一歩である。

第二に、記録集合は「中身を集めた**だけ**のもの」ではない。文集にたとえるとよい。同じ 10 篇の文章を、ある編者は「追悼文集」として、別の編者は「地域史料集」として編んだとする。収録作品がまったく同じでも、この2つの文集は別のものである。誰が、いつ、何のために編んだかが違うからだ。RiC の記録集合も同じである。原文が「記録集合の同一性はメンバーに依存して派生する」と述べる (§2.2.2) のは、記録集合がメンバーなしには成り立たないという意味であって、メンバーの顔ぶれ**だけ**で同一性が決まるという意味ではない。まとめるという行為は、まとめられる記録を作る行為とは別の、独自の目的を持つ活動だからである (同 §2.2.2 脚注 14。「同じエージェントが記録集合とそのメンバーの双方に責任を負う場合でも、それぞれを存在させるのは目的の異なる2つの活動の遂行である」)。だから同じ記録群から複数の記録集合が作られてよいし、逆に、追加受入でメンバーが増えても同じ記録集合であり続けられる。

第三に、1件の記録だけを収めた記録集合も作れる (定義は「1つ**以上**の記録」) が、それはその記録自体とは別の実体である。1篇だけを収めた文集が、その1篇と同じものでないのと同じである。文集には編者と編集の事情があり、それは収められた文章の作者や執筆の事情とは別に記述される。

囲みの文集のたとえを図にしたのが図 4.5 である。上段の白抜き三角は実体型どうしの分類を、下段の矢印は個々の実例どうしの関係を表す。「記録集合が記録を含む」は下段にだけ現れ、上段の分類には現れない。

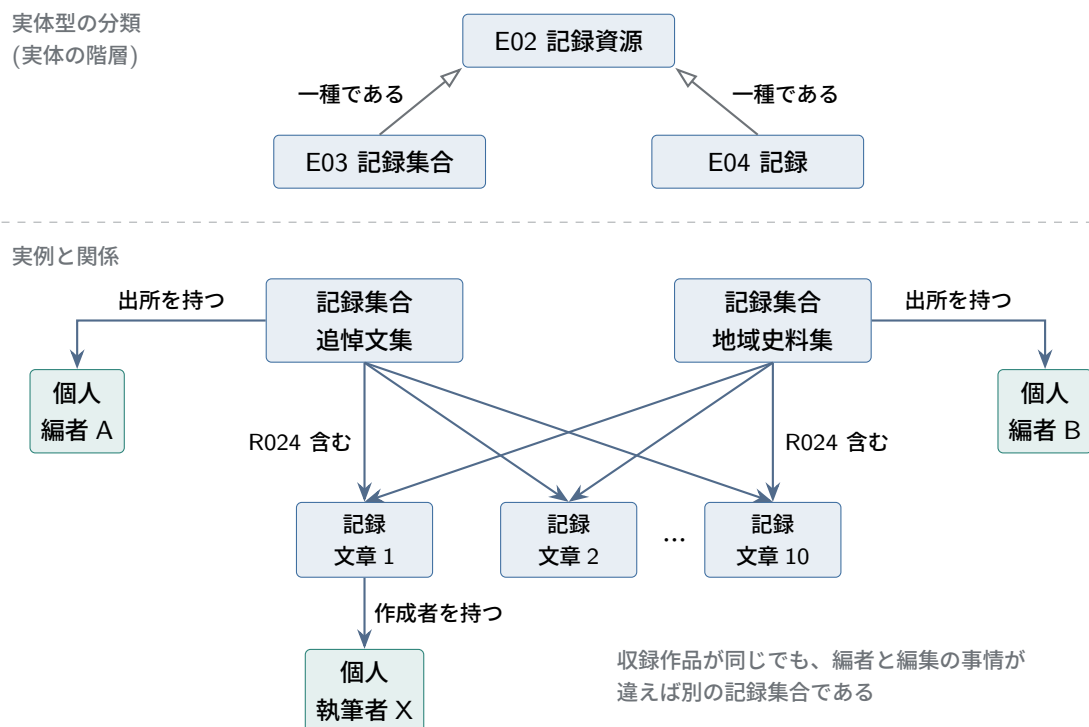


図 4.5 記録と記録集合は分類上の兄弟であり (上段)、「含む」(R024) は実例どうしを結ぶ関係である (下段)。記録集合から記録への 6 本の矢印はすべて R024「含む」である。同じ記録が 2 つの記録集合に含まれてよく (R024 のスコープノート)、記録集合の出所 (編者) は記録の作成者とは別に記述される。「出所を持つ」は R026、「作成者を持つ」は R027 である。編者を作成者 (R027) と見るか集積者 (R028) と見るかは記述の方針しだいなので、ここでは両者の上位の R026 で描いた。

4.2.3 具現化 (Instantiation, E06)

「時間と空間を越えて情報を伝達する手段として、エージェントが担体上に行った、何らかの持続的で復元可能な形で情報の記録」。第 3 章 3.5 節で述べたとおり。「担体への記録」と聞くと紙やフィルムを思い浮かべがちだが、スコープノートの例はむしろデジタル側に重心がある。同じ記録が DOCX と PDF/A と紙出力という 3 つの並存する具現化を持つ例、大きな記録集合 (フォンドやシリーズ) が 1 つのデータベースとして維持される例、ケースファイル一式が 1 つの PDF にまとめてスキャンされる例が挙がる。ビニールレコード、ビデオカセット、デジタルファイルのように、情報の伝達に機器 (メディアエータ) を要する具現化があるという指摘も、デジタル保存の課題と直結する。

4.2.4 エージェント (Agent, E07) とその下位実体

「世界の中で活動を遂行するもの」。人間とその集まりに限らず、物理的な姿を持たない役割 (職位) や、ソフトウェアとして存在するもの (メカニズム) まだがエージェントになれる。下位実体は次の 4 系統である。

個人 (Person, E08) 人間個人。スコープノートの大半は**ペルソナ** (社会的に構成された同一性) の議論に割かれている。筆名が本名を凌駕する場合 (Mark Twain)、複数人が 1 つのペルソナを共有する

場合、人が生涯の中で同一性を変える場合など、「1人=1つの同一性」が成り立たない現実への対応が明記されている。

集団 (Group, E09) 「エージェントとして共に行動する2以上のエージェント」。下位に家族 (E10) と団体 (E11)。ISAAR(CPF) の3区分 (団体・個人・家族) はエージェントの下に引き継がれている。そのうえで RiC は家族と団体を「集団」の下位に位置づけ直し、有権者団のような、団体とも家族とも言えない集団も扱えるようにした。

職位 (Position, E12) 「集団の中での個人の機能的役割」。学長、県知事、文書課長など。個人と集団の交点を独立の実体としたことが新しい。ある職位が生んだ記録集合を、歴代の在職者個人を特定しないまま記述する、という実務上よくある状況を正確に表現できる。

メカニズム (Mechanism, E13) 「個人または集団が作成した、活動を遂行するプロセスまたはシステム」。ソフトウェア、ロボット、探査機など。設計者が定めた規則に基づいて活動し、記録を作成・改変しうる。ウェブクローラや火星探査車が例に挙がる。自動処理が記録を量産する現代への対応である。

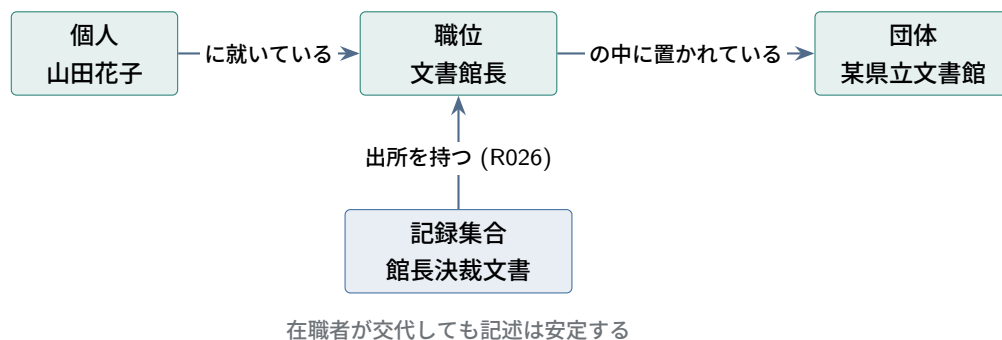


図 4.6 職位 (Position) の使いどころ。記録を「個人」でも「団体」でもなく「職位」に結びつけることで、人事異動に左右されない記述ができる (RiC-E12 スコープノートに基づく)。職位が記録を作成したとはっきり言えるなら、より狭い「作成者を持つ」(R027) で書けばよい。

4.2.5 イベント (Event, E14) と活動 (Activity, E15)

イベントは「時間と空間の中で起こる、または生じるもの」。自然現象 (地震、洪水) も、エージェントが引き起こすもの (選挙、文書のフォーマット変換) も、両者の複合 (水損記録の救済) も含む。活動はイベントの下位実体である。定義欄は「エージェントが設計した目的のために何かを行うこと」と短く、§2.2.4 の本文が「エージェントが設計し遂行する、意図された目的を持つイベント」と言い換えている。団体の文脈では従来の「機能 (function)」と同じ範囲を指す。団体・職位・メカニズムの活動は機能として記述してよいが、個人や家族の活動には「機能」という語がなじまないものもあるため、より一般的な「活動」が採用された (RiC-CM §2.2.4)。E15 の例には詩作も挙げられている。ISDF が扱っていた領域はここに吸収される。

活動には2つの見方があるとスコープノートは言う。目的 (何のために) から見る見方と、過程 (どのように) から見る見方である。組織の機能分類 (目的の階層) も、業務フロー (過程の連なり) も、どちらも活動の記述として表現できる。

4.2.6 規則 (Rule, E16) とマンドート (Mandate, E17)

規則は「エージェントの存在・責任・権限を支配し、あるいはエージェントによる活動の遂行を支配し、あるいはエージェントが作成・管理するものの際立った特性に寄与する条件」。成文の法令・規程・標準だけでなく、**不文**の社会規範や慣習も含むことが明記されている。マンドートは規則の下位実体で、「あるエージェントが別のエージェントに、活動を遂行する責任または権限を委任すること」。スコープノートはこの活動を「特定の活動」と限定し、あわせて場所 (管轄) と時間の限定も伴うのが普通だと述べる。

初学者が注意すべき点が2つある。第一に、規則・マンドートはそれを証拠立てる文書 (記録) とは別の**実体**である (「A documentary source is a record」と原文は釘を刺す)。たとえば公文書館法という法律は規則であり、その条文を載せた官報などの文書が、それを証拠立てる記録である。法が定める権限をマンドートとして別に立てることもできる。第二に、記述という活動自体も規則 (RiC-CM そのもの、各国の目録規則) に支配されており、規則実体は記述の記述 (§4.5) でも使われる。

4.2.7 日付 (Date, E18) と場所 (Place, E22)

日付は「実体の識別と文脈化に寄与する年代情報」。なぜ属性ではなく実体なのかという当然の疑問には、「年代の記述は本質的に複雑だから」と答えられている (RiC-CM §2.2.6)。自然言語表記 (「元禄8年」) と機械可読な標準表記 (ISO 8601 と、その拡張で不確かな日付や範囲も書ける EDTF。第7章 7.7 節) の併存、確実性や精度の度合い。これらを担うには、値1つの属性では足りない。場所は「境界を持つ、名前のついた地理的領域」。管轄区域 (jurisdiction)、人工構造物、自然地形を含み、歴史的実体なので、開始日や終了日を持ったり、境界が変わったりしうるとされる。座標への参照も可能である。

4.3 属性

属性 (attribute) は実体の特性である。RiC-CM 1.0 は 42 の属性 (RiC-A01～A45、欠番あり) をアルファベット順に定義し (§3.6)、続く RiC-CM の第4章 (§4) で、どの実体にどの属性が使えるかを一覧にしている。名称、識別子、一般記述 (General Description)、歴史 (History)、言語、法的地位、記録資源の数量 (Extent)、範囲と内容 (Scope and Content) など、ISAD(G) の記述要素の多くはここに対応物を見出せる。42 の全体は付録 A.2 節に原文と対照して挙げた。実体 19 や関係 85 に比べて属性は本文で扱う機会が少ないが、実際に記述を設計するときに最初に開くのはこの一覧である。

属性を読むときの要点は3つある。

■(1) 値スキーマは「指示的」であって規範ではない 各属性には値スキーマ (Value Schema) として、自由テキスト、モデル準拠テキスト、統制値、規則準拠値の4種のいずれかが示される (§3.5)。ただし原文は「値スキーマは指示的 (indicative) なものであり、規範 (prescriptive) でも禁止 (proscriptive) でもない」と明言する。ここでも RiC は内容規則を定めない。何をどの統制語彙で書くかは実装の決定事項である。4種の違いと属性の例を表 4.2 にまとめた。表の例では、統制値の属性は RiC-O で個体を参照する形になり、ほかの3種は文字列で書かれている。ただし統制値でも文字列で書くもの (§4.3.1 の末尾) があり、逆に文字列のほかにクラスでも書ける属性もある。名称・識別子 (第5章の表 5.3) や、

rico:RecordResourceExtent などのクラスがある数量がそうである。

表 4.2 値スキーマの 4 区分 (RiC-CM §3.5・§3.6 に基づく)。どれも指示的な目安で、どの語彙・書式を使うかは実装が決める。表中のプロパティ名とクラス名は rico: を省いた。

値スキーマ	原文の説明 (要約)	属性の例	RiC-O での形
自由テキスト Free text	自然言語の決まりのほかに制約のない語や文	A43 一般記述	文字列 (generalDescription)
モデル準拠テキスト Model-based text	言語上・内容上のモデルに従う語や文。表現を揃え、検索できるようにする	A01 追加受入 (Non-accruing など決まった語で書く)	文字列 (accruals)
統制値 Controlled value	統制された語彙リストから選ぶ語。シソーラスとして維持されることが多い	A36 記録集合種別	個体への参照 (hasRecordSetType で RecordSetType の個体を指す)
規則準拠値 Rule-based value	機械処理のための規則に従う値。日付や座標の国際標準など	A29 正規化日付	文字列 (normalizedDateValue)

■(2) 属性か実体かは実装で入れ替わる 概念的に「統制値」をとる属性 (活動種別、記録集合種別、職業種別など) は、実装では実体 (クラス) として扱うほうが便利なのが多いと RiC-CM 自身が述べる (§3.3)。共有語彙 (シソーラス) を作れば複数の記述から参照でき、LOD 環境では外部の語彙と接続できるからである。実際 RiC-O はこれらを rico:ActivityType・rico:RecordSetType・rico:OccupationType のようなクラスとして実体化した (第 5 章)。「CM で属性だったものが O ではクラスになっている」という対応関係は、両文書を往復するときの最重要の読み替え規則である。

■(3) 記録集合の属性には 2 種類の意味がある 第 3 章で述べた区別がここで形になる。記録集合に「言語: 日本語」と書いたとき、それは集合それ自体の属性ではなく、**メンバーである記録たちの属性の要約**である。原文は「記録集合それ自体は言語も文書形式種別も持たない」と明言している。RiC-CM §3.4 はこの問題を正面から論じ、厳密には「集合はすべてのメンバーが属性 X を持つ」「集合は一部のメンバーが属性 X を持つ」と関係を分化させるべきだが、ERM の枠内で属性を倍増させるのは煩雑なので、CM では * 印 (メンバーの記述である属性) と + 印 (集合とメンバー双方の記述でありうる属性) を付けるにとどめたと説明する。RiC-O は、* 印の属性のうち 5 つ (内容種別・文書形式種別・言語・法的地位・状態) について、rico:hasOrHadAllMembersWithLanguage / rico:hasOrHadSomeMembersWithLanguage のように全メンバーと一部のメンバーを書き分けるプロパティを用意した。「この記録集合は日本語」という日常の言い方の曖昧さを、機械可読なレベルで解消する仕組みである。一方、アクセス条件 (A08) のような文章で書く属性には rico:conditionsOfAccess という文字列のプロパティが 1 つあるだけで、この書き分けはない。原文が日常語法の例に挙げる「この記録集合は閲覧制限あり」の曖昧さは、文面で補うか、利用規程を規則として立てて結ぶことで解く (第 7 章 7.8 節)。

記録集合に使える 19 の属性を、原典の印ごとに分けたのが表 4.3 である。* 印の 10 属性のうち、RiC-O に書き分けのプロパティがあるのは上から 3 段目の 5 つだけであることが読み取れる。

表 4.3 記録集合に使える属性の 3 区分 (RiC-CM §4.2.1 の印) と RiC-O での書き方。* 印の属性のうち、全メンバーと一部のメンバーを書き分けるプロパティがあるのは 5 つだけである。表中のプロパティ名は `rico:` を省いた。

区分 (§4.2.1 の印)	属性	RiC-O での書き方
無印 (集合それ自体の属性)	A43 一般記述、A22 識別子、A28 名称、A35 記録資源の数量、A01 追加受入、A36 記録集合種別	集合に直接書く (<code>generalDescription · identifier · name · recordResourceExtent · accruals · hasRecordSetType</code>)
+(集合にもメンバーにも使える)	A21 歴史、A38 範囲と内容、A40 構造	集合に直接書く (<code>history · scopeAndContent · structure</code>)。集合とメンバーのどちらについての記述かは文面で示す
*(メンバーの記述)	A10 内容種別、A17 文書形式種別、A25 言語、A26 法的地位、A39 状態	<code>hasOrHadAllMembersWith...</code> と <code>hasOrHadSomeMembersWith...</code> で書き分ける (... は <code>ContentType · DocumentaryFormType · Language · LegalStatus · RecordState</code>)
*(メンバーの記述)	A03 真正性注記、A07 分類、A08 アクセス条件、A09 使用条件、A24 完全性注記	書き分けのプロパティはない。文字列のプロパティが 1 つずつある (<code>authenticityNote · classification · conditionsOfAccess · conditionsOfUse · integrityNote</code>)

4.3.1 属性を実際に付けてみる

以上の 3 点は、1 つの例に同時に現れる。シリーズ「伝染病予防関係綴」と、その中の 1 件「腸チフス発生届」に属性を付けたのが図 4.7 である。

RiC-O で書くと、この区別がプロパティ名の違いとして現れる。まず記録集合の側である。

```
ex:series1950s a rico:RecordSet ;
  rico:name "伝染病予防関係綴" ; # 集合自体の名称
  rico:identifier "P-83/2/1" ;
  rico:hasRecordSetType rst:Series ;
  rico:accruals "追加なし" ;
  rico:recordResourceExtent "簿冊12冊" ; # 集合そのものの大きさ
  rico:hasOrHadAllMembersWithLanguage ex:ja ; # 全メンバーが日本語
  rico:hasOrHadSomeMembersWithDocumentaryFormType ex:todokede ;
  rico:directlyIncludes ex:typhoid1 . # R024 の下位。直接のメンバー
```

同じ「言語」と「文書形式種別」を、記録の側では次のように書く。参照されている `ex:ja` と `ex:todokede` は、リテラルではなく独立した個体である。前の例の `rst:Series` も同じで、`rst:` は記録集合種別の語彙を指す接頭辞である (第 5 章)。

```
ex:typhoid1 a rico:Record ;
  rico:name "腸チフス発生届" ;
  rico:hasOrHadLanguage ex:ja ; # この記録の言語
  rico:hasDocumentaryFormType ex:todokede .

ex:ja a rico:Language ; rico:name "日本語" .
ex:todokede a rico:DocumentaryFormType ; rico:name "届出書" .
```

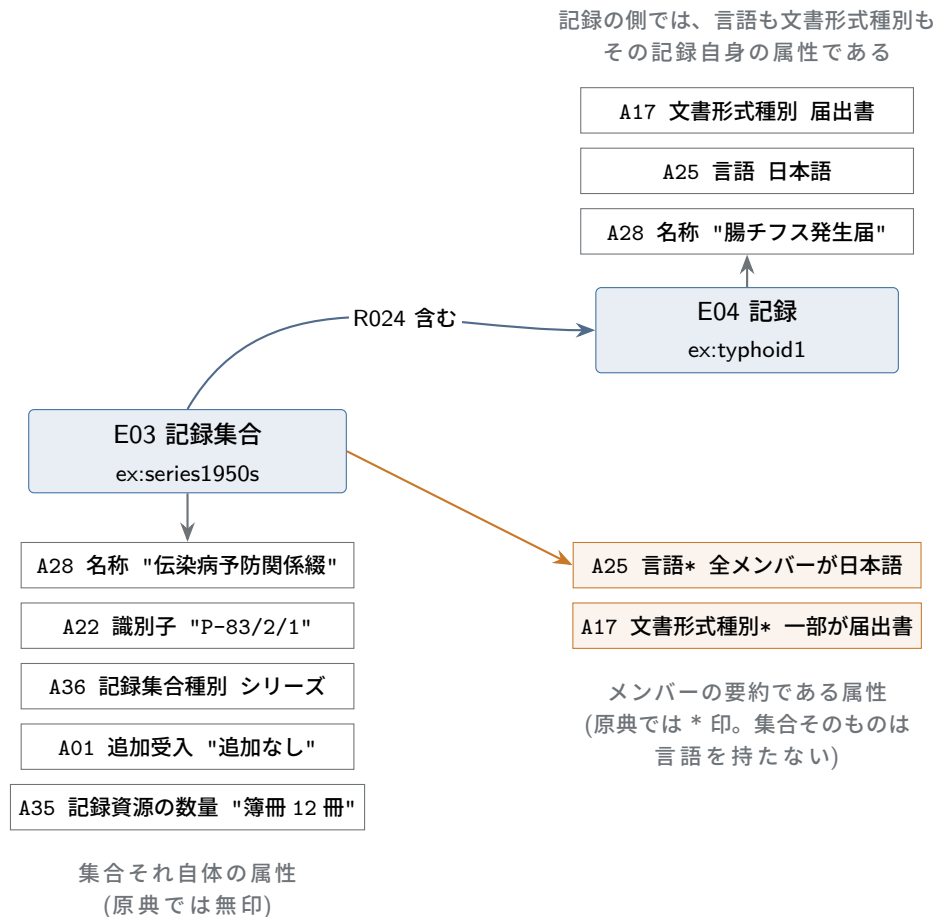


図 4.7 属性の付き方 (RiC-CM §4.2.1 の記録集合の属性一覧に基づく)。同じ「言語」でも、記録に付けばその記録の属性、記録集合に付けばメンバーの要約である。原典は後者に * 印を添えて区別し、RiC-O はプロパティ名を分けて実装する。数量 (A35) が無印であることに注意。簿冊 12 冊はメンバーの要約ではなく、その集合そのものの大きさである。

注目してほしいのは、記録集合の言語を、そのメンバーの言語として書き分けていることである。`rico:hasOrHadLanguage` の定義域は「エージェント・記録・記録部分」の和集合であり、記録集合は含まれない。`rico:hasDocumentaryFormType` も記録・記録部分を対象とする。ただし、OWL の定義域は入力を拒む制約ではない。記録集合に前者を書けば、その集合が定義域のいずれかのクラスにも属すると推論される (第 5 章)。意図した区別を保つには、上のように全メンバーか一部のメンバーかを選んで記述し、入力の実装が必要なら SHACL (第 8 章) で条件を定める。

この例には (1) と (2) も現れている。`rico:accruals` と `rico:recordResourceExtent` の値は自由テキストで、書き方の規則は RiC の側にはない (値スキーマは指示的である)。一方 `rst:Series` と `ex:today` は、リテラルではなく **個体への参照** になっている。記録集合種別も文書形式種別も、CM では統制値をとる属性だが、RiC-O ではクラスのインスタンスとして実体化されるからである。属性一覧 (付録 A.2 節) で値スキーマが「統制値」になっている属性は、実装ではたいていこの形をとる。ただし例外もあり、日付の限定 (A13) と分類 (A07。値スキーマにはモデル準拠テキストも併記されている) は、RiC-O では文字列を値にとるプロパティ (`rico:dateQualifier`、`rico:classification`) である。

4.4 関係

関係 (relation) は実体同士のつながりであり、RiC の「Contexts」を実際に担う部分である。RiC-CM 1.0 は 85 の関係を定義する (§5.4)。ID は RiC-R001 から RiC-R086 まで振られ、R043 は欠番である。以下では実体の ID (RiC-E01 など) と同様に、接頭辞を省いて R026 のように書く。ほとんどの関係には逆向きの関係 (Rnnni) が対で用意されており、逆向きを別に数えれば総数はさらに増える。

実体の階層 (図 4.4) は「どんな種類のものがあるか」を示すだけで、それらがどう結ばれるかは何も語らない。関係の体系がその部分を受け持つ。以下、13 の型による分類 (§4.4.1)、実体どうしを結ぶ実際の配線 (§4.4.2)、関係自身の階層 (§4.4.3) の順に見ていく。

原典で関係を数えるときの注意

§5.6 「List of Relations」の一覧表 (定義域の順に並ぶ) には、日付関係の R080～R086 (「～の作成日である」など) が載っていない。載っているのは R079 までである。これらは §5.3 の階層チャートにも §5.4 の個別定義にも現れる。一覧表だけを見て数えると 7 つ足りなくなる。

4.4.1 13 の型

RiC-CM はすべての関係を 13 の概念的な型 (types of relations) に分類する (§5.2)。1 つの関係が複数の型に属することもある。表 4.4 に、13 の型と、各型でもっとも広い関係を、定義域 (出発点になれる実体) と値域 (到達点になれる実体) つきで挙げる。

本書では以降、関係を日本語の短い名前と呼ぶ。原文の名称と対照できるよう、表 4.4 には原文名を併記し、全 85 関係の対照表を付録 A.4 節に置いた。本文と図表で使う訳語は、この対照表と一致させてある。型の名称の原語は付録 A.3 節にある。

値域の列を縦に見ると、13 のうち 8 つが「もの」になっている。「もの」はすべての実体の上位だから (§4.2)、これらの関係は到達点をどの実体にとってもよい。うち全体－部分と順序の 2 つは定義域も「もの」で、両端とも限定がない。残りの 5 つは定義域も値域も特定の実体に限られる。つまり関係の体系は、実体を特定して配線する関係と、相手を限定しない汎用の関係の 2 種類でできている。前者が記録の生成をたどる骨格をつくり、後者が時間・場所・規則・主題といった文脈を任意の実体に添える役を果たす。

4.4.2 実体間の関係マップ

型の一覧だけでは、どの実体からどの実体へ矢印が伸びるかが見えにくい。特定の実体を結ぶ関係を図 4.8 に、相手を限定しない関係を図 4.9 に描き分ける。

図 4.8 は、第 3 章で見た RiC の世界観をそのまま配線図にしたものである。活動がエージェントによって遂行され (R060)、その活動が記録資源として記録され (R033)、記録資源はエージェントを出所として持ち (R026)、担体の上では具現化として存在する (R025)。この 4 つの関係 (矢印は 5 本) を押さえれば、記録の生成をたどる経路は引ける。残りは同種の実体同士をつなぐ関係で、「この記録集合はあの記録集合と関連する」 (R022) 「この職員はあの職員の部下である」 (R044 の下位) のように、同じ

表 4.4 関係の 13 の型と、各型の最上位の関係 (RiC-CM §5.2・§5.3・§5.4 に基づく。原文の has or had・is or was は「現在または過去において」の意味で、訳では省いた)

型	最上位の関係 (訳)	原文名	定義域	値域
全体－部分	R002 部分を持つ	has or had part	もの	もの
順序	R008 先行する	precedes or preceded	もの	もの
主題	R019 主題を持つ	has or had subject	記録資源	もの
記録資源間	R022 記録資源として関連する	is record resource associated with record resource	記録資源	記録資源
記録資源－具現化	R025 具現化を持つ	has or had instantiation	記録資源	具現化
出所	R026 出所を持つ R033 文書化している	has provenance documents	記録資源・具現化	エージェント活動
具現化間	R034 具現化として関連する	is instantiation associated with instantiation	具現化	具現化
管理	R036 に権限を持つ	has or had authority over	エージェント	もの
エージェント間	R044 エージェントとして関連する	is agent associated with agent	エージェント	エージェント
イベント	R057 イベントとして関連する	is event associated with	イベント	もの
規則	R062 規則として関連する	is rule associated with	規則	もの
日付	R068 日付として関連する	is date associated with	日付	もの
空間	R074 場所として関連する	is place associated with	場所	もの

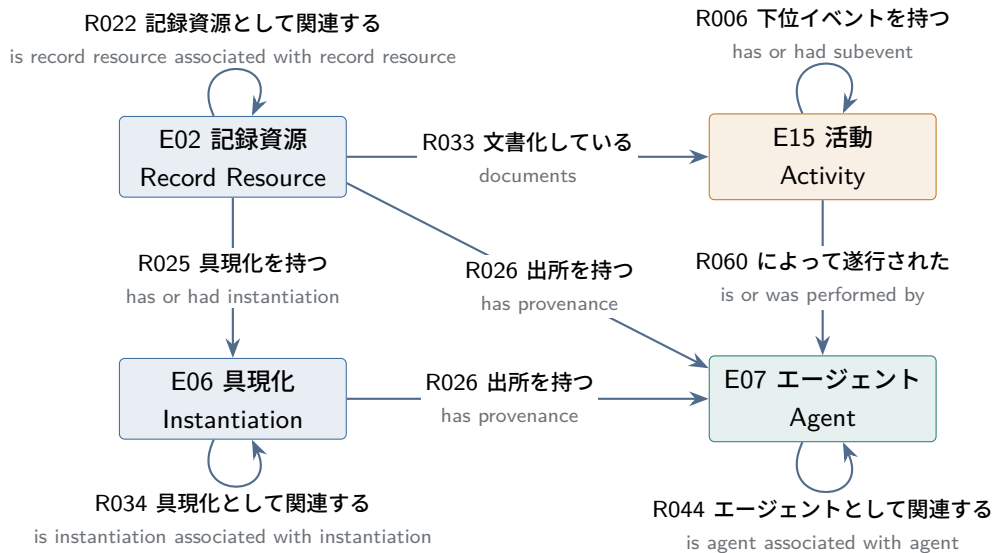


図 4.8 中核実体を結ぶ関係 (RiC-CM §5.3・§5.4 に基づく)。矢印は定義域から値域へ向かう。同じ箱に戻る矢印は、その実体同士を結ぶ関係 (記録資源と記録資源、エージェントとエージェントなど) を表す。R006 の定義域・値域はイベントだが、活動はイベントの一種なので活動どうしにも使える (R006 のスコープノート)。逆向きの関係も対で定義されているので、実装ではどちらの向きからも記述できる。関係名の原文は付録 A.4 節の対照表にある。

種類のもののあいだの横のつながりを担う。

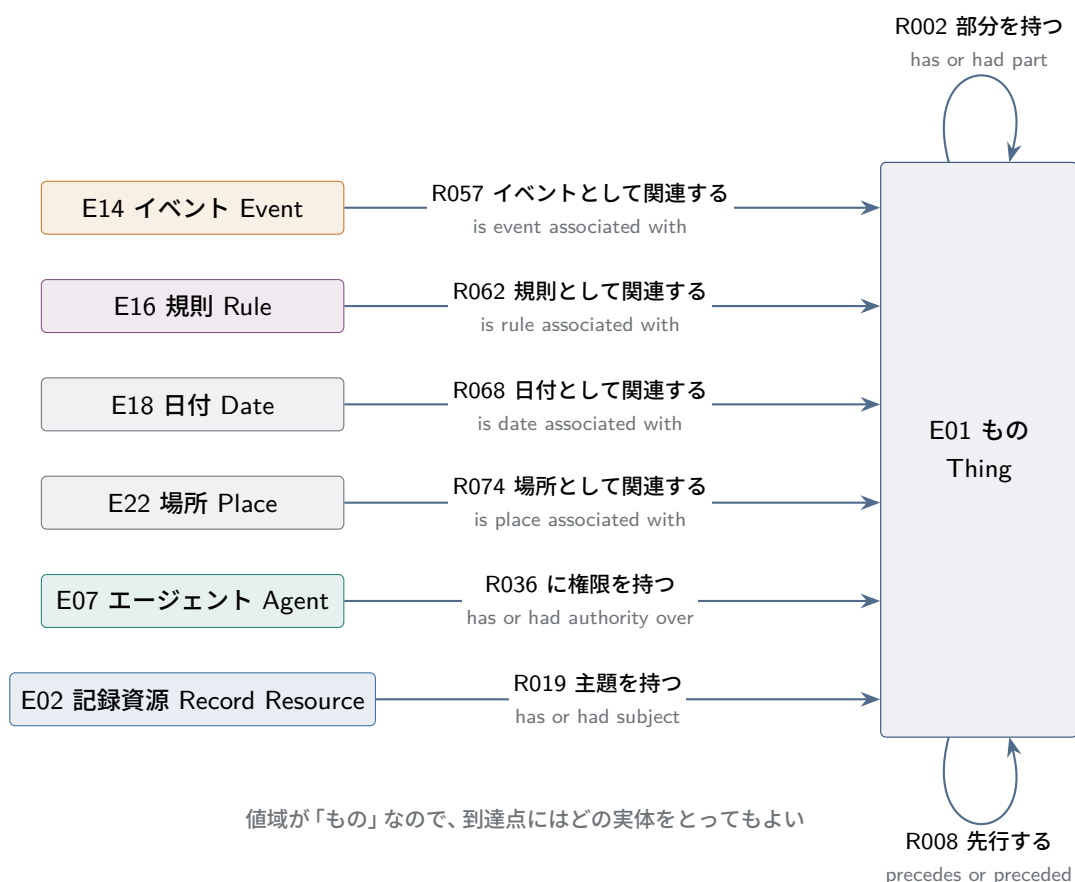


図 4.9 値域が「もの」である関係 (RiC-CM §5.3 に基づく)。イベント・規則・日付・場所は、記録資源にもエージェントにも活動にも、そして互いにも結びつけられる。「もの」に戻る 2 本の矢印は、全体－部分と順序が実体の種類を問わずに成り立つことを表す。関係名の原文は付録 A.4 節の対照表にある。

図 4.9 を読むときの要点は矢印の向きである。日常語では「記録に日付を付ける」と言うが、RiC-CM の R068 は日付を定義域、ものを値域にとり、「日付が～に関連する」と定義されている。イベント・規則・場所も同じで、文脈を担う実体の側が出发点になっている。ただし RiC-CM は各関係に逆向きの関係を用意しており (R068i)、RiC-O では `rico:isDateAssociatedWith`(日付→もの) と `rico:isAssociatedWithDate`(もの→日付) の両方がプロパティとして定義されている。記述の現場では書きやすい向きを選べばよく、推論器が逆向きを補う (第 6 章 6.3 節)。図の向きは、原典の定義域・値域の表記に合わせたものと理解しておけばよい。

4.4.3 関係自身の階層

関係は平らなリストではなく、広い関係から狭い関係へと階層をなす。頂点は「ものはものに関係する (R001 is related to)」で、そこから 13 の型の最上位関係 (表 4.4 の第 2 列) へ枝分かれし、さらに細分されていく。原典のチャート (§5.3) は 5 階層分の欄を持つ。ほとんどの枝は第 3～4 階層で止まり、最も深いところだけが第 6 階層に達する (R018 「子を持つ」の 1 本。欄が足りないので、チャートでは第 5 階層の欄に「第 6 階層」と注記して押し込まれている)。この階層はポリヒエラルキーで、1

つの関係が複数の親を持ち、チャートの複数の場所に現れる。原典のチャートで同じ関係に ”(see also above/below)” と注記されているのはそのためである。

出所の型を例にとると、階層は図 4.10 のようになっている。

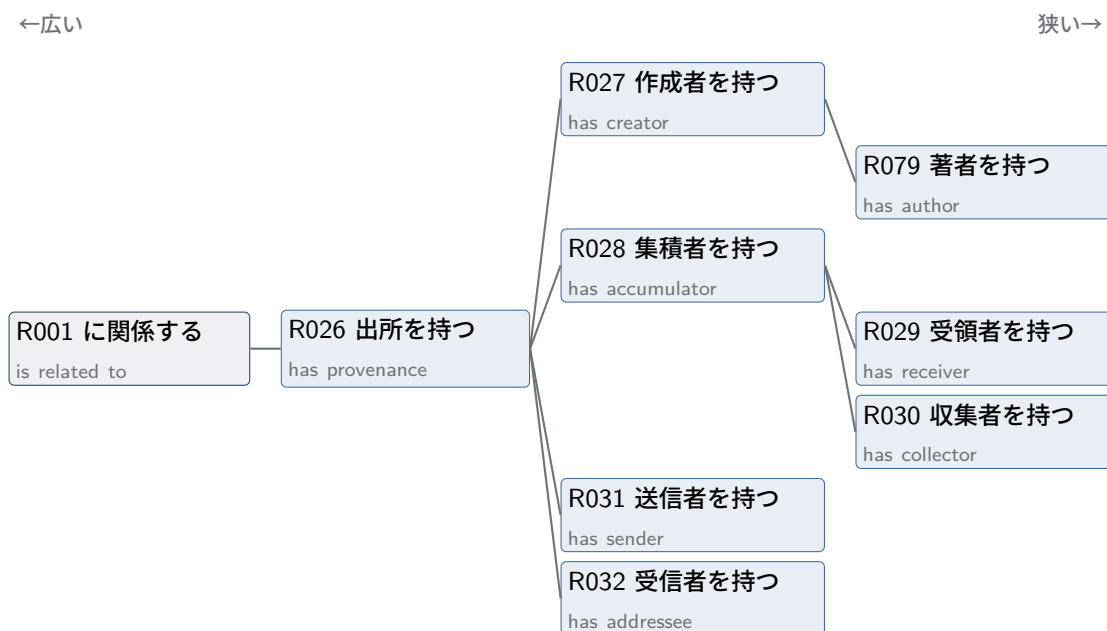


図 4.10 出所の型に属する関係の階層 (RiC-CM §5.4 の Broader/Narrower relations 欄に基づく)。各箱の下段が原文の関係名である。線は「～より狭い関係である」を表す。R079 だけは定義域が記録に、値域が個人・集団・職位に限られ、他は定義域が記録資源・具現化、値域がエージェント全体である。

この階層構造は、記述と検索の両方を支える仕掛けである。狭い関係 (「この書簡は山田花子を送信者として持つ」R031) が成り立てば、その上位の広い関係 (「出所を持つ」R026、「に関係する」R001) も自動的に成り立つ。記述者は分かる範囲で最も狭い関係を選べばよく、検索者は広い関係で横断的に問い合わせられる。粒度の自由 (第3章) を支えるのはこの設計であり、RiC-O ではこれが下位プロパティ宣言として形式化され、推論器が上位関係を補う (第6章 6.4 節)。

4.4.4 関係そのものに属性がある

RiC-CM §5.5 は、関係そのものに付く属性を定義する (RA01 確実性、RA02 日付、RA03 記述、RA04 識別子、RA05 出典、RA06 場所) の 6 つ。原語の名称はいずれも Certainty of Relation のように of Relation を伴う。「A が B を作成した」という関係に対して、「いつ」「どのくらい確実か」「典拠は何か」を付記できるということである。たとえば作成関係に「日付:1873～1875 年頃」「確実性: 不確実」「出典: 表紙の書き込み」を添える。ISAAR(CPF) にも関係の説明と日付を書く欄があった (5.3.3・5.3.4)。RiC-CM の「関係の属性」は RDF で素直に表現できないため、RiC-O では関係の実体化 (reification) という技法で実装される (第5章 5.5.3 項)。

4.4.5 読み方の実際

85 の関係の各定義 (§5.4) は、ID、名称 (逆関係の名称を併記)、定義域 (domain: 関係の出発点になれる実体)、値域 (range: 到達点になれる実体)、多重度、定義、スコープノート、例、関係の型、上位関係、下位関係からなる。たとえば出所の型の最上位にある R026 は、名称が `has provenance`、定義域が「記録資源または具現化」、値域が「エージェント」、型が出所、上位関係が R001、下位関係が R027・R028・R031・R032 である。R026 のスコープノートは、より狭い関係 (たとえば作成者を持つ) が使えないときにだけ使うよう注意している。定義域と値域という言葉は RDF/OWL でもそのまま使われるので、ここで慣れておくとよい。R026 の定義欄を実際に並べ、各欄の読み方を添えたのが表 4.5 である。

CM と O を往復するとき、名称のずれに注意が要る。R026 に対応する RiC-O のプロパティは `rico:hasOrganicProvenance`(`has organic provenance`) で、`organic`(有機的) が加わっている。RiC-O は CM に対応するプロパティに `rico:RiCCMCorrespondingComponent` という注釈で対応する CM の関係 ID を記録しているので、名前ではなく ID で突き合わせるのが確実である。表 4.5 の最下行がその例である。

表 4.5 RiC-CM §5.4 の関係定義の欄 (R026 の例) と、対応する RiC-O のプロパティ。名称がずれても、注釈に書かれた CM の ID で突き合わせられる。

欄	R026 の記載 (原文)	読み方
ID	RiC-R026	関係の番号
Name	<code>has provenance</code> inverse relation: is provenance of	名称と、逆向きの関係の名称
Domain/Range	Record Resource or Instantiation / Agent	定義域 (出発点) と値域 (到達点) になれる実体
Cardinality	M to M	多重度 (多対多)
Definition	Connects a record resource or an instantiation to an agent that creates or accumulates the record resource or receives or sends it.	定義
Scope Notes	This is the most general provenance relation. Use it ... only if it is not possible to use a narrower, more specific relation, for example has creator.	使い方の注意。ここでは「狭い関係を優先せよ」
Examples	C. T. Lanham Papers on Ernest Hemingway [record set] has provenance Lanham, C. T. (Charles Trueman), 1902- [person]	実例 (括弧内は実体の種類)
Relation types	Provenance relations	13 の型のどれに属するか
Broader relations	RiC-R001 is related to	1 つ上の関係
Narrower relations	RiC-R027 has creator, RiC-R028 has accumulator, RiC-R031 has sender, RiC-R032 has addressee	1 つ下の関係
(RiC-O 側)	<code>rico:hasOrganicProvenance</code> 。注釈 <code>rico:RiCCMCorrespondingComponent</code> の値は RiC-R026 ('has provenance' relation)	名称は違うが、ID で対応が分かる

4.5 記述を記述する

RiC-CM 第6章の主題は「記述という行為そのものの記述 (documenting description)」である。記述レコードもまた記録であり、エージェント (文書館、アーキビスト、変換プログラム) が活動 (記述) を遂行して作成したものだから、RiC の実体と関係でそのまま記述できる。専用の語彙は要らない、というのが主張である。

原文は4つの層を「地図のズームイン」にたとえる (§6.1)。所蔵機関 (どんなマンドートと規則の下で記録を保存・記述しているか)、職位 (どの担当がその作業を行うか)、記述レコード (この目録データ自体がいつ誰により作成・改訂されたか)、そして記述内の個々の言明の検証 (この生没年はどの史料に基づくか) である。原典は各層を図4~8で例示しており、それぞれが通常の実体と関係だけで描かれている (表4.6)。

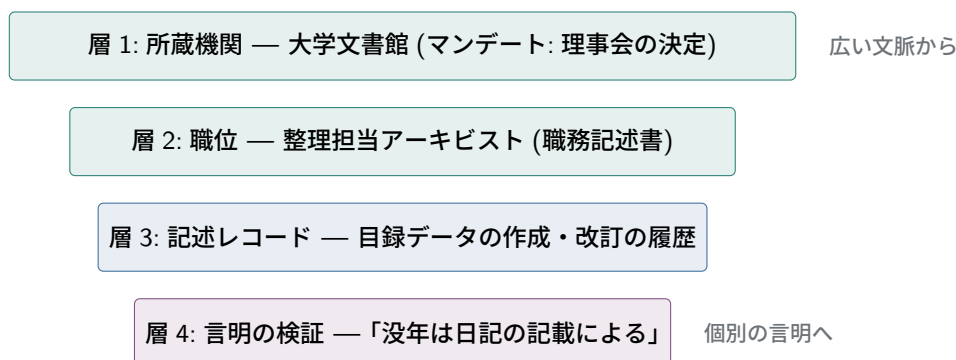


図 4.11 記述の記述の4層 (RiC-CM §6.1)。それぞれの層が、より特定の層の文脈になる。

表 4.6 記述の記述の4層を、原典の図4~8がどの実体と関係で描いているか (RiC-CM §6.2~§6.5)。専用の語彙はなく、通常の実体・関係・属性の属性だけで書かれている。関係は原典の図の表記のまま示す。

層	原典の図	主な実体	図に現れる関係
1 所蔵機関	図4 (§6.2)	団体 (大学文書館)、マンドート (理事会の決定)、規則 (記録管理方針・記述手順)、記録 (保存期間表)、活動 (記述)	R067 authorizes、R065i is responsible for issuing、R064 is or was expressed by、R063 regulates or regulated
2 職位	図5 (§6.3)	団体、マンドート (職務記述書)、規則 (記述手順)、職位、個人、活動 (記述)	R065i、R067、R063、R054i is or was occupied by、R060i performs or performed
3 記述レコード	図6・図7 (§6.4)	職位、個人、活動 (記述・記述の修正)、記述レコード (修正前と修正後)、記述対象の記録	R054i、R060i、R061 results or resulted in、R021 describes or described、R009 precedes in time、R059 affects or affected。図7は R060i に関係の属性 RA02(日付) を添える
4 言明の検証	図8 (§6.5)	個人、日付	R070i has birth date に関係の属性 RA05(出典) を添える

この章は、メタデータの伝統的な二分法に対する RiC の回答でもある。従来の実務では、**記述メタデータ** (記録の内容・文脈を利用者の発見のために記述するもの) と **管理メタデータ** (その記述自

体を誰がいつどの規則で作り、いつ改訂したか) を区別し、後者には専用の置き場が用意されてきた。ISAD(G) の第 7 エリア (記述コントロール: アーキビストノート、規則・慣行、記述作成日)、EAD の eadheader/control、EAC-CPF の control である。つまり 2 種類のメタデータは、**同じ文書の別区画**に住んでいた。

RiC はこの区別を、区画の違いではなく**階の違い**として扱う。管理メタデータとは「記述レコードという記録についての、記述メタデータ」である。だから専用の語彙は要らず、同じ実体と関係を一段上に適用すればよい。記述したアーキビストは `rico:Person` (歴史上の人物とまったく同じクラス)、準拠した目録規則や適用プロファイルは `rico:Rule`、記述という仕事は `rico:Activity`、作成・改訂の時期は `rico:Date` である。具体的に書いてみる (図 4.12)。次の例に出てくる角括弧は、名前を付けずにその場で個体を 1 つ作る Turtle の書き方である (第 5 章 5.1 節のコラム)。

```
# 記述レコード: この目録データそのものを1個の記録として立てる
ex:desc83 a rico:Record ;
    rico:describesOrDescribed ex:fondsP83 ; # 何を記述したものか
    rico:hasCreator ex:sato ; # 記述したアーキビスト
    rico:isOrWasRegulatedBy ex:profile2026 ; # 準拠した規則
    rico:hasOrHadLanguage ex:ja ; # 属性の例と同じ個体
    rico:hasCreationDate [ a rico:Date ; # 記述の作成日
        rico:normalizedDateValue "2026-07" ] .

ex:sato a rico:Person ; # 記述者も Person 実体
    rico:name "佐藤一郎" ;
    rico:occupiesOrOccupied ex:pos1 . # 職位(層2)への接続

ex:pos1 a rico:Position ;
    rico:name "整理担当アーキビスト" ;
    rico:existsOrExistedIn ex:archives . # 機関(層1)への接続

ex:archives a rico:CorporateBody ;
    rico:name "某県立文書館" .

ex:profile2026 a rico:Rule ;
    rico:name "某県立文書館 記述適用プロファイル 2026年版" .
```

表 4.7 ISAD(G) 第 7 エリア (記述コントロール) の要素と、上の作例での書き方。専用の欄が、記述レコードに付く通常の関係になる。

ISAD(G) の要素	作例のプロパティ	値になる実体
3.7.1 アーキビストノート (誰が作成したか。「どう作成したか」は作例では扱わない)	<code>rico:hasCreator</code>	<code>rico:Person(ex:sato)</code>
3.7.2 規則・慣行	<code>rico:isOrWasRegulatedBy</code>	<code>rico:Rule(ex:profile2026)</code>
3.7.3 記述作成日	<code>rico:hasCreationDate</code>	<code>rico:Date</code>

要点は 2 つある。第一に、`rico:describesOrDescribed` という 1 本の関係が、記述レコード (desc83) と記述対象 (fondsP83) という**階の違う 2 つの記録資源**をつなぐ。ISAD(G) 第 7 エリアに書かれてい

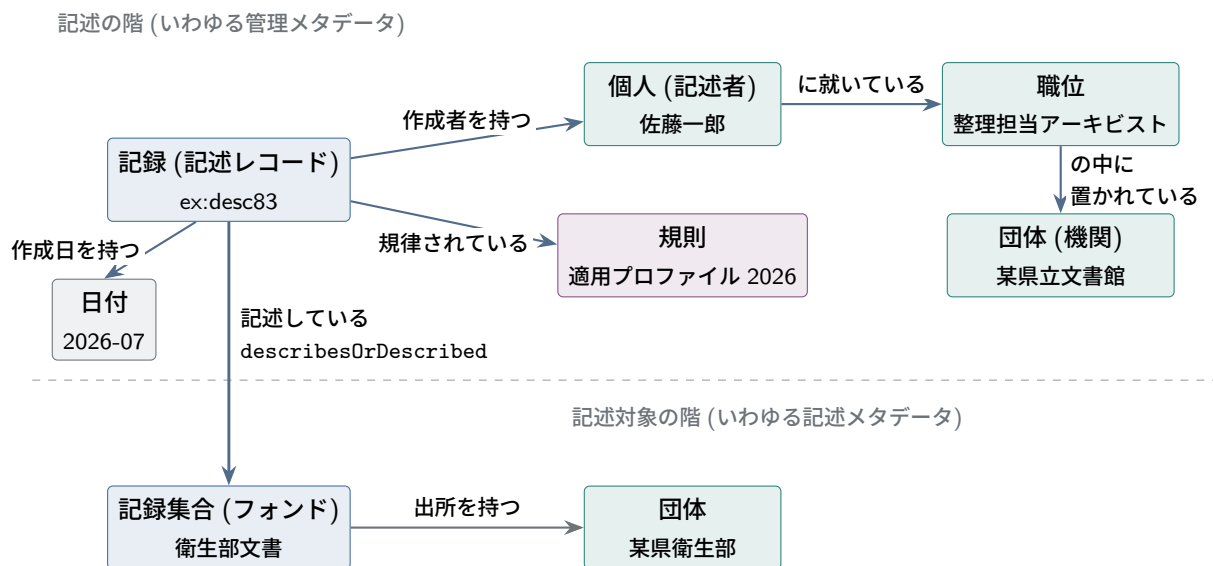


図 4.12 記述の記述の仕組み。目録データそのもの (記述レコード) が一段上の階の記録になり、記述者・規則・日付はその記録に関係で結ばれる。2つの階は `rico:describesOrDescribed` という通常の関係でつながる。記述者・職位・機関は、記述対象の階の実体とまったく同じクラスである。矢印のラベルは付録の対照表の訳語で、「規律されている」「作成日を持つ」は R063・R080 の逆向き (R063i・R080i) である。

た内容は、すべて desc83 の側の属性・関係として表現される (表 4.7)。第二に、**記述者は特別扱いされない**。アーキビスト佐藤一郎は、記録を作成した歴史上の行為者たちと同じ `rico:Person` であり、職位 (`rico:Position`) を介して機関 (層 1、`rico:CorporateBody`) につながる。これは語彙の節約であると同時に、考え方の表明でもある。記述もまた歴史的な行為であり、記述者もいずれ記録の歴史の登場人物になる。RiC-FAD §7 は、どの記録を残し、どう記述したかというアーカイブズとアーキビストの役割を記述することを、記録を理解し解釈するための重要な文脈に数えている。その考えがモデルの形で実装されている。

原典の図はもう少し細かい。上の作例と違うところを 3つ補っておく。

- 原典の図は**記述という活動**を実体として立てる。個人が活動を遂行し (R060i)、その活動が記述レコードをもたらす (R061「をもたらした」) という三段である。上の作例が `rico:hasCreator` で個人へ直結しているのは、入門のための短縮である。工程を活動として立てる書き方は第 7 章 7.9 節でまとめて扱う。
- 層 1 の図 (原典 図 4) には**保存期間表**が現れる。「記録管理方針」という規則が「保存期間表」という記録によって表現される (R064)、という形である。規則と、それを証拠立てる記録を別の実体として立て、両者を結ぶ。この型は第 7 章 7.9 節でそのまま使う。
- 層 4 の図 (原典 図 8) は、個人と日付を結ぶ関係 (R070i。R070「の生年月日である」の逆向き) に、関係の属性 RA05 出典を添えた形である (§4.4.4)。「この生年はどの史料による」という根拠は、個人の属性ではなく、関係に付ける出典として書く。

なお実務では、記述レコードをここまで実体化する代わりに、名前付きグラフ (第 7 章 7.4.4 項) の単位で作成者・日付・準拠規則を記録する近道もある。粒度は粗いが安価である。両者は排他的でないの

で、グラフ単位の管理情報から始めて、必要な部分だけ記述レコードに精密化するという段階的な進め方ができる。

この章が意味するところは大きい。記述の作成者・根拠・改訂履歴が構造化されて残るなら、目録は「無署名の権威」であることをやめ、検証可能な言明の集まりになる。誰がいつどんな根拠でそう記述したのかを利用者が確かめられるようにすることは、先に見た RiC-FAD §7 の考え（アーキビストの役割の記述は記録を理解するための重要な文脈である）を、実際に形にすることでもある。ただし、すべての言明に出典を付ける記述は高コストであり、どこまでやるかはやはり実装の判断である。

確認問題 (ヒントは付録 A.6)

- 問 4.1** 記録集合に記録が「属する」とことと、記録が記録資源の「一種である」ことの違いを、本章の文集のたとえを使って説明せよ。
- 問 4.2** 身近な出来事（たとえば授業のレポート提出）を、RiC-CM の実体（記録・エージェント・活動・日付）と関係で図示せよ。
- 問 4.3** ある目録データそのものを記述レコードとして実体化すると、何ができるようになるか。2つ挙げよ。
- 問 4.4** 図 4.12 で、アーキビストと歴史上の人物が同じクラス (Person) であることの意味を述べよ。

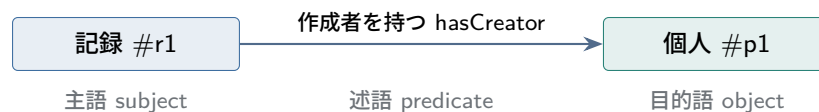
第 5 章

RiC-O — オントロジーという実装

本章では RiC-O を扱う。前半は前提知識 (RDF とオントロジー) の入門、後半は RiC-O 本体の設計の読解である。

5.1 RDF 入門 — 3 語の文でできた世界

RDF(Resource Description Framework) は、W3C が標準化したデータ表現の枠組みである。発想は単純で、あらゆる情報を「主語・述語・目的語」の 3 つ組 (トリプル) の集まりとして表す。



トリプルを何本も集めると、ノード (主語・目的語) がアーク (述語) で結ばれたグラフができる。第 2 章の図 2.3 右側は、実はトリプルの束である。RDF の重要な約束事は次の 3 つだけである。

1. **ものには IRI で名前を付ける。**IRI(Internationalized Resource Identifier) は、世界規模で対象を識別するための名前である。本書ではウェブのアドレスと同じ形の HTTP IRI を使う。管理するドメインの下で名前を発行すれば、他機関の名前空間と区別できる。ただし、同じ IRI を別の対象に使い回さない責任は発行者にある。IRI 自体が対象の同一性や永続性を保証するわけではない。また、IRI は「ブラウザで開けるページ」を必須としない。公開時に説明を取得できるようにすることは、名前の構文とは別の運用上の約束である。
2. **値はリテラルで書ける。**目的語の位置には、別のものへの参照 (IRI) のほかに、**リテラル (literal)** を置ける。リテラルとは「調査旅行日誌」「1868」「2026-07-31」のような、**それ以上たどる先を持たない生の値**である。「#r1 — 名称 → “調査旅行日誌”」のように書く。IRI との違いは、たどれるかどうかにある。IRI は「その名前と呼ばれるもの」を指し、その先にさらに記述が続く。リテラルはそこで行き止まりで、文字列や数値としてそのまま読むだけである。本書の図では、IRI で名指すものを角の丸い色付きの箱で、リテラルを白い四角で描く。色付きの箱からはさらに矢印を出せるが、白い四角は矢印を受けるだけの終端である (図 5.7 の “日誌” など)。だから「作成者」を IRI で書けばその人物の生没年もたどれるが、文字列で書けば名前が読めるだけになる。この選択が第 3 章 3.6 節の「簡便法か精密か」であり、第 4 章 4.1.2 項の設計 A と設計 B の違いでもある。

3. 述語も IRI である。「作成者を持つ」という関係自体にも世界規模の名前が付く。この述語の語彙集を提供するのがオントロジーであり、RiC-O の仕事である。

コラム: トリプルは「積み上げ型」の表である

表計算やデータ分析に馴染みがあれば、トリプルという発想は**表の持ち方の転換**として理解できる。成績のデータを例にする。ふつうに思い浮かべるのは左の形、行が生徒、列が科目の表だろう。同じ情報は、右の形でも持てる。

氏名	数学	国語
佐藤	40	88
鈴木	98	70

氏名	科目	得点
佐藤	数学	40
佐藤	国語	88
鈴木	数学	98
鈴木	国語	70

右の積み上げ型は、1 行が「誰の・何の・いくつ」という 3 つ組である。ここまで来れば気づくだろう。トリプルの集まりとは、列を「主語・述語・目的語」と読み替えた、積み上げ型の表にほかならない。氏名が主語、科目が述語、得点が目的語である。RDF を格納して SPARQL で検索できるようにしたデータベースをトリプルストア (triplestore) と呼ぶ。その内部が実際にこの 3 列の巨大な表であることは、第 6 章 6.8 節で見る。

積み上げ型の利点は、そのまま RDF の利点である。第一に、**新しい科目が増えても列を足さなくてよい**。行を足すだけである。RDF が、スキーマ変更なしに新しい述語 (語彙) を受け入れられるのはこの性質による。第二に、**空欄が存在しない**。鈴木が英語を受けていなければ、その行を書かないだけで、列展開型のような空セル (NULL) は生じない。「書かれていないことは不明」という開世界の解釈 (5.2 節) とも、この持ち方は相性がよい。第三に、**個体ごとに持つ属性がばらばらでも平気**である。記録には記録の、人物には人物の述語だけが並べばよく、全個体に共通の列を設計する必要がない。多様な実体が混在するアーカイブズ記述に都合がよい。

逆方向も見ておく。見慣れた列展開型の表は、積み上げ型からピボット (pivot・unstack) という操作で**生成できる**。つまり従来型の一覧表は、トリプルの集まりから作る出力の 1 つである。第 3 章で見た「入力と出力の分離」の、最小の実例がここにある。なおこの持ち方は、データ分析では wide/long 形式、データベース実務では横持ち・縦持ち、設計パターンの名では EAV (Entity-Attribute-Value) と呼ばれてきた、実績のある技法の親戚である。

RDF はモデル (考え方) であって、書き方 (構文) は複数ある。本書では人間に読みやすい Turtle 記法を使う。フォンドとその中の 1 記録を Turtle で書くところなる。

```

PREFIX rico: <https://www.ica.org/standards/RiC/ontology#>
PREFIX rst: <https://www.ica.org/standards/RiC/vocabularies/recordSetTypes#>
PREFIX ex: <https://archives.example.jp/>

ex:fonds01 a rico:RecordSet ;
    rico:hasRecordSetType rst:Fonds ;
    rico:name "山田家文書" ;
    rico:directlyIncludes ex:r1 .
    # 「a」は「～のインスタンスである」
    # 種別: フォンド(同梱の個体)
    # 記録r1を直接含む

ex:r1 a rico:Record ;
    rico:name "調査旅行日誌" ;
    rico:hasCreator ex:p1 ;
    rico:hasOrHadDigitalInstantiation ex:r1scan . # デジタル具現化

ex:r1scan a rico:Instantiation . # スキャン画像(TIFF)

ex:p1 a rico:Person ;
    rico:name "山田花子" .

```

記録 `ex:r1` は情報内容としての日誌であり、そのスキャン画像は別の個体 (具現化) として現れる。第3章で見た区別が、データの形にそのまま写っていることを確かめてほしい。Turtle の文法規則は少ないので、次のコラムにまとめて示す。ここにある規則だけで、本書の例も、Turtle で公開されている RDF の多くもおおむね読める。

コラム: Turtle の文法 — これだけ知れば読める

文とピリオド: Turtle の1文は「主語 述語 目的語」の3語で、ピリオド `.` で終わる。`ex:r1 rico:hasCreator ex:p1 .` は「`r1` の作成者は `p1` である」という1つの文である。文の切れ目を決めるのはこのピリオドだけで、改行ではない。

空白と改行: 語と語の区切りには、半角スペース・タブ・改行のどれを何個使ってもよい。処理系にとってはすべて同じ「1つ以上の区切り」である。したがって次の3つは、まったく同じ1文である。

```

ex:r1 rico:hasCreator ex:p1 .
ex:r1    rico:hasCreator    ex:p1 .
ex:r1
    rico:hasCreator
        ex:p1 .

```

つまり字下げや桁揃えは、人間が読みやすいように付けるだけの飾りである。Python のように字下げが意味を持つことはなく、XML のように閉じタグで範囲を示すこともない。範囲を決めるのは区切り記号 (ピリオド、セミコロン、コンマ、角括弧) だけである。本書の例で述語の位置を揃えているのも、読みやすさのためにすぎない。

逆に、**語の内部に区切りを入れてはいけない**。`rico:hasCreator` のように接頭辞の後ろで改行や空白を入れると、構文の誤りになって読み込めない。長い IRI が行からはみ出すときは、途中で折り返さず、行そのものを長くするか、接頭辞を使って短く書く。

ピリオドは直前の語にくっつけて `ex:p1.` と書いても通るが、`ex:p1 .` と 1 つ空けるのが慣習で、そのほうが誤読も誤記も減る。

名前空間:IRI を見ると、`https://www.ica.org/standards/RiC/ontology#Record` のように、前半と後半に分かれている。後半の `Record` が個々の名前で、前半の `https://www.ica.org/standards/RiC/ontology#` の部分を**名前空間** (namespace) と呼ぶ。「どの機関の、どの語彙集に属する名前か」を表す部分である。IRI の各部分と、この後で述べる `#` や接頭辞との関係を図 5.1 にまとめた。

名前空間が要る理由は、名前の衝突を避けることにある。`Record` という語は、アーカイブズでは「記録」、データベースでは「表の 1 行」、音楽ではレコード盤を指す。もし短い名前だけで済ませれば、別の分野の `Record` と区別がつかない。名前空間を前に付ければ、「ICA の RiC オントロジーの `Record`」と「別の語彙の `Record`」は違う IRI になり、同じデータの中に混ざっても取り違えられない。日本語の姓名で同姓同名を所属で区別するのに似ている。名前空間にドメイン名を使うのは、本節冒頭の約束事 1 で述べたとおり、自分が管理するドメインの下で名前を発行すれば、他機関の名前空間と重ならないからである。

名前空間の末尾の `#:RiC-O` の名前空間は `.../ontology#` と、シャープ記号で終わる。この記号はウェブの URL で「文書内の位置」を指す**フラグメント** (fragment) の区切りで、RDF ではこれを名前空間と個々の名前の境目に流用している。`rico:Record` を展開すると `.../ontology#Record` になり、`#` の前が語彙集、後が語彙集の中の 1 語という読み方になる。この書き方には理由がある。HTTP はフラグメントより後ろをサーバに送らない決まりなので、`.../ontology#Record` を取りに行くと、実際に取得されるのは `.../ontology` という 1 つのファイルである。つまり**語彙全体が 1 回の通信でまとめて手に入る**。まとまりが小さく、要素が一緒に更新されていく語彙にはこの方式が向き、オントロジーはその典型である (W3C の指針 *Cool URIs for the Semantic Web*)。

区切りに `/` を使う流儀もある。本書の例で個々の記録に `https://archives.example.jp/r1(ex:r1)` という IRI を与えているのがそれで、この場合は名前ごとに別々に取得できる。何万件もの記録を公開する側では、1 件だけ見たい利用者に全件を渡すわけにいけないので、`/` が向く。語彙は `#`、実データは `/` という使い分けは、この違いから来ている。

接頭辞: 名前空間は長く、しかも同じデータの中で何度も繰り返される。そこで Turtle では、冒頭の PREFIX 宣言で「この長い前半を、以後は短い呼び名で略記する」と約束する。辞書の凡例が「【日国】 = 『日本国語大辞典』」と定めるのと同じ仕組みである。上の例の 1 行目の宣言により、以後 `rico:Record` と書けば `<https://www.ica.org/standards/RiC/ontology#Record>` と書いたのとまったく同じ意味になる。つまり `rico:` は「ICA の RiC オントロジーという語彙集の中の」という所属の表示であり、`ex:` は本書の例のために仮に立てた架空機関の名前空間である。略さずに書くときは、IRI 全体を山括弧 `<>` で囲む。宣言の書き方には 2 通りある。PREFIX `rico: <...>` という形 (ピリオドは不要) と、@prefix `rico: <...> .` という形 (末尾にピリオドが要る) である。前者は SPARQL に合わせて Turtle 1.1 で追加されたもので、後者が古くからの書き方である。意味は同じで、公開データではどちらも見かける。ピリオドの有無だけが違うので、書き写すときはそこに注意する。

述語 `a:ex:rel01 a rico:OrganicProvenanceRelation` のような文に現れる `a` は、それ自

体が1つの述語で、「～というクラスのインスタンスである」を意味する `rdf:type` の略記である (英語の *is a* に由来する)。つまりこの文は「`rel01` は有機的出所関係 (というクラス) のインスタンスである」と読む。なお `a` を他の述語と同じ調子で「2つのものの関係」と読もうとすると、目的語がものではなくクラスだという点で少しねじれる。このねじれは5.2節で論理と突き合わせるときに改めて扱う。

セミコロンとコンマ: 同じ主語について複数の文を書くときは、主語を繰り返す代わりにセミコロン ; で「同じ主語のまま、次の述語へ」と続ける。さらに、述語まで同じで目的語だけが複数あるときは、コンマ , で目的語を並べる。次の3通りはまったく同じ意味である。

```
# (1) 全部を別の文で書く
ex:f rico:includesTransitive ex:a .
ex:f rico:includesTransitive ex:b .
ex:f rico:name "フォンドF" .

# (2) セミコロンで主語を共有する
ex:f rico:includesTransitive ex:a ;
    rico:includesTransitive ex:b ;
    rico:name "フォンドF" .

# (3) さらにコンマで述語も共有する
ex:f rico:includesTransitive ex:a , ex:b ;
    rico:name "フォンドF" .
```

リテラル: 文字列は `"..."` で囲む。言語タグを `"名主"@ja` のように付けられ、`"1868"^^xsd:gYear` のようにデータ型を付けることもできる。引用符の中では、空白も改行も**意味のある文字**として扱われる点が、引用符の外との大きな違いである。ただし `"..."` の中に生の改行は書けないので、改行を含む文章を書きたいときは三重引用符 `"""..."""` を使う。引用符そのものを値に含めたいときは `\` と書く。

空白ノード: 角括弧 `[ ... ]` は「名前 (IRI) を付けない、その場限りの個体」を作る。括弧の中身は、その個体を主語とする「述語 目的語」の並びである。

```
ex:r1 rico:isAssociatedWithDate [ a rico:Date ;
                                rico:expressedDate "慶応4年" ] .
```

これは「`r1` はある日付と結びつく。その日付は `Date` のインスタンスで、表記は慶応4年」と読む。グラフに描けば図5.2のように、名前のないノードが1つでき、そこからさらに矢印が出ている。日付や数量のような「わざわざ世界規模の名前を与えるほどではない」補助的な個体に便利で、RiC-O の公式サンプルでも多用される。ただし空白ノードは IRI を持たないため外部から参照できない。長く維持し共有するデータでは、主要な実体には IRI を与えるのが原則である。

コメント: `#` から行末までは注釈で、データとしては無視される。範囲が行末までなので、複数行にわたる注釈は各行の先頭に `#` を置く。

文字コードと大文字小文字: ファイルは UTF-8 で書く。日本語をそのまま IRI にもリテラルにも書けるが、IRI に日本語を使うと処理系や公開先で扱いが揺れやすいので、実務では IRI を ASCII に保ち、日本語はリテラルに置くのが安全である。IRI と接頭辞名は大文字小文字を区別

する (rico:Record と rico:record は別物)。キーワードの a・true・false も小文字しか通らない。区別しないのは PREFIX(と BASE) 形の宣言だけで、これは大文字でも小文字でも書ける。



図 5.1 IRI の分解と接頭辞による略記。rico:Record は、名前空間 .../ontology# に Record を続けた IRI の略記である。語彙は # で、個々のデータは / で区切るのが通例である。ここでは IRI を文字列として分解して見せるので、分解した部分は白い四角で描いた。

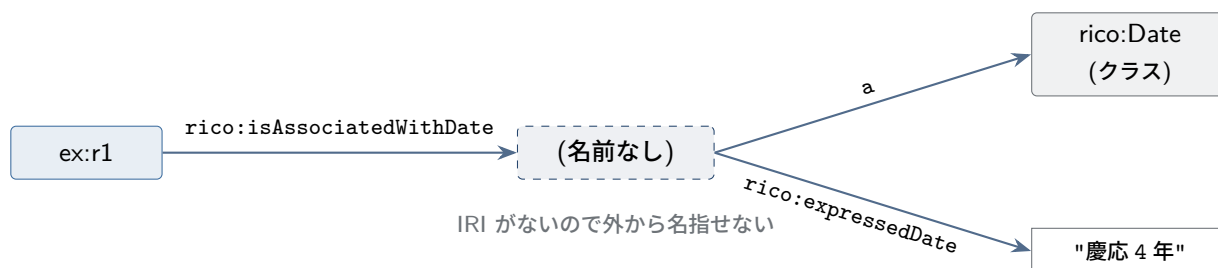


図 5.2 空白ノード。角括弧 [...] は名前 (IRI) を持たないノードを 1 つ作り、括弧の中はそのノードを主語とする「述語 目的語」の並びになる。

5.1.1 RDF の書き方は 1 つではない — Turtle・JSON-LD・RDF/XML

RDF は抽象的なデータモデルであり、同じトリプルの集合を複数の構文 (直列化形式、serialization) で書き表せる。どの形式で書いても意味は同一で、ツールで機械的に相互変換できる。主なものを挙げる。

Turtle 上で使った形式。名は Terse RDF Triple Language (簡潔な RDF トリプル言語) の略である。最も簡潔で、人間が読み書きするのに向く。本書の例は Turtle で書く (RiC-O の公式サンプルは大半が RDF/XML で、Turtle のものは 5.6 節で読む docuteam 社の例に限られる)。学習・例示・手書きにはこれを使えばよい。

JSON-LD JSON (JavaScript Object Notation) の形をした RDF。JSON は「鍵と値の組」を波括弧で囲んで並べ、値の位置にさらに同じ形を入れ子にできるデータ記法である^{*1}。LD は Linked

^{*1} もとはウェブブラウザ上で実行される言語 JavaScript の書き方だったが、いまは言語を問わず、ウェブサービスどうし

Data(第1章の Linked Open Data の LD) を指す。ふつうの JSON との違いは、鍵や値に現れる名前を IRI に対応づける仕組み(後の図 5.3 の @context) を備え、それによって全体が RDF のトリプルとして読める点にある。ウェブ開発で標準的な JSON と同じ道具(あらゆるプログラミング言語のライブラリ、Web API) で扱えるため、開発者との接点やシステム間連携ではしばしば第一候補になる。RiC-AG も、既存のメタデータを変換する先の形式の例に「RDF や JSON-LD」を挙げている (§3.6)。

RDF/XML 最も古い形式。RiC-O のオントロジー本体 (RiC-O_1-1.rdf) はこの形式で配布されている。XML ツールで処理できる利点はあるが、人間が読むには冗長で、新規に選ぶ理由は少ない。

N-Triples 1 行に 1 トリプルを略記なしで書き並べる形式。冗長だが構造が自明なので、大量データの交換や行単位の処理に向く。

見た目の違いを示すために、上の `ex:r1` についての 2 つのトリプル (Record のインスタンスであること、名称が「調査旅行日誌」であること) を 4 つの形式で書いたのが図 5.3 である。JSON-LD では @context が接頭辞の宣言に、@id が主語に、@type が述語 `a` に当たる。RDF/XML はクラス名を要素名に、主語の IRI を `rdf:about` 属性に置く。N-Triples は略記を使わないので、どの行も IRI を省略なしで書く。

つまり「どれが正しい形式か」という問いは立たず、「誰が(何が)読むか」で選ぶ。人が読むなら Turtle、ウェブアプリケーションと話すなら JSON-LD、既存の XML 基盤に載せるなら RDF/XML、機械同士の大量転送なら N-Triples、という使い分けが实际的である。本書は以後も Turtle を使う。

5.1.2 矢印の向きと参照の向き — 図とデータの読み方

本章冒頭の図をはじめ、本書はここまで関係を矢印で描いてきた。ここで一度立ち止まって、矢印の向きが何を意味するのかを正面から扱っておく。作図の流儀の話に見えて、実はデータ設計の中身に直結する。矢印の実体は参照 (reference)、つまりある記述の中から別のものを名指すことであり、参照はデータ設計の骨格である(その歴史的経緯は第2章 2.4 節で追った)。どちらの側が相手を名指すか、名指しの手段は何か(文字列か、キーか、IRI か)、逆向きにたどれることは保証されるか。この3点はシステムの検索能力と保守性を直接決める。ところが図解では、向きは「なんとなく」で描かれ、読む側も雰囲気ですりかちである。同じ1つの事実「山田花子が日誌を作成した」を、3つの流儀で描き分けたのが図 5.4 である。

■流儀 1: 現実の行為の向き 「作成者が記録を作成する」という出来事の向きである。因果や手順を説明する概念図はふつうこの向きをとり、直感にもよく合う。ただしこの矢印は、データの中のどの記載にも直接は対応しない。描かれているのは世界の側の出来事であって、記述の側の構造ではないからである。

がデータをやりとりするときの共通形式になっている。XML(第2章)と役割は近く、開始タグと終了タグの対ではなく波括弧と鍵で構造を表すぶん、記述量が少ない。アーカイブズの実務にも入っている。米国の記述規則 DACS に基づく記述を扱う管理システム ArchivesSpace は、「記録を JSONModel オブジェクトとして表現しており、これがシステムとやりとりする単位である」と述べ、エージェントの名称には適用した規則を示す `"rules": "dacs"` という項目が現れる (API 文書、バージョン 4.2.0)。ただしここで使われるのは JSON であって JSON-LD ではない。DACS の標準そのものは出力形式を定めず、本文は Markdown で管理されている。

Turtle

```
PREFIX rico: <https://www.ica.org/standards/RiC/ontology#>
PREFIX ex:   <https://archives.example.jp/>
ex:r1 a rico:Record ;
      rico:name "調査旅行日誌" .
```

JSON-LD

```
{ "@context": { "rico": "https://www.ica.org/standards/RiC/ontology#" },
  "@id":      "https://archives.example.jp/r1",
  "@type":    "rico:Record",
  "rico:name": "調査旅行日誌" }
```

RDF/XML

```
<rdf:RDF
  xmlns:rico="https://www.ica.org/standards/RiC/ontology#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
  <rico:Record rdf:about="https://archives.example.jp/r1">
    <rico:name>調査旅行日誌</rico:name>
  </rico:Record>
</rdf:RDF>
```

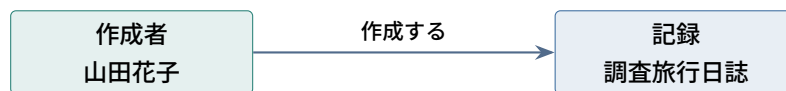
N-Triples(1 行に 1 トリプル。紙幅の都合で $\hookrightarrow$ の位置で折り返した)

```
<https://archives.example.jp/r1> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
   $\hookrightarrow$  <https://www.ica.org/standards/RiC/ontology#Record> .
<https://archives.example.jp/r1> <https://www.ica.org/standards/RiC/ontology#name>
   $\hookrightarrow$  "調査旅行日誌" .
```

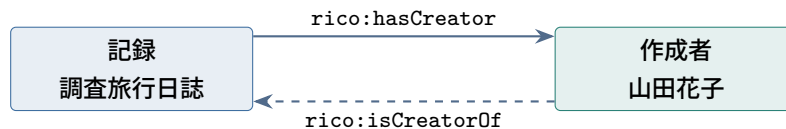
図 5.3 同じ 2 つのトリプル (記録 r1 は Record のインスタンスであり、名称は「調査旅行日誌」である) を 4 つの形式で書いたもの。意味はどれも同じである。

■流儀 2: 名前付きプロパティの向き RDF のトリプルは主語→目的語の向きを必ず持つ。だからプロパティの矢印は、そのプロパティ名が定める向きに描く。rico:hasCreator は「記録が作成者を持つ」だから記録→作成者で、流儀 1 と逆になる。ここで思い出してほしいのは、RiC-CM がほぼすべての関係を両方向の名前で与えていることである (“Record has creator Agent” と “Agent is creator of Record”)。RiC-O はこれを owl:inverseOf で実装しており (208 組の逆関係を、それぞれ両側から宣言した 416 個)、どちらの向きで書いても、もう一方は推論で得られる (第 6 章場面 2)。したがって矢印の向きそのものに正解はなく、ラベルと向きが対応しているかだけが正誤を決める。「作成する」と書いた矢印は作成者から、rico:hasCreator と書いた矢印は記録から出ていれば、どちらも正しい。なお、向きに意味がない関係もある。rico:hasSibling(きょうだいである) や rico:hasOrHadSpouse(配偶者である) など、RiC-O の 16 個のプロパティは対称プロパティ (owl:SymmetricProperty) として宣言されており、どちら向きに書いても同じ意味になる。

流儀 1: 現実の行為の向き (出来事・因果を描く)



流儀 2: 名前付きプロパティの向き (主語 → 目的語)



2 つは owl:inverseOf で結ばれ、片方を書けばもう片方は推論で得られる

流儀 3: 文書間の参照の向き (どの文書がその記載を持つか)

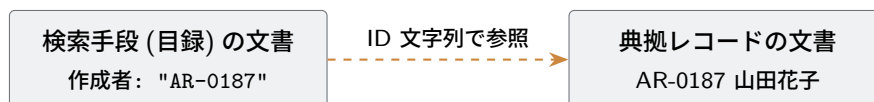


図 5.4 同じ 1 つの事実「山田花子が日誌を作成した」を描く、3 つの矢印の流儀。向きはそれぞれ別の理由で決まっており、互いに逆でも矛盾ではない。

■流儀 3: 文書間の参照の向き 検索手段の文書の中に典拠レコードの ID を書き込むとき、参照の矢印は検索手段→典拠である。これは関係の意味の向きではなく、**どのファイルのどの欄にその記載があるか**という置き場所の向きである。文書ベースの記述では、この向きは書き手が選べるものではなく、標準と文書構造が決めてしまう。この点が従来標準と RiC の重要な分かれ目になるので、第 7 章 7.4 節で図とともに詳しく扱う。

■実務上の帰結 流儀 2 に関して 1 つ、実務的な注意がある。逆向きが「推論で得られる」のは推論を使う場合の話で、推論を使わないストアでは、**実際に書いた向き**のトリプルしか検索に現れない。SPARQL には述語の前に `~` を付けて逆向きに辿る記法があるので検索側で吸収もできるが、データを受け取る側がそれを知らなければ「作成者が見つからない」事故になる。機関の適用プロファイル (第 7 章) で「この関係はこの向きで書く (または両方書く)」と決めておくのが確実である。

5.2 オントロジーとは何か

オントロジー (ontology) という語は哲学の存在論に由来するが、情報科学では「ある領域の概念 (クラス) と関係 (プロパティ) を、機械が処理できる形で定義した語彙体系」を指す。RDF の上でオントロジーを記述するための言語が RDFS(RDF Schema) と OWL(Web Ontology Language) であり、RiC-O は OWL 2 で書かれている。

オントロジーが定義する主な部品は次のとおりである。

クラス (class) ものの種類。RiC-CM の実体に対応する。rico:Record、rico:Person など。クラス

間には下位クラス関係 (`rdfs:subClassOf`) が定義でき、「`rico:Person` は `rico:Agent` の下位クラス」のように実体階層 (図 4.4) をそのまま表現する。

オブジェクトプロパティ (object property) もの同士を結ぶ述語。RiC-CM の関係に対応する。`rico:hasCreator`、`rico:isOrWasIncludedIn` など。定義域 (domain) と値域 (range)、逆プロパティ (`owl:inverseOf`)、上位プロパティ (`rdfs:subPropertyOf`)、推移性 (transitivity) などの論理的性質を宣言できる。

データタイププロパティ (datatype property) ものとりテラル値を結ぶ述語。RiC-CM の (自由テキスト系の) 属性に対応する。`rico:name`、`rico:scopeAndContent` など。

個体 (individual) クラスのインスタンス。RiC-O は、あらかじめ IRI を与えた個体を 7 つ同梱している (5.4 節)。

5.2.1 オントロジーを支える論理 — 記述論理

ここまでは「`rico:Person` は `rico:Agent` の下位クラス」のような宣言を日本語で書いてきた。機械に推論をさせるには、この日本語を、読み手によって解釈が変わらない形式言語に置き換えなければならない。その言語が**記述論理** (Description Logic、DL) であり、OWL の意味論はこの体系に基づいている。こうして書き下した宣言の 1 つ 1 つを**公理** (axiom) と呼ぶ。ここでは、個々の事実と区別して、語彙の一般的な約束事をこう呼んでいる。OWL の正式な用語法では、個体についての所属や関係の言明も公理に含まれる。推論は両者を元手にして進む (第 6 章)。

以下は、論理式を見るのが初めての読者を想定して、道具を 1 つずつ組み立てる。本書で記述論理の記法が実際に必要になるのは第 6 章の補足だけなので、読み飛ばして先へ進んでもさしつかえない。RiC-O の仕様書 (HTML 版) は、公理を「Has domain `rico:Instantiation` or `rico:RecordResource`」や「`proxyFor 1 RecordResource`」のような英語混じりの略記で示している。記号の意味を押さえておけば、これらも同じ公理として読める。

■登場人物は 3 種類だけ 記述論理が扱うものは 3 つに限られる。

個体 (individual) 「山田花子」「甚兵衛」「フォンド 01」のような、名前のついた個々のもの (甚兵衛は第 7 章の通し例に出てくる村の名主である)。第 4 章の言葉では実体インスタンスに当たる。

クラス (class) 個体の**集まり**。`Person` というクラスは、甚兵衛や花子といった個体を要素とする集合である。

プロパティ (property) 個体 2 つの組の**集まり**。`hasCreator` というプロパティは、(日誌, 山田花子) のような組の集合である。

クラスを集合と見る点が要である。「`rico:Person` は `rico:Agent` の下位クラス」は、集合の言葉では「`Person` という集合は `Agent` という集合にすっかり含まれる」となり、記述論理では

$$\text{Person} \sqsubseteq \text{Agent}$$

と書く。 $\sqsubseteq$ は数学の $\subseteq$ (部分集合) に対応する記号で、「左は右に含まれる」と読む。図 5.5 のとおり、包含が成り立っていれば、個人であるものは自動的にエージェントでもある。「甚兵衛は個人である。ゆえに甚兵衛はエージェントである」という推論は、この包含を 1 歩たどることで得られる。



内側の4つの箱は、どれも外側の Agent にすっぽり収まっている。だから「甚兵衛は Person に属する」と書けば、「甚兵衛は Agent にも属する」は書かなくても成り立つ

図 5.5 クラスを集合と見る。下位クラス関係 $\sqsubseteq$ は集合の包含である。RiC-CM がエージェントの下に4つの実体を置くのは(図 4.4)、集合の言葉では $\text{Person} \cdot \text{Group} \cdot \text{Position} \cdot \text{Mechanism}$ の4つが Agent の部分集合だと言っているに等しい。

■一階述語論理に翻訳して意味を確かめる 記述論理の記号は簡潔だが、意味を確かめたいときは一階述語論理に翻訳するのが確実である。一階述語論理は、「どの～についても」「～であるものがある」という言い方(量化)と、「かつ」「または」「ならば」を、読み手によって解釈の変わらない形で書き表すための記法である。記述論理はその断片にあたる。使える述語を1項と2項に限り、量化の現れ方も限られた型に制限したもので、記述論理のどの公理も機械的に一階述語論理の式へ翻訳できる。翻訳の記法そのものに馴染みがなければ、次のコラムを先に読んでほしい。

準備: 述語論理の式を読む

述語とは、ものについての主張の型である。引数の数で2種類ある。 $\text{Person}(x)$ のような1項述語は「 x は個人である」という性質を表し、クラスに対応する。 $\text{hasCreator}(x, y)$ のような2項述語は「 x は y を作成者として持つ」という関係を表し、プロパティに対応する。引数に実際の個体名を入れた $\text{Person}(\text{甚兵衛})$ は、真偽の定まった1つの主張になる。

x, y は変数である。中学校の数学で x が「まだ決まっていない数」を指したのと同じで、ここでは「まだ決まっていない1つの個体」を指す仮の名前にすぎない。名前は何でもよく、 x を u に書き換えても式の意味は変わらない。

変数を含む式は、そのままでは真偽が決まらない。「 x は個人である」は x が誰かを言わないうちは真とも偽とも言えない。そこで量子化で範囲を指定する。

- $\forall x(\dots)$ — 「どの x についても、 $\dots$ 」(全称量化)
- $\exists x(\dots)$ — 「 $\dots$ を満たす x が少なくとも1つある」(存在量化)

主張どうしをつなぐのが論理結合子である。 $\wedge$ は「かつ」、 $\vee$ は「または」、 $\rightarrow$ は「ならば」、 $\leftrightarrow$ は「ならばが両向きに成り立つ(同値)」、 $=$ は「同じものである」。 $\rightarrow$ だけは注意がいる。日常で使う「ならば」は前後の結びつき(理由や順序)を含んでいるが、ここでの $\rightarrow$ は両辺の真偽だけで決まる。 $A \rightarrow B$ が偽になるのは A が真で B が偽のときだけで、 A が偽のときは B が何であれ真になる。だから $\forall x(C(x) \rightarrow D(x))$ は、 C でない個体については何も要求しない。 C である

個体については「 D でもある」と結論するだけで、 D でないことが書かれていない限り、矛盾として拒否する理由は生じない。定義域や値域の公理が入力チェックにならないのはこのためである(後のコラム「RDB の世界地図で読むオントロジー」)。

以上で式は読める。声に出して読む練習をしておく。

式	読み
$\forall x (\text{Person}(x) \rightarrow \text{Agent}(x))$	どの x についても、 x が個人ならば x はエージェントである
$\exists y \text{ hasCreator}(r1, y)$	$r1$ が作成者として持つ相手 y が、少なくとも 1 つある
$\forall x \forall y (\text{hasCreator}(x, y) \rightarrow \text{Agent}(y))$	どの x, y についても、 x が y を作成者として持つならば、 y はエージェントである

読み方の型は決まっている。 $\forall$ で始まる式は「どれについても～」、 $\exists$ で始まる式は「～なものがある」。 $\rightarrow$ は、左が当てはまる個体についてだけ右を要求すると読む。

なお「一階」とは、量化できるのが**個体**だけで、述語そのものを量化できないという制限を指す。「すべてのクラス C について…」とは書けない。この制限のおかげで、一階述語論理には、導ける文なら必ずいつかは見つけ出す証明の手続きがある(次に述べる半決定可能性)。どんな問いにも必ず答えが出る決定可能性まで得るには、記述論理のようにさらに表現力を絞る必要がある。

表現力を絞る見返りは**決定可能性**である。決定可能とは、どんな入力に対しても有限の手数で必ず答えが出る手続きが存在する、という保証である。一階述語論理の全体には、この保証がない。ある文が別の文から導けるかどうかを必ず判定する手続きは存在しない(1936年に Church と Turing がそれぞれ独立に示した、決定問題が解けないという結果)。ただし片側だけは保証されている。実際に導ける場合には、証明を順に作り出していけばいつかそれに行き当たり、「はい」と答えられる(Gödel が 1930 年に発表した完全性定理から従う)。保証がないのは「導けない」と結論するほうで、こちらは手続きが終わらないまま走り続けることがある。答えの片側だけが必ず返るこの性質を**半決定可能**という。記述論理は使える構文を制限して、どちらの答えも有限の手数で出る状態を取り戻している。実務の言葉に直せば、推論器が答えを返さずに走り続ける事態を、体系の設計の段階で排除できるということである。第 6 章 6.8.4 項で見る OWL 2 のプロファイルは、この保証をさらに強め、計算時間の上限まで押さえた部分集合である。

もう 1 つ、記法が目立つ特徴を先に押さえておく。記述論理の式には**変数が現れない**。 $\text{Person} \sqsubseteq \text{Agent}$ には x がない。これは省略ではなく設計である。記述論理は、1970 年代の意味ネットワークやフレーム(第 3 章のコラムで触れた第二次 AI ブームの知識表現)を論理として厳密化する研究から生まれ、その際、個体を主役にする代わりに**クラスを集合として直接演算する**代数的な書き方を採った。この書き方には実利がある。公理が変数なしの 1 行で書いて見通しがよいこと、そして使う演算子の種類を選ぶことで表現力と計算量を細かく調整できることである*2。以下、主な記号を 1 つずつ、一階述語論理への翻訳と RiC-O の実際の公理を添えて読んでいく。クラスは「個体の集まり」、プロパティは「個体と

*2 採用した演算子の頭文字を並べたものが *ALC* や *SROIQ* といった体系の名前になっている。OWL 2 DL の意味論は *SROIQ* に基づく。

個体の二項関係」と解釈する。

■**下位関係** $\sqsubseteq$ 最も基本の記号で、「左は右に含まれる」と読む。クラスの公理 $C \sqsubseteq D$ を一階述語論理に翻訳すると

$$\forall x (C(x) \rightarrow D(x))$$

「どの個体 x についても、 C ならば D でもある」となる。RiC-O の実例は $\text{Person} \sqsubseteq \text{Agent}$ (個人はエージェントである)。プロパティの公理 $R \sqsubseteq S$ は変数2つで

$$\forall x \forall y (R(x, y) \rightarrow S(x, y))$$

と翻訳され、実例は $\text{directlyIncludes} \sqsubseteq \text{includesTransitive}$ (直接含むなら、含む) である。

■**逆関係** $-$ と**同値** $\equiv$ 肩に付く $-$ は、関係の向きをひっくり返す演算である。

$$R^-(x, y) \leftrightarrow R(y, x)$$

($\leftrightarrow$ は「どちらからでも導ける」)。hasCreator は「記録 $\rightarrow$ 作成者」という向きの関係だから、hasCreator $^-$ はその逆向き、「作成者 $\rightarrow$ 記録」の関係になる。 $\equiv$ は「左右がまったく同じ意味」で、逆向きの関係に名前を与える定義に使われる。RiC-O の公理 $\text{isCreatorOf} \equiv \text{hasCreator}^-$ の翻訳は

$$\forall x \forall y (\text{isCreatorOf}(x, y) \leftrightarrow \text{hasCreator}(y, x))$$

である。

■**連鎖** $\circ$ $R \circ S \sqsubseteq T$ は「 R で進み、続けて S で進めるなら、 T で一足飛びに行ける」と読み、翻訳は

$$\forall x \forall y \forall z (R(x, y) \wedge S(y, z) \rightarrow T(x, z))$$

である。推移性はこの形の特別な場合 $R \circ R \sqsubseteq R$ で、RiC-O の実例を翻訳すれば

$$\text{includesTransitive}(x, y) \wedge \text{includesTransitive}(y, z) \rightarrow \text{includesTransitive}(x, z)$$

(含むものの中のものは、含まれている) となる。

■**クラスを作る演算** $\exists R.C \cdot \forall R.C \cdot \sqcup$ ここからが記述論理らしい部分である。これらの記号は文ではなく、**新しいクラス (個体の集合) を作る演算**である。Person や Agent のようにあらかじめ名前のついたクラスだけでなく、「作成者として個人を持つもの」のような**その場で定義した集合**を式で書けるようにする道具立てである。

$\exists R.C$ は「 R の相手として C に属するものを少なくとも1つ持つ個体」の全体を表す。たとえば $\exists \text{hasCreator}.\text{Person}$ は「作成者として個人を持つもの」というクラスである。 $\forall R.C$ は「 R の相手があれば、それがすべて C に属する個体」の全体、 $\sqcup$ はクラスの合併(「または」)である。3つとも、ある個体 x がその集合に属する条件を一階述語論理で書けば意味がはっきりする。

$$x \in \exists R.C \iff \exists y (R(x, y) \wedge C(y))$$

$$x \in \forall R.C \iff \forall y (R(x, y) \rightarrow C(y))$$

$$x \in C \sqcup D \iff C(x) \vee D(x)$$

記号の $\exists$ と $\forall$ が、翻訳では相手 y にかかる量子子として現れる。これが記述論理の変数なしの記法と、変数を明示する一階述語論理の記法の接点である。

$\forall R.C$ は、読み方に一点だけ注意が要る。翻訳の右辺は $\rightarrow$ の形をしているので、 R の相手が **1 つもない** 個体についても真になる。条件が成り立つ場面が存在しなければ、「そのすべてが C である」という主張は破られようがないからである。つまり $\forall \text{hasCreator.Agent}$ は「作成者が 1 人以上いて、その全員がエージェント」ではなく、「作成者として書かれているものがあれば、それはエージェント」を意味する。「少なくとも 1 つある」ことまで言いたければ $\exists$ と組み合わせる必要がある。

■**T** —「すべての個体」のクラス T (トップと読む) は、どの個体も無条件に属するクラスである。所属の条件が常に真のクラスと考えればよい。それ自体は当たり前のものに見えるが、変数を持たない記述論理では、 T が 2 つの決まり文句の部品として働く。

第一の使い方は $\exists R.T$ 、「相手は何でもよい」である。 $\exists R.C$ の C の位置に「何でも属するクラス」を置くことで、相手の種類を問わない「とにかく R の相手を持つ個体」のクラスを作る。 $\exists \text{hasCreator.T}$ は「(誰でもよいから) 作成者を持つもの」である。

第二の使い方は $T \sqsubseteq X$ 、「すべての個体について X 」である。 $\sqsubseteq$ は「左は右に含まれる」だから、この式は「すべての個体のクラスが X に含まれる」、すなわち「どの個体も X に属する」という全称の主張になる。一階述語論理の $\forall x(\dots)$ に当たる言い回しを、変数なしで書くための定型である。

この 2 つを使うと、定義域と値域の公理が書ける。RiC-O の定義域公理

$$\exists \text{hasCreator.T} \sqsubseteq \text{RecordResource} \sqcup \text{Instantiation}$$

は「作成者を (誰でもよいから) 持つ個体は、記録資源か具現化である」と読み、一階述語論理では

$$\forall x (\exists y \text{hasCreator}(x, y) \rightarrow \text{RecordResource}(x) \vee \text{Instantiation}(x))$$

となる。値域公理 $T \sqsubseteq \forall \text{hasCreator.Agent}$ は、2 つ目の決まり文句で外側を読み、 $\forall$ で内側を読む。「どの個体についても ($T \sqsubseteq$)、その作成者はあればすべてエージェントである ($\forall \text{hasCreator.Agent}$)」。翻訳は

$$\forall x \forall y (\text{hasCreator}(x, y) \rightarrow \text{Agent}(y))$$

「作成者の側は必ずエージェント」である。見た目は暗号めいているが、 $\exists R.T \sqsubseteq C$ (定義域は C) と $T \sqsubseteq \forall R.C$ (値域は C) は、この形のまま慣用句として覚えてよい。

■**数の制約** $=1$ $=1 R.C$ は「 C に属する R の相手をちょうど 1 つ持つ個体」の全体である。RiC-O の実例 $\text{Proxy} \sqsubseteq =1 \text{proxyFor.RecordResource}$ は「Proxy はちょうど 1 つの記録資源を代理する」。一階述語論理に翻訳するには「少なくとも 1 つ存在し、2 つあれば実は同じ」という等号付きの式が要る:

$$\begin{aligned} \forall x (\text{Proxy}(x) \rightarrow \exists y (\text{proxyFor}(x, y) \wedge \text{RecordResource}(y)) \\ \wedge \forall y \forall z (\text{proxyFor}(x, y) \wedge \text{RecordResource}(y) \\ \wedge \text{proxyFor}(x, z) \wedge \text{RecordResource}(z) \rightarrow y = z)) \end{aligned}$$

一意性を述べる後半にも RecordResource の条件が要ることに注意したい。この条件を落とすと「 proxyFor の相手はどんな種類のものも含めてただ 1 つ」という別のもっと強い主張になる。「ちょうど 1 つ」「1 つ以上」のように個数を制限する公理を総称して**基数制約** (cardinality restriction) と呼ぶが、そのうち数を数える対象をクラスで限定するこの形を**限定数制約** (qualified number restriction) と呼び、 $SR\mathcal{OIQ}$ の Q がこれを指す。等号を使う分だけ表現力が強く、体系の計算量を左右する。

まとめると、 $\sqsubseteq$ の記法は公理を変数なしで短く一覧するのに向き、一階述語論理への翻訳 (規則の形) は意味を確かめるのに向く。以下では両方を場面に応じて使う。ここまでに出た記号を表 5.1 にまとめる。本文で扱わなかったクラスの同値 $C \equiv D$ と共通部分 $C \sqcap D$ (「かつ」) も参考に加えた。

表 5.1 記述論理の記号早見表 (C, D はクラス、 R, S はプロパティ、 x, y, z は個体を表す変数)

記号	読み	一階述語論理への翻訳
$C \sqsubseteq D$	C は D に含まれる	$\forall x (C(x) \rightarrow D(x))$
$R \sqsubseteq S$	R は S に含まれる	$\forall x \forall y (R(x, y) \rightarrow S(x, y))$
$C \equiv D$	C と D は同じ	$\forall x (C(x) \leftrightarrow D(x))$
R^-	R の逆向き	$R^-(x, y) \leftrightarrow R(y, x)$
$R \circ S \sqsubseteq T$	R から S とたどれるなら T	$\forall x \forall y \forall z (R(x, y) \wedge S(y, z) \rightarrow T(x, z))$
$\exists R.C$	C の相手を 1 つ以上持つもの	$\exists y (R(x, y) \wedge C(y))$
$\forall R.C$	相手があればすべて C であるもの	$\forall y (R(x, y) \rightarrow C(y))$
$C \sqcup D$	C または D	$C(x) \vee D(x)$
$C \sqcap D$	C かつ D	$C(x) \wedge D(x)$
$\top$	すべての個体のクラス	常に真
$=1 R.C$	C の相手をちょうど 1 つ持つもの	C の相手が存在し、 C の相手が 2 つあれば等しい

■RDF トリプルとの対応 この記法と 5.1 節で学んだトリプルの対応は、2 段に分けると正確に言える。第一に、**データのトリプルは、論理の原子式** (それ以上分解できない事実の文) に対応する。

Turtle	記述論理	一階述語論理
ex:r1 rico:hasCreator ex:p1 .	hasCreator(r1, p1)	hasCreator(r1, p1)
ex:p1 a rico:Person .	Person(p1)	Person(p1)
rico:Person rdfs:subClassOf rico:Agent .	Person $\sqsubseteq$ Agent	$\forall x (\text{Person}(x) \rightarrow \text{Agent}(x))$

上 2 行が**事実**である。1 行目のようなふつうのトリプルは、素直に 2 項述語の原子式になる。

ところが 2 行目、a のトリプルには見過ごせない**ねじれ**がある。1 行目と同じ調子で述語化するなら $a(p1, \text{Person})$ 、すなわち $\text{type}(p1, \text{Person})$ という 2 項述語の原子式になるはずである。しかし表では $\text{Person}(p1)$ 、つまり **1 項述語の適用**として読んだ。2 つの読みでは Person の立場が違う。前者では**引数** (ものの名前) の位置に、後者では**述語**の位置に現れている。要するに a だけは、目的語の位置に述語の**名前**が入るという点で、他の述語と対称でないのである。論理の標準的な読み方は後者 ($\text{Person}(p1)$) であり、この読みをとる限り、事実の水準では記述論理と一階述語論理の記法は**同形**である (記述論理の文献には $p1 : \text{Person}$ と書く流儀もある)。ただし前者の読み方 (type という 2 項述語として読むこと) も間違いではない。それが後のコラム「クラスが主語になる書き方」の主題である。

3 行目の**公理**で、はじめて 3 欄が別の顔になる。一階述語論理は量化された変数を明示し、記述論理は前項で見たとおり変数なしの 1 行に畳み、Turtle はそれを 1 本のトリプルに符号化する (この符号化については後述)。つまり記述論理らしさは公理の水準に現れ、事実の水準の RDF グラフとは、原子式の集合、つまり事実の集まりにほかならない。記述論理はこの事実の集まりを **ABox**(assertion box) と呼ぶ。

第二に、公理の側もトリプルである。これらの公理は、RiC-O のファイルの中ではそれ自身が RDF トリプルとして書かれている。オントロジーも RDF データの一種なのである。上で読んだ公理のいくつかを Turtle で書くと、次のようになる (RiC-O 本体はこれと同内容の RDF/XML である)。

```
# 下位クラス:Person は Agent の一種
rico:Person rdfs:subClassOf rico:Agent .

# 下位プロパティ:直接含むなら、含む
rico:directlyIncludes rdfs:subPropertyOf rico:includesTransitive .

# 逆プロパティ:「～の作成者である」は「作成者を持つ」の逆
rico:isCreatorOf owl:inverseOf rico:hasCreator .

# 推移性(連鎖公理と同じ意味)
rico:includesTransitive a owl:TransitiveProperty .

# 3歩の連鎖の近道(丸括弧は経路のリスト)
rico:hasOrganicProvenance owl:propertyChainAxiom
    ( rico:thingIsSourceOfRelation
      rico:organicProvenanceRelation_role
      rico:relationHasTarget ) .
```

記号と Turtle は同じことの2つの表記であり、人が考えるときは記号が、機械に渡すときは Turtle(や RDF/XML) が使われる、という分担になる。公理の集まりのほうは **TBox**(terminology box) と呼ぶ^{*3}。

コラム: クラスが主語になる書き方 — 2つの読み方

上のリスティングをよく見ると、奇妙なことに気づくかもしれない。`rico:Person rdfs:subClassOf rico:Agent` を素直に述語として読むと `subClassOf(Person, Agent)` となり、`Person` が項 (ものの名前) の位置に現れる。しかし `Person` は `Person(x)` のように使う述語だったはずだ。述語を項にして量化や主張の対象にするのは、一階述語論理の枠を越える (二階論理の) 操作に見える。先ほど `a` のトリプルで見たねじれ (述語の名前が項の位置に入ること) と同じ問題が、公理のトリプルでは全面化するわけである。この扱いには標準的な答えが2つある。

読み方 1: トリプルは公理の符号化にすぎない (OWL 2 DL の立場)。この立場では、`subClassOf` のトリプルは論理式そのものではなく、公理を運ぶための構文である。推論器は読み込みの段階でこれを公理 $\text{Person} \sqsubseteq \text{Agent}$ として読み取る。その意味は一階の文 $\forall x (\text{Person}(x) \rightarrow \text{Agent}(x))$ と同じである。項の位置に現れる `Person` は、述語の名前を引用しているだけで、クラス全体にわたる量化はどこにも起きない。だから一階の範囲に収まる。

読み方 2: すべてを「もの」として扱いきる (RDF 意味論・規則エンジンの立場)。逆の方向もある。クラスも個体の一種とみなし、所属を `type(a の正体)` という普通の2項述語で書くのである。すると `subClassOf(C, D)` はそのまま一階の原子式であり、次の橋渡しの規則を公理として

^{*3} *SROIQ* の文献では、プロパティについての公理 (下位プロパティ・逆・推移性・連鎖) を **RBox**(role box) として **TBox** と分けることが多い。本書では両方をまとめて **TBox** と呼ぶ。

足せば、読み方1と同じ推論が得られる。

$$\text{type}(x, C) \wedge \text{subClassOf}(C, D) \rightarrow \text{type}(x, D)$$

ここで量化されている変数 C, D が走る範囲は「クラスという名の**個体**」であって述語ではないから、これも一階に収まる。推論機能を持つトリプルストアの多くは、実際にこちらの発想で作られている。データ全体を $\text{triple}(s, p, o)$ という1つの3項述語の巨大な表とみなし、この種の規則(データベース分野で Datalog と呼ばれる規則言語の形)を回して導出トリプルを蓄える。第6章で「推論器が破線のトリプルを追加する」と述べる処理は、多くの場合この方式で実行される(同章 6.8.2 項のマテリアライゼーション)。そして、この立場をとれば、 a を2項述語 type と素直に読むことも正当化される。 $\text{Person}(p1)$ と $\text{type}(p1, \text{Person})$ は、同じ事実の2つの表記として、橋渡しの規則を介して行き来できる。

この例は、どちらの読み方でも一階論理の範囲で扱える。なお、OWL 2 には同じ IRI をクラスと個体の両方に使う**パニング (punning)**もある。その場合、クラスとしての解釈と個体としての解釈は独立しており、一方の性質が自動的に他方へ移るわけではない。クラスの IRI が rdfs:subClassOf の主語に現れるだけでは、パニングにはならない。また、本書の rst:Fonds は $\text{rico:RecordSetType}$ と skos:Concept の両方に属する**個体**である。これは複数のクラスへの所属であって、クラスと個体の兼用ではない。

推論(第6章)とは、TBox(公理)を ABox(事実)に適用して、新しい原子式、すなわち新しいトリプルを導く計算にほかならない。たとえば「 $\text{Person} \sqsubseteq \text{Agent}$ で、甚兵衛は Person である。ゆえに甚兵衛は Agent である」という三段論法が、その最も単純な形である。第6章の各場面には、使われている公理をこの記法で添える。

コラム:RDB の世界地図で読むオントロジー

リレーショナルデータベースに馴染みがあれば、次の対応で概観できる。クラス $\approx$ テーブル定義、個体 $\approx$ 行、データタイププロパティ $\approx$ 列、オブジェクトプロパティ $\approx$ 外部キー(結合)、IRI $\approx$ 全世界で一意な主キー。ただし決定的な違いが3つある。第一に、RDB の行は必ずどれか1つのテーブルに属するが、RDF の個体は複数のクラスに同時に属せる(公式サンプルには $\text{rico:Instantiation}$ かつ $\text{premis:Representation}$ という個体が実際に登場する)。第二に、RDB のスキーマは制約(違反データは拒否される)だが、OWL の公理は推論の根拠である(「 rico:hasCreator の値域は rico:Agent 」とは「作成者と書かれたものはエージェントだと**推論してよい**」という意味であり、入力チェックではない)。第三に、RDB は閉世界仮説(書かれていないことは偽)に基づくが、OWL は開世界仮説(書かれていないだけでは偽としない)で解釈される。「作成者のトリプルがない」ことは「作成者がいない」ことを意味しない。この違いは検索結果の解釈に直接影響する(第6章)。

5.3 なぜ1つのデータに混ぜて書けるのか — RDF・OWL・SKOS の層構造

ここまでに RDF、RDFS、OWL が登場し、後の章では SKOS や Dublin Core という名前も出てくる。似た顔の英字略語が並ぶので、互いの関係と、1つのデータの中に混ぜて書ける理由を、ここで整理しておく。結論を先に言えば、これらは層をなす**1組の積み木**であり、積み木をつなぐ約束はただ1つ、「すべてをトリプルで書く」である (図 5.6)。

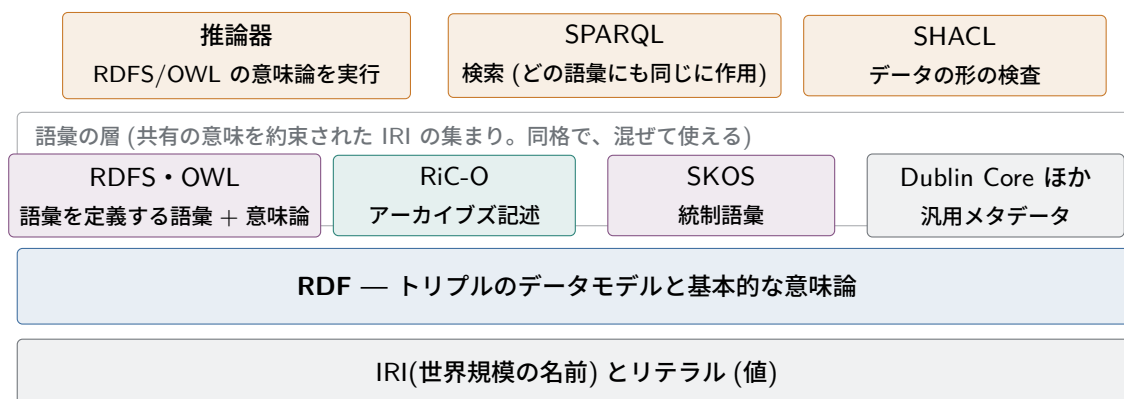


図 5.6 層構造の見取り図。基盤は RDF(トリプル) だけ1つで、その上に載る語彙はすべて同格である。RDFS・OWL は語彙の定義に使われ、RDF の基本的な意味論を拡張し、SPARQL や SHACL はどの語彙のトリプルにも同じに働く。

■**基盤は RDF だけ1つ** 最下層の約束は、名前を IRI で付けること (5.1 節)。その上の RDF は、データを主語・述語・目的語のトリプルで書くという**データモデル**であり、その基本的な意味論も定める。rdf:type などの語彙も持つ。ただし、記録や作成者といった分野固有の概念までは決めない。それを担うのが RiC-O などの語彙である。

■**語彙はすべて同格** その上の層が**語彙 (vocabulary)** である。語彙とは、突き詰めれば**共有の意味を約束された IRI の集まり**にすぎない。rico: も skos: も dcterms:(Dublin Core) も、そして rdfs:・owl: 自身も、この意味ではすべて語彙であり、同格である。トリプルの述語や型の位置には、どの語彙の IRI を置いてもよい。トリプルストアは述語の所属を区別しないから、1つのデータの中に rico: と skos: と自前の myo: が同居できる (第 7 章 7.5 節の例がそうだった)。日本語という1つの文法の上で、法律用語と医学用語を同じ文章に混ぜて使えるのと同じで、文法は用語の混在を妨げない。

■**語彙の定義に使う RDFS と OWL** RDFS と OWL は語彙の定義に使われる。rdfs: と owl: は「語彙を定義するための語彙」であり (subClassOf、inverseOf…)、さらに数学的な**意味論** (この形のトリプルからは何が帰結するか、という規定) を持つ。推論器とは、この意味論の実行系である (第 6 章)。RiC-O は「OWL で定義された語彙」なので、OWL の意味論をまるごと受け継ぐ。SKOS は逆に、意味論を意図的に軽くした語彙である (skos:broader から逆関係などは導出されるが、クラスの包含は導出されない。第 7 章 7.5 節のコラム)。つまり OWL と SKOS は別の層にいてではなく、同じ語彙の層の中での**意味の重さ**の違いである。

■**道具はどの語彙にも同じに働く** 最上段は言語・道具の層である。SPARQL は検索言語、SHACL は検査言語で、SHACL の条件自体は `sh:` 語彙を使う RDF として書かれる。どの語彙のトリプルに対しても同じに作用する。だから「SKOS のデータを SPARQL で検索する」ことにも「RiC-O のデータを SHACL で検査する」ことにも、何の追加の仕掛けも要らない。

■**意味を解釈する主体** 統語の統一は以上のとおりだが、`skos:broader` や `rico:hasCreator` の意味を解釈する主体は別に押さえておく必要がある。語彙ごとに専用の処理系があるわけではない。推論器が組み込みで知っているのは RDFS と OWL の意味論だけで、RiC-O や SKOS の意味は、それ自身の定義ファイル (OWL の公理の集まり) として**データの側**に書かれている。推論器にデータと一緒にこの定義を読み込ませれば、OWL の意味論を実行するだけで各語彙の帰結が得られる。形式意味論に載らない意味 (どのラベルを画面に出すか、規約が守られているかの検査) はアプリケーションと SHACL が受け持ち、散文の仕様を読むのは人間である。

コラム: プログラミングにたとえると — 語彙はライブラリである

プログラミングの経験があれば、次の対応で見通しがよくなるはずである。

RDF の世界	プログラミングの世界
IRI	名前空間付きの識別子
RDF(トリプル)	言語の文法とデータモデル
語彙 (<code>rico:</code> 、 <code>skos:</code>)	ライブラリ
語彙の公理 (OWL ファイル)	ライブラリの実装コード
推論器	言語処理系
散文の仕様書	ドキュメント

NumPy を使うのに専用のインタプリタは要らない。動作は NumPy 自身のコードに書かれていて、処理系は言語の意味論でそれを実行するだけだからである。同じ理由で RiC-O 専用の推論器は必要ない。SKOS にも逆関係などを導く公理があり、汎用の推論器で扱える。たとえの限界は、コードが手続きであるのに対し公理は宣言であること (第6章のコラム「公理と手続き」)。それでも「意味は同梱物の側に書かれ、処理系は汎用のものが1つあればよい」という仕組みは同じである。

この層構造が見えると、「RiC-O を使う」の意味が正確になる。RDF というデータモデルを選び、その上で RiC-O という語彙 (と、それが継承する OWL の意味論) を主たる用語集として使い、必要な場面で SKOS や Dublin Core の用語を混ぜる。それだけのことである。標準たちは競争していない。分業している。

5.4 RiC-O の全体像

RiC-O 1.1 の規模を数字で押さえておく (RiC-O 公式ドキュメントによる)。名前空間 IRI は `https://www.ica.org/standards/RiC/ontology#`、推奨接頭辞は `rico:` である。

- ・ クラス:107(RiC-CM の実体 19 に対し、大幅に多い。差分の由来は次節)
- ・ データタイププロパティ:75
- ・ オブジェクトプロパティ:480(RiC-CM の 85 関係に対応するものを含む)
- ・ 名前付き個体:7。記録集合種別の Fonds・Series・File・Collection(定義は ISAD(G) の用語集から採られている) と、文書形式種別の FindingAid・AuthorityRecord・IIIFManifest である。いずれも `skos:Concept` でもあり、IRI は `rico:` とは別の名前空間 (本章冒頭の例の `rst:` など) に置かれている

プロパティの命名には規則性がある。過去と現在の両方をカバーする関係は `hasOrHad.../isOrWas...` と命名される (例:`rico:hasOrHadMember` は、集団が現在または過去に持つメンバーを指す)。アーカイブズ記述は本質的に歴史記述であり、「かつてそうだった」関係を常に扱うための工夫である。ほかにも、名前の形から向き・推移性・関係ノードとのつながりが読み取れる。主な型を表 5.2 にまとめた。`rico:directlyIncludes` は `rico:includesTransitive` の、`rico:includesTransitive` は `rico:includesOrIncluded` の下位プロパティなので、細かい名前を書いておけば、推論を使うとき粗い名前での検索にも応答する (第 6 章)。全クラス・全プロパティに英仏西のラベルが付き (1.1 ではクラスに独語ラベルが加わった)、定義は英語で与えられ、RiC-CM のどの構成要素に対応するかも、対応があるものには注記される。CSV 一覧と併せれば辞書として使える。

表 5.2 RiC-O のプロパティ名の型。名前の形から、向き・時制・推移性・関係ノードとのつながりが読み取れる (名前の `rico:` は省いた)

名前の形	意味	例 (→ 逆のプロパティ)
<code>has...</code> 、 <code>is...Of</code>	基本の対	<code>hasCreator</code> → <code>isCreatorOf</code>
<code>hasOrHad...</code> 、 <code>isOrWas...</code> 、 <code>...OrIncluded</code>	現在または過去に成り立つ (成り立った)	<code>hasOrHadMember</code> → <code>isOrWasMemberOf</code> 、 <code>includesOrIncluded</code> → <code>isOrWasIncludedIn</code>
<code>directly...</code>	間に何もはさまない直接の関係だけ	<code>directlyIncludes</code> → <code>isDirectlyIncludedIn</code>
<code>...Transitive</code>	推移的。間接の関係も含む	<code>includesTransitive</code> → <code>isIncludedInTransitive</code>
<code>thing...Relation</code> 、 <code>relation...</code>	実体化した関係ノードへの出入り (5.5.3 項)	<code>thingIsSourceOfRelation</code> → <code>relationHasSource</code> 、 <code>relationHasTarget</code> → <code>thingIsTargetOfRelation</code>
<code>...Relation_role</code>	関係ノードを自分自身に結ぶ補助 (5.5.3 項)	<code>organicProvenanceRelation_role</code> (逆はない)
<code>proxy...</code>	Proxy に使う変種 (5.5.4 項)	<code>proxyFor</code> → <code>hasProxy</code> 、 <code>proxyIn</code> (逆はない)

設計原則としては 5 つが掲げられている (RiC-O ドキュメント)。領域参照オントロジーであること (現段階では他分野のオントロジーから借用せず、機関が理解・実装・保守しやすいことを優先する。他のモデルとの対応づけは今後の課題とされている)、即座に使えること (既存メタデータからの変換で情報を失わない)、柔軟な枠組みであること (粗くも細かくも書ける)、記述の新しい可能性を開くこと

(SPARQL と推論)、拡張可能であること (下位クラス・下位プロパティの追加と、統制語彙を RDF で表すための標準である SKOS の語彙との接続。第 7 章 7.5 節) である。とりわけ「即座に使える」ことを優先して**同じ情報に複数の表現方法を意図的に残している点**は、利点 (移行が容易) と引き換えに一貫性の維持を実装側に委ねる設計判断であり、第 3 章のトレードオフ論とつながる。

5.5 RiC-CM から RiC-O へ — 何がどう対応するか

RiC-CM の構成要素が RiC-O でどう書かれるかを、表 5.3 にまとめる。多くの項目には、クラスを使う精密な書き方と、リテラルや直接のプロパティで済ませる簡便な書き方の両方がある。以下の項では、この表の行を上から順に見ていく。

表 5.3 RiC-CM の構成要素と RiC-O での書き方 (名前の `rico:` は省いた)。多くの項目に、精密な書き方と簡便な書き方の両方がある

RiC-CM の構成要素	RiC-O の精密な書き方	RiC-O の簡便な書き方
実体 (19)	同名のクラス (<code>Record</code> 、 <code>Person</code> など)	—
属性: 自由テキスト	データタイププロパティ (<code>scopeAndContent</code> 、 <code>history</code> など)	—
属性: 統制値 (記録集合種別・担体種別など)	多くは <code>Type</code> の下位クラスのインスタンスを <code>has...Type</code> などで結ぶ (<code>RecordSetType</code> と <code>hasRecordSetType</code> など。言語は <code>Language</code> を <code>hasOrHadLanguage</code> で結ぶ)	<code>type</code> に文字列を書く
属性: 名称・識別子	<code>Appellation</code> の下位クラス (<code>Name</code> 、 <code>Identifier</code>) を <code>hasOrHadName</code> 、 <code>hasOrHadIdentifier</code> で結ぶ	<code>name</code> 、 <code>identifier</code> に文字列を書く
実体のうち日付	<code>Date</code> のインスタンスを <code>isAssociatedWithDate</code> やその下位プロパティで結ぶ	<code>date</code> とその下位 (<code>beginningDate</code> 、 <code>endDate</code> など) に文字列を書く
関係 (85)	<code>Relation</code> の下位クラス (<code>OrganicProvenanceRelation</code> など) で関係を実体化する	直接のオブジェクトプロパティ (<code>hasOrganicProvenance</code> など)。精密な書き方とは連鎖公理で結ばれる
(対応なし)	RiC-O 独自のクラス (<code>Proxy</code>)	—

5.5.1 実体はクラスへ

RiC-CM の各実体には同名のクラスが 1 対 1 で対応し、階層もそのまま写される。ここは素直である。

5.5.2 属性はデータタイププロパティまたはクラスへ

自由テキスト系の属性 (一般記述、歴史、範囲と内容など) はデータタイププロパティになる。一方、統制値系の属性 (担体種別、記録集合種別、活動種別など) はクラスに昇格する (rico:CarrierType、rico:RecordSetType など、rico:Type の下位クラス群)。名称と識別子も rico:Appellation(呼称) の下位クラスとして実体化されている。RiC-CM §3.3 は、統制値をとる属性は実装では実体として扱われうると述べ、RiC-O がそうしていることも明記している。

ただし、ここで RiC-O の「即座に使える」原則が顔を出す。クラス化された概念のほぼすべてに、リテラルで済ませる簡便法も併設されているのである。種別をクラスのインスタンスへの参照で書く代わりに rico:type プロパティに文字列を書いてよいし、日付を rico>Date 実体にせず rico:date(および多数の下位プロパティ) に文字列で書いてもよい。名称も rico:Name インスタンスを作らず rico:name に書けばよい。これらの簡便プロパティには「対応するクラスを使わない場合にのみ使う」という注記 (skos:scopeNote) が付いている。rico:type と rico:date の注記はさらに「使いやすさのために用意したもので、将来は非推奨にして削除することがありうる」と述べ、移行期の妥協であることを明示している。

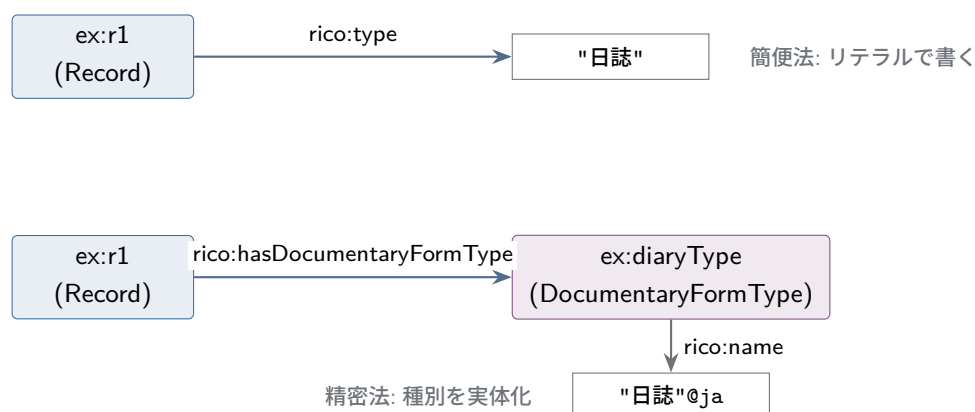


図 5.7 同じ情報の 2 つの書き方 (上: 簡便法、下: 精密法)。精密法なら種別自体を SKOS 語彙として整備し、多言語ラベルや上位下位関係を持たせ、機関を越えて共有できる。

5.5.3 関係はオブジェクトプロパティ + 関係クラスへ

RiC-CM の関係の実装は 2 本立てである。

第一に、各関係に対応する**直接のオブジェクトプロパティ**がある。R026(出所を持つ) なら rico:hasOrganicProvenance である (名称のずれは第 4 章 4.4 節末尾で述べたとおり)。トリプル 1 本で書いて簡単だが、関係そのものに属性 (日付・確実性・出典。RiC-CM §5.5) を付けられない。RDF のトリプルは 3 語で完結しており、「トリプルについてのトリプル」を素直には書けないからである^{*4}。

第二に、関係を**クラスとして実体化** (reification) した関係クラス群がある (rico:Relation を頂点に、rico:CreationRelation、rico:OrganicProvenanceRelation など 48 種)。関係を 1 個のノー

^{*4} RiC-O も rico:Relation の注記で、関係に日付などを記録するために関係クラスを使うのは「RDF-Star が W3C 勧告になり、より軽い方法を提供するまで」のことだと述べている。

ドにしてしまえば、そのノードに日付や確実性をぶら下げられる。複数の主体が1つの関係に関与する場合 (n 項関係) も表現できる。

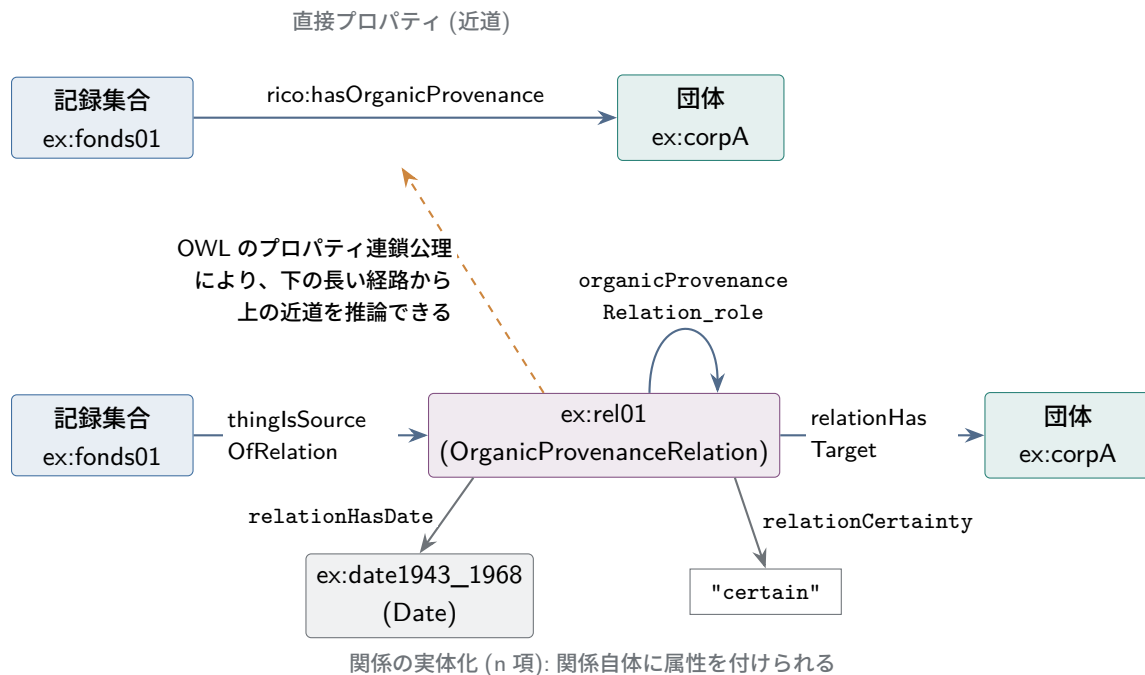


図 5.8 同じ出所関係の2つの表現。RiC-O では多くの直接プロパティが「実体化された関係を経由する長い経路の近道」として形式的に定義されている。長い経路と、関係ノードを自分自身に結ぶ `_role` の1行 (上に曲がった矢印) を書けば、近道は推論で得られる (本文と第6章)。

どちらで書くべきかは、また実装の判断である。関係について語るべきこと (日付、根拠) があるなら実体化し、なければ直接プロパティで十分、というのが実際的な指針になる。重要なのは、両者が無関係に重複しているのではなく、OWL 2 のプロパティ連鎖公理 (property chain axiom) によって形式的に結ばれていることである。`rico:hasOrganicProvenance` は「`rico:thingIsSourceOfRelation` → `rico:organicProvenanceRelation_role` → `rico:relationHasTarget` という経路の近道」として定義されている。

経路の中央にある `rico:organicProvenanceRelation_role` は、関係ノードを自分自身に結ぶだけの補助のプロパティである。RiC-O はクラス `rico:OrganicProvenanceRelation` を「このプロパティで自分自身に結ばれているもの」として定義している (`owl:hasSelf` という書き方)。ところがこの書き方は、OWL 2 RL (規則で推論する処理系の多くが対象とする範囲。第6章 6.8.4 項) に入っていない。そのため、関係ノードが `rico:OrganicProvenanceRelation` であると書いただけでは、OWL 2 RL の推論器は中央の1歩を補わず、近道も導かない。RiC-O はこのプロパティの注記で、関係ノードを1つ作るたびにそれ自身へ結ぶ1行を書くよう求めている。図 5.8 の曲がった矢印がこの1行で、Turtle では次のように書く。

```
ex:fonds01 rico:thingIsSourceOfRelation ex:rel01 .
ex:rel01 a rico:OrganicProvenanceRelation ;
    rico:organicProvenanceRelation_role ex:rel01 ;    # 自分自身に結ぶ
    rico:relationHasTarget ex:corpA .
```

この 1 行を書いておけば、推論器を有効にした環境で、長い経路から近道が導かれる。導かれるのは `rico:hasOrganicProvenance` と、逆向きの `rico:isOrganicProvenanceOf` の 2 つである。他の関係クラスにも、それぞれに対応する `_role` のプロパティがあり、作成関係なら `rico:creationRelation_role` である。

5.5.4 RiC-O にしかないもの — Proxy

RiC-CM に対応物がない代表的なクラスが `rico:Proxy` である。定義は「ある記録資源を、特定の別の記録資源の中にあるものとして代理するもの」。用途は、文脈ごとに別の情報を持たせることにある。

1 つの記録が複数の記録集合に属せる (多重所属) ことを思い出そう。ある書簡が「山田家文書」の一部であると同時に、研究者が編んだ「震災関係資料コレクション」にも属するとする。このとき「山田家文書の中での整理番号や配列順序」と「コレクションの中での配列順序」は別々の情報である。記録そのものに順序を持たせると、2 つの文脈の情報が衝突する。そこで「文脈ごとの分身」として Proxy を立て、**Proxy** に順序や文脈固有の情報を持たせる。

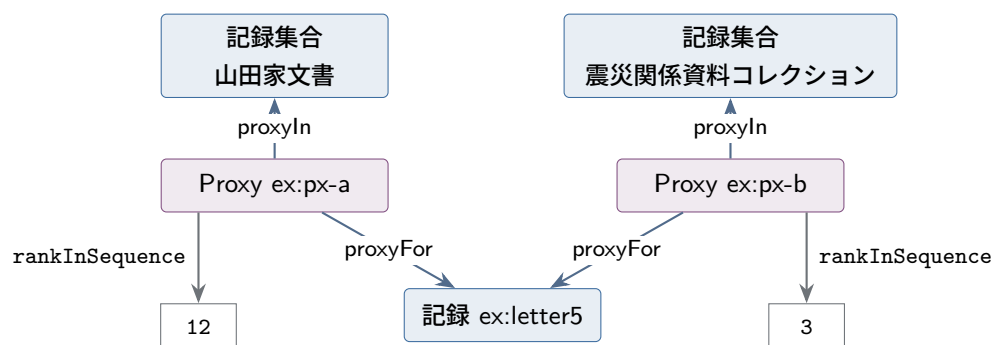


図 5.9 Proxy の仕組み。同じ記録が文脈 (所属する記録集合) ごとに別の「分身」を持ち、配列順序のような文脈固有の情報は分身の側に記録される。原秩序 (第 3 章) を、他の文脈での秩序と衝突させずに保存できる。順位の数値 (`rico:rankInSequence`) は、前後関係のプロパティに添える補助である (本文参照)。

順序そのものは `rico:precedesOrPreceded/rico:followsOrFollowed` を頂点とする系列のプロパティ (直接の前後を表す `rico:directlyPrecedesInSequence`、推移的な `rico:precedesInSequenceTransitive`、および Proxy 用の変種) で表す。RiC-O 1.1 では補助的に順位を数値で持つ `rico:rankInSequence` も追加されたが、順序プロパティと併用する場合に限ると注記されている。EAD 経験者なら、この Proxy は EAD の構成要素の並び (文書順) をグラフの世界に持ち込むための仕組みだと理解すればよい。なお RiC-O は、この Proxy を OAI-ORE という別の標準の Proxy に近いものとして注記している (`rico:closeTo`)。

5.6 実例を読む

RiC-O の公式サンプルのうち、スイスの docuteam 社が用意したデジタル保存パッケージの例 (RiC-O のリポジトリの `examples/examples_v1-1/Matterhorn-Switzerland`) を簡略化して読んでみよう。デジタル保存の場面で、PREMIS の語彙と組み合わせて使った例である。

読む前に、見慣れない名前の正体を説明しておく。以下に現れる `<_20201118115821577>` や `<700>` は、**リテラルではなく IRI** である (文法コラムのとおり、引用符 `"..."` で囲まれていればリテラル、山括弧 `<>` で囲まれていれば IRI である)。ただしこれまでの例と違って接頭辞を使わず、**相対 IRI** で書かれている。相対 IRI とは、基底となる IRI に対する相対的な名前で、データの読み込み時に完全な IRI に補完される。名前の中身は、作成時刻 (2020 年 11 月 18 日 11 時 58 分 21 秒 577) をもとに機械的に作ったとみられる機関内の識別子で、読む人にとっての意味は持たない。「一意でありさえすれば、名前は無意味な記号列でよい」という IRI の性格がよく現れた実例である。長いので、以下では `<_2020...577>` のように中略して示す。

```
PREFIX rico: <https://www.ica.org/standards/RiC/ontology#>
PREFIX premis: <http://www.loc.gov/premis/rdf/v3/>

<_2020...577> a rico:Record ;
    rico:title "Logo docuteam" ;
    rico:scopeAndContent "Enthaelet eine Logo-Datei" ;
    rico:isOrWasRegulatedBy <700> ; # 閲覧制限などの規則
    rico:thingIsSourceOfRelation [ # 作成関係を実体化
        a rico:CreationRelation ;
        rico:type "archivist" ;
        rico:relationHasTarget <101> ] ;
    rico:hasOrHadDigitalInstantiation <_2020...159> .

<_2020...159> a rico:Instantiation, premis:Representation .

<700> a rico:Rule ;
    rico:type "accessRestrictionsClosureYear" ;
    rico:endDate "2025" .

<101> a rico:Person ;
    rico:name "Tobias Wildi" .
```

この listing をグラフに描いたのが図 5.10 である。注目してほしい点は次の 3 つで、図の丸数字と対応している。

- ① 1 つの個体が `rico:Instantiation` と `premis:Representation` の両方のクラスに属している。RiC-CM は具現化の記述も扱うが、具現化を物理的に管理するのに要する属性や関係をすべて扱うわけではなく、物理的な管理と保存に要する部分は既存の標準で補えるように作られている (RiC-CM §1.2)。この例を作った docuteam 社は、その部分に PREMIS のオントロジーを使った。
- ② 閲覧制限が `rico:Rule` 実体として表現されている。ISAD(G) では「アクセス条件」というテキスト要素だったものが、終了年を持つ独立の実体になり、機械的な判定 (2025 年を過ぎたか) に使える形になっている。
- ③ 作成関係が実体化され、関与の種別 (“archivist”) が関係に付与されている。なお、この実例には 5.5.3 項で述べた `_role` の 1 行 (ここでは `rico:creationRelation_role`) がない。そのため OWL 2 RL の推論器にかけても、近道の `rico:hasCreator` は導かれない。

前節までの道具立てが全部使われていることが分かるだろう。

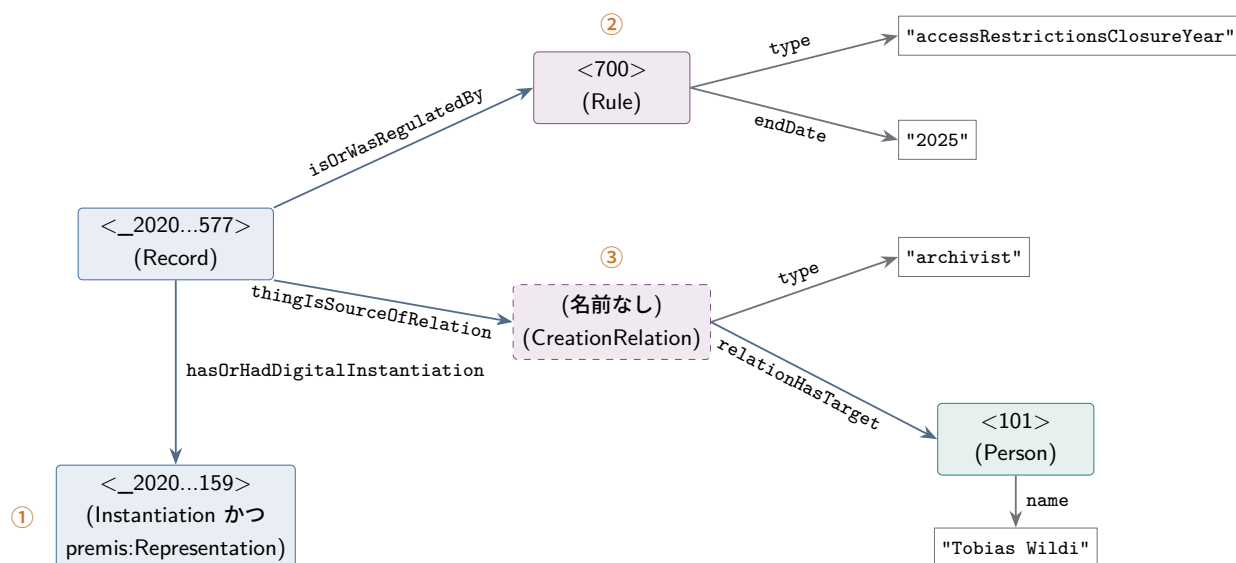


図 5.10 docuteam の実例をグラフに描いたもの (プロパティ名の rico: は省いた)。① 2 つのクラスに属する具現化、② 終了年を持つ規則、③ 実体化された作成関係。破線の箱は名前のない空白ノードである。

確認問題 (ヒントは付録 A.6)

問 5.1 次の Turtle を日本語で読み下せ。

```
ex:s2 a rico:RecordSet ;
    rico:name "防疫統計系列" ;
    rico:isOrWasIncludedIn ex:fondsP83 .
```

問 5.2 問 5.1 の内容を、セミコロンとコンマを使わず、3 つの独立した文 (トリプル) として書き直せ。

問 5.3 hasCreator⁻ はどんな関係か。規則の形 (→ を使った形) で書け。

問 5.4 「クラスが主語になる」文の例を本章から 1 つ挙げ、コラムの 2 つの読み方で説明せよ。

第 6 章

推論 — グラフだからできること

RiC-O の公式ドキュメントは、RiC-O が支えるものとして 3 つを挙げる。Linked Data としての RDF データセットの公開、SPARQL による問い合わせ、そして「オントロジーの論理による推論 (inference)」である。本章が扱うのは 3 つめで、推論とは何かを説明し、アーカイブズ記述で推論が実際に役立つ場面を、具体例で 1 つずつ確かめる。

6.1 推論とは何か

推論とは、明示的に記述されたトリプルとオントロジーの公理 (定義) から、書かれていないが論理的に導けるトリプルを機械が導出することである。

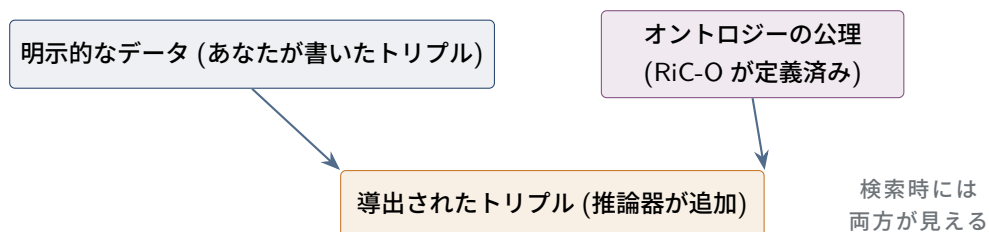


図 6.1 推論の基本的な仕組み。記述者は最小限を書き、残りは推論器が補う。

推論の元手になる公理は、RiC-O にあらかじめ組み込まれている。主なものは 5 種類で、いずれも第 5 章で登場した。下位クラス (subClassOf)、下位プロパティ (subPropertyOf)、逆プロパティ (inverseOf:RiC-O には 416 の逆プロパティ宣言がある)、推移性 (TransitiveProperty)、プロパティ連鎖 (propertyChainAxiom:121 件) である。つまり RiC-O は、用語のリストであると同時に推論の規則集でもある。この規則集を活用するには、推論機能を持つトリプルストア (GraphDB、Fuseki など。第 7 章) に RiC-O 本体とデータを載せ、推論を有効にすればよい。ただし推論器が扱える公理の範囲には限りがあり、データの側で補わなければならない点もある (場面 4 と 6.8.4 項)。5 種類の公理が何を導き、本章のどの場面で使われるかを表 6.1 にまとめておく。

本章の例では、推論の結果を確かめる道具として SPARQL (RDF の検索言語) の式がたびたび登場する。初見でも次のことだけ知っていれば読める。

表 6.1 RiC-O に組み込まれた主な公理と、本章で使う場面

公理の種類	OWL での書き方	RiC-O の例	導かれるもの	場面
下位クラス	<code>rdfs:subClassOf</code>	<code>Person $\sqsubseteq$ Agent</code>	個人は Agent でもある	場面 3
下位プロパティ	<code>rdfs:subPropertyOf</code>	<code>hasOrHadSpouse $\sqsubseteq$ hasFamilyAssociationWith</code>	細かい関係から粗い関係	場面 1・3
逆プロパティ	<code>owl:inverseOf</code> (416 件)	<code>hasCreator</code> と <code>isCreatorOf</code>	逆向きのトリプル	場面 2
推移性	<code>owl:TransitiveProperty</code> (28 件)	<code>includesTransitive</code>	間接の「含む」	場面 1
プロパティ連鎖	<code>owl:propertyChainAxiom</code> (121 件)	関係ノードを経由する 3 歩 $\sqsubseteq$ <code>hasOrganicProvenance</code>	長い経路の近道	場面 4

コラム:SPARQL を読むための最小限

SPARQL の基本形は `SELECT 変数 WHERE { パターン }` である。読み方は 3 つ覚えればよい。

第一に、`?r` のように `?` で始まる語は**変数** (まだ分かっていない穴) である。`SELECT` の後に並べた変数が、答えとして返してほしいものを指す。

第二に、`WHERE` の波括弧の中身は、Turtle とまったく同じ書き方の「主語 述語 目的語」パターンの並びである (ピリオドやセミコロンの規則も Turtle と同じ)。いちばん大きな違いは、任意の位置に変数を置けることである。たとえば `?r a rico:Record` は「`?r` は記録である」という条件を表す。

第三に、同じ変数が複数のパターンに現れたら、それらは**同じもの**を指す。SPARQL の処理系はグラフの中から、すべてのパターンを同時に満たす変数の割り当てを探し、その一覧を返す。これだけである。なお実際の式には Turtle と同様の `PREFIX` 宣言が先頭に付くが、本書の例では紙幅の都合で省略している。

この 3 点で本章の例はすべて読める。より進んだ機能 (条件を絞る `FILTER`、「`~`が存在しない」ことを調べる `FILTER NOT EXISTS`、検索結果から新しいトリプルを作る `CONSTRUCT` など) は、登場する箇所ですべて説明する。

以下、推論の効用を場面別に見る。

6.2 場面 1: 階層をまたぐ検索 — 推移性

フォンド→シリーズ→ファイル→記録という 4 層の編成を、各層のあいだの「直接含む」だけで記述したとする。

```
ex:fonds01    rico:directlyIncludes  ex:series02 .
ex:series02   rico:directlyIncludes  ex:file07 .
ex:file07     rico:directlyIncludes  ex:record31 .
ex:record31   a    rico:Record .
```

4 行目の `a` は「`~`のインスタンスである」を表す述語で、`ex:record31` が記録であることを述べている (第 5 章の Turtle 文法コラム)。

利用者が知りたいのは多くの場合「フォンド 01 に (直接だろうと間接だろうと) 含まれる記録の全部」である。RiC-O では `rico:directlyIncludes` が推移的プロパティ `rico:includesTransitive` の

下位プロパティとして定義されているため、推論器は図 6.2 の破線のトリプルを自動で導出する。記述論理の記法 (第 5 章 5.2.1 項) で書けば、使われている公理は次の 2 本である。

$$\text{directlyIncludes} \sqsubseteq \text{includesTransitive}$$

$$\text{includesTransitive} \circ \text{includesTransitive} \sqsubseteq \text{includesTransitive}$$

上の式は「直接含むなら含む」、下の式は「含むものが含むものは、含む」(推移性)と読む。この 2 本から導かれる「含む」の全体を**推移閉包** (transitive closure) と呼ぶ。直接の関係を出発点に、たどれるかぎりたどってできる関係の集まりのことである。フォンドからシリーズ、シリーズからファイルとたどれるなら、フォンドからファイルへの関係も閉包に入る。下の公理を変数を使った規則の形 (第 5 章 5.2.1 項) で書けば

$$\text{includesTransitive}(x, y) \wedge \text{includesTransitive}(y, z) \rightarrow \text{includesTransitive}(x, z)$$

である。

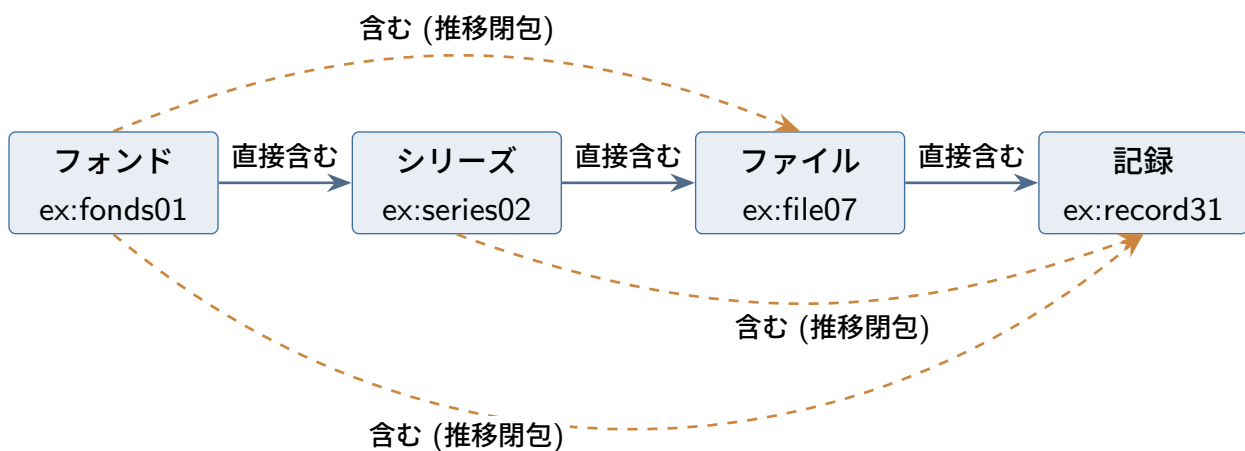


図 6.2 推移性による導出。実線は記述するトリプル (rico:directlyIncludes)、破線は推論で得られるトリプル (rico:includesTransitive)。書くのは隣り合う階層の 3 本だけで、階層をまたぐ「含む」は推論器が補う (隣り合う階層どうしにも rico:includesTransitive は導かれるが、図では省いた)。

推論による導出(書かなくてよい)

```
ex:fonds01 rico:includesTransitive ex:series02 , ex:file07 , ex:record31 .
ex:series02 rico:includesTransitive ex:file07 , ex:record31 .
ex:file07 rico:includesTransitive ex:record31 .
```

したがって、階層が何段あっても、階層をたどる部分は SPARQL の 1 行で書ける。

```
SELECT ?r WHERE {
  ex:fonds01 rico:includesTransitive ?r .
  ?r a rico:Record .
}
```

読み下すと「フォンド 01 が (推移の意味で) 含んでいる ?r であって、かつ記録であるものを、すべて挙げよ」。2 つのパターンに同じ変数 ?r が現れているので、両方の条件を同時に満たすものだけが答えになる。この例のデータでは ex:record31 だけが返る。シリーズとファイルもフォンド 01 に含まれるが、記録であるとは書かれていないので 2 つめの条件で外れる。

EAD/XML では、入れ子になった要素の子孫をたどる式 (XPath の子孫軸など) を書くことになる。RDB では再帰共通表式 (recursive CTE) という比較的高度な SQL が要る。グラフ + 推論では、モデルの側が「含む関係は連鎖する」と知っているので、問い合わせが単純になる。全体-部分 (`rico:hasPartTransitive`)、イベントの部分 (`rico:hasSubeventTransitive`)、場所の包含 (`rico:containsTransitive`)、先祖 (`rico:hasAncestor`)、時間的先行 (`rico:precedesInTime`) にも同様の推移的プロパティが用意されている。

コラム: 公理と手続き — 宣言と計算のあいだ

プログラムを書いた経験があると、推移性を次の 2 本の規則で書きたくなるかもしれない。親子関係 `parent` から祖先関係 `ancestor` を作る、おなじみの再帰である。

$$\text{parent}(x, y) \rightarrow \text{ancestor}(x, y)$$

$$\text{parent}(x, y) \wedge \text{ancestor}(y, z) \rightarrow \text{ancestor}(x, z)$$

これは計算の手順 (元データ `parent` に 1 歩ずつ継ぎ足して閉包を作る) が透けて見える書き方である。規則の左側 (前提の側) に導出対象の `ancestor` が高々 1 回しか現れないので、この形を**線形再帰**と呼ぶ。対して本文の公理は $\text{parent} \sqsubseteq \text{ancestor}$ と $\text{ancestor} \circ \text{ancestor} \sqsubseteq \text{ancestor}$ 、つまり結果の性質だけを述べる。

書き手が手続きを意識する必要は、原則としてない。DL/OWL は**宣言的**な言語で、公理は「何が帰結するか」だけを定め、どんな順序でどう計算するかは推論器に任される。決定可能性 (第 5 章) は、答えを有限の手順で判定できる算法が存在するという保証であり、任意の計算方法が必ず停止するという意味ではない。この保証を満たす算法を処理系が実装する。意味は書き手が、手順は処理系が受け持つ。この分業こそが宣言的言語の眼目である。ちなみに上の 2 本も OWL 2 で書ける (1 本目は下位プロパティ、2 本目は $\text{parent} \circ \text{ancestor} \sqsubseteq \text{ancestor}$ という連鎖公理)。

ただし、但し書きが 2 つある。第一に、2 つの書き方は**意味が微妙に違う**。`parent` の事実だけから出発するなら帰結は同じだが、RiC のデータでは「親子は不明だが祖先であることは分かる」という粗い事実を `ancestor` として**直接**主張できる (粒度の自由)。推移公理版は、そうして直接主張された祖先関係同士も合成する ($\text{ancestor}(a, b)$ と $\text{ancestor}(b, c)$ から $\text{ancestor}(a, c)$)。線形再帰版は `parent` を経由しない合成をしない。RiC-O では `rico:hasAncestor` が推移的プロパティとして定義されているので、粗い主張が混在していても、それら同士の合成まで導かれる。2 つの書き方の違いは**意味**の違いであって、手続きの都合の違いではない。

第二に、意味が同じでも**計算の重さ**は公理の形で変わる。線形再帰は「導出済みの事実と元データの結合」だけで閉包が作れるのに対し、推移公理を素朴に適用すると「導出済み同士の結合」が発生し、重複した導出が増えうる。もっともこれは処理系の側で吸収される話で、トリプルストアは推移閉包を専用の処理で計算するのが普通である。OWL 2 にはさらに、「この範囲の公理なら規則エンジンで実装できる」「この範囲なら多項式時間」といった**計算の保証つき部分集合** (プロファイル。RL・EL・QL) が定義されており、RiC-O の主要な公理 (下位クラス・下位プロパティ・逆・推移・連鎖) は規則エンジン向きの範囲に収まる (この範囲に収まらない公理もある。6.8.4 項)。記述者としては意味だけを書けばよく、性能はストアの設定と、導出したトリプルを事前に計算して書き込んでおくかどうかの選択 (6.8 節) の問題として切り分けるのが、この技術圏の作法である。

6.3 場面 2: どちら向きに書いても見つかる — 逆プロパティ

記述の現場では「記録の側に作成者を書く」機関と「作成者の側に作成物を書く」機関がありうる。RiC-O では対になるプロパティが `owl:inverseOf` で宣言されているため^{*1}、どちら向きに書かれていても、推論器が逆向きを補い、両方向から検索できる。機関を越えてデータを集約するとき (LOD の本領) には、書き方の向きの違いを吸収するこの仕組みが実的な意味を持つ。

6.4 場面 3: 詳しく書いた人の損にならない — 下位プロパティのロールアップ

ある機関は人間関係を細かく「配偶者 (`rico:hasOrHadSpouse`)」「教師 (`rico:hasOrHadTeacher`)」と記述し、別の機関は単に「関係があった」としか記述していないとする。RiC-O ではこれらのプロパティが階層をなしている。記述論理で書けば、上が配偶者、下が教師の系統で、どちらも同じ頂点に行き着く (図 6.3)。

`hasOrHadSpouse` $\sqsubseteq$ `hasFamilyAssociationWith` $\sqsubseteq$ `isAgentAssociatedWithAgent`

`hasOrHadTeacher` $\sqsubseteq$ `knows` $\sqsubseteq$ `isAgentAssociatedWithAgent`

推論器は下位プロパティの言明から上位プロパティの言明を導出するので、「山田花子と何らかの関係があった人物」を上位プロパティ 1 つで問い合わせれば、細かく書かれたデータも粗く書かれたデータも同じ網ですくえる。細かい言明が上位のプロパティへこうして吸い上げられることを**ロールアップ** (roll-up) と呼ぶ。以降の章でもこの語を使う。

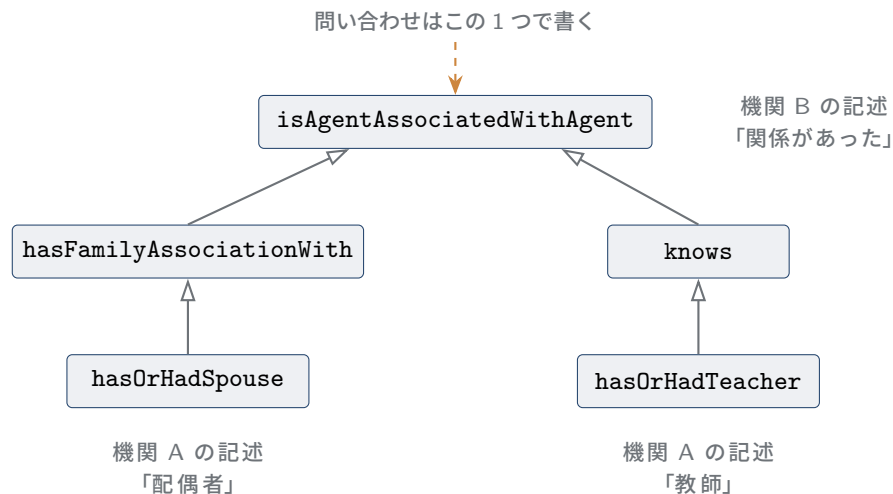


図 6.3 下位プロパティのロールアップ。白抜きの三角は「下位プロパティである」を表す。細かく書いた言明 (下の段) も粗く書いた言明 (頂点) も、上位のプロパティ 1 つで問い合わせられる。

図 6.3 では、配偶者と教師が別々の枝を通して同じ頂点に集まっている。機関 A が細かく書いた言明も、機関 B が頂点のプロパティで粗く書いた言明も、頂点で問い合わせれば同じ答えの中に現れる。

^{*1} たとえば `rico:hasCreator` と `rico:isCreatorOf` が対になる。記述論理で書けば `isCreatorOf` $\equiv$ `hasCreator` である。

これは第 3 章で述べた「粒度の自由」の技術的な裏付けである。詳しく書けば詳しい検索に応えられ、粗くしか書けなくても横断検索からは漏れない。記述の濃淡がそのまま検索の分断になる、という事態をオントロジーの階層が防いでいる。クラス側も同様に、`rico:Person` のインスタンスは自動的に `rico:Agent` のインスタンスでもあるから、「エージェント全部」という問い合わせに個人も団体もメカニズムも応答する。

6.5 場面 4: 近道の自動生成 — プロパティ連鎖

第 5 章 5.5.3 項で見たとおり、関係に日付や確実性を付けたい場合は関係を実体化して書く。

```
ex:fonds01 rico:thingIsSourceOfRelation ex:rel01 .
ex:rel01 a rico:OrganicProvenanceRelation ;
rico:organicProvenanceRelation_role ex:rel01 ;
rico:relationHasTarget ex:corpA ;
rico:relationHasDate ex:date1943_1968 ;
rico:relationCertainty "certain" .
```

この「長い経路」に対して、RiC-O は `rico:hasOrganicProvenance` をプロパティ連鎖の近道として定義している。記述論理で書けば

$$\text{thingIsSourceOfRelation} \circ \text{organicProvenanceRelation_role} \circ \text{relationHasTarget} \\ \sqsubseteq \text{hasOrganicProvenance}$$

であり、変数を使った規則の形で書けば

$$\text{thingIsSourceOfRelation}(x, y) \wedge \text{organicProvenanceRelation_role}(y, z) \\ \wedge \text{relationHasTarget}(z, w) \rightarrow \text{hasOrganicProvenance}(x, w)$$

つまり「関係ノードの入口から出口まで 3 歩でたどれるなら、直接 1 本で結んでよい」である (図 6.4)。

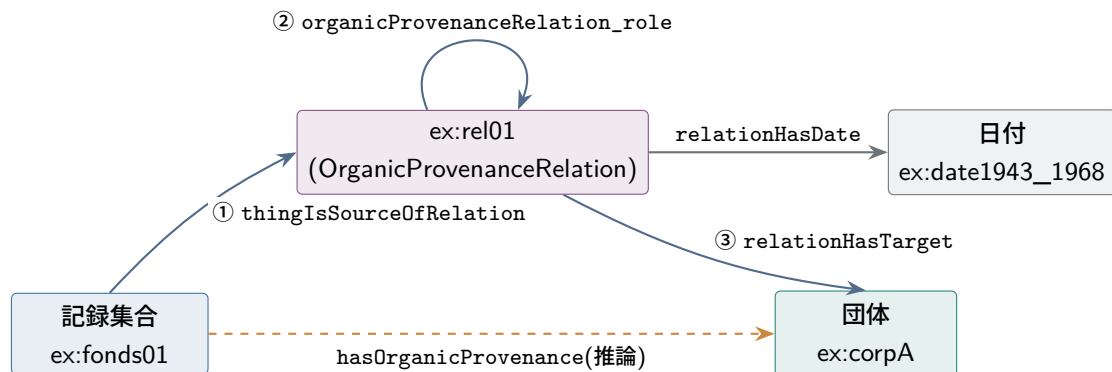


図 6.4 プロパティ連鎖による近道。関係ノードを経由する 3 歩 (自分自身への 1 歩を含む) がそろると、推論器は記録集合から団体への直接の 1 本 (破線) を導く。日付は連鎖に加わらない。

真ん中の `organicProvenanceRelation_role` は、関係ノードを自分自身に結ぶ補助のプロパティである。「このノードは有機的出所関係である」ということを、経路の 1 歩として表すために置かれている。RiC-O の定義では、有機的出所関係のインスタンスはどれも自分自身とこの関係にある。しかしこの形の定義 (OWL の `hasSelf`) は、トリプルストアが推論によく使う OWL 2 RL の規則 (6.8.4 項) の

範囲の外にあり、推論器はこの1歩を自分では補わない。そこで上の listing の3行目のように、自分自身への1本をデータに書いておく。RiC-O もこのプロパティの説明 (スコープノート) で、OWL 2 RL とプロパティパスで2つの近道 (`rico:hasOrganicProvenance` とその逆) を得るために、各インスタンスをこのプロパティで自分自身に結ぶよう述べている。この1行があれば、推論器は次を導出する。

```
# 推論による導出
ex:fonds01 rico:hasOrganicProvenance ex:corpA .
```

つまり、丁寧に書かれたデータと簡便に書かれたデータが、検索時には同じ形に揃う。検索する側は常に簡便な近道プロパティで問い合わせればよく、データがどちらの流儀で書かれたかを知らなくてよい。RiC-O 1.1 では日付についても同様の近道 (`rico:hasCreationDate` など) が連鎖として定義された。この仕組みがなければ、「表現方法が複数ある」という RiC-O の柔軟性は検索の悪夢になっていたはずである。柔軟性と検索可能性を両立させる要が、プロパティ連鎖公理だと言ってよい。

6.6 場面 5: 組織の変遷をたどる — アーカイブズらしい総合問題

RiC-O の公式ドキュメントが挙げる問い合わせ例は、アーカイブズ利用の実感によく合っている。「(データセットによれば) ある実体の消滅から現在までに、(ある活動について) それを継いだ団体はどれか」「この写真にはどんな具現化が存在するか」「この原本の現存する写しは何か」。ここでは組織変遷の例を通して確かめる。

県の衛生行政を例にとる。1948 年に衛生部が発足し、1994 年に保健福祉部へ改組、2010 年に健康福祉部へ再編されたとする。RiC ではこれを、3 つの団体 (CorporateBody。Agent の一種)、1 つの活動 (Activity: 衛生行政)、後継関係と活動遂行関係のトリプルで記述できる。

```
ex:eisei      a rico:CorporateBody ; rico:name "衛生部" ;
              rico:hasSuccessor ex:hokenfukushi .
ex:hokenfukushi a rico:CorporateBody ; rico:name "保健福祉部" ;
              rico:hasSuccessor ex:kenkofukushi .
ex:kenkofukushi a rico:CorporateBody ; rico:name "健康福祉部" .

ex:eiseiGyosei a rico:Activity ; rico:name "衛生行政" .
ex:eisei      rico:performsOrPerformed ex:eiseiGyosei .
ex:hokenfukushi rico:performsOrPerformed ex:eiseiGyosei .
ex:kenkofukushi rico:performsOrPerformed ex:eiseiGyosei .

ex:series1950s a rico:RecordSet ; rico:name "伝染病予防関係綴" ;
              rico:hasOrganicProvenance ex:eisei ;
              rico:documents ex:eiseiGyosei .          # この活動を文書化する
```

このデータをグラフとして描くと図 6.5 になる。ラベルは日本語に意識してある (対応するプロパティ名は上の Turtle のとおり)。

このデータに対して、たとえば次のことが可能になる。1950 年代のシリーズから出発して、有機的出所の組織 (衛生部) → 後継の連鎖 → 現在の担当部局 (健康福祉部) へと機械的にたどれる。逆に、現在の健康福祉部から出発して「歴代の前身組織が同じ活動について残した記録集合の全部」を集めることもできる。

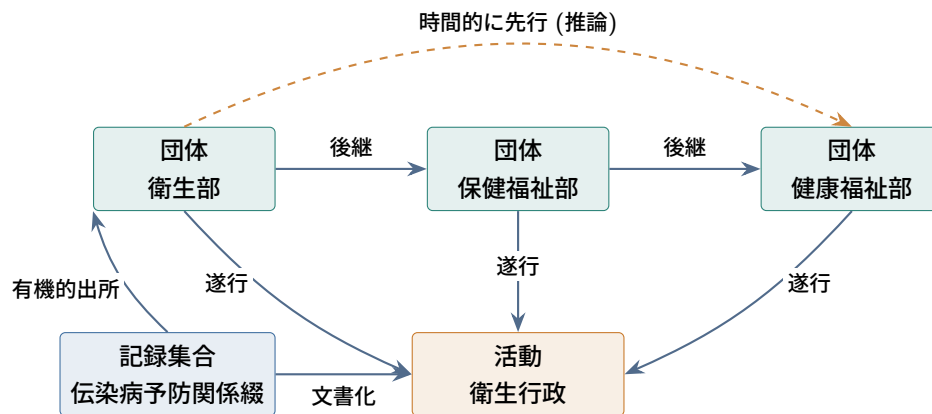


図 6.5 組織変遷の記述のグラフ表現。シリーズから旧機関へ、後継の鎖をたどって現機関へ、また活動を軸に歴代機関の記録集合へと、どの向きにもたどれる。

後継をたどるには `rico:hasSuccessor+` というプロパティパスを使える (+ は「1 回以上の繰り返し」の意味。6.8.3 項で改めて扱う)。

```
SELECT ?later ?name WHERE {
  ex:eisei rico:hasSuccessor+ ?later .
  ?later rico:name ?name .
}
```

答えは保健福祉部と健康福祉部の 2 つである。図の破線の時間的先行関係 (`rico:precedesInTime`) も推論で導かれるが、これは後継以外の先後関係も含むので、後継の連鎖と同一視してはいけない。次の問い合わせは、同じ活動を文書化する集合とその有機的出所を集めるもので、後継の連鎖を条件にはしていない。

```
# 衛生行政を文書化する記録集合と、その有機的出所を一覧する
SELECT ?rs ?agent WHERE {
  ?rs rico:documents ex:eiseiGyosei ;
  rico:hasOrganicProvenance ?agent .
}
```

読み下すと「衛生行政という活動を文書化しており、かつ有機的出所 `?agent` を持つ記録集合 `?rs` について、記録集合と出所の組をすべて挙げよ」。セミコロンは Turtle と同じく「同じ主語 `?rs` で続ける」の意味である。答えは (伝染病予防関係綴、衛生部) のような対の一覧になり、歴代組織のデータが揃っていれば、活動を軸にした通史的な一覧がこの 4 行で得られる。

ISAD(G) の世界では、この情報は各フォンドの「管理歴 (administrative history)」に散文で書かれ、照合は人間の仕事だった。RiC では変遷が構造化データになるので、機械が答えられる。行政文書のアーカイブズにとって、これは推論とグラフの効用が最も直接的に現れる場面だろう。

6.7 場面 6: 自分の規則は足せるか — 拡張性と、OWL では書けない規則

ここまでは RiC-O に組み込み済みの公理の話だった。本節では、自分の記述対象に固有の語彙や規則を追加する場面を扱う。「二人の人物の共通の祖先が書いた文書を探したい」という要求を例に、できること・できないことを切り分けてみる。

6.7.1 語彙の追加は明示的に認められている

RiC-O の設計原則の 1 つは拡張可能性であり、独自の下位クラス・下位プロパティを追加することは、公式に想定された使い方である (RiC-O ドキュメント設計原則 5)。RiC-AG も FAQ「How to extend RiC」を設けてこれを正面から扱っている。推奨される作法は次の 4 つである。

- 独自の名前空間と識別子を用意する。
- 新しい構成要素を RiC の既存要素の下位として接続する。
- 拡張を文書化して公開する。
- 需要が一般的なら、EGAD への提案 (GitHub の Issue や利用者メーリングリスト) も検討する。

FAQ には、公証文書の「遺言者 (has testator)」を `rico:hasAuthor` の下位プロパティとして定義する実例も付いている。同じ形で、たとえば宗門人別改帳の記述で「寺請をした寺」(記載された人々の宗旨を証明して証印を押した檀那寺)を区別したければ、次のように書ける (第 7 章 7.5 節で詳しく扱う)。

```
PREFIX myo: <https://archives.example.jp/ontology#>
myo:hasCertifyingTemple rdfs:subPropertyOf rico:hasAuthor .
```

下位プロパティとして宣言しておけば、独自の細かい語彙で記述しても、`rico:hasAuthor` やその上位の `rico:hasCreator` での横断検索 (場面 3 のロールアップ) から漏れない。拡張が「RiC からの逸脱」にならないための要は、この既存階層への接ぎ木である。孤立した独自プロパティを作ると、標準語彙での検索と切れてしまう。

6.7.2 「共通の祖先」の例 — 問い合わせで済むもの

冒頭の要求は、実は新しい規則を追加しなくても実現できる。RiC-O には `rico:hasAncestor` (祖先を持つ) が推移的プロパティとして定義済みなので、親子関係 (`rico:hasChild` など) を書いておけば、世代をまたぐ祖先関係は推論で得られる。`rico:hasChild` は `rico:hasDescendant` の下位プロパティで、`rico:hasDescendant` は `rico:hasAncestor` の逆である。推論器はこの 2 つの公理で親子関係を「祖先を持つ」に読み替え、推移性で世代をつなぐ。場面 1~3 の仕組みを組み合わせで使っていることになる。あとは SPARQL で「花子の祖先でもあり、太郎の祖先でもある人物が作成した記録」を問い合わせればよい。

```
SELECT ?doc ?x WHERE {
  ex:hanako rico:hasAncestor ?x .    # 花子の祖先(推移閉包)
  ex:taro   rico:hasAncestor ?x .    # 太郎の祖先でもある
  ?x rico:isCreatorOf ?doc .          # その人物が作成した記録
}
```

読み下すと「花子の祖先であり、かつ太郎の祖先でもある人物 ?x と、その ?x が作成した記録 ?doc の対を、すべて挙げよ」。3つのパターンすべてに同じ変数 ?x が現れていることが要で、この変数の共有が「2本の祖先の系譜が同一人物で合流する」という条件そのものを表している。hasAncestor は推移的なので*2、親子関係を書いておくだけで曾祖父母以遠も検索対象に入る。

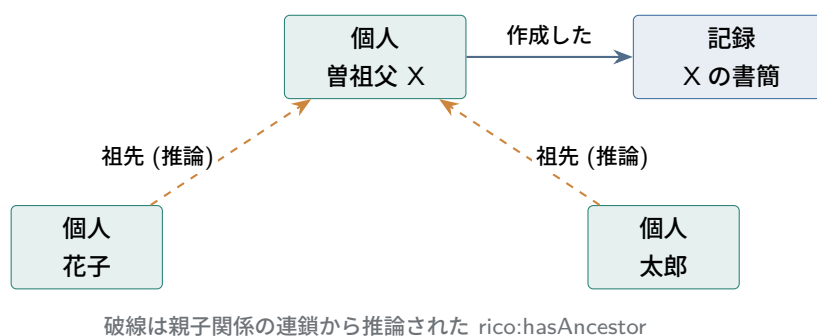


図 6.6 共通祖先の文書を探す。2本の祖先経路が同じ人物 X で合流するかどうかは、SPARQL の変数共有 (2つのパターンが同じ ?x を指すこと) で表現される。

つまりこの要求は RiC-O の対象範囲内であり、必要な道具は「語彙 (定義済み)+ 推論 (定義済み)+ 問い合わせ (自分で書く)」である。

6.7.3 連鎖で書ける規則と、問い合わせで書く条件

「共通の祖先を持つ」という関係を、新しいプロパティとして導出することも OWL 2 でできる。A から祖先 X へ進み、X から子孫 B へ進めば、1本の経路になるからである。rico:hasDescendant は rico:hasAncestor の逆プロパティなので、次の連鎖公理で足りる。

```
myo:hasCommonAncestorWith a owl:ObjectProperty ;
  owl:propertyChainAxiom
    ( rico:hasAncestor rico:hasDescendant ) .
```

図 6.7 のように、花子から祖先 X へ上り、X から子孫の太郎へ下りる 2 歩が 1 本の「共通の祖先を持つ」になる。

この公理は、祖先がわかっている人物とその人物自身の組も導出する。X の子孫には花子自身も含まれるからである (図の灰色の矢印)。異なる人物の組だけを表示したいなら、問い合わせで FILTER(?a != ?b) とする (FILTER は 6.8.3 項で扱う)。

*2 記述論理で書けば $\text{hasAncestor} \circ \text{hasAncestor} \sqsubseteq \text{hasAncestor}$ である。

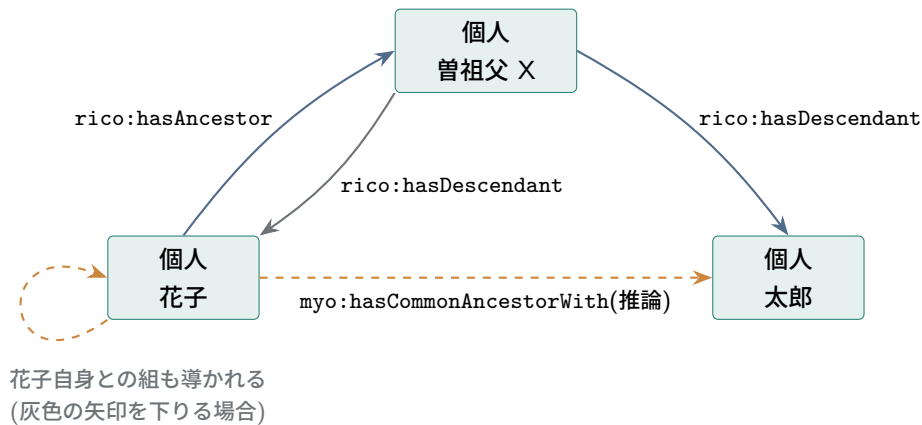


図 6.7 連鎖公理による「共通の祖先を持つ」の導出。祖先へ上って子孫へ下りる 2 歩が 1 本になる。下りる先が出発点と同じ人物でも成り立つので、自分自身との組も導かれる。`rico:hasAncestor` と `rico:hasDescendant` も親子関係から推論されたものだが、ここでは連鎖の入力として実線で描いた。

```
SELECT ?a ?b WHERE {
  ?a myo:hasCommonAncestorWith ?b .
  FILTER(?a != ?b)
}
```

ただしこれは RDF の項の違いを調べる条件であり、2 つの IRI が別人を指すという OWL の主張ではない。

一方、「共通祖先があり、かつ出生地も同じ」というように、同じ両端について複数の経路の条件を同時に課す規則は、1 本のプロパティ連鎖では書けない。こうした条件は SPARQL で問い合わせ、結果をトリプルとして保存したければ CONSTRUCT で生成できる。単に答えが欲しい場合と、関係を公理として追加したい場合を分けて考えるとよい。

整理すると、独自の要求は 3 つに振り分けて実現する。

1. **語彙の拡張** (下位クラス・下位プロパティの追加)—OWL の範囲内。RiC-O が公式に想定。
2. **答えが欲しいだけの要求**—規則を追加せず SPARQL で問い合わせる。今回の共通祖先はこれで足りる。
3. **OWL で表現できない導出規則**—規則エンジンや CONSTRUCT で実装する。RiC-O の守備範囲の外だが、RiC-O のデータの上で動かすことは通常の技術でできる。

なお「そもそも分野固有の知識を拡張として持つべきか、別の場所に置くべきか」という一段手前の設計判断は、第 7 章 7.5 節で扱う。

6.8 処理系の中身 — 書いた記述を誰が実行するか

ここまで「推論器」「トリプルストア」とひとまとめに呼んできた処理系の内部を、少しだけ覗いておく。語彙や公理をいくら学んでも、それを実行する機械の姿が見えないままでは、公理・検索・性能の関係が宙に浮いてしまうからである。

6.8.1 トリプルストアの構造

トリプルストアの内部は、リレーショナルデータベースを知っていれば見通しやすい (第 8 章 8.3 節)。おおよそ次の部品からなる。

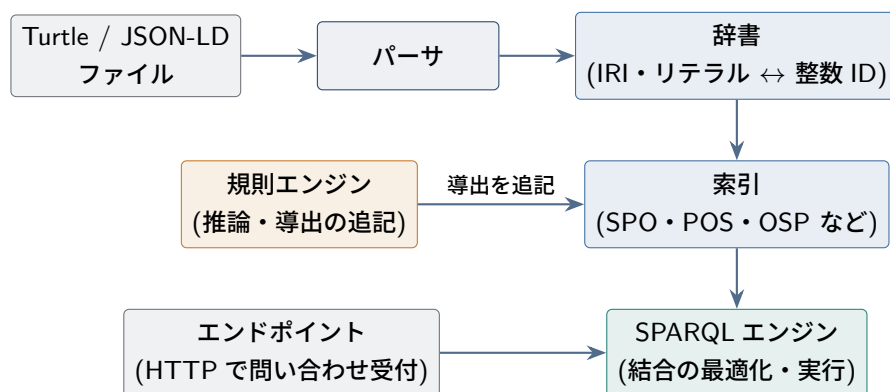


図 6.8 トリプルストアの内部構成のあらまし。RDB でいえば、索引は B 木索引、SPARQL エンジンを実行計画の最適化器に当たる。辞書は、長い文字列を短い番号に置き換える対応表である。

読み込まれたトリプルは、まず**辞書**で IRI やリテラルが整数の内部 ID に置き換えられる (長い IRI を毎回文字列比較しては遅いからである)。次に**索引**に格納される。トリプルは主語 (S)・述語 (P)・目的語 (O) の 3 つ組なので、SPO 順・POS 順・OSP 順など複数の並びで冗長に索引を持っておくと、「主語が分かっているとき」「述語だけ分かっているとき」のどの形の問い合わせにも速く答えられる。SPARQL エンジン、WHERE 節のパターン群をどの順に索引照合すれば結合が小さく済むかを見積もって実行する。RDB の実行計画の最適化と同じ発想である。推論を有効にすると、**規則エンジン**が公理に対応する規則 (第 5 章のコラムで見た Datalog 型の規則) を繰り返し適用し、導出トリプルを索引に追記していく。

6.8.2 推論の実行方式 — 事前計算か、検索時か

推論の実装には大きく 2 方式ある。**マテリアライゼーション** (前方連鎖) は、データの投入時に規則を適用し尽くし、導出トリプルをすべて索引に書き込んでおく方式である。検索は速い (すでに書いてあるものを引くだけ) が、格納量が増え、データを削除したときに「その事実に依存していた導出」を取り消す再計算が要る。**検索時展開** (後方連鎖・クエリ書き換え) は逆に、問い合わせが来た時点で規則をさかのぼって適用する方式で、更新は軽いが検索のたびに計算する。

製品でいえば、GraphDB は前者の代表で、適用する規則の集合 (RDFS のみ、OWL-Horst、OWL 2 RL など) を設定で選ぶ。Apache Jena(Fuseki) は汎用の規則推論器と格納系 (TDB2) の組み合わせで、両方式を構成できる。ほかに Virtuoso、Amazon Neptune、軽量な Oxigraph などがある。いずれも SPARQL エンドポイントという HTTP の窓口を持ち、アプリケーションはそこへ問い合わせを送る。

これらと系統の違う道具として、**タブロー法**系の推論器 (HermiT、Pellet など。無償のオントロジーエディタ Protégé から呼び出せる) がある。タブロー法は、公理の集まりが矛盾しないかを、ありうる

解釈を場合分けして枝分かれさせながら調べる証明法である。これは大量データの導出よりも、**オントロジーそのものの検査** (公理群に矛盾がないか、クラス階層はどう整理されるか) に向く。語彙の検査 (TBox 推論) とデータの導出 (ABox 推論) は同じ「推論」でも性格の違う仕事であり、道具も分かっている。本章の注意点 (6.9 節) で触れる RiC-O 1.0 の不具合 (1.0.2 で修正) は、この種の検査で見つかる性質の誤りである。

6.8.3 SPARQL をもう少し — 現場で使う 4 つの機能

基本形 (コラム参照) に加え、実務で頻出する機能を 4 つ、これまでの例データの上で見しておく。

■**OPTIONAL** — **あれば取る、なくても捨てない** 名前があれば添え、なくても一覧から落とさない検索を書く (場面 1 と同じく推論を有効にしたストアを前提にする。推論がなければ、後で見るプロパティパスで書き換える)。

```
SELECT ?r ?name WHERE {
  ex:fonds01 rico:includesTransitive ?r .
  OPTIONAL { ?r rico:name ?name }
}
```

OPTIONAL の中は「一致すれば値を返し、なくても行自体は残す」。名前のない記録も空欄つきで一覧に現れる。開世界仮説の世界では「名前が書かれていない」ことは「名前がない」ことを意味しないから、この控えめな挙動を明示して選べることに意味がある。OPTIONAL を付けずに書くと、名前の書かれていない記録は一覧から黙って消える。

■**FILTER** — **値で絞る** 年代で絞り込んでみる。

```
SELECT ?r ?d WHERE {
  ?r      a rico:Record ;
          rico:isAssociatedWithDate ?date .
  ?date   rico:normalizedDateValue  ?d .
  FILTER(?d < "1900")
}
```

関連づけられた日付の年が 1900 年より前の記録に絞る。値の比較・文字列照合・言語タグの検査などが書ける (この例は 4 桁の年を文字列のまま比較しており、桁が揃っている場合にだけ正しい。日付全体や不確実な日付を扱う場合は、その形式に応じた比較が必要になる)。日付を 2 段でたどっているのは、rico:normalizedDateValue の定義域が rico>Date であって記録資源ではないからである (第 4 章 4.2.7 項)。rico:normalizedDateValue は日付実体に付ける。記録に直接付けても OWL の推論器はそれを拒まず、その記録を日付の一種と推論するだけである。誤りとして検出するには SHACL などの検査が要る (第 8 章 8.4 節)。

■**プロパティパス** — **推論なしの推移閉包** 述語の繰り返しを、クエリの側で直接書くこともできる。

```
SELECT ?r WHERE {
  ex:fonds01 rico:directlyIncludes+ ?r .
}
```

述語に付いた + は「1 回以上の繰り返し」で、`directlyIncludes` を何段でもたどる。つまり推論を使わずに、問い合わせの側で推移閉包を計算する道もある。使い分けはこうである。推移性を公理にすれば「含むは連鎖する」という意味がデータの一部になり、すべての利用者・すべての検索が恩恵を受ける。パスで書けば、その場のその問い合わせ限りの計算で済み、ストアの推論設定に依存しない。公開エンドポイントに推論があるとは限らないので、パスの書き方は覚えておく価値がある。

■CONSTRUCT — 検索結果から新しいトリプルを作る 検索結果を、表ではなくトリプルの形で返させる。

```
CONSTRUCT { ?rs rico:hasOrganicProvenance ?a }
WHERE {
  ?rs rico:thingIsSourceOfRelation ?rel .
  ?rel a rico:OrganicProvenanceRelation ;
       rico:relationHasTarget ?a .
}
```

SELECT が表を返すのに対し、CONSTRUCT は新しいトリプルの集合を返す。この例は、実体化された出所関係から近道プロパティを生成する。つまり場面 4 の連鎖公理と同じ導出を、推論器なしで手動実行している。データ変換 (第 7 章) や、OWL で書けない規則の実装 (場面 6) を実際に担うのは、多くの場合この CONSTRUCT である。

「たどれるかぎりたどる」結果を得る方法は 3 つある。推移性の公理、プロパティパス、そしてプロパティパスを WHERE に書いた CONSTRUCT (上の 2 つの機能の組み合わせ) である。表 6.2 に、それぞれをどこに書き、何に依存し、結果がどう残るかを並べる。

表 6.2 「たどれるかぎりたどる」を実現する 3 つの方法

方法	書く場所	ストアの推論 設定	結果の残り方	向く場面
推移性の公理 (場面 1)	オントロジー (RiC-O に定義済み)	要る	マテリアライズすれば 索引に残る	すべての利用者・問い合わせに同じ結果を返した いとき
プロパティパス (<code>directlyIncludes+</code>)	問い合わせ	要らない	その問い合わせ限り	推論のない公開エンド ポイント
CONSTRUCT (パスと組み 合わせる)	問い合わせ	要らない	生成したトリプルを保 存すれば残る	データ変換、OWL で書 けない規則

6.8.4 OWL 2 プロファイル — 計算の保証つき部分集合

最後に、公理の表現力と処理系の関係を制度として整理したものが OWL 2 のプロファイルである。表現力を上げるほど推論は重くなるので、用途別に「この範囲に収めれば、この方式・この計算量で処理できる」という部分集合が規格化されている。

RiC-O が実際に使っている公理のうち、名前付きクラスどうしの下位クラス・下位プロパティ・逆・推移・連鎖は OWL 2 RL の範囲に収まる。だから実務では「RL 対応の規則エンジンを持つトリプルストアに載せてマテリアライズする」のが標準の構成になる。ただし RiC-O には RL の外の公理もあ

表 6.3 OWL 2 DL と 3 つのプロファイル

名称	性格	主な処理方式・用途
OWL 2 DL	決定可能な範囲で表現力最大 (SROIQ)	タブロー法 (HermiT 等)。語彙の検査向き。最悪計算量は大きい
OWL 2 RL	規則で書ける範囲	規則エンジンでマテリアライズ (GraphDB 等)。大量データの導出向き
OWL 2 EL	大規模な語彙の分類向け	多項式時間。医学用語集 (SNOMED CT) などの巨大オントロジー
OWL 2 QL	RDB の上での問い合わせ向け	クエリ書き換えで SQL に翻訳できる範囲

る。場面 4 で見た、関係ノードを自分自身に結ぶ定義 (`hasSelf`) がその例で、RL の推論器はこれを使わない。そのため近道を得るには、自分自身への 1 本をデータに書いておく。基数制約 (Proxy の「ちょうど 1 つ」) のような公理も RL の外にあり、データ導出にはあまり使われず、語彙検査の場面で意味を持つ。第 3 章のコラムで、RiC-O の推論は構造的な種類のものが中心だと述べたのは、この事情のことである。

6.9 推論は魔法ではない — 注意点

推論は、書いてある公理と事実から導けることだけを導く。使う前に押さえておきたい注意点が 4 つある。

■**開世界仮説に注意** OWL の推論は「書かれていないことは不明」という前提に立つ。「作成者が記録されていない記録の一覧」のような欠如の問い合わせは、論理的な否定ではなく「トリプルが存在しない」ことの検査 (SPARQL の `FILTER NOT EXISTS`) として書く必要があり、意味の違いを理解して使い分けなければならない。しかも答えは推論の有無で変わる。作成者が人物の側から `rico:isCreatorOf` で書かれている記録や、`rico:hasAuthor` だけで書かれている記録は、推論がなければ「`rico:hasCreator` がない」ものとして一覧に出てしまう。

■**推論はデータの誤りも増幅する** 誤って張られた 1 本の関係から、多数の誤った導出トリプルが生まれる。特に推移的プロパティは誤りを遠くまで運ぶ。推論の前提は入力データの品質管理である。

■**計算コストは無料ではない** 導出トリプルを事前に全部作る方式 (マテリアライゼーション。6.8.2 項) は検索が速い代わりに格納量と更新コストが増え、検索のたびに導出する方式は逆の性質を持つ。RiC-O 1.0 の初期版では公理の組み合わせが原因で推論器が矛盾を検出する不具合が見つかり (近道の推論を成り立たせるために関係クラスの個体を自分自身に結ぶ 48 の補助プロパティ (場面 4 の `organicProvenanceRelation_role` の仲間) が、どの個体も自分自身とこの関係にあると宣言されていた)、1.0.2 で修正された、という経緯もある。推論を使う実装は、この種の技術的細部と付き合うことになる (だからこそ、それを引き受けるのは通常システムの側であり、記述者ではない)。

■**推論が有効なのは公理が意味を捉えているときだけ** RiC-O の公理は EGAD の設計判断の産物であり、あなたの機関の業務ルール (たとえば「同一フォンド内の整理番号は一意」) までは知らない。そうした制約の検証には SHACL のような別の技術を使う (第 8 章 8.4 節)。

確認問題 (ヒントは付録 A.6)

問 6.1 `rico:hasCreationDate` は次の連鎖公理で定義されている。これを変数を使った規則の形で書け。

$$\text{thingIsSourceOfRelation} \circ \text{creationRelation_role} \circ \text{relationHasDate}$$
$$\sqsubseteq \text{hasCreationDate}$$

問 6.2 SPARQL の OPTIONAL は、開世界仮説とどうなじみがよいのか。OPTIONAL がない場合に起きることと合わせて説明せよ。

問 6.3 マテリアライゼーション (前方連鎖) と検索時展開の得失を、更新頻度の高いデータを例に比較せよ。

問 6.4 場面 5 のシリーズから「保健福祉部」へたどる関係の鎖を書け。その経路の探索と、図の破線で示した時間的先行関係の推論は、どう違うか。

第 7 章

実践の手順

実践の公式の手引きとしては、2025 年 10 月に RiC-AG の草案 0.1 が公開された。ただし AG 自身が実装戦略の章 (§3.1) で「利用場面は多様すぎるため、段階的な実装手順書の提供は不可能」と明言し、検討しうる行動のリストと事例を示すという方針をとっている。本章では、RiC-AG の実装戦略の章と Getting Started の事例、RiC-CM §1.6.4 の移行論、RiC-O の設計原則と公式サンプル、先行機関の経験を踏まえて、現実的な手順に組み立て直して示す。

7.1 記述を始める前に用意するもの

ISAD(G) は、標準を手にとればそのまま記述を書ける文書だった。26 の記述要素それぞれに「目的」と「規則」が置かれ、何をどう書くかがそこに書いてあったからである。RiC-CM は違う。第 1 章 1.2 節で見たとおり、RiC-CM は記述内容の書き方を定める標準ではないと自ら述べており、属性の値スキーマについても「指示的 (indicative) であって、規範でも禁止でもない」と断っている (§3.5)。実体・属性・関係の一覧はあるが、ISAD(G) でいう規則の欄が空いている状態だと考えればよい。

RiC-CM 自身の想定も、そこに現れている。§1.6.2 は、RiC-CM が定めるのは「システムへの入力を標準化する枠組み」であり、出力や画面の作りは縛らないと述べ、続けて、記述ソフトウェアの開発者が RiC-CM を実装してくれることへの期待を書く。実務者は、そうしてできた道具を通じて RiC-CM の恩恵を受ける、という筋書きである。主な読者はアーカイブズ界とされているが (§1.3)、実務者向けの手引きは別に作ると EGAD は述べており (§1.6.2)、RiC-CM 自身は記述者が日々の記述でそのまま手引きにできる形には書かれていない。

したがって、RiC-CM から記述に入るまでには、3 つを自分の側で用意することになる。

適用プロファイル 19 の実体、42 の属性、85 の関係のうち、自機関で何を使い、どれを必須とするか。

この設計が 7.3 節の工程 2 にあたる。

内容規則 名称をどの形で書くか、日付をどう正規化するか、どの統制語彙を使うか。ISAD(G) に対する DACS(米国) や、図書館でいえば日本目録規則に当たる部分である。RiC にはまだこれがない。RiC-AG 0.1 もこの部分を埋めてはおらず、規則ではなく助言と事例を示す方針をとっている (冒頭で「あらゆる利用文脈に対応する個別の指針を示すものではない」と断っている)。

記述を保持する仕組み 入力と保管を担うシステム、あるいはそれに代わる表計算の設計 (7.11 節)。

ここで 3 つに分けたのは、どこが手当てされていないかを見るためであって、標準がこの 3 つにきれ

いに分かれるという意味ではない。ISAD(G) は記述要素の一覧と書き方の規則を1つの文書で兼ねていたし、EAD は符号化の形式を定めながら、どの要素を持つかという構造も同時に決めている。実在の標準は複数の役目を重ねて持つ。RiC-CM が変わっているのは、そのうち1つだけを引き受けると明言した点である。

この3つが未整備であることが、本章で各機関が繰り返し判断を求められる理由であり、同時に、次に述べるシナリオの選び方を左右する条件でもある。

7.2 3つのシナリオ

RiC への関わり方は、機関の状況に応じておおよそ3つに分かれる。

シナリオ A: 既存記述の変換・公開 すでに EAD/Excel/データベースにある目録データを RiC-O/RDF に変換し、LOD として公開したり機関横断検索に供したりする。現時点で最も実績が多い。フランス国立公文書館の実例がこの型である。

シナリオ B: RiC ネイティブの記述 新規の記述を最初から実体・関係の形で作成する。対応システムがまだ少ないため先進事例に限られるが、組織変遷の激しい行政文書などでは効果大きい。

シナリオ C: 将来に備えた設計 当面は ISAD(G)/EAD の実務を続けつつ、将来の変換を容易にするようデータの規律を高めておく(作成者の典拠管理、識別子の一意化、日付の正規化など)。ほとんどの機関にとって、当面の現実解はこれである。

RiC-AG の実装戦略の章も、この整理と両立する助言を与えている。目的の特定から始めること(「RiC で何が最も役に立つか」を機関の必要と結びつける)、小さく始めて段階的に育てること(パイロットプロジェクト、失敗するなら最小のコストで)、チーム全体で RiC を学ぶこと(アーキビスト・IT・データ分析者では見る角度が違う)、そして資源の見積もりと業務上の根拠(ビジネスケース)を作ることである。ソフトウェア導入の順序についても、公開ポータルを先に RiC 化する道、既存データの一対一変換を先行させてから記述の高度化に進む二段構えの道、既存機能と並行して RiC ベースの機能を足していく道、という複数のシナリオを描いている。

シナリオ C が「何もしない」とことと違う点に注意してほしい。RiC への変換で最も苦勞するのは、RiC の複雑さではなく、**既存データの不規律**(同一作成者の表記ゆれ、識別子の重複、日付の自由記述)である。これらは今の実務の中で改善でき、その改善は RiC と無関係にも価値がある。

コラム: 導入例はどこまで進んでいるか (2026 年)

「実績が最も多いのはシナリオ A」という本章の見立てには、裏づけがある。先頭を走るのはフランスである。国立公文書館(AnF)は、国立図書館・文化省との典拠相互運用の実証 PIAAF(2018 年 2 月公開)を経て、Sparna 社とともに RiC-O Converter を開発し(第 1 版 2020 年 4 月)、パリの公証人役場 122 のうち 40 の記録から作った知識グラフ(実体どうしを関係で結んだネットワークとして作られたデータセットを、この分野ではこう呼ぶ)を、視覚的な検索インタフェース(Sparnatural)付きで公開している。EGAD の資源一覧はこのデータセットを 5,790 万トリプルと記載するが、うち約 3,700 万は推論で生成されたものである。ポータル FranceArchives は 2023 年に、SPARQL で問い合わせできる RDF リポジトリを公開した。2025 年 9 月の報道によれば、

10.1 万件の検索手段から生成した 5.04 億トリプル (RiC-O 0.2 準拠) を収めており、ポータルのプロジェクト責任者は「RiC-O 0.2 準拠のアーカイブズメタデータの RDF リポジトリとしては最大」と述べている^a。

スイスでは視聴覚遺産ポータル Memobase が RiC-O ベースのデータモデルで本番運用され、docuteam 社 (第5章で読んだ公式サンプルの作成元) の目録ソフトウェア docuteam context が RiC-O ネイティブの記述を支えている。舞台芸術の SAPA 財団の導入例もスイスである。欧州横断では、ホロコースト関係資料の研究基盤 EHRI が RiC-O を用いた試験的な知識グラフを構築した。このほかポルトガル (科学者書簡)、ギリシャ (裁判所文書)、韓国 (芸術記録)、中国 (配給切符資料のオントロジー拡張) など、各地の適用研究が続いている。これらを集めた一覧 RiC-ResourceList が、利用者コミュニティの手で作られ EGAD の管理下で公開されているので (付録 A.8 節)、最新の事例はそこで追える。

2026 年時点では、本番運用は少数の先行機関に集中しており、多くはまだ研究・実証の段階にある。

^a Kaelig Alléaume 「RiC (Records in Contexts) : un nouveau langage pour les archives à l'ère numérique」 Archimag, 2025 年 9 月 16 日。発言は FranceArchives のプロジェクト責任者 Manonmani Restif による。

7.3 シナリオ A の標準的な工程

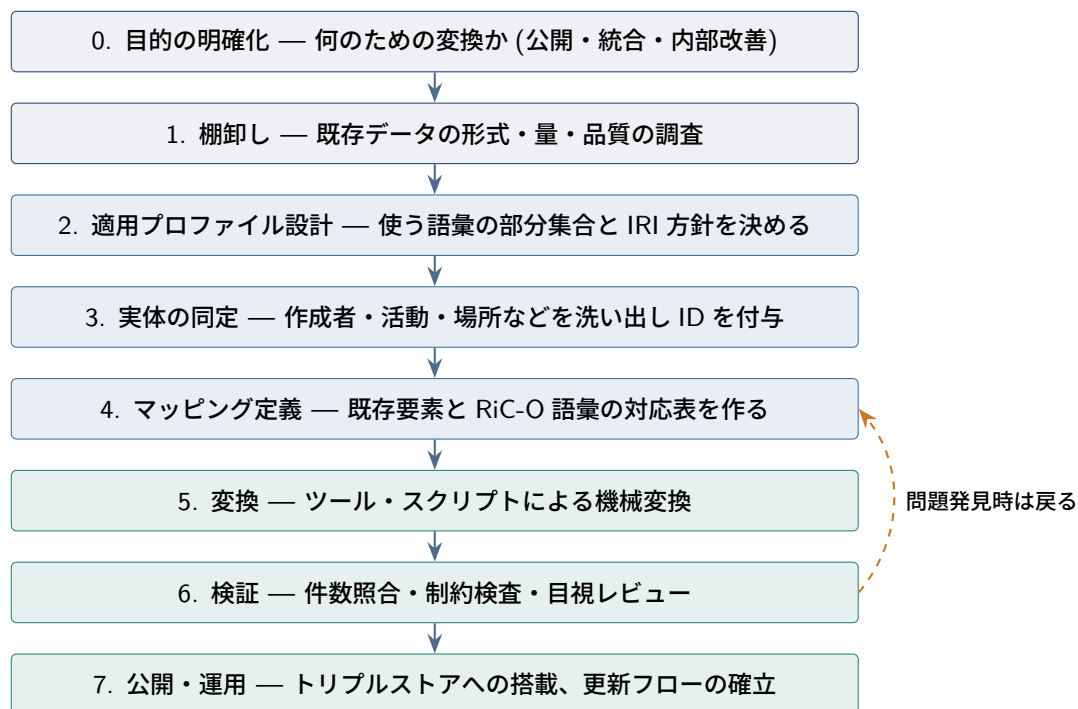


図 7.1 既存記述を RiC-O/RDF へ変換する標準的な工程。2～4 が知的な中心作業であり、アーキビストの判断が最も要る。

7.3.1 工程 2: 適用プロファイルの設計

RiC-O の 480 のオブジェクトプロパティと 75 のデータタイププロパティは、全部を使うことを想定していない。RiC-O 自身が「たいていの用途では、これらのプロパティの一部だけで足りるはずだ」と書き、実装機関は必要に応じた部分集合を使えばよいとしている。そこで最初に決めるべきは、自機関で使うクラス・プロパティの一覧 (適用プロファイル) である。決めることは主に次の 4 つである。

1. **使う実体の範囲**。最小構成なら Record Set/Record、Agent、Activity 程度から始められる。具現化を分けるか、規則・イベントをどこまで実体化するか。
2. **簡便法と精密法の使い分け** (第 5 章 5.5)。日付・種別・名称をリテラルで書くか実体化するか。本書が勧めるのは、外部と共有したい概念 (作成者、記録集合種別、場所) は実体化し、内部的な注記はリテラルにするという切り分けである。
3. **直接プロパティと関係クラスの使い分け** (第 5 章 5.5.3)。関係の日付や典拠を記録する必要がある関係 (出所など) だけ実体化する。
4. **IRI 設計**。ドメイン、パス構造 (/records/、/agents/ など)、変更しないという約束。一度公開した IRI は永続させるのが LOD の倫理である。

このプロファイルは文書化して機関の規則として維持する。RiC の用語で言えば、これは記述という活動を支配する Rule であり、それ自体が RiC で記述できる (RiC-CM 第 6 章)。

7.3.2 工程 4: マッピングの実際

この工程には公式の手がかりができた。RiC-AG 0.1 とともに、既存 4 標準 (ISAD(G)、ISAAR(CPF)、ISDF、ISDIAH) から RiC-CM への包括的なクロスワーク (対応表) と、EAD 2002 DTD から RiC-O 1.1 へのクロスワークが、表計算ファイルとして公開された (EAC-CPF 対応も準備中とされる)。AG は同時に、対応が一意でない場合があること (たとえば ISAD(G) の「日付」は複数の解釈がありうる)、機関の符号化慣行に合わせた調整が前提であることも明記している。次の表は、その要点を簡便法での一例として要約したものである。

表の「記述レベル」の行は、記録集合 (Record Set) について使う。記録集合種別はフォンド・シリーズ・ファイルなど、集合の種別を表すものだからである。記述レベルがアイテムの場合は、原則として記録 (Record) を立てる。ファイルも、それが最小の記述単位として使われてきた伝統では、記録に当てるほうがよいことがある (RiC-AG §5.3.1)。

表を見て気づくとおり、多くの要素には「散文のまま持っていく道」と「実体に分解する道」の両方がある。全部を一度に実体化する必要はない。まず散文のまま変換して器を移し、価値の高いところ (作成者、組織変遷) から段階的に実体化していく、という 2 段階構えが現実的であり、RiC-O が簡便法を残しているのはこの進め方を可能にするためである。

7.3.3 工程 5~6: 変換と検証

変換の道具としては、フランス国立公文書館が Sparna 社とともに開発したオープンソースの RiC-O Converter (EAD 2002 と EAC-CPF のファイルを RiC-O 準拠の RDF に変換する。2020 年に第 1 版

表 7.1 ISAD(G) 記述要素から RiC-O への対応の例 (簡便法での一例)

ISAD(G) 要素	RiC-O での主な対応
3.1.1 レファレンスコード	<code>rico:identifier</code> (または <code>rico:Identifier</code> 実体)
3.1.2 タイトル	<code>rico:title</code> / <code>rico:name</code>
3.1.3 日付	<code>rico:date</code> 系プロパティ (または <code>rico:Date</code> 実体 + 関係)
3.1.4 記述レベル	<code>rico:hasRecordSetType</code> (Fonds/Series/File 等の個体へ)
3.1.5 数量と媒体	具現化 (Instantiation) の側の <code>rico:instantiationExtent</code> ・ <code>rico:carrierExtent</code> など (公式クロスワークの対応)。記録資源の数量として持つなら <code>rico:recordResourceExtent</code>
3.2.1 作成者名	<code>rico:hasOrganicProvenance</code> (または広義の <code>rico:hasOrganicOrFunctionalProvenance</code>) で Agent 実体へ
3.2.2 管理歴・伝記歴	Agent 側の <code>rico:history</code> (散文) または Event 群
3.2.3 伝来	<code>rico:history</code> (または保管・移管の Event 群)
3.3.1 範囲と内容	<code>rico:scopeAndContent</code>
3.4.1 アクセス条件	<code>rico:conditionsOfAccess</code> (または <code>rico:Rule</code> 実体)
3.5.3 関連記述単位	各種の記録資源間関係プロパティ
階層 (上位・下位)	<code>rico:directlyIncludes</code> (逆は <code>rico:isDirectlyIncludedIn</code>)。上位の <code>rico:includesOrIncluded</code> で書くと推移の推論が働かない

公開) が出発点になる。ただし、EAD の使い方は機関ごとに癖があるため、既製の変換規則をそのまま使えることは少なく、自機関の符号化慣行に合わせた調整が普通である。自作するなら、XML を別の形式へ変換するための W3C 標準言語である XSLT か、Python を使うことになる。表計算やデータベースからの変換であれば、Python(rdfib ライブラリ) によるスクリプトが定番である。

検証は少なくとも 3 水準で行う。件数の照合 (元データの記述単位数・作成者数と、生成された個体数が合うか)、機械的な制約検査 (必須プロパティの欠落、IRI の重複。SHACL という制約記述言語が使える)、そして標本の目視 (変換結果を SPARQL で引き出し、原本の目録と突き合わせる) である。第 6 章で述べたとおり、推論は誤りも増幅するので、検証を省いた公開は危うい。

7.4 通し例: 目録と典拠レコードの連携は、RiC でどうなるか

移行の意味を最も実感しやすいのは、多くの機関がすでに実践している記述 (ISAD(G)/EAD の階層記述と、ISAAR(CPF)/EAC-CPF の典拠レコード) が、RiC でどう書き直されるかを、具体的なデータで見比べることだろう。第 6 章の衛生行政の例を素材に、従来形と RiC 形を並べてみる。順序は、まず階層 (記録の側の構造)、次に典拠との連携 (作成者の側) である。

7.4.1 従来形:2 種類の文書を ID 文字列でつなぐ

従来の実務では、記録の記述 (ISAD(G) に基づく検索手段、日本の実務でいえば目録。符号化は EAD) と、作成者の記述 (ISAAR(CPF) に基づく典拠レコード。符号化は EAC-CPF) を別々の文書として維持し、両者を識別子で結びつける。簡略化して示すところである。

```

<!-- 検索手段(EAD 2002)。ISAD(G) 3.1.1~3.1.4, 3.2.1, 3.2.2 に対応 -->
<archdesc level="fonds"><did>
  <unitid>P-83</unitid>
  <unittitle>衛生部文書</unittitle>
  <origination>
    <corpname authfilenumber="AR-0125">某県衛生部</corpname>
  </origination></did>
  <bioghist><p>衛生部は1948年に設置され、1994年の機構改革で
    保健福祉部に改組された。…</p></bioghist>
  <dsc> <!-- 下位の記述。入れ子の深さが階層を表す -->
    <c01 level="series"><did>
      <unitid>P-83/1</unitid><unittitle>庶務系列</unittitle></did>
      <c02 level="file"><did>
        <unitid>P-83/1/12</unitid>
        <unittitle>予防接種関係綴</unittitle>
        <unitdate>昭和23年度</unitdate></did></c02>
      </c01>
      <c01 level="series"><did>
        <unitid>P-83/2</unitid><unittitle>防疫統計系列</unittitle></did>
      </c01>
    </dsc>
  </archdesc>

```

多段階記述の階層(フォンド→シリーズ→ファイル)は、c01・c02 という要素の入れ子で表現されている。ここに注目してほしい。上位と下位の間には参照の記載がない。XML 文書の中での位置(どの要素の中にあるか)が、親子関係の表現を兼ねているのである。第2章2.4節の言葉で言えば、階層については「位置が参照を代行している」。これは紙の目録で、字下げや配列が構造を表していたことの直接の継承である。

```

<!-- 典拠レコード(EAC-CPF)。ISAAR(CPF) に対応 -->
<cpfDescription>
  <identity><entityType>corporateBody</entityType>
    <nameEntry><part>某県衛生部</part></nameEntry></identity>
  <description>
    <existDates><dateRange>
      <fromDate standardDate="1948">1948</fromDate>
      <toDate standardDate="1994">1994</toDate>
    </dateRange></existDates>
  </description>
  <relations>
    <cpfRelation cpfRelationType="temporal-later">
      <relationEntry>某県保健福祉部</relationEntry>
    </cpfRelation></relations>
</cpfDescription>

```

前の断片は EAD 2002、この断片は EAC-CPF 2010 の構文である。典拠レコードの識別子 AR-0125 は、省略した control/recordId に置く。identity/entityId は記述対象の団体自体の識別子であり、典拠レコードの識別子とは役割が異なる。

この二本立て自体が ISAAR(CPF) の成果であり、作成者情報を複数のフォンドで使い回せるようにした (第2章)。しかし連携の実態を見ると、次の4点が**ゆるい**。

- 検索手段から典拠への参照は `authfilenumber="AR-0125"` という **ID 文字列**であり、その ID が指す典拠レコードの実在も一意性も、機械的には保証されない。
- 「作成者である」という関係の性質 (作成したのか蓄積したのか、いつからいつまでか) は、`origination` という要素名と散文 (`bioghist`) に埋まっている。
- 後継組織への関係も、典拠側では `relationEntry` の中の**文字列**「某県保健福祉部」であって、相手の典拠レコードへの機械可読な参照ですらないことが多い。
- 同じ事実 (1948 年設置、1994 年改組) が検索手段の散文と典拠レコードの両方に書かれ、二重管理と不同期の温床になる。

参照の**向きが固定されている**ことも、この構造の帰結として押さえておきたい (図 7.2。参照手段の歴史的経緯は第2章 2.4 節)。文書ベースの標準では、関係はどちらかの文書のフィールドとして記録されるから、矢印の向きは「どの文書がそのフィールドを持つか」で決まってしまう。EAD の `origination` は検索手段→典拠の向きである。実は逆向きも用意されていて、EAC-CPF の `resourceRelation` を使えば典拠レコードから記録資源を参照できる。しかし両者は別々の文書の中の別々の記載であり、片方だけ書かれたり、内容が食い違ったりしても、それを止める仕組みはない。双方向にしたければ2回書き、両者が食い違わないようにするのは人手の仕事だった。

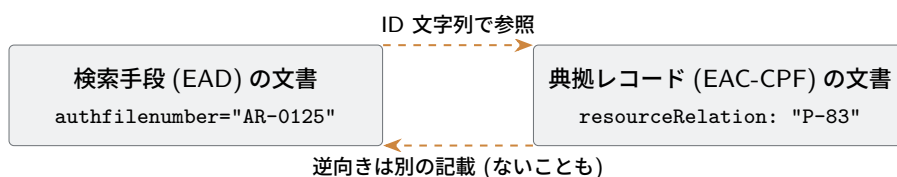
この設計は、関係データベースの定石とも比べられる (図 7.2(b))。RDB では「多数の記述が1つの典拠を指す」関係を、**多の側** (ここでは記述の側) のテーブルに外部キー列を置いて表す。向きは意味ではなく設計の定石で固定されるが、文書ベースと違って参照整合性制約があり、存在しない典拠を指す参照はデータベースが拒否してくれる。文書ベースの ID 参照は、「外部キーはあるが整合性制約のない」状態にあたる。

RiC ではこの事情がもう一段変わる (図 7.2(c))。関係はどちらの記述の所有物でもない、グラフの独立した辺 (あるいは関係実体) になり、向きは構造の制約ではなく名前の選び方 (`rico:hasCreator` か `rico:isCreatorOf` か) という記法上の問題に退く (第5章 5.1.2 項)。参照先は ID 文字列ではなく IRI で、名指しの範囲が世界規模へ広がる。ただし公平のために言えば、RDF は開世界仮説をとるため、RDB のような参照整合性の**自動的な強制**はない。指した先の実体が記述されていなくても誤りにはならない (「まだ書かれていないだけ」と解釈される)。存在検査が必要なら SHACL などで明示的に検査する (本章の工程 6)。つまり3つのアーキテクチャは、参照の**置き場所** (文書の中/多の側の列/独立した辺) と、参照の**保証** (なし/DB が強制/世界規模の名前 + 任意検査) の組み合わせとして整理できる。

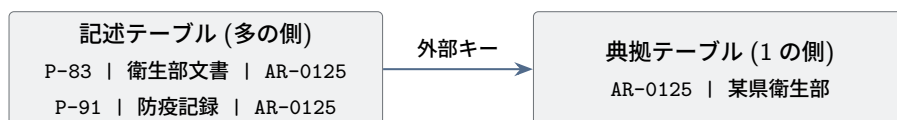
7.4.2 階層の移し方 — 入れ子から明示的な関係へ

まず記録の側の階層を RiC に移そう。原則は1つだけである。**入れ子で暗黙に表現されていた親子関係を、1本ずつ明示的な関係 (トリプル) にする**。この例の各記述単位は記録集合として独立し、その種別は `rico:hasRecordSetType` の値 (公式語彙の個体 `rst:Fonds`・`rst:Series`・`rst:File`) で表す。

(a) 文書ベース: 参照は文書の中の記載。2 本は独立で、同期の保証がない



(b) 関係データベース: 外部キーは多の側に置く。向きは定石で固定、整合性は DB が強制



(c) RiC(RDF): 関係は独立した辺。向きは名前の選び方、参照は IRI

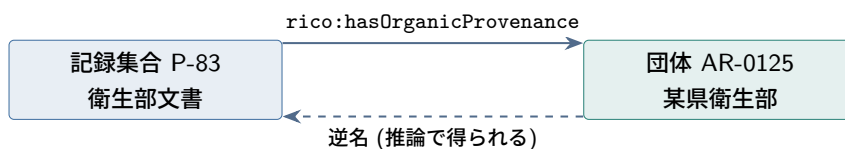


図 7.2 参照アーキテクチャの 3 段階。参照の置き場所は、文書の中の記載 (a) から、多の側の外部キー列 (b)、どちらにも属さない独立した辺 (c) へと移り、向きの固定は構造の制約から記法の選択に変わる。

```
ex:fondsP83 a rico:RecordSet ;
  rico:hasRecordSetType rst:Fonds ;
  rico:identifier "P-83" ;
  rico:name "衛生部文書" ;
  rico:directlyIncludes ex:s1 , ex:s2 . # 入れ子だったものを明示

ex:s1 a rico:RecordSet ;
  rico:hasRecordSetType rst:Series ;
  rico:identifier "P-83/1" ;
  rico:name "庶務系列" ;
  rico:directlyIncludes ex:f112 .

ex:s2 a rico:RecordSet ;
  rico:hasRecordSetType rst:Series ;
  rico:identifier "P-83/2" ;
  rico:name "防疫統計系列" .

ex:f112 a rico:RecordSet ; # 簿冊も「記録の集合」
  rico:hasRecordSetType rst:File ;
  rico:identifier "P-83/1/12" ;
  rico:name "予防接種関係綴" ;
  rico:isAssociatedWithDate [ a rico:Date ;
    rico:expressedDate "昭和23年度" ] .
```

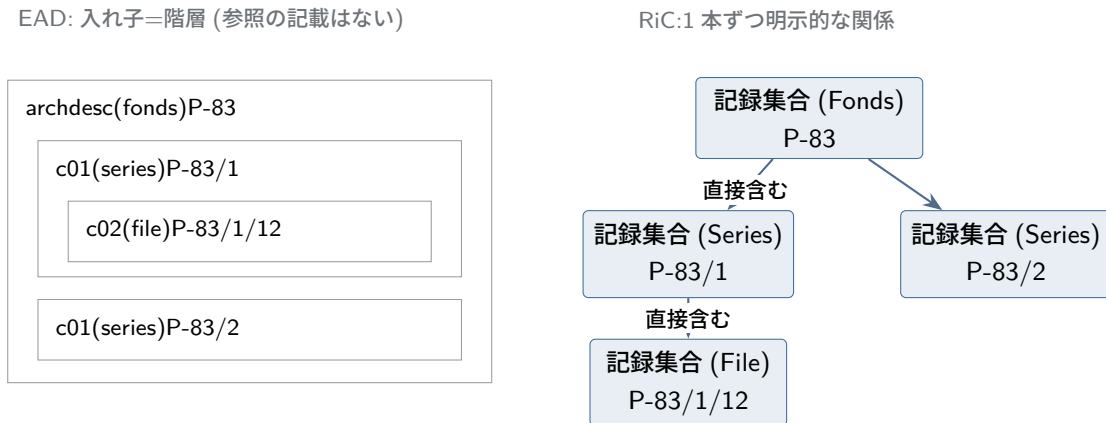


図 7.3 階層記述の移行。EAD では入れ子 (文書内の位置) が階層を表していたが、RiC では各記述単位が独立の実体になり、親子は `rico:directlyIncludes` という明示的な関係で 1 本ずつ主張される。

見た目には同じ木である。しかし性質は次の 3 つの点で変わっている。

- **下位の単位が独立の実体になった。**EAD の `c02` 要素はフォンドの文書の一部でしかなく、外から直接指すことができなかった (できても文書内 ID どまりである)。RiC では簿冊 `ex:f112` が自分の IRI を持ち、典拠からも、他機関のデータからも、利用者の引用からも直接指せる。
- **階層が検索可能なデータになった。**「このフォンドの下全簿冊」は、EAD では文書構造をたどるプログラムの仕事だったが、RiC では `rico:includesTransitive` の推論 (第 6 章場面 1) または SPARQL のプロパティパス 1 行で得られる。
- **単一の木という制約が外れた。**同じ簿冊を複数の集合 (元のシリーズと、たとえば「予防接種関係」という主題別コレクション) に属させることが、関係を 1 本足すだけで表せる (第 3 章の多次元記述)。

一方で、変わらないものもある。ISAD(G) の多段階記述の規則 (記述は全体から部分へ、情報は該当するレベルに置き、上位で述べたことを下位で繰り返さない) は、RiC でもそのまま良い設計である。各事実を 1 つの実体にだけ書くという原則は、むしろグラフでこそ徹底できる (上位の情報は関係をたどれば得られるので、繰り返す理由が消える)。RiC-CM が「ISAD(G) に基づく記述は RiC-CM に受け入れられる」と述べ (§1.6.2、§1.6.4)、階層記述そのものは退けないと断っている (§1.6.2.2) のは、この意味である。階層は否定されるのではなく、**数ある関係の 1 種類**になる。

7.4.3 典拠との連携:1 つのグラフに実体として置く

次に作成者の側である。同じ内容を RiC で書くと、検索手段と典拠レコードという 2 種類の文書の区別が消え、いま作った階層と同じ 1 つのグラフに、Agent 実体とその関係として合流する。

```

ex:fondsP83 a rico:RecordSet ;
  rico:hasRecordSetType rst:Fonds ;
  rico:identifier "P-83" ;
  rico:name "衛生部文書" ;
  rico:thingIsSourceOfRelation ex:prov83 .    # 出所関係を実体化

ex:prov83 a rico:OrganicProvenanceRelation ;
  rico:organicProvenanceRelation_role ex:prov83 ;    # 近道の推論用
  rico:relationHasTarget ex:eisei ;
  rico:relationHasDate [ a rico:Date ;
    rico:expressedDate "1948-1994" ] .

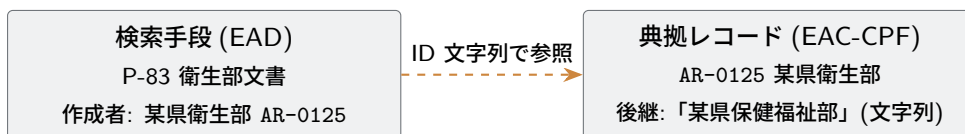
ex:eisei a rico:CorporateBody ;
  rico:identifier "AR-0125" ;
  rico:name "某県衛生部" ;
  rico:hasSuccessor ex:hokenfukushi .    # 後継も実体への参照

ex:hokenfukushi a rico:CorporateBody ;
  rico:name "某県保健福祉部" .

```

ex:prov83 の 2 行目は、関係ノードを自分自身に結ぶ補助の 1 本である。これを書いておくと、推論器は第 6 章 6.5 節の連鎖から、記録集合から団体への近道 rico:hasOrganicProvenance を導く。トリプルストアがよく使う OWL 2 RL の推論器は、この 1 本を自分では補わない。

従来形:2 種類の文書 + ゆるい参照



RiC 形:1 つのグラフ。参照はすべて IRI



図 7.4 目録と典拠の連携の before/after。従来 2 つの文書に分かれ ID 文字列でつながっていたものが、RiC では実体と関係の同じグラフに置かれる。

図 7.4 の変化を要素の単位で追ったのが表 7.2 である。従来形の各要素が、RiC 形のどのトリプルになったかを確かめられる。表は本書の作例での選択であり、AG の EAD 2002 → RiC-O 1.1 クロスウォークは unitid を具現化の識別子に、unittitle を rico:title に写すなど、別の選択も示している。

変わった点を数えあげると、次の 4 つになる。

- 参照がすべて **IRI による直接参照**になった。「AR-0125 という ID の典拠がどこかにあるはず」ではなく、ex:eisei という実体そのものを指す (この例では、従来の典拠 ID を移行先の団体の

表 7.2 通し例 P-83 の従来形の要素と、本書の RiC 形での書き方

従来形の要素	本書の RiC 形	備考
検索手段 (EAD 2002)		
archdesc・c01・c02 の level 属性	rico:hasRecordSetType(値は rst:Fonds など)	各記述単位が独立の記録集合になる
unitid	rico:identifier	
unittitle	rico:name	
unitdate	rico:isAssociatedWithDate と日付の実体 (rico:expressedDate)	
c01・c02 の入れ子	rico:directlyIncludes	親子を 1 本ずつ明示する
origination の corpname と authfilenumber	出所関係の実体 (rico:thingIsSourceOfRelation → rico:OrganicProvenanceRelation → rico:relationHasTarget)。近道は rico:hasOrganicProvenance	ID 文字列が IRI になる
bioghist	団体の側に 1 回だけ書く (散文なら rico:history)	AG のクロスワークも団体の rico:history に写す
典拠レコード (EAC-CPF 2010)		
control/recordId(AR-0125)	団体の rico:identifier	この例で採った方針
nameEntry/part	rico:name	
existDates	団体の日付 (本文の作例では省略)	
cpfRelation(temporal-later)	rico:hasSuccessor	文字列が実体への関係になる
resourceRelation	書かない (逆向きの rico:isOrganicProvenanceOf は推論で得られる)	2 回書かずに済む
control のその他の管理情報	記述の記述 (記述を記録として立てる)	RiC-CM 第 6 章

ローカル識別子として再利用する方針を採った)。

- 関係が**意味と期間を持つ**ようになった。「作成者」欄という書式上の位置ではなく、有機的出所という型の関係で、1948–1994 年という期間付きで主張される。
- 後継関係が文字列から実体間の関係 (rico:hasSuccessor) になり、第 6 章で見た変遷の推論・検索がそのまま使える。
- 二重管理が原理的に解消される。1948 年設置という事実は衛生部という実体に 1 回だけ書けばよく、検索手段風の表示も典拠レコード風の表示も、同じグラフからの**出力**として生成される (第 3 章の入力と出力の分離)。

言い換えれば、「典拠レコードの管理」という仕事は RiC では消えるのではなく、**Agent 実体とその関係の管理**へと昇格する。ISAAR(CPF) が定めた典拠の内容 (名称、存在日付、履歴、関連) は Agent の属性と関係にそのまま対応し (公式クロスワークが対応表を与える)、ISAAR の 4 種の関連 (階層的・時間的・家族・連想的) は、RiC の豊富な Agent 間関係 (上下、後継、合併・分割、家族、雇用…) へ精密化される。区分ごとの対応の例を表 7.3 に挙げる (団体の合併・分割を表す rico:wasMergedInto・rico:wasSplitInto のように、RiC-O にだけあって RiC-CM に対応する関係のないものもある)。ISAAR の管理情報 (コントロールエリア) だけは、Agent の属性には対応しない。典拠レコードそれ自体のメンテナンス情報は記述という活動の記録なので、記述を 1 つの記録 (Record) として立て、多くは記述の活動 (Activity) と結んで書く。これが AG の示す一案である (記述の記述。RiC-CM 第 6 章、RiC-AG §5.3.4)。

表 7.3 ISAAR(CPF) の関連の 4 区分と、対応する RiC-O のプロパティの例。AG の公式クロスウォーク (ISAAR(CPF) 5.3.1) が挙げる関係から採った。括弧内は RiC-CM の関係番号

ISAAR(CPF) の区分	RiC-O のプロパティ
階層的	rico:isOrWasControllerOf(R041)、rico:isOrWasLeaderOf(R042)、rico:hasOrHadSubordinate(R045)
時間的 家族	rico:precedesInTime(R009)、rico:hasSuccessor(R016) rico:hasDescendant(R017)、rico:hasChild(R018)、rico:hasFamilyAssociationWith(R047)、rico:hasSibling(R048)、rico:hasOrHadSpouse(R049)
連想的	rico:isAgentAssociatedWithAgent(R044)、rico:hasOrHadWorkRelationWith(R046)、rico:knows(R051)、rico:hasOrHadCorrespondent(R052)、rico:hasOrHadTeacher(R053)、rico:hasOrHadMember(R055)

7.4.4 混ぜって混沌としないか — 論理の統合と、管理の分割

ここで管理の見通しを立てておきたい。従来は、目録は目録、典拠は典拠として**独立**に管理できていた。別々の文書だから、担当も、更新のサイクルも、品質管理も別々に回せた。それが 1 つのグラフに合流したとき、この独立性がどうなるかは、実務の関心事である。

見通しを与えるのは、**統合されるのは論理 (意味のモデル) であって、管理の単位ではない**という区別である。RiC は「検索手段と典拠レコードは同じ世界の記述であり、意味の上では 1 つのグラフをなす」と言っているのであって、「1 つのファイル・1 人の担当・1 つの更新フローで管理せよ」とは言っていない。そして RDF の技術スタックには、管理を分割したまま論理を統合するための標準的な道具が、最初から用意されている。道具は 3 つある。名前付きグラフ、IRI による分散した参照、そして記述の記述である。

■1 つ目の道具: 名前付きグラフ トリプルストアは実際には、トリプルを「どのグラフに属するか」というラベル付きの 4 つ組で格納できる。このラベルで区切られたグラフを**名前付きグラフ (named graph)**という (SPARQL 1.0(2008 年) の RDF データセットとして標準化され、RDF 1.1(2014 年) でデータモデルの側にも取り込まれた。直列化形式は TriG)。これを使うと、従来の文書の区分けを、そのままグラフの区分けとして再現できる。次の例は TriG という形式で書いてある。Turtle の記述を GRAPH グラフ名 {…} で囲んだだけの形である。

```
GRAPH ex:agents {      # 典拠系: エージェント記述の担当が管理
  ex:eisei a rico:CorporateBody ;
    rico:name "某県衛生部" ;
    rico:hasSuccessor ex:hokenfukushi .
}
GRAPH ex:catalog {     # 目録系: 目録担当が管理
  ex:fondsP83 a rico:RecordSet ;
    rico:name "衛生部文書" ;
    rico:hasOrganicProvenance ex:eisei .      # グラフ境界を越えて参照
}
```

典拠系のグラフと目録系のグラフは、別々に更新し、別々にバックアップし、別々に品質検査できる

(SHACL の検査もグラフ単位でかけられる)。検索時には GRAPH や FROM で対象のグラフを指定するか、両者の和をデフォルトグラフとして扱うよう処理系を設定する。名前付きグラフが自動的に1つの検索対象になるわけではない。つまり従来の「文書の独立性」は失われるのではなく、**物理的な偶然** (たまたま別ファイルだった) から **意識的な設計** (どう分割して管理するか) へと性格を変える。境界を越える関係トリプル (上の `rico:hasOrganicProvenance`) をどこに置くかは、次に見るとおり、それ自体が設計問題である。

■**境界を越える関係はどこに置くか** 境界を越える**関係の記述**は、目録と典拠のどちらに属するのか、それとも独立なのか。論理のレベルでは答えがはっきりしている。**独立**である。関係は RiC の第一級の記述対象であり、実体化すれば両端を IRI で指す第三の実体になる (第5章 5.5.3 項)。関係がどちらかの記述の所有物でなくなったことこそ、7.4 節で見た従来形からの変更点だった。管理のレベルでは3つの置き方があり (図 7.5)、決め手は**その言明を誰が作り、誰が維持するか**である。

1. **目録グラフに置く**。整理の過程で目録担当が確定する関係 (このフォンドの出所は衛生部である、など) は、目録作業の産物だから目録側に置く。「多の側が指す」という外部キーの定石と一致し、EAD からの変換で生まれる関係も自然にここに落ちる。
2. **典拠グラフに置く**。両端とも Agent の関係 (`rico:hasSuccessor`、合併・分割) は典拠作業の産物だから典拠側である。ここは迷う余地がない。
3. **独立の関係グラフに置く**。関係が独自の調査・維持のサイクルを持つ場合である。先例は確立している。RDB では多対多の関係を、どちらのテーブルにも属さない**結合表** (第2章 2.4 節のコラム。中間テーブルとも呼ばれる) として独立させるのが定石であり、LOD には**リンクセット** (`linkset`。データセット自体を記述するための語彙 VoID が定める) という、2つのデータセットをつなぐリンクの集合を第三の管理単位として公開する作法が普及している。機関横断の同定リンク (`owl:sameAs`、7.6 節) も、この形で別に管理・公開されることがある。シリーズシステム (次項) の CRS も、実態は「シリーズと機関の関係の台帳」を中央で独立に管理する仕組みだった。

目録と典拠をまたぐ出所関係について言えば、第一候補は1の目録グラフである。多くの機関では出所の確定は整理・目録作成の仕事の一部であり、言明の責任者と置き場所が一致するからである。ただし、関係を実体化して期間・根拠・確実性を持たせ、出所調査として目録更新とは別のサイクルで育てていくなら、3へ昇格させるのが筋になる。なお直接プロパティ形 (1本のトリプル) はどこか1つのグラフに入れるほかないが、実体化形は関係実体とその属性の一式をまとめて移せるので、グラフを分けても壊れない。関係クラス (第5章 5.5.3 項) には、グラフ分割と相性が良いという利点もあるわけである。いずれにせよ、どの型の関係をどのグラフに置くかは、適用プロファイルに明記すべき規約事項である。

■**2つ目の道具:IRI による分散した参照** そもそも同じストアに同居させる必要すらない。IRI による参照は、参照先が同じデータベースにあることを要求しない。典拠系は国レベルの共同典拠サービスが、目録系は各文書館が、それぞれ別のサーバで管理・公開し、IRI で指し合う。これは妥協ではなく、LOD の設計思想そのものである (7.6 節)。SPARQL には複数のエンドポイントをまたいで問い合わせる仕組み (SERVICE) もある。

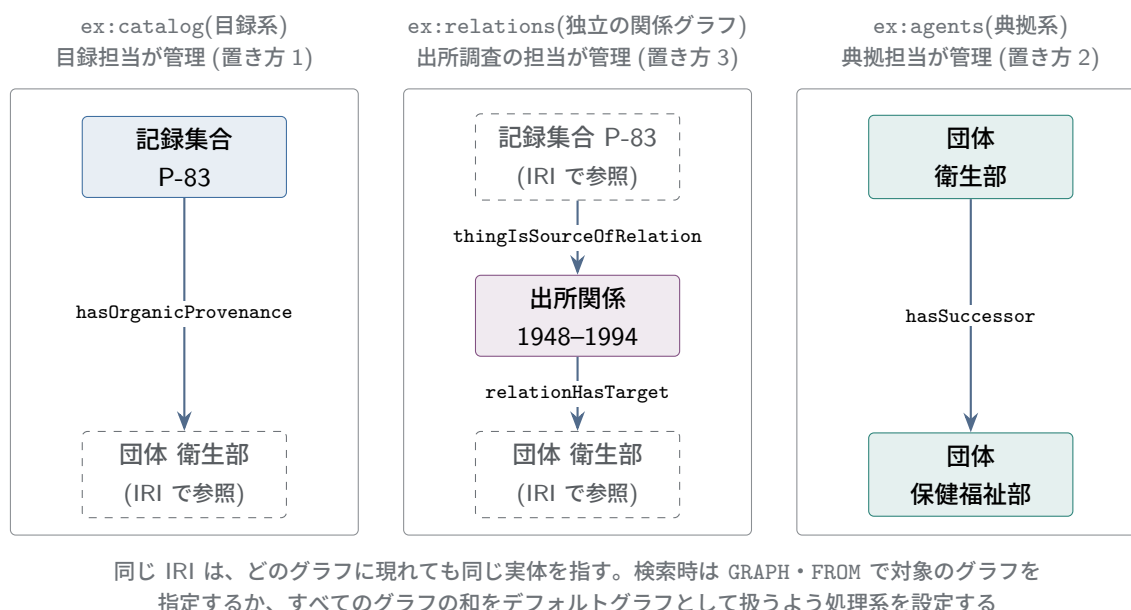


図 7.5 名前付きグラフで管理を分割した例。枠は管理の単位、矢印はトリプルで、各トリプルはそれが置かれたグラフの枠の中に描いた (破線の箱は、別のグラフにある実体を IRI で指しているもの)。出所を直接プロパティで書くなら目録グラフに (置き方 1)、関係実体として書くなら独立の関係グラフに (置き方 3) 置く。実際にはどちらかを選び、適用プロファイルに書いておく。

■**3 つ目の道具: 記述の記述** 「どの記述を、誰が、どの規則に基づいて維持しているか」自体を RiC で記録できる (記述の記述、RiC-CM 第 6 章)。ISAAR(CPF) のコントロールエリアが果たしていた管理情報の役割は、ここに引き継がれる。

■**限界** 限界も述べておく。名前付きグラフは格納と検索の仕組みであって、OWL の意味論の一部ではない。推論をどのグラフの範囲で走らせるかは処理系の設定であり (第 6 章 6.8 節)、書き込み権限や承認ワークフローに至っては標準の外で、機関の規程とシステムの実装が受け持つ。第 8 章 8.3.1 項の言葉で言えば、RiC が標準化するのは概念スキーマ (対象領域そのものの論理設計) までである。データをどの単位に分けて持つかには名前付きグラフという標準的な道具があるが、誰が書き換えてよいかといった運用の統制は運用の仕事である。まとめれば、**論理は標準が統合し、格納は名前付きグラフで分割でき、統制は運用が受け持つ**。混沌は必然ではない。ただ、従来は文書という物理単位が黙って与えてくれた独立性を、今度は自分で設計する必要がある。

7.4.5 シリーズシステムと RiC — 60 年越しの合流

この before/after を見て、オーストラリアのシリーズシステムを思い出した読者もいるだろう。1960 年代にピーター・スコットが提唱した方式で、fond という固定した頂点を立てることをやめ、記録はシリーズ単位で、組織は組織で独立に記述し、両者を期間付きの関係で結ぶ。組織改編のたびに記録群の帰属を書き換えるのではなく、関係を足していく。オーストラリア国立公文書館の CRS(Commonwealth Record Series) として半世紀以上運用されてきた。

シリーズシステムが 2 種類の記述 (シリーズ・機関) と期間付き関係で実現していたことを、RiC は

任意の実体と関係で書ける。逆に言えば、シリーズシステムの運用経験は、そのまま RiC の実践の型になる。整理すると次のとおりで、これは第6章の場面5(衛生行政) で実際に組んだ形でもある。

- シリーズを `rico:RecordSet` として**独立**に記述し、特定のフォンドの「中」に固定しない。
- 作成・蓄積の関係は実体化 (`rico:OrganicProvenanceRelation` 等) して**期間**を持たせ、担い手が交代したら関係を**追加**する (過去の関係は書き換えない。 `hasOrHad...` という命名もこの考え方による)。
- 組織の変遷は `rico:hasSuccessor`、合併・分割は `rico:wasMergedInto`・`rico:wasSplitInto` で機関側のグラフとして持つ。
- フォンドが必要ななら (利用者への提示や交換のために)、**二次的な集合**として後から構成する。多重所属が使えるので、構成をめぐる立場の違い (フォンド派とシリーズ派) は排他的でなくなる。

3つの方式の組み立て方を表7.4に並べる。

表7.4 フォンド型の記述 (ISAD(G))・シリーズシステム・RiC の組み立て方

	ISAD(G)(フォンド型)	シリーズシステム	RiC
記述の単位	フォンドを頂点とする階層の各レベル	シリーズと機関を、それぞれ独立に記述する	任意の実体を独立に記述する
フォンド	頂点に置く	立てない	必要なら二次的な集合として後から構成する
作成者との結び方	作成者の要素 (3.2.1) と典拠レコードへの参照	シリーズと機関のあいだの期間付きの関係	関係 (<code>rico:hasOrganicProvenance</code>)。期間を持たせるなら関係実体 (<code>rico:OrganicProvenanceRelation</code>)
担い手が交代したとき	作成者を並べ、経緯は管理歴 (3.2.2) の散文に書く	関係を足す	関係を足す (過去の関係は書き換えない)

フォンドを基礎とする ISAD(G) と、フォンドを立てないシリーズシステムは、長らく別の学派として扱われてきた。RiC のグラフはどちらも表現できる。折衷ではなく、同じ現実を別の仕方で切り取っていた両者を、1つのモデルで書けるということである。

7.5 ドメイン知識をどこに置くか — 3つの置き場所

適用プロファイルの設計で最も判断に迷うのは、おそらく自分の分野に固有の知識 (パフォーマンスアートの上演をめぐる概念、日本近世文書の文書型や村役人の役職、といったドメイン知識) を、RiC の記述のどこに持たせるかという点である。大きく言えば、既存の RiC-O 語彙の範囲で事実だけを記述する道と、RiC-O に自分の分野の語彙を追加してしまう道がある。この2つは見た目以上に性格が違うので、指針を示しておく。

実際には、置き場所は3つある。

置き場所 1: データの中 (既存語彙の範囲で書く) 分野固有の情報を、既存プロパティの**値**として書く。種別は `rico:type` に文字列で、説明は `rico:scopeAndContent` や `rico:generalDescription` の散文で、主題は `Thing` への主題関係で表す。RiC-O の規則には一切手を加えない。

置き場所 2: 統制語彙として分離する (SKOS) 分野固有の**概念の体系** (文書型の一覧、役職の一覧、演

目の一覧) を、RiC-O とは別の統制語彙 (SKOS) として整備し、RiC-O の種別クラス (rico:DocumentaryFormType など、名前が Type で終わるクラス) や主題関係から参照する。RiC-O の規則にはやはり手を加えない。RiC-AG の FAQ 「How to use RiC with vocabularies」が扱うのはこの道である。

置き場所 3: オントロジーの拡張 (下位クラス・下位プロパティ) 分野固有の**関係の意味**を、RiC-O の既存要素の下位として自分の名前空間に追加する (第 6 章 6.7 節)。RiC の規則体系そのものを (接ぎ木の形で) 広げる道である。

コラム:SKOS — 統制語彙を RDF で書くための標準

ここから SKOS という名前が繰り返し登場するので、まとめて説明しておく。SKOS(Simple Knowledge Organization System) は、シソーラス・件名標目表・分類表といった**統制語彙** (知識組織化体系) を RDF で表すための W3C 標準である (2009 年勧告)。オントロジー (OWL) と同じく RDF の上に載る語彙だが、役割が違う。

道具立ては少ない。語彙の 1 項目を skos:Concept(概念) という**個体**として IRI で立て、名前をラベルで付ける。skos:prefLabel が優先形 (1 言語につき 1 つ)、skos:altLabel が別名・別表記 (いくつでも) である。概念同士は skos:broader(上位の概念へ)・skos:narrower(下位へ)・skos:related(連想) で結び、意味の範囲は注記 skos:scopeNote で補う。語彙全体は skos:ConceptScheme という単位にまとめる。部品の全体を、小さな語彙の実例で示す (図 7.6 に同じ内容を図示する)。

```
vocab:kinsei a skos:ConceptScheme ;
  skos:prefLabel "近世村方文書型語彙"@ja .

vocab:register a skos:Concept ;
  skos:prefLabel "帳簿"@ja ;
  skos:inScheme vocab:kinsei .

vocab:shumon a skos:Concept , rico:DocumentaryFormType ; # 両方を宣言
  skos:prefLabel "宗門人別改帳"@ja ;
  skos:altLabel "人別帳"@ja , "宗門改帳"@ja ;
  skos:broader vocab:register ; # 上位概念へ
  skos:scopeNote "近世の戸口・宗旨調査の帳簿"@ja ;
  skos:inScheme vocab:kinsei .

vocab:kenchi a skos:Concept , rico:DocumentaryFormType ;
  skos:prefLabel "検地帳"@ja ;
  skos:broader vocab:register ;
  skos:inScheme vocab:kinsei .

# データの側は、種別として概念を参照するだけ(ex:r45 は後の通し例の記録)
ex:r45 rico:hasDocumentaryFormType vocab:shumon .
```

概念に rico:DocumentaryFormType を併記しているのは、rico:hasDocumentaryFormType の値域がそのクラスだからである。SKOS の概念であることと RiC の文書形式種別であることは

両立し、同じ IRI が 2 つの型を持つ。

OWL と混同しやすいのは**階層の意味**である。skos:broader は rdfs:subClassOf ではない。「宗門人別改帳 broader 帳簿」は主題の地図としての上下であって、論理的な包含(下位のインスタンスはすべて上位のインスタンスでもある)を主張しない。skos:Concept 自体はクラスだが、そのインスタンスとして記述した概念は、それだけでは OWL のクラスにはならない。rst:Fonds もこのような個体である(第5章)。この意味論の軽さは手抜きではなく設計方針である。軽いからこそ、図書館の件名標目表のような既存の統制語彙を、重い論理を持ち込まずにそのまま RDF 化して公開できる。

使い分けはこう覚えればよい。**世界の構造**(実体の種類と関係)は OWL で(それが RiC-O である)、**分野の概念の地図**(ものの分類・主題)は SKOS で。公開済みの SKOS 語彙も多く、国立国会図書館の件名標目(Web NDL Authorities)や Getty の AAT(芸術・建築シソーラス)は IRI で参照できる。自前の語彙を作る前に、既存の公開語彙で足りないかを確認するのが定石である。なお、OWL の語彙と SKOS の語彙を同じデータに混ぜて書ける理由(すべてが RDF のトリプルであり、語彙は同格だから)は、第5章 5.3 節で説明した。

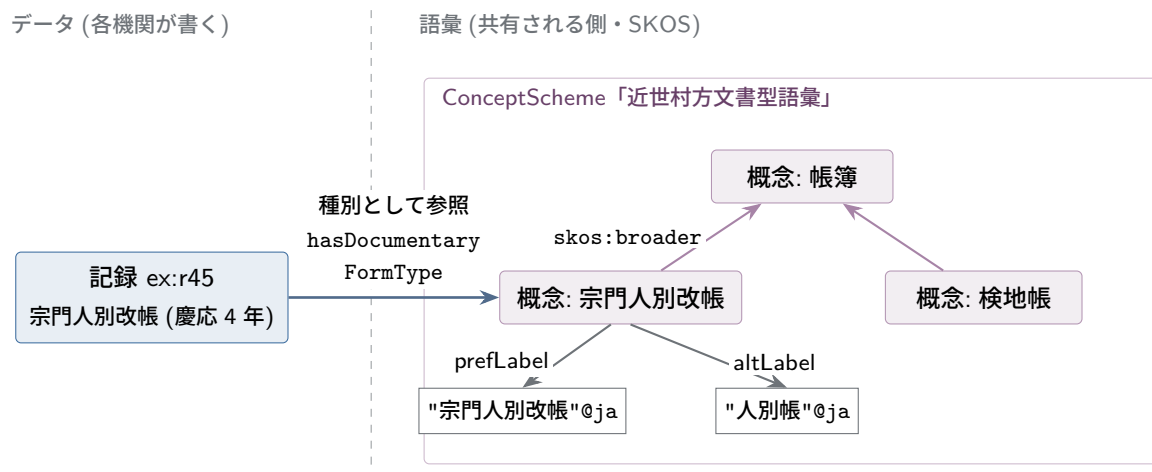


図 7.6 SKOS 語彙の構造。概念は IRI を持つ個体で、ラベル (prefLabel/altLabel) と概念間の上下 (broader) を持ち、語彙全体は ConceptScheme にまとまる。データの側は種別として概念を参照するだけでよく、表記ゆれは語彙側が吸収する。

7.5.1 使い分けの目安

判断の鍵は、その知識が「個別の事実」か「概念の体系」か「関係の種類」かである。

■**個別の事実なら、データに書く** 「この記録は 1868 年に作成された」「この上演は野外で行われた」は、特定のものについての事実であり、既存の属性・関係で書けばよい。和暦・元号はその典型で、rico:expressedDate(表記のままの日付「慶応 4 年」と rico:normalizedDateValue(正規化した日付) という既存の仕組みがそのまま使える。新しい規則は要らない。

■**概念の体系なら、統制語彙に分離する** 「宗門人別改帳」「検地帳」「触書」という文書型の一覧、「名主」「組頭」「百姓代」という村役人の体系、能・狂言の演目一覧。これらは個別の記録についての事実ではなく、分野で共有される**概念の地図**である。こうした知識は RiC-O の中に書き込むのではなく、SKOS 語彙として独立に作り、rico:DocumentaryFormType や rico:ActivityType のインスタンス、あるいは主題として参照するのがよい。分離しておく利点は共有にある。同じ「宗門人別改帳」語彙を複数の機関が参照すれば、機関を越えた横断検索が成立するし、語彙側に多言語ラベルや上位下位関係（「宗門人別改帳は帳簿の一種」）を持たせて、分野の知識を分野の専門家が育てていける。RiC-O 側は「種別がある」ことしか知らなくてよい。ドメイン知識の大半（ものの分類）は、実はこの道で足りる。

■**関係の種類なら、拡張を検討する** 既存の関係では意味が粗すぎて、検索や推論で使いたい区別が失われる場合に、はじめて拡張を考える。RiC-AG の公式例は公証文書の「遺言者 (has testator)」である。遺言状と人物を rico:hasAuthor(著者) で結ぶだけでは「遺言者を探す」検索ができないので、hasTestator を rico:hasAuthor の下位プロパティとして追加する。同型の判断で、宗門人別改帳の「寺請をした寺」(村方文書。rico:hasAuthor の下位)、「初演した/再演した」(パフォーマンスアート。rico:performsOrPerformed の下位)なども拡張の候補になる。ただし拡張には設計・文書化・維持の費用がかかり、RiC の改訂に追随する義務も生じる。第 6 章で述べた作法（既存要素への接ぎ木）を守れば、拡張を知らない他機関でも rico:hasAuthor の水準では検索できるが、拡張の意味そのものは共有した相手にしか伝わらない。だから拡張は「本当に関係の意味が要るとき」に限り、作ったら公開して共有可能にする、が原則である。

もっとも、この「公開して共有」を支える制度はまだない。RiC-AG 自身、拡張のレジストリ（登録簿）の維持が「将来は重要になるかもしれない」と述べるにとどまり、当面は RiC-ResourceList に載せてほしいという善意ベースの運用である。この問題は次節で正面から扱う。

7.5.2 通し例: 宗門人別改帳を 3 つの書き方で

抽象論では掴みにくいので、同じ資料を 3 つの置き場所で書き比べる。素材は「信濃国某村の宗門人別改帳。慶応 4 年、名主の甚兵衛が作成。各人の檀那寺が記され、寺の証印がある」という近世村方文書である。

■**置き場所 1 で書くと** まず、既存語彙の値としてだけ書いてみる。

```
ex:r45 a rico:Record ;
  rico:name "宗門人別改帳" ;
  rico:type "宗門人別改帳" ; # 文書型を文字列で
  rico:isAssociatedWithDate [ a rico:Date ;
    rico:expressedDate "慶応4年" ; # 和暦は表記のまま
    rico:normalizedDateValue "1868" ] ; # 正規化した年
  rico:hasCreator ex:jinkei .

ex:jinkei a rico:Person ;
  rico:name "甚兵衛" ;
  rico:generalDescription "某村名主(嘉永6年～明治2年)" . # 役職は散文
```

これで事実が残る。和暦が rico:expressedDate(表記のまま) と rico:normalizedDateValue(正

規化した値) という既存の仕組みで扱えているように、この水準で十分な部分も多い。しかし弱点も見える。「名主が作成した文書」は検索できない(その知識は散文の中に埋まっている)。文書型の検索は文字列一致だけなので、他機関が「宗門改帳」「人別帳」と書いていれば別物になる。

■置き場所2で書くと 文書型と役職を SKOS 語彙として別ファイルに切り出し、データからは参照する。文書型の概念は、コラムで定義した vocab:shumon(宗門人別改帳) をそのまま使う。

```
# --- 語彙(共有される側。分野の専門家が育てる) ---
# 文書型の vocab:shumon と vocab:register はコラムの例のとおり
vocab:nanushi a skos:Concept , rico:Type ;
    skos:prefLabel "名主"@ja ;
    skos:altLabel "庄屋"@ja ;      # 地域による呼び名も同じ概念に
    skos:scopeNote "近世村落の代表者。地域により庄屋・肝煎とも"@ja .

# --- データ(各機関が書く側) ---
ex:r45 rico:hasDocumentaryFormType vocab:shumon ;
    rico:hasAuthor ex:nanushi45 .      # 名主という職位が作成
ex:nanushi45 a rico:Position ; rico:name "某村 名主" ;
    rico:hasOrHadType vocab:nanushi .   # 職位の種別として概念を参照
ex:jinbei rico:occupiesOrOccupied ex:nanushi45 .
```

「名主(庄屋)」「宗門人別改帳(人別帳)」という分野の知識は語彙側に1回だけ書かれ、どの機関のデータからも参照できる。表記ゆれは語彙が吸収するので、「人別帳」で検索しても同じ概念に着地する。複数の機関が同じ語彙を参照した瞬間に、機関横断の検索が成立する。役職が Position 実体になったので、「甚兵衛が名主だった」ことも構造化された。RiC-O には職位の種別に専用のクラスがないので、職位から概念への参照には汎用の rico:hasOrHadType を使い、概念の側には rico:Type を併記した。

「名主として作成した」ことも、ここで書けている。帳面の作成者として職位を立て、rico:hasAuthor で結ばばよい。この関係の値域には人物・集団と並んで職位が含まれており、RiC-O の Position の注記も、職位にあった人々の活動から生じた記録集合を、人物を必ずしも特定せずに職位で示す使い方を挙げている。職位をたどれば甚兵衛にも行き着く。

■置き場所3で書くと ここまでで書けていないのは、この帳面に固有の関係である。宗門人別改帳には各人の檀那寺が記され、その寺の証印によってキリシタンでないことが証明されている(寺請)。寺は帳面の内容の一部に責任を持つので、既存の関係で最も近いのは rico:hasAuthor(内容に寄与した者。署名した者を含む) である。しかしそれだけでは名主と寺が同じ関係で並び、「寺請をした寺」を選んで引くことができない。RiC-O にも、作成の関係を関係実体(rico:CreationRelation) として立て、rico:creationWithRole で役割の種別(rico:RoleType) を添える書き方はある。ただしそれは関係ごとに役割を値として持たせる書き方で、1本の述語では引けない。この区別を1本の関係として検索や推論に使いたいなら、AG の遺言者の例と同じ形で拡張する。

```
# --- 拡張(1回だけ定義し、文書化して公開する) ---
myo:hasCertifyingTemple a owl:ObjectProperty ;
    rdfs:subPropertyOf rico:hasAuthor ;
    rdfs:domain rico:Record ;
    rdfs:range rico:CorporateBody ;
    rdfs:label "寺請をした寺"@ja ;
    rdfs:comment "記載された者の宗旨を証明して証印を押した檀那寺"@ja .

# --- データ ---
ex:tera45 a rico:CorporateBody ; rico:name "某寺" .
ex:r45 myo:hasCertifyingTemple ex:tera45 .
```

これで「寺請をした寺」が1つのプロパティで検索でき、しかも下位プロパティのロールアップ(第6章)により、この拡張を知らない相手にも `rico:hasAuthor`(さらにその上位の `rico:hasCreator`) の水準では見える。逆に言えば、ここまでの必要(既存の関係では粗すぎて困る具体的な検索・推論の要求)がないなら、置き場所2で止めるのが正しい。

7.5.3 迷ったときの順序

以上から、指針は次の順序になる。まず置き場所1で書けないか(たいてい書ける)。次に、それが分野で共有される概念なら置き場所2へ切り出す。関係の意味がどうしても必要なときだけ置き場所3へ進む。そして拡張する場合も、いきなり作る前に、`RiC-O` にすでにないか(`RiC-O` は `RiC-CM` より語彙が多い。置き場所2の「名主として作成した」のように、職位や役割の種別で書けることも多い)、`EGAD` に提案できないか(`FAQ` が推奨する)を確かめる。この順序は「最小の手段を選ぶ」原則であると同時に、「ドメイン知識はできるだけ `RiC-O` 本体から分離し、共有できる形に置く」原則でもある。`RiC` はアーカイブズ記述の**共通語**であって、各分野の百科事典ではない。分野の知識は語彙や拡張として分離して公開すれば他機関の資産になるが、散文やローカルな独自規則に埋め込めば、その機関の中に閉じる。

表 7.5 ドメイン知識の3つの置き場所

	1. データの中	2. 統制語彙 (SKOS)	3. 拡張
向く知識	個別の事実	概念の体系(分類)	関係の意味
<code>RiC-O</code> への変更	なし	なし(参照のみ)	接ぎ木で追加
例	和暦の日付、上演の場所	文書型・役職・演目の一覧	遺言者、寺請をした寺
費用	最小	語彙の整備・維持	設計・文書化・改訂追随
共有性	低い(散文は再利用不能)	高い(語彙自体が資産)	公開すれば共有可能

7.6 共有の基盤は誰が用意するのか — 識別子と拡張のインフラ問題

前節の「共有できる形に置く」という指針には、暗黙の前提がある。共有を受け止める**基盤**が存在する、という前提である。この前提は現在ほとんど満たされていない。本節はこの不都合な事実を確認し、

その上で、基盤なしで RiC-O が成り立つ範囲を示す。

7.6.1 何が欠けているか

第一に、拡張の共有は制度化されていない。前節で見たとおり、AG が推奨するのは「文書化して公開し、RiC-ResourceList に一項目を加える」ことまでであり、拡張の登録・審査・調停・版管理を担う仕組み（プログラミング言語のパッケージレジストリや、図書館界の語彙管理に相当するもの）は存在しない。同じ分野（たとえば日本近世文書）で2つの機関が別々に拡張を作れば、それらを突き合わせて統合する手続きはどこにもない。

第二に、より深刻なのは識別子の共有基盤である。機関横断の網という RiC の眼目は、「同じものには同じ IRI(または対応付けられた IRI) が使われる」ときに初めて成立する。ところが図書館界に VIAF(各国の著者典拠を束ねる国際基盤) や ISNI(著作物に関わる個人・団体に識別子を与える国際基盤。ISO 27729) があるのに対し、アーカイブズ界にこれへ相当するものはほぼない。作成者（エージェント）については、米国を中心とする共同典拠事業 SNAC、フランスの国立公文書館が公開するエージェント典拠データ、そして事実上のハブとして使われている Wikidata(誰でも編集・参照できる共有の構造化データ基盤) が部分的に機能しているが、網羅性も国際的な調整も VIAF の水準には遠い。さらにアーカイブズ固有の実体（活動、規則、職位、そして記録集合そのもの）に至っては、図書館典拠の守備範囲の外であり、共有識別子の仕組みは構想段階ですらない。各機関は自前のドメインで IRI を铸造するほかになく（7.3 節）、その結果、同一人物・同一組織が機関の数だけ別の IRI を持つことが常態になる。事後的に owl:sameAs や skos:exactMatch(語彙の概念どうしが一致すると宣言する) で対応付けることは技術的には可能だが、その照合（同定）作業こそが高コストで不確実な仕事であり、誰が担うのかは決まっていない。実体の種類ごとに、本書で触れた基盤をまとめると表 7.6 のようになる。

表 7.6 実体の種類ごとに見た共有識別子の基盤（2026 年。本章で触れたものに限る）

実体	国際的な基盤	国内	状況
人物・団体・家族	VIAF、ISNI(図書館界)。SNAC、フランス国立公文書館のエージェント典拠データ、Wikidata	Web NDL Authorities(ジャパンサーチも一部を独自に定義)	図書館典拠が中心。アーカイブズ側は部分的
場所	既存のジオ語彙・地名典拠	ジャパンサーチの場所の実体	部分的
時間（元号を含む）	—	ジャパンサーチの時代・元号の実体	国内に固有の資源がある
概念（種別・主題）	Getty の AAT などの公開 SKOS 語彙	国立国会図書館件名標目 (Web NDL Authorities)	公開語彙がある
記録集合・活動・規則・職位	なし	なし	共有の仕組みは構想段階ですらない

7.6.2 共有基盤なしで RiC-O はどこまで成り立つか

結論は「半分は成り立ち、半分は成り立たない」である。RiC-O が約束する価値を2つに分けると見通しがよい。

機関の内側で閉じる価値（記述の構造化、多次元の記述、入力と出力の分離、機関内での検索と推論）は、共有基盤なしで成立する。自機関の IRI だけで閉じたデータセットでも、第6章で見た推論の利益

はすべて得られる。現在の先行実装の多くも、価値の中心はこちらにある。外部の典拠への対応付けを張っている例もあるが、それは各実装が個別に張ったもので、共有の基盤に支えられたものではない。単独の機関にとって RiC-O を採用する理由は、共有基盤の有無とは独立に存在する。

機関を越えてつながる価値 (文脈の網が機関境界をまたいで伸びること、LOD としての相互接続) は、共有識別子か事後照合なしには成立しない。そして RiC の一次資料はこの問題を解いていない。IRI の設計と維持は各機関の責任とされ (LOD の一般原則の再確認にとどまる)、AG も「典拠レコードや統制語彙が整っている機関は有利だ」と述べるにとどまる。つまり RiC は**語彙の共通化** (同じ概念に同じ名前) までを提供し、**指示対象の共通化** (同じ人物に同じ識別子) は制度の側に残した。ここは標準の欠陥というより守備範囲の線引きだが、線の外側が空き地のままであることは事実である。

歴史的な先例はある。図書館界でも、目録規則と MARC の標準化が先にあり、VIAF のような国際基盤が実際に動き出すまでには数十年を要した。標準が先行し、インフラが後から (標準があるからこそ) 成立する、というのが通常の順序ではある。アーカイブズ界で同じ順序が再現される保証はないが、RiC という共通語彙の存在は、将来の「アーカイブズ版 VIAF」の前提条件を 1 つ満たしたとは言える。

7.6.3 当面の実務解

インフラの成立を待たずにできることは、方向としては 1 つに集約される。**自機関の IRI を鑄造しつつ、既存の外部識別子への対応付けを最初から記録しておく**ことである。エージェントには Wikidata・ISNI・VIAF・国内典拠 (国立国会図書館の Web NDL Authorities など) の ID があれば対応付けを張る。場所には既存のジオ語彙や地名典拠を参照する。対応付けは owl:sameAs 等のトリプル数本であり、将来どんな共有基盤が現れても、この対応付けが橋になる。ただし owl:sameAs は「同一のもの」という強い宣言なので、相手の IRI が典拠レコード (記述) ではなく人物・団体そのものを指しているかを確かめる (第 2 章 2.6 節)。逆に、対応付けのない孤立 IRI の集合は、基盤が現れたときに照合作業を丸ごと負うことになる。前節の表 7.5 の言葉で言えば、これは「共有性への投資を、インフラの成立に先行して行っておく」ことに当たる。

本書はこの識別子・拡張インフラの不在を、第 3 章で述べた表現の一意性の問題と並ぶ、RiC のもう 1 つの未解決の構造的リスクとして明記しておく。前者が「書き方が収束するか」の問題だとすれば、こちらは「書かれたものがつながるか」の問題である。どちらも標準文書の内側では解決せず、コミュニティと制度の側の仕事として残されている。

7.6.4 日本の基盤に照らすと

以上は国際的な状況だが、日本には日本の条件がある。本章の末尾で触れるとおり、AG の事例は英国・北欧・スペインの資料に偏っている。同じ偏りは、例を架空の県庁と近世村方文書だけで通し、ここまで国内の基盤に触れてこなかった本書にもある。確認できた範囲を書いておく。

ジャパンサーチは分野横断の統合ポータルで、連携機関から提供されたメタデータを共通形式 (JPS 利活用スキーマ) に変換し、RDF の LOD として公開している。2026 年 3 月現在、連携 310 データベースのうち 240 データベース由来の約 2,980 万件、約 16 億トリプルが変換済みである。SPARQL エンドポイントがあり、コンテンツごとの「記述情報」は <https://jpsearch.go.jp/data/> で始まる URI

を持ち、拡張子または**コンテンツネゴシエーション**^{*1}で JSON-LD・RDF/XML・Turtle が得られる。語彙はウェブページのマークアップに広く使われる汎用語彙 Schema.org を基本とし、「いつ・どこ・だれ・何の一部か・何と関連するか」の5つを独自語彙(接頭辞 jps:)の構造化記述プロパティが担う。時間・場所・名称の値は正規化されてすべて URI を与えられ、正規化できたものは LOD ハブ(ウェブ上で他のデータから多くリンクされている URI)にリンクされる。

RiC 側から見ると、この最後の点が使える。ただし内訳は分けて見たほうがよい。人や組織については、ジャパンサーチ自身が「国立国会図書館名称典拠にあるものはその URI を用い」、ないもののうち出現頻度の高いものだけを独自に実体として定義すると説明している。つまり人名・団体名の共有 IRI 基盤は Web NDL Authorities のほうであり(7.7 節でも触れる)、ジャパンサーチの独自性はむしろ、**時代・元号・場所を実体として正規化し、分野横断のコンテンツに一律に振っているところにある**。和暦の元号を含む時間の実体に IRI が振られているのは、近世・近代の記録を扱う立場からは見過ごせない。いずれにしても、7.6.3 項の「対応付けを最初から記録しておく」の相手として、Wikidata や VIAF と並べてよい候補である。

ただしジャパンサーチのデータモデルはコンテンツ 1 件を単位とした記述であり、RiC の実体と関係の網とは粒度も目的も違う。記録資源・エージェント・日付・場所・上位下位についてはおおよその対応づけを考えられるが、活動・規則・マンドート・職位に当たる受け皿はない。公開されている公式の対応表も見当たらない。

国立公文書館デジタルアーカイブについては、識別子の永続性という主題そのものが実例になっている。館は 2021 年の第 4 世代システムで、資料と画像に固有の URI を付与した。簿冊には file、件名・細目には item という表記を用い、ISAD(G) の記述レベルの表記に揃えたと説明されている。目録情報は CC0 で公開され、当時の報告では **OAI-PMH**^{*2}による一括提供に対応してジャパンサーチへの連携を週次で自動化したとされる。そしてその報告は、この URI を「あくまでアクセスのための識別子」と述べ、システム更新や仕様の都合で変わらないよう継続性への配慮が必要だとし、長期のアクセス保証には DOI のような恒久識別子の利用も考えられると書いていた。

その懸念は現実になった。2026 年 3 月 23 日、館はシステムを次の世代へ切り替えた(正式運用開始は 4 月 1 日。アジア歴史資料センターの資料提供システムも同時である)。館の案内は「新システムでは、各資料の目録詳細・ビューア画面へのリンクは、英数字の ID(URI) を利用した方式に統一されます。また、旧システムの一部の URL はご利用いただけません」と述べている。file/item という形式そのものは引き継がれ、システムを構築した事業者は、一部資料を除く全画像を IIIF に対応させたと発表している^{*3}(第 4 世代の試行運用から前進している)。それでも、システム更新のたびに一部の URL が失われるという事実は残る。永続性が方針として保証された識別子ではない、ということである。本節冒頭で見た識別子の問題が、国内でもそのまま現れている。

現在地は率直に書いておく。RiC-ResourceList を 2026 年 9 月に確かめたところ、全 52 件のうち日本に関わる項目は本書自身の登録 1 件だけで、国内機関による実装や実証の報告は載っていなかった(同じアジアでは韓国・中国・タイの事例が登録されている)。公開運用されている国内の RiC-O データ

^{*1} content negotiation。同じ URI に対して、要求側が希望する形式を伝え、対応する形式で返ってくる HTTP の仕組み。

^{*2} Open Archives Initiative Protocol for Metadata Harvesting。所蔵機関のメタデータを外部から一括して取り込むための、GLAM で広く使われる HTTP のプロトコル。

^{*3} NTT データ「国立公文書館の新デジタルアーカイブシステムに「AMLAD」を導入、本格運用開始」2026 年 4 月 28 日。

セットも確認できない。本書が架空の例で通しているのは、参照できる国内の実装がないという事情も反映している。

条件のほうは悪くない。分野横断の LOD 基盤があり、国立公文書館の目録情報は CC0 で機械可読に出ている。欠けているのは、記録集合・活動・規則といったアーカイブズ固有の実体に共有 IRI を与える仕組みと、それを運用として引き受ける機関である。

7.7 日本語データの実務 — 和暦・字体・多言語

一次資料の例はヨーロッパの資料が中心なので、日本語データに固有の論点をここでまとめておく。どれも RiC が新しく生んだ問題ではないが、RDF に載せるときの書き方には定石がある。

7.7.1 多言語・多表記の名称

RDF のリテラルには言語タグ (BCP 47) を付けられ、同じ述語に複数並べてよい。

```
ex:eisei rico:name "某県衛生部"@ja ,
               "Boken Eiseibu"@ja-Latn ,      # ローマ字翻字
               "ボウケンエイセイブ"@ja-Kana .  # 読み
```

日本の目録実務で必須級の「読み (ヨミ)」は、カタカナ表記に @ja-Kana タグを付けて併記するのが簡便法である。名称の種別 (本名・筆名・旧姓・翻字…) や使用期間まで管理したい場合は、名称そのものを実体化する精密法がある。RiC-O には rico:AgentName クラスが用意されている。

```
ex:saito a rico:Person ;
        rico:hasOrHadAgentName ex:saitoName1 .
ex:saitoName1 a rico:AgentName ;
        rico:textualValue "齋藤茂吉"@ja ;
        rico:type "本名" .
```

簡便法と精密法の使い分けの判断は、日付や種別のとき (第 5 章 5.5 節) と同じである。

7.7.2 旧字体・異体字

「齋藤/斎藤/斉藤」「廣島/広島」のような字体のゆれは、**文字列の統一で解くのではなく、IRI の同一性で解く**のが、RiC-O を含む LOD での定石である。実体は 1 つ (ex:saito) で、表記の違いは、1 つの実体に複数の名称を持たせること (上の rico:AgentName、または SKOS の altLabel) で吸収する。識別を文字列一致に頼らずに済むのは、参照を IRI 化したことの直接の見返りである (第 2 章 2.4 節)。そのうえで、データ投入時には Unicode 正規化 (NFC) を統一しておく。見た目が同じで符号位置が違う文字は、気づかれないまま検索の取りこぼしを生む。旧字と新字の検索時の同一視 (「広島」で「廣島」も見つける) は、データではなく索引・アプリケーション層の仕事である。

7.7.3 和暦の正規化

日付の二本立て (原表記 + 機械可読値。第 5 章) が最も活きるのが和暦である。rico:expressedDate には資料の表記をそのまま、rico:normalizedDateValue には機械可読値を書く。RiC-O 自身は後者

を「公的な技術標準に基づく機械可読な表記」と定義したうえで、スコープノートで ISO 8601 と、その拡張である **EDTF**(Extended Date/Time Format. ISO 8601-2 として標準化) を名指ししている。EDTF は目録実務で頻出する「あいまいな日付」を書ける。不確実は 1868?、およそは 1868~、年代までなら 186X、期間は 1948-04/1949-03(昭和 23 年度) である。

```
ex:f112 rico:isAssociatedWithDate [ a rico:Date ;
    rico:expressedDate      "昭和23年度" ;
    rico:normalizedDateValue "1948-04/1949-03" ] .

ex:r1 rico:isAssociatedWithDate [ a rico:Date ;
    rico:expressedDate      "慶応4年閏4月" ;
    rico:normalizedDateValue "1868-05/1868-06" ] .
```

換算には 2 つの注意がある。第一に、明治 5 年 12 月 2 日 (グレゴリオ暦 1872 年 12 月 31 日) 以前の和暦は太陰太陽暦 (天保暦など) なので、月日まで正規化するなら暦の換算が必要である (閏月はその典型で、上の慶応 4 年閏 4 月はグレゴリオ暦の 5 月下旬~6 月中旬にまたがる)。年単位の近似 (慶応 4 年 ≈ 1868) で足りる場面は多いが、旧暦 12 月は翌西暦年にかかることがある。第二に、改元は年の途中で起きる。大正以降は改元の日から新しい元号になり、たとえば昭和 64 年 1 月 7 日の翌日が平成元年 1 月 8 日である。明治改元は例外で、慶応 4 年 9 月 8 日に行われ、同年 1 月 1 日に遡って明治元年とされた。そのため同じ時期の日付が、慶応 4 年とも明治元年とも書かれる。原表記を保存してあれば、正規化の方針を後から変えることもできる。これが二本立ての利点である。表 7.7 に、原表記と正規化値の組の例をまとめる。

表 7.7 和暦・年度・あいまいな日付の原表記と正規化値の例

原表記 (rico: expressedDate)	正規化値 (rico: normalizedDateValue)	注意
昭和 23 年度	1948-04/1949-03	年度は 4 月から翌年 3 月までの期間
慶応 4 年閏 4 月	1868-05/1868-06	太陰太陽暦の閏月。グレゴリオ暦では 5 月下旬~6 月中旬
明治 5 年 12 月 2 日	1872-12-31	太陰太陽暦の最後の日。翌日が明治 6 年 1 月 1 日 (1873 年 1 月 1 日)
昭和 64 年 1 月 7 日	1989-01-07	翌日の 1989 年 1 月 8 日から平成元年
1868 年ごろ	1868~	EDTF の「およそ」
1868 年か	1868?	EDTF の「不確実」
1860 年代	186X	EDTF の「桁の不明」

7.7.4 外部典拠との接続

日本語の人物・団体なら、国立国会図書館の Web NDL Authorities が IRI 付きの典拠を提供しており、VIAF にも連携している。自機関の Agent 実体からこれらへ owl:sameAs で橋を架けるのが、7.6 節で述べた「当面の実務解」の日本語版である。Web NDL Authorities は、典拠データの URI(…/auth/ndlna/…) と、名称実体 (人物・団体そのもの) の URI(…/auth/entity/…) を分けて

いる。owl:sameAs で結ぶ相手は後者である。

7.8 個人情報とアクセス制限 — どこまで公開するか

RiC で記述が精密になるほど、個人に関する構造化データは増える。しかも本書で利点として紹介してきた仕組み (IRI による名寄せ、逆プロパティ、推移閉包) は、そのままプライバシーのリスク要因でもある。散らばっていたために事実上見えなかった情報が、推論と検索で集約されるようになるからである。LOD として公開したデータは複製・収集され、公開の取り消しは事実上できない。だから「どこまで書くか」と「どこまで公開するか」を、別の問いとして設計する必要がある。

道具立ては次の 4 つである。

- **記録への閲覧制限の記述**。簡便法は `rico:conditionsOfAccess` に散文で書くこと、精密法は利用規程そのものを `rico:Rule` 実体にして `rico:isOrWasRegulatedBy` で結ぶことである (規程が変われば 1 箇所の更新で済む)。
- **公開の制御はグラフの分割で行う** (7.4.4 項)。内部グラフと公開グラフを分け、公開グラフは内部データからの生成物として作る。すべてを 1 つに書いてから隠すのではなく、公開してよいものを選んで出す設計にする。制限の記述 (機械可読になった) と制限の執行 (アクセス制御の実装) は別物であり、後者は名前付きグラフ単位の権限管理などシステムの仕事である。
- **存命個人の实体は最小限に**。名称と役割程度にとどめる、あるいは個人を立てず職位 (`rico:Position`) までで止める、という選択がある (第 4 章 4.5 節の階層がここでも役立つ。なお記述者であるアーキビスト自身の作業履歴も個人データであり、同じ配慮の対象になる)。
- **削除の扱い**。内容を除き、削除済みであることだけを IRI の応答に残す方式 (tombstone) がある。ただし識別子自体が個人を特定する場合には、それを残してよいかも判断が要る。参照切れを避ける措置と個人情報の削除は別の問題であり、自機関から削除しても外部の複製まで消えるわけではない。

日本の文脈では、個人情報保護法・情報公開法・公文書管理法の下での利用制限区分を `rico:Rule` として機械可読に記述できることが、RiC の実務的な貢献になり得る。閲覧審査の根拠規程と対象記録の対応がグラフとして残れば、審査の一貫性の検証も、制限期限の到来の検出 (第 6 章の検索) も自動化の対象になる。

7.9 保存期間・評価選別・廃棄 — 記録管理の工程を記述する

ここまでの本章の記述は、すでにアーカイブズに入った記録を対象にしてきた。だが記録の一生は移管より前に始まる。保存期間の設定、評価選別、保存期間が満了したときの措置 (移管または廃棄) は現用段階の記録管理の中核であり、アーカイブズに何が届くかを決めているのはこの工程である。RiC-CM は主たる想定読者をアーカイブズ界に置きながら、レコードマネジメント界にも関心を持たれることを意図すると明言しており (§1.3)、AG は FAQ の 1 つ (§6d) で、デジタル化と並べてこの主題を扱っている。第 4 章 4.2 節で定義だけを見たマンドート (E17) が、ここで初めて働く。規則 (E16) のほうは第 4 章 4.5 節と本章 7.8 節ですでに使っている。保存期間を規則として書く型には、原典の側にも先例がある。RiC-CM は記述の記述の章で、「記録管理方針」という規則が「保存期間表」という記録に表現

される図を挙げている (§6.2 図 4)。

7.9.1 工程は活動として立てる

AG が示す骨格は単純で、拍子抜けするほど汎用的である。工程を 1 つの**活動** (Activity) として立て、周囲を 4 本の関係で結ぶ。工程が影響を及ぼす対象は `rico:affectsOrAffected`(逆向きは `rico:isOrWasAffectedBy`)、決定や成果物は `rico:resultsOrResultedIn`、遂行主体は `rico:isOrWasPerformedBy`、時期は `rico:occurredAtDate` である。1 本目の値域は「もの」一般なので、対象は記録資源でも具現化でもよい(この点は後で働く)。もう 1 つ、個別の活動を一般的な活動の下位イベント (`rico:isOrWasSubeventOf`) として置く手もある。AG は評価選別の例でこれを取り、そうすることで規則を介して、評価選別の結果を登録簿に記載するという行為へ橋を架けられると述べている。AG はこれを、活動種別を与える書き方の**代わり**として示している。

同じ型がデジタル化にも評価選別にも使え、「他の多くの種類の工程にも一般化する」と AG は言う (§6d.1)。逆に言えば、RiC には評価選別のための専用の道具立てがない。42 の属性に保存期間に当たるものはなく、廃棄については日付を結ぶ `rico:hasDestructionDate`(および日付実体を立てない場合の簡便版 `rico:destructionDate`) があるだけである。工程の記述は、汎用の活動の型を組み合わせで自分で作ることになる。RiC-O が統制語彙を同梱しているのも記録集合種別と文書形式種別の 2 つだけである。活動種別 (`rico:hasActivityType` の値) の統制値は RiC の側から与えられないので、既存の標準語彙を使うか、自分で用意することになる。

7.9.2 ベルギーの例 — 登録簿の記載を廃棄の根拠と見る

AG §6d.3 が挙げるのは、RiC のメーリングリストで議論された、ベルギーのフランドル地域の記録管理機関の例である^{*4}。ある機関がシリーズ R を評価選別し、その結果として、たとえば 10 年後にシリーズ内のファイル F を廃棄するという決定が出る。R は政府の登録簿に記載があり、評価選別の結果 (F の廃棄を含む) を登録簿に記載することが法で義務づけられている。この構造を図 7.7 に示す。

AG 自身が「要点」として挙げるのは、評価選別を活動として定式化し、前項の関係で囲むという、デジタル化と「完全に同型」(entirely analogously) の書き方である。ただし AG がここで名前を挙げるのは 3 本で、うち 2 本が逆向きの語形である (`rico:isOrWasAffectedBy` と `rico:performsOrPerformed`。 `rico:resultsOrResultedIn` は前項の列挙と同じ向きである)。日付は列挙から落ちているが、図には現れる。向きは逆プロパティなのでどちらでもよい。そのうえで AG は、登録簿の記載を「廃棄のマンデートを与えるものと**見なすことができる**」と、留保つきで述べている。本書としてはこの後者に注目したい。廃棄してよいという権限が、評価選別の結果が法定の登録簿に記載されることに結びつけられている。この構造は、日本の制度と比べるときの足がかりになる。なお AG は、この事例では評価選別を行った機関がそのまま廃棄も実施する見込みであると述べている。AG の図でも、評価選別・記載・廃棄の 3 つの活動をすべて同じ機関が遂行している(本書の図では、評価選別以外の 2 本を省いた)。

図をよく見ると、前項で触れた「規則を介した橋」の中身が分かる。橋になっている規則は**記載義務**

^{*4} フランドル (Flanders, オランダ語で *Vlaanderen*) はベルギー北部のオランダ語圏の地域で、独自の議会と政府を持つ。以下に出てくる登録簿への記載義務について、AG は「法で義務づけられている」と書くだけで、根拠となる法令名を示していない。

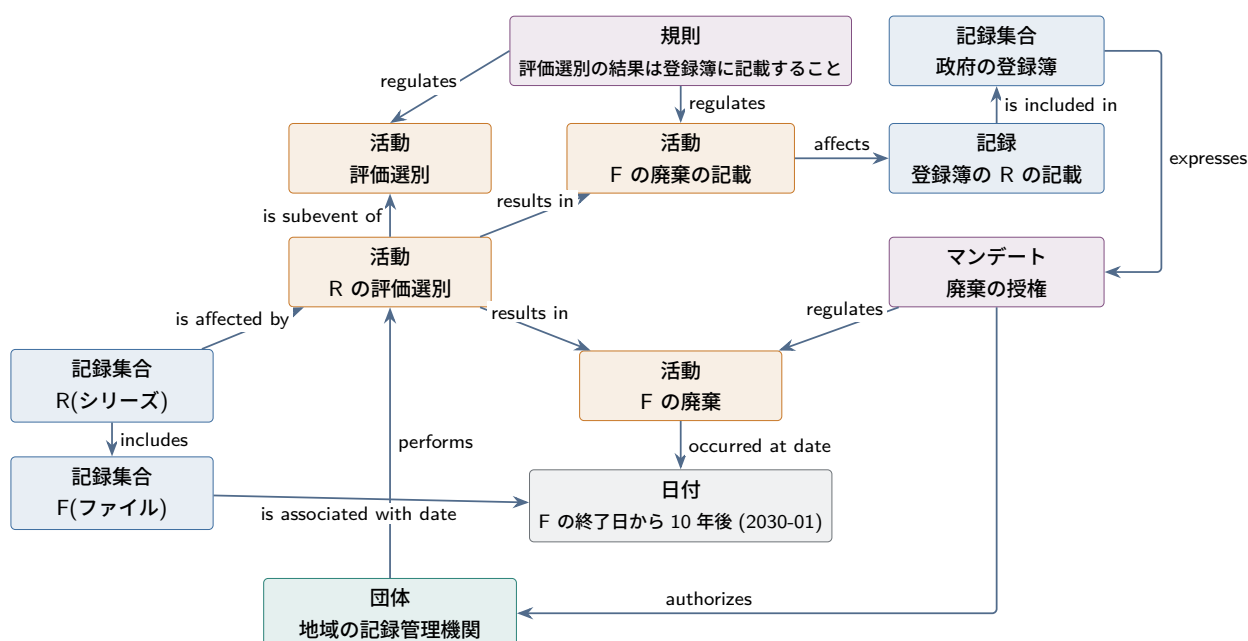


図 7.7 ベルギーの例の構造 (AG §6d.3 の図を本書の記法に移したもの)。実体の型と属性 (記録集合種別、正規化した日付値)、ファイル F の終了日、記録管理機関を管理者とする関係、および記録管理機関が「F の廃棄の記載」と「F の廃棄」も遂行するという 2 本の関係は、繁雑になるので省いた。廃棄の権限は、登録簿がマンドートを表現し、そのマンドートが廃棄という活動を支配し、かつ機関に権限を与える、という三段で書かれている。AG の本文は登録簿の記載 (register log) を廃棄のマンドートを与えるものと見なせると書くが、図では「表現する」の矢印は登録簿 (記録集合) から出ている。本書の図は AG の図に従った。

そのものである。「評価選別の結果は登録簿に記載しなければならない」という法の要求を 1 つの規則として立て、それが「評価選別」という一般の活動と「F の廃棄の記載」という活動の両方を支配すると書かれている。個別の評価選別を一般の評価選別の下位イベントに置いたのは、この規則に届かせるためだったのである。そして権限の側は三段になっている。図では登録簿 (記録集合) がマンドートを表現し、そのマンドートが F の廃棄という活動を支配し、あわせて記録管理機関に権限を与える。マンドートが行き先として持つのは記録ではなく活動とエージェントであって、この点は次項の作例で内閣総理大臣の同意を書くときにそのまま使える。

AG は登録簿を記録集合、R についての記載を記録としてモデル化している。この文脈では関心が R の側にあるからで、登録簿を「その地域における評価選別の記録」として主題にするなら記録集合と記録の切り方は変わる (AG §6b が長く論じているのはこの種の判断である)。

技術的な注記が 2 つある。rico:hasDestructionDate は rico:hasEndDate の下位プロパティで、定義域は記録資源と具現化、値域は日付である。AG は、RiC-O を使うのなら R068 「日付として関連する」で書くよりも「少し精密」だと述べている。R068 は日付を定義域にとる向きで、RiC-O のプロパティ名では rico:isDateAssociatedWith にあたる。その逆 (もの→日付) が rico:isAssociatedWithDate(R068i) で、本書の作例はこちらを使う。図のほうは汎用の日付関係で書かれており、rico:hasDestructionDate は注記での提案にとどまる。なお AG の図では、この矢印のラベルが R068 の語形なのに矢印は記録集合から日付へ向いている。ラベルと向きが食い違うため、本書の図では R068i の語形に直した。

`rico:hasDestructionDate` は RiC-O 1.0 で追加されたもので、RiC-CM に対応する要素はない。前項で「専用の道具立てがない」と書いたが、廃棄の日付だけはオントロジーの側が先に細かくなっている。同じ考えで、次項の作例では保存期間の満了する日を `rico:hasEndDate(rico:isAssociatedWithDate` の下位で、`rico:hasDestructionDate` の親) で書いた。使えるうちでいちばん狭い関係を選ぶ、というのが原則である。

技術的な注記の2つ目は、**将来の日付を書いてよい**ということである。一部の関係名が過去形を併記するのは RiC の命名の癖であって制約ではなく、AG はこの語形が将来の版で変わり得ることと、RiC-O 1.1 でその一步が入っていることを述べている。

7.9.3 日本の制度に置く

日本の国の行政機関では、この構造がさらに明示的である。公文書管理法 (公文書等の管理に関する法律、平成 21 年法律第 66 号) の骨組みは次のようになっている。

職員が行政文書を作成し、または取得したときは、行政機関の長が、分類し名称を付し、保存期間および保存期間の満了する日を設定しなければならない (第 5 条第 1 項)。作成・取得するのは職員、設定するのは長である。単独で管理することが適当なものを除き、相互に密接な関連を有する行政文書 (保存期間を同じくすることが適当なものに限る) は行政文書ファイルにまとめなければならない (第 5 条第 2 項)、そのファイルについても同じ設定を行う (第 5 条第 3 項)。設定した保存期間と満了する日は延長できる (第 5 条第 4 項)。

そして満了前のできる限り早い時期に、満了したときの措置 (歴史公文書等に該当するものは政令の定めにより国立公文書館等へ移管、それ以外は廃棄) を定めなければならない (第 5 条第 5 項)。これらの事項、すなわち分類・名称・保存期間・満了する日・満了したときの措置・保存場所その他必要な事項は、情報公開法第 5 条の不開示情報に該当するものを除いて、行政文書ファイル管理簿に記載しなければならない (第 7 条第 1 項)。管理簿は事務所に備えて一般の閲覧に供するとともに、情報通信技術を利用する方法で公表される (第 7 条第 2 項)。

保存期間が満了したものは第 5 条第 5 項の定めに基づいて移管または廃棄され (第 8 条第 1 項)、廃棄しようとするときはあらかじめ内閣総理大臣に協議してその同意を得なければならない。同意が得られなければ、新たに保存期間と満了する日を設定することになる (第 8 条第 2 項)。

条文ごとの工程と主体を、本節の作例での書き方と並べると表 7.8 のようになる (作例は後に示す)。

ベルギーの例との違いが 2 つある。措置を定めるのは第 5 条第 5 項の「定め」であり、管理簿の記載 (第 7 条) はその公表の側にある。そしてもう 1 つ、廃棄には内閣総理大臣の同意という手続が挟まる。ただしここは慎重に見たい。移管または廃棄の義務そのものは第 8 条第 1 項が行政機関の長に直接課しているもので、廃棄の権限の源泉は法律であって内閣総理大臣ではない。同意は権限の委任ではなく、消極的な統制である。マンドートの定義、すなわち「あるエージェントが別のエージェントに、活動を遂行する責任または権限を委任すること」に厳密に当てはまるかは、議論の余地がある。それでも、「別のエージェントの行為なしにこの活動は遂行できない」という関係を明示的に持つ点で、マンドートとして書く実益はベルギーの例の登録簿の記載より大きい。以下の作例ではそう書く。第 4 章でマンドートの定義だけを見たときに実感の湧かなかったものが、ここで書く形をとる。

この構造は法の他の箇所にも現れる。国立公文書館等の長が、特定歴史公文書等として保存している文書を歴史資料として重要でなくなったと認めて廃棄する場合も、内閣総理大臣への協議と同意が要る

表 7.8 公文書管理法の工程と、本節の作例での書き方

条文	工程	主体	作例での書き方
第 5 条第 1 項・第 3 項	分類・名称・保存期間・満了する日の設定	行政機関の長	規則 <code>ex:retentionA12</code> がファイルを <code>rico:regulatesOrRegulated</code> で支配し、満了する日を <code>rico:hasEndDate</code> で持つ 記録集合 <code>ex:fileA12</code> (種別は <code>rst:File</code>)
第 5 条第 2 項	行政文書ファイルへのまとめ	長	
第 5 条第 4 項	保存期間・満了する日の延長	長	作例では扱わない
第 5 条第 5 項	満了したときの措置の定め	長	活動 <code>ex:appraisalA12</code> が <code>rico:resultsOrResultedIn</code> で規則 <code>ex:dispositionA12</code> を生む 記録 <code>ex:registerEntryA12</code> を管理簿 <code>ex:fileRegister</code> に含め、 <code>rico:expressesOrExpressed</code> で 2 つの規則を表現させる
第 7 条第 1 項・第 2 項	管理簿への記載と公表	長	
第 8 条第 1 項・第 2 項	移管または廃棄。廃棄には内閣総理大臣の同意が要る	長 (同意は内閣総理大臣)	活動 <code>ex:destroyA12</code> 。同意はマンドート <code>ex:consentA12(rico:authorizes・rico:regulatesOrRegulated)</code> 。ファイルに <code>rico:hasDestructionDate</code>

(第 25 条)。

適用範囲には注意が必要である。この法律が直接の義務を課すのは、国の行政機関 (第 2 章)、独立行政法人等 (第 3 章)、国立公文書館等 (第 4 章) である。第 8 条第 2 項の協議・同意については会計検査院が除かれている (同項ほか、条文が指定する数箇所と同じ)。地方公共団体は、法の趣旨にのっとって必要な施策を策定し実施する努力義務を負うとされ (第 34 条)、対応する仕組みは各団体の条例と文書管理規程の側にある。本書が通し例に使ってきた某県もそちらに属するので、以下の作例は国の行政機関を想定して書く。

作例の日付は制度に合わせてある。保存期間の起算日は、施行令が原則とする「ファイル作成日の属する年度の翌年度の 4 月 1 日」(施行令第 8 条第 7 項) に置いた。令和 8 年度に作成したファイルなら、起算日は 2027 年 4 月 1 日、保存期間 10 年の満了する日は 2037 年 3 月 31 日になる。保存期間の値と満了時の措置は、行政文書の管理に関するガイドライン (平成 23 年 4 月 1 日内閣総理大臣決定、令和 7 年 2 月 14 日一部改正) の別表から採った。保存期間は別表第 1 (行政文書の保存期間基準)、満了時の措置は別表第 2 (保存期間満了時の措置の設定基準) である。作例の 10 年は 13 の項 (1) 「職員の任免、進退、身分、賞罰又は恩給及び給与に関する文書」で、措置は廃棄。サンプルデータ (7.12 節) に置いた比較用の 2 件は、30 年が 1 の項「法律の制定又は改廃及びその経緯」で措置は移管、3 年が 26 の項「情報の収集及び分析その他の調査の結果に関する文書」で措置は廃棄である。移管先が国立公文書館になるのは施行令第 10 条本文の帰結である (宮内庁長官と外務大臣には別の定めがある)。

この別表は、令和 9 年 4 月 1 日の改正施行に向けて大幅な見直しが進んでいる。別表第 1 と別表第 2 を統合し、移管と廃棄を区別して書く形に変わる予定である^{*5}。作例の項番号は現行版のものなので、

^{*5} 国立公文書館統括公文書専門官室評価選別担当「行政文書の管理に関するガイドライン別表の見直しに伴う移管基準の変更について」『アーカイブズ』99 号、2026 年 2 月。見直しの動機が、行政文書のメタデータによるライフサイクル情報の管理 (起算日・保存期間・満了時の措置を機械可読に持ち、処理を自動化すること) にあると述べられている点は、本書の主題と直接に重なる。

実務では必ず現行の別表を確認すること。

```
# 保存期間そのものは記録の属性ではなく、記録を支配する「規則」
ex:retentionA12 a rico:Rule ;
    rico:name "行政文書ファイル A-12 の保存期間(10年)" ;
    rico:regulatesOrRegulated ex:fileA12 ;
    rico:hasEndDate [ a rico:Date ; # 満了する日
        rico:expressedDate "令和19年3月31日" ;
        rico:normalizedDateValue "2037-03-31" ] .

# 満了時の措置を決める活動(法第5条第5項)
ex:appraisalA12 a rico:Activity ;
    rico:name "A-12 の保存期間満了時の措置の決定" ;
    rico:isOrWasSubeventOf ex:appraisalGeneral ; # 一般の「評価選別」
    rico:hasActivityType ex:atAppraisal ;
    rico:isOrWasPerformedBy ex:ministry ;
    rico:occurredAtDate [ a rico:Date ;
        rico:normalizedDateValue "2027-04-01" ] ;
    rico:affectsOrAffected ex:fileA12 ;
    rico:resultsOrResultedIn ex:dispositionA12 .
```

活動が生んだ「定め」は規則であり、それが支配するのは記録ではなく将来の廃棄という活動である。管理簿の記載はその定めを**表現する記録**として、rico:expressesOrExpressed で結ぶ。第4章で注意したとおり、規則とそれを証拠立てる文書は別の実体だからである。

```
ex:dispositionA12 a rico:Rule ;
    rico:name "A-12 は保存期間満了時に廃棄する" ;
    rico:issuedBy ex:ministry ;
    rico:regulatesOrRegulated ex:destroyA12 .
```

管理簿そのものは記録集合、個々のファイルについての記載はその中の記録である。1つの記載が、保存期間の規則と措置の定めを表現する。この切り方はAGのベルギーの例と同じで、関心が個々のファイルの側にあるからこうなる。ただしAGの図では「表現する」の矢印を登録簿(記録集合)の側から出していた。ここでは記載ごとに表現する規則が違うので、記載(記録)の側から出す。「某省における文書管理の記録」を主題にするなら、管理簿の側を主役にした別の書き方になる。

なお、保存期間が政令で定める期間(1年)未満のファイルは管理簿への記載義務の対象外である(第7条第1項ただし書き、施行令第12条)。管理簿の記載を措置の根拠の在りかとして書くモデルは、記載義務のあるファイルについてのものである。

```
# 行政文書ファイル管理簿(法第7条)
ex:fileRegister a rico:RecordSet ;
    rico:name "行政文書ファイル管理簿(某省)" ;
    rico:directlyIncludes ex:registerEntryA12 .

ex:registerEntryA12 a rico:Record ;
    rico:name "A-12 の記載" ;
    rico:expressesOrExpressed ex:retentionA12 , ex:dispositionA12 .
```

残るのは廃棄そのものである。廃棄はまだ起きていないが、活動として立てておく。そこに内閣総理大

臣の同意を結びつけ、規則の下位実体であるマンドートとして書く。同意の相手方は `rico:authorizes` で指す。ただしこのプロパティの定義は「マンドートが行為する権限を与えるエージェント」であって、さきほど述べたとおり、この同意は権限の付与そのものではない。定義から少しはみ出した使い方をしているという自覚は要る。同意を書かず `rico:regulatesOrRegulated` だけで済ませる選択もあり、そのほうが安全ではある。

```
# 廃棄という(まだ起きていない)活動と、その権限
ex:destroyA12 a rico:Activity ;
    rico:name "A-12 の廃棄" ;
    rico:isOrWasPerformedBy ex:ministry ;
    rico:affectsOrAffected ex:fileA12 .

ex:consentA12 a rico:Mandate ;           # 規則の下位実体
    rico:name "廃棄についての内閣総理大臣の同意(法第8条第2項)" ;
    rico:issuedBy ex:primeMinister ;
    rico:authorizes ex:ministry ;
    rico:regulatesOrRegulated ex:destroyA12 .

ex:fileA12 a rico:RecordSet ;
    rico:hasRecordSetType rst:File ;
    rico:identifier "A-12" ;
    rico:hasDestructionDate [ a rico:Date ;           # 将来の日付でよい
                                rico:normalizedDateValue "2037-04" ] .
```

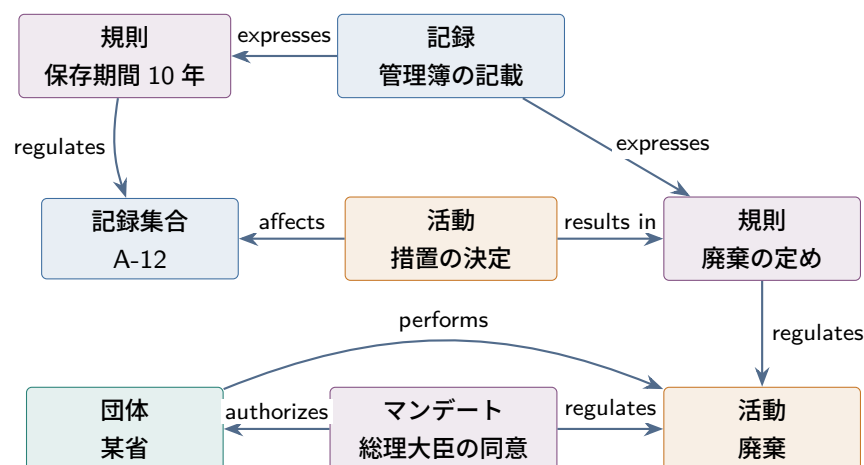


図 7.8 保存期間と廃棄の構造。保存期間も廃棄の定めも記録の属性ではなく規則である。廃棄という活動には、内閣総理大臣の同意をマンドートとして結びつけた（同意は権限の委任そのものではないので、定義から少しはみ出した使い方である。本文参照）。管理簿の記載は規則を表現する記録として置かれる。

このグラフに何ができるようになるかは、第 6 章の言葉で言える。保存期間満了の検出そのものは推論ではなく、日付を比較する問い合わせである（6.8.3 項の FILTER）。だが「満了したもののうち、移管の定めがあるものと廃棄の定めがあるものを分けて出す」「同意が未取得の廃棄予定を洗い出す」「ある規則が支配している記録を全部挙げる」といった問いは、措置が散文の欄ではなく実体と関係になっていれば、そのまま SPARQL で書ける。組織の階層をまたいで集計するなら、`rico:hasDirectSubdivision`

で書いた下部組織の関係から、推移的な `rico:hasSubdivisionTransitive` を推論で得られる (第6章場面1の `rico:directlyIncludes` と同じ仕組み)。逆に、これらが管理簿の自由記述欄にとどまっているかぎり、目で数えるしかない。この3つの問い合わせは、本書のサイトで配布しているサンプルデータに実行できる形で入れてある (7.12 節)。

訳語と制度の距離

RiC の `appraisal` は日本のアーカイブズ学で「評価選別」と訳される。`disposition` は難しい。公文書管理法はこれに当たる概念を「保存期間が満了したときの措置」と表現し、その内容を移管または廃棄としている。`disposition` を「処分」と訳すと行政法の処分 (行政処分) と衝突し、「廃棄」と訳すと移管の選択肢が落ちる。本節では制度側の言い方に寄せた。

AG の例がベルギーの一地域の法制度を前提にしていることも忘れないほうがよい。「登録簿の記載を廃棄のマネートと見なす」という読みは、記載が法定義務である制度でこそ成り立つ。制度が違えば、どの文書がどの権限を生むかは変わる。モデリングの型は共有できるが、型に入れる中身は制度ごとに読み直す必要がある。

7.9.4 デジタル化も同じ型で書ける

AG §6d が評価選別と並べてデジタル化を扱うのは、両者が同じ型で書けることを示すためである。ただしデジタル化には固有の論点が1つある。デジタル化は記録資源に対する活動ではなく、**具現化に対する活動**として書く。理由は AG §6d.2 の注記が述べている (背景の議論は §6c.2 にある)。記録資源の知的な側面 (それが保持する情報) は、デジタル化によって本質的に変わらない。デジタル化は、その情報の1つの記録を別の記録に移す作業である。より広く使えるようになる、あるいはより堅牢に保存されるという効果はあるが、それは情報の具体的な記録の使われ方の話であって、情報そのものは影響を受けない。第3章で記録と具現化を区別した理由が、ここで実務上の帰結を持つ。

AG の例にはもう1つ読みどころがある。活動種別 (`rico:hasActivityType` の値) としては「スキャン」1つを与えたうえで、「デジタル化」と「スキャン」を**活動そのもの**としても別に立てているのである。活動種別は分類のための属性で、RiC-CM は繰り返し不可と定めている (RiC-A02)。AG によれば、これは使えるうちでいちばん狭い分類を選ぶことを期待した定めであり、AG の図もそうしている。一方、デジタル化とスキャンの両方を活動として立てておくと、「一方が他方の特殊な場合である」という含みが書けるうえに、個別のデジタル化をずっと広い文脈に接続できる。AG がより重要だと言うのはこちらである。関係も属性も、デジタル化とスキャンのどちらの側からでも伸ばせるようになるからである。分類と個体を書き分ける、という RiC の作法の一例である。

```
ex:digitization2024 a rico:Activity ;
    rico:name "宗門人別改帳のスキャン" ;
    rico:isOrWasPerformedBy ex:archives ;
    rico:occurredAtDate [ a rico:Date ;
        rico:normalizedDateValue "2024" ] ;
    rico:affectsOrAffected ex:paperInst ; # 対象は原本という具現化
    rico:resultsOrResultedIn ex:digitalInst .
```

記録の側には、2つの具現化への関係が並ぶだけである。デジタル化の前後で、記録それ自体について

て書いたことは1文字も変わらない。

```
ex:r45 a rico:Record ;                                # 記録それ自体は変わらない
rico:hasOrHadAnalogueInstantiation ex:paperInst ;
rico:hasOrHadDigitalInstantiation   ex:digitalInst .
```

図 7.9 に、この2つの listing を合わせた形を示す。活動が当たるのは原本という具現化で、その結果として別の具現化が生まれる。

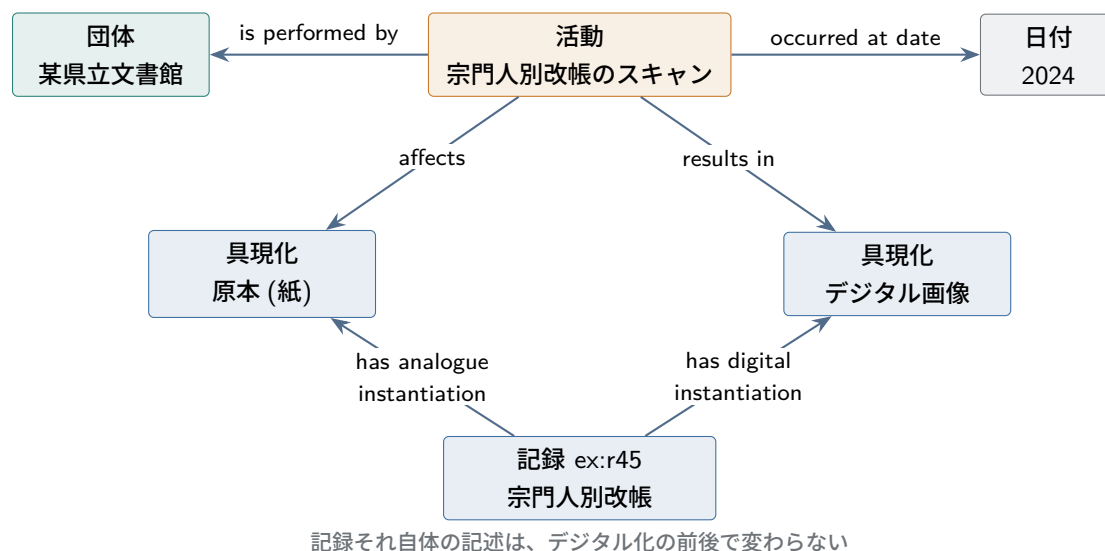


図 7.9 デジタル化は具現化に対する活動として書く (AG §6d.2 の型)。活動は原本という具現化に影響し、デジタルの具現化を生む。記録の側は具現化への関係が増えるだけである。

`rico:hasOrHadAnalogueInstantiation` と `rico:hasOrHadDigitalInstantiation` は `rico:hasOrHadInstantiation` の下位プロパティで、AG はこれらが表現種別 (Representation Type) 属性とうまく補い合うと述べている。下位プロパティなので、`rico:hasOrHadInstantiation` で検索する側は書き分けを気にしなくてよい (第 6 章 6.4 節のロールアップ)。

もっとも、デジタル化の結果が新しい記録になる場合もある。AG が挙げるのは、デジタル化した具現化が注記を加えられる形式であり、実際に注記が加えられた場合である。その判断の目安は §6c.3 が論じ、§6d.2 はその場合の図を別に示している。「デジタル化は具現化への活動」は既定の読みであって、例外がないという主張ではない。

7.10 シナリオ B: 新規記述のワークフロー

RiC ネイティブの記述では、作業の単位が「1つの検索手段を書き上げる」ことから「実体を登録し、関係を張る」ことに変わる。受入 (アクセッション) から公開までを実体の言葉で言い直すと、次のようになる。

1. **受入の時点**で、移管元の団体 (Agent)、移管という活動 (Activity/Event)、受け入れた記録群 (Record Set) を登録し、関係で結ぶ。この時点では記録集合1個と関係数本の粗い記述でよい。
2. **整理の過程**で、編成 (シリーズ、ファイル) を記録集合の入れ子として作り、内容から判明した作成者・活動・場所を実体として追加し、関係を張る。既存の典拠 (他機関・国レベルの典拠、

Wikidata 等) に同じ実体があれば、自前で作らず参照するか、対応関係を張る。

3. **記述の各時点**で、誰がいつ何を根拠に記述したか (記述の記述。RiC-CM 第6章) を、システムが自動記録するのが理想である。
4. **公開**は、蓄積された実体と関係のネットワークからの出力生成である。従来型の検索手段の体裁も、この出力の1つとして生成できる。

最初の一步の水準も AG の Getting Started が示している。RiC に必須項目はないが、ほとんどの実体は名称と識別子を持つと役立つこと、ISAD(G) が必須とした6要素 (レファレンスコード・タイトル・作成者・日付・数量・記述レベル) が RiC でも出発点として適切なこと、範囲内容や歴史のような散文はそのまま持ち込んだ上で、主題やイベントを構造化した関係として追加していけばよいことである。英国国立公文書館の実目録を RiC-CM に写す通し例が、使った実体・属性・関係の一覧つきで掲載されており、雛形として使える。

重要な発想の転換は、**記述が「完成」しなくなる**ことである。検索手段という完成品の代わりに、常に成長し改訂されるネットワークがあり、出力はその時点のスナップショットになる。版管理と改訂履歴 (それ自体が記述の記述である) の重要性が増す。

7.11 道具箱

2026 年時点で実務に使える主な道具を挙げる (EGAD が公開し、利用者が項目を登録していく一覧 RiC-ResourceList も参照)。

- **オントロジーの閲覧**: RiC-O 公式の HTML ドキュメントと CSV 一覧。本格的に読むなら Protégé (スタンフォード大学の無償オントロジーエディタ) で RDF ファイルを開くと、階層や公理を対話的にたどれる。
- **変換**: RiC-O Converter (EAD/EAC-CPF から。フランス国立公文書館の3万件超の EAD 検索手段を RiC-O 1.1 データセットに変換した実績を持つ 3.0.0 が公開されている)、rdflib (Python)、XSLT。マッピングの定義には RiC-AG 付属の公式クロスワークが出発点になる。
- **格納と検索**: トリプルストア。GraphDB (商用、無償版あり、推論に強い)、Apache Jena Fuseki (オープンソース) が定番である。いずれも SPARQL エンドポイントを提供する。
- **検証**: SHACL 処理系 (GraphDB 内蔵、pySHACL など)。
- **可視化**: RiC 専用の可視化ツールはまだなく、RDF 汎用の道具を使い分ける。最新のツール状況は RiC-ResourceList で確認できる。
 - 対象がオントロジー自体なら、WebVOWL (OWL ファイルを読み込むとクラスとプロパティの図を描くウェブツール) や Protégé の可視化プラグインで RiC-O の構造を眺められる。
 - 対象がデータなら、GraphDB の Visual graph 機能 (SPARQL 結果や近傍をインタラクティブなグラフとして表示)、LodView・LodLive のような Linked Data ブラウザ (IRI をたどりながら近傍を可視化) が手軽である。
 - 検索インターフェースとしては、Sparnatural (SPARQL を書かずに視覚的に問い合わせを組み立てるツール) を RiC-O データに載せた例がフランス国立公文書館関係のデモで公開されている。

－ 1 つ注意として、アーカイブズのグラフは巨大になるため「全体を 1 枚に描く」ことは実用にならない。起点を決めて近傍を数ホップ表示する使い方 (本書の図がしているのもこれである) が現実的である。

- **記述システム**: docuteam context (スイス。RiC-O をメタデータ標準に据えたアーカイブズ情報システム) など、対応製品が少しずつ現れている。既存の主要目録システムの代表である AtoM の RiC 対応について、RiC-AG の FAQ は率直な見通しを述べている。ソフトウェア開発は EGAD の直接の守備範囲外であること、AtoM の基盤であるリレーショナルデータベースは RDF/OWL 系のデータ形式と相性の課題があること、そして本格的なアプリケーションよりも API (プログラムから機能やデータを呼び出すための窓口) 群の整備のほうが現在のオープンソース生態系では実り多いかもしれないこと、である。導入判断の際には RiC-ResourceList で最新状況の確認が必要である。

コラム: ツールが揃うまで — Excel で RiC を書く

現場の実感として、こう思った読者は多いはずである。ISAD(G) の目録は、専用システムがなくても Excel で書けた。1 行が 1 記述単位、列が記述要素、そして階層は「親 ID」の列に上位のレファレンスコードを書けば表現できた。RiC 対応のツールが揃わない現状でも、同じように手を動かす方法はある。

従来の台帳は、たとえばこういう 1 枚の表である。

レファレンスコード	タイトル	作成年代	記述レベル	親のレファレンスコード
P-83	衛生部文書	1948–1994	フォンド	
P-83/1	庶務系列	1948–1994	シリーズ	P-83
P-83/1/12	予防接種関係綴	昭和 23 年度	ファイル	P-83/1
P-83/2	防疫統計系列	1948–1994	シリーズ	P-83

これで目録が書けた理由を確かめておく。木構造では根以外の各点の親が**ちょうど 1 つ**だから、つながりの記述が「親のレファレンスコード」という **1 列**で済んだのである。行が実体、列が属性、参照は ID。第 2 章の言葉で言えば、現場の Excel 台帳はすでに立派な「表形式のグラフ記述」だった。RiC で変わるのは関係の側である。関係が多対多・型付き・期間付きになり、**1 列に収まらなくなる**。

答えは、列を増やすことではなく、**シートを増やす**ことである。第一に、実体の種類ごとに 1 シートを立てる (記録集合シート、エージェントシート、場所シート…。) 行は実体 1 つ、先頭に必ず ID 列を置く。第二に、関係を独立の**関係シート**に出す。行は関係 1 本で、「源 ID・関係の型・先 ID・開始・終了・根拠」の列を持つ。

記録集合シート

ID	種別	名称	直接含まれる先
P-83	Fonds	衛生部文書	
P-83/1	Series	庶務系列	P-83
P-83/1/12	File	予防接種関係綴	P-83/1

エージェントシート

ID	クラス	名称
AR-0125	CorporateBody	某県衛生部
AR-0230	CorporateBody	某県保健福祉部

関係シート

源 ID	関係の型	先 ID	開始	終了	根拠
P-83	hasOrganicProvenance	AR-0125	1948	1994	整理時の調査
AR-0125	hasSuccessor	AR-0230			1994 年の機構改革

多対多は行が増えるだけで書ける。そして関係シートの形をよく見てほしい。源・型・先という3列の**積み上げ型**であり、第5章のコラムで見たトリプルの表そのものである。気づいた読者もあるだろうが、これは RDB の結合表であり、RiC の関係実体 (第5章 5.5.3 項) であり、7.4.4 項の「独立の関係グラフ」の表計算版である (親が1つに決まっている素直な階層だけは、上のように実体シートの1列に残す簡便法で足りる)。変換のときは、開始・終了や根拠の欄が埋まった行を関係クラス (出所なら `rico:OrganicProvenanceRelation`) に展開し、それらが空の行は直接プロパティのトリプル1本にすればよい。しかも多くの機関の台帳には、すでに作成者列や関連資料列という形で「アドホックな多重関係」が生えていたはずで、それらを関係シートへ移すのは正規化の作業にすぎない。この形の Excel は、グラフの**編集フロントエンド**として十分に機能する。

機械変換に耐えるための規律は5つある。(1)1行1実体・1行1関係、1セル1値 (複数値はコンマで詰めず行を分ける)。(2)見た目に意味を持たせない (結合セル・色・字下げは変換で消える)。(3)ID を全行に与え、参照は名前でなく ID で書く。(4)列名を固定し、「列名→RiC-O のプロパティ」の対応表を先頭シートに置く (これがそのままミニ適用プロファイルになる)。(5)日付は Excel の自動変換に注意し、和暦・年度は文字列として持つ (7.7 節)。

ここまで整っていれば、RiC-O への変換は `rdflib` の短いスクリプトや `OpenRefine` (表形式データの整形と名寄せに使う無償ツール) で機械的にできる。そして将来、記述システムが成熟したときも、この規律で育てた台帳は最小のコストで移行できる。シナリオ C の「将来に備えたデータの規律」を、表計算の上で実践していることになるからである。失われたのは「木ならば1列で書けた」という幸運だけで、行指向の作業様式そのものは、RiC と両立する。

7.12 最初の一步の例 — 演習として

学習目的なら、機関規模の計画は要らない。次の演習を勧める。

1. 手近な記録群 (ゼミ資料、家族文書、サークルの文書箱) を選び、実体を10個前後洗い出す (記録集合2~3、記録2~3、人・団体2~3、活動1~2、規則や場所があれば加える)。
2. 紙の上でグラフを描く (本書の図の見よう見まねでよい)。どの関係を使うかを RiC-CM §5.6 の一覧から選ぶ。
3. それを Turtle で書き下ろす (第5章の例を雛形に。IRI は `ex:` で仮置きしてよい)。
4. 無償のトリプルストア (Fuseki) に読み込ませ、SPARQL で「この集合に含まれる記録の一覧」

「この人物が関わる実体の一覧」を出してみる。

5. 余力があれば GraphDB で推論を有効にし、書いていないトリプル (逆向き、推移閉包) が検索に現れることを確かめる。

この一巡で、本書の第 4~6 章の内容が手を動かす形で確認でき、「RiC の記述とは実体と関係のデータを作ることだ」という感覚が身につく。

白紙から始めるのが重い場合は、本書のサイトで配布しているサンプルデータ^{*6}を出発点にできる。本書に出てくる作例をそのまま読み込めるファイルにしたもので、本章の通し例 (某県衛生部フォンドと村方文書)、7.9 節の保存期間と廃棄、第 6 章の系譜の例と SPARQL の問い合わせ 15 件 (本書に出てくる 14 件と、近道の推論を確かめる 1 件)、第 8 章 8.4 節のシェイプと検査対象データが入っている。Fuseki と pySHACL での動かし方は同ディレクトリの README.md にある。本書と同じ CC BY 4.0 なので、改変して授業に使ってよい。同梱の check.py は、全ファイルの構文、rico: 名の実在、定義域・値域、問い合わせの返す件数 (推論あり・なしの両方)、SHACL の報告内容、そして本文の Turtle listing すべてとの一致と整合を機械的に検査する。

現時点で欠けているもの

RiC-AG の草案公開で、実践の手引きは大きく前進した。それでもなお、AG は自らを草案かつ「生きた文書」と位置づけてフィードバックを募っている段階であり、意図的に「段階的な手順書」であることを避けてもいる。記録集合の粒度の目安や必須とすべき属性の下限のような「答えの表」を、AG は与えていない。AG 自身が、記録か記録集合かのようなモデル化の判断に決定的な正解・不正解はないとし (§6b)、事例と判断材料を示す方針をとっている。また現状の事例は英国・北欧・スペインの資料に偏っていることを AG 自身が認めており、日本の実務への当てはめは検証が要る。本章の手順は AG と先行事例から組み立てた現時点の再構成であり、AG の改訂や国内の適用指針が出れば、それに従って更新されるべきものである。学習者としては「確定した正解を暗記する」のではなく、「公式文書がどこまでを定め、どこからが実装の判断かを見分ける」読み方を身につけることが、いちばん息の長い投資になる。

確認問題 (ヒントは付録 A.6)

- 問 7.1 架空の小さな文書館を想定し、適用プロファイルの最小構成 (使う実体、実体化する関係、IRI 方針) を箇条書きで設計せよ。
- 問 7.2 本章 7.4 節の EAD 断片のうち、シリーズ「防疫統計系列」だけを Turtle に手で変換せよ (親フォンドとの関係も含めること)。
- 問 7.3 名前付きグラフを使って典拠系と目録系を分割するとき、グラフ境界を越える関係トリプルをどちらに置くか。理由とともに決めよ。
- 問 7.4 「昭和 8 年度」という日付を、原表記 (rico:expressedDate) と EDTF の機械可読値 (rico:normalizedDateValue) の 2 本立てで書け。

^{*6} <https://tkub.github.io/edu/ric/ric-introduction-ja-examples.zip>。展開すると ric-introduction-ja-examples/ ができる。RiC-O 1.1 の RDF と CSV も同梱してあるので、これだけで推論と検査が動く。

問 7.5 存命個人が作成者である記録群を LOD 公開する場合の設計方針を、本章 7.8 節の道具立てから 3 つ選んで述べよ。

第 8 章

どこまで技術を学ぶ必要があるか

「RiC をやるにはプログラミングができないといけないのか」—最もよく聞かれる問いに、本章で答える。結論を先に言えば、**役割による**。全員に必要なのは概念の理解であり、技術の実装は分業できる。ただし分業が成り立つためには、境界を越えて会話できる程度の共通言語が要る。本章はその共通言語の目録である。

8.1 役割別の必要水準

表 8.1 役割別に見た技術素養の目安 (◎=習得すべき、○=読める・使える程度、△=概要を知っていればよい、—=不要)

要素技術	目録担当	データ設計担当	開発者	管理職
RiC-CM の概念 (実体・関係)	◎	◎	○	○
Turtle を読む	○	◎	◎	△
Turtle を書く	△	◎	◎	—
SPARQL(読む・簡単な検索)	○	◎	◎	△
OWL の公理を読む	—	○	◎	—
XML/EAD	○	◎	◎	△
IRI・永続識別子の設計	△	◎	◎	○
トリプルストアの運用	—	△	◎	△
スクリプト (Python 等)	—	○	◎	—
SHACL などの検証技術	—	○	◎	—

表 8.1 の「目録担当」の列を見てほしい。書く技術はほとんど要らない。必要なのは、概念モデルを理解していること、システムが出力するデータ (Turtle) や検索式 (SPARQL) を読んで意味が分かること、そして自分の記述判断 (記録か記録部分か、実体化するか) がデータにどう反映されるかを想像できることである。RiC-CM 自身も、システムの開発者は複雑さを覆い隠すユーザインタフェースを作ること、データの作成と維持に伴う知的・技術的・経済的な負担を和らげられると述べている (§1.3)。一方「データ設計担当」(機関の適用プロファイルを作り、マッピングを定義する人) には、読み書き両方の素養が要る。アーカイブズ学と情報技術の両方が分かるこの中間人材が、RiC 実装の成否を握る。

8.2 要素技術の地図

第5～7章に登場した技術を、学ぶ順序を意識して図8.1に整理する。上から、データの基礎、意味の記述、識別・語彙・検証、既存資産と運用の4段に並べた。

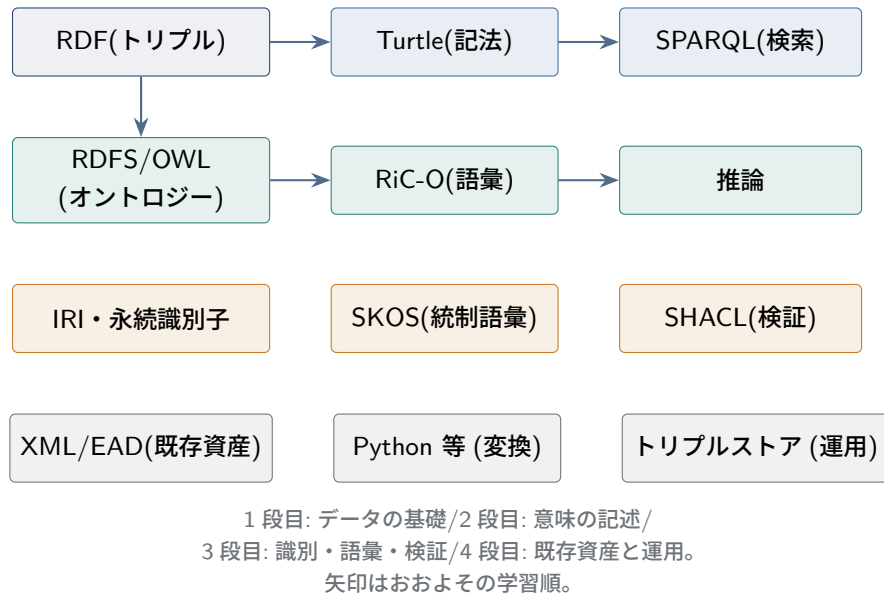


図 8.1 RiC 実装を支える要素技術の地図。RiC-O と EAD を除けば、W3C 標準か、広く普及した汎用の技術である。

各技術の要点と、アーカイブズ実務との接点を短く述べる。

RDF と Turtle 第5章で学んだ。データの原子が「主語・述語・目的語」であること、ものには IRI で名前を付けること。この2点が腹に落ちていれば十分で、構文の細部は都度調べればよい。

SPARQL RDF 用の検索言語。SQL に似た見た目を持つ (`SELECT ... WHERE`) が、表ではなくグラフのパターン照合である。目録担当も「読める」と、公開エンドポイントでの調べ物や、開発者への要望の伝達が格段に楽になる。

RDFS/OWL 語彙を定義する側の言語。使う (RiC-O を利用する) だけなら深入り不要だが、`subClassOf`、`domain/range`、`inverseOf`、推移性、プロパティ連鎖の5語は第6章の推論を理解する最低限として押さえない。

IRI と永続識別子 LOD の前提となる仕組み。DOI や ARK などの永続識別子の考え方、「一度公開した識別子は変えない・消さない」という運用規律を含む。技術というより制度設計であり、アーキビストの職能と地続きである。

SKOS 統制語彙 (シソーラス) を RDF で表す標準。RiC-O の `*Type` クラスのインスタンス群は SKOS 語彙として整備することが想定されている (公式の Fonds などの個体も SKOS concept を兼ねる)。件名標目や資料種別の管理経験がそのまま活きる領域である。

SHACL データが機関のルール (必須項目、値の形式) を満たすか検査する制約言語。OWL の推論 (第6章) が「意味を広げる」向きの技術であるのに対し、SHACL は「形を締める」向きの技術で、

品質管理の道具である。実物は 8.4 節で見る。

XML/EAD と変換スクリプト 既存資産からの移行の道具。EAD の構造理解は変換設計の前提であり、Python(rdfliib) や XSLT は変換の実装手段である。

トリプルストア RDF 専用のデータベース。SPARQL エンドポイントと推論機能を提供する。運用(バックアップ、性能、アクセス制御)は通常の情報システム運用と同種の仕事である。

8.3 リレーショナルデータベースの経験はどう活かすか

RDB と SQL に触れた経験のある読者は、その経験の大部分をそのまま活かせる。対応と相違を表 8.2 に整理する。

表 8.2 RDB の概念と RDF/RiC-O の概念

RDB	RDF/RiC-O	注意点
テーブル定義	クラス	個体は複数クラスに属せる
行	個体 (資源)	
列	データタイププロパティ	同じプロパティを複数回使える (繰り返し可)
外部キー・結合	オブジェクトプロパティ	向きがあり、逆プロパティが定義される
主キー	IRI	世界規模で一意。公開後は不変が原則
スキーマ (制約)	オントロジー (公理)+SHACL(制約)	OWL の公理は制約ではなく推論規則
NULL(値なし)	トリプルの不在	閉世界と開世界の違い (第 5 章コラム)
SQL	SPARQL	発想は近い。再帰的な探索は SPARQL+ 推論が得意
正規化 (実体の分離)	実体化 (作成者・種別のクラス化)	発想はほぼ同じ。RDB 設計の勘がそのまま使える

とりわけ「正規化」の経験は貴重である。「作成者名を各行に文字列で繰り返し書くのではなく、作成者テーブルに分離して ID で参照する」という RDB 設計の定石は、RiC-CM が作成者をはじめてからエージェントという実体として立てていることに当たる (手順の具体例は第 2 章 2.4 節のコラム)。種別のような統制値の属性についても、RiC-CM §3.3 は、実装では実体として扱うことが多いとし、その理由に、共有の語彙を作れることや既存の共有語彙を使えることを挙げている。これも同じ発想である。多対多の関係を結合表で表し、その表に「役割」のような列を足す設計も、RiC の関係の実体化とそのまま対応する。ER 図を描いた経験があれば、RiC-CM の実体・属性・関係の章は既視感をもって読めるはずである。逆に、RDB 経験者が戸惑うのは開世界仮説と「スキーマがデータを拒否しない」ことの 2 点であり、そこだけ意識して学び直せばよい。

8.3.1 従属性・独立性の概念はどう対応するか

データベース理論の従属性 (dependency) をめぐる概念群も、RiC の読解にそのまま使える。以下で述べる対応を、先に表 8.3 にまとめておく。右端の列は、その性質が RiC-O の公理で書かれているのか、別の検査に任されているのかを示す。

表 8.3 データベース理論の従属性・独立性と、RiC での対応

データベース理論	RiC での対応	どこで担保されるか
存在従属・弱実体	記録部分 (記録に依存)、記録集合 (メンバーに依存)、記録と具現化、rico:Proxy	多くは RiC-CM の定義 (散文)。rico:Proxy だけは OWL の基数制約 (rico:proxyFor・rico:proxyIn がちょうど 1 つ)
関数従属	業務上の一意性 (1 つの識別子は 1 つの記録だけを指す、など)	オントロジーではなく SHACL などの検査 (RiC-O 1.1 は関数的プロパティを使っていない)
データ独立性	入力と出力の分離、記録と具現化の分離	モデルの設計 (何を別の実体に切り分けるか)

■**存在従属と弱実体** ある実体が別の実体なしには存在できないことを**存在従属** (existence dependency) という。ER モデルでは、他の実体が存在しなければ存在できず、同一性 (キー) も親実体に依存する実体を**弱実体** (weak entity)、その親子を結ぶ関係を識別関係 (identifying relationship) と呼ぶ。RiC-CM は記録資源の 3 区分の違いを次のように説明している。「記録の同一性は記録自体に由来する。記録部分の同一性は、属する記録に依存して派生する。記録集合の同一性は、メンバーに依存して派生する」 (§2.2.2)。これは弱実体の概念の言い換えとして読める。記録は強実体、記録部分は記録を親とする弱実体に相当し、記録集合はメンバーに存在従属する (ただし親が複数という点で古典的な弱実体より緩い)。具現化も、「記録は少なくとも 1 つの具現化に表されるまで存在しない」 (§2.2.2) という条件で記録と結ばれている。ただしその具現化は、記述の時点で失われていてもよい (E06 のスコープノート)。RiC-O ではこの種の従属が OWL の基数制約として書かれる場合がある。たとえば rico:Proxy は「ちょうど 1 つの記録資源を代理し (rico:proxyFor)、ちょうど 1 つの記録資源の中にある (rico:proxyIn)」と公理化されており、代理する相手なしには意味をなさない存在従属実体である。ER 設計で弱実体を見分ける勘は、RiC のどの実体が単独で立ち、どの実体が他に寄りかかるかを見分ける勘として、そのまま通用する。

■**関数従属と一意性** 正規化理論の関数従属 (functional dependency: X の値が決まれば Y の値が一意に決まる) に相当する宣言は、RiC-O にはほとんどない。OWL には関数的プロパティ (値を高々 1 つに限る) という道具があるが、RiC-O 1.1 はこれを使っていない。名称も日付も作成者も原則として繰り返し可能であり、一般に「高々 1 つ」という制限はない。先に見た Proxy などの局所的な基数制約は、これとは別である。これは設計の欠落ではなく、アーカイブズ記述の性質 (別名・改名・複数の作成者・不確かな日付の併記が常態であること) と、開世界仮説の帰結 (同じものが別の IRI で二度登場するかもしれず、値の一意性を論理で強制すると意図しない同一視の推論を招くこと) を踏まえた選択と読める。したがって「この目録では 1 つの識別子は 1 つの記録だけを指す」のような業務上の関数従属は、オントロジーではなく SHACL などの検査の層で守ることになる。RDB でスキーマが担っていた整合性維持の仕事が、RiC の世界では公理 (推論) と制約 (検査) の 2 つに分かれる。この対応を押さえておくと、両者の役割分担が見通しやすくなる。

■**データ独立性** ANSI/SPARC は、データベースの設計を、利用者ごとの見え方を定める外部スキーマ、対象領域そのものの論理設計を定める概念スキーマ、実際の格納方式を定める内部スキーマの 3 層に分けた。この分け方があってはじめて、下の層を変えても上の層を書き直さずに済むという「データ独立性」が言える。この発想も、RiC の設計に対応物を持つ。入力と出力の分離 (第 3 章) は、出力の形 (外部スキーマに当たる) を変えても記述そのもの (概念スキーマに当たる) を書き換えずに済むという点で、データ独立性の一種であり、記録 (知的内容) と具現化 (担体上の記録) の分離は、内容の記述を担体・フォーマットの変転から隔離する仕組みである。フォーマット移行が起きても記録の記述は書き換えずに済み、新しい具現化を追加するだけでよい。RDB の教科書で学ぶ「変わりやすいものから変わりにくいものを切り離す」という設計原理が、アーカイブズの時間スケール (数百年) に引き伸ばされて適用されていると見ることができる。

8.4 形を締める — SHACL を 1 つ書いてみる

前節の最後で、業務上の一意性のような制約は「オントロジーではなく検査の層で守る」と述べた。その検査の層を担うのが **SHACL**(Shapes Constraint Language。2017 年の W3C 勧告) である。本書では第 5~7 章で何度も名前を挙げてきたが、実物を見ないと OWL との違いが腹に落ちない。短い例で確かめておく。

次のデータのうち `ex:rec1` は正しい記述で、残りの 2 件に問題がある。`ex:rec2` には名称がなく、`ex:rec3` は作成者として**場所**を指している。

```
ex:rec1 a rico:Record ;                                # 正しい記述
    rico:name "赤痢発生届" ;
    rico:hasCreator ex:nakamura .
ex:nakamura a rico:Person ; rico:name "中村正" .

ex:rec2 a rico:Record .                                # 名称がない

ex:rec3 a rico:Record ;
    rico:name "検疫記録" ;
    rico:hasCreator ex:yokohama .                      # 作成者が場所

ex:yokohama a rico:Place ;
    rico:name "横浜" .
```

まず、これを RiC-O の推論器にかけるとどうなるか。`rico:hasCreator` の値域は `rico:Agent` だと宣言されている。しかし OWL の値域宣言は**検査ではなく推論の根拠**だった (表 8.2)。だから推論器は誤りを報告せず、代わりに「`ex:yokohama` はエージェントでもある」という**新しいトリプルを導出する**。RiC-O は個人・集団・職位・メカニズムの 4 つを互いに素と宣言しているが、場所とエージェントは互いに素だと宣言していないので、矛盾も検出されない。誤ったデータが、静かに誤ったまま増える。名称がないことも、開世界仮説の下では「名称がない」ではなく「まだ書かれていない」でしかなく、問題として扱われない。

SHACL は逆向きの道具である。検査したい形を**シェイプ** (shape) として書き、それに合わないノードを違反として報告する。

```

PREFIX sh: <http://www.w3.org/ns/shacl#>
PREFIX rico: <https://www.ica.org/standards/RiC/ontology#>
PREFIX ex: <https://archives.example.jp/>

ex:RecordShape a sh:NodeShape ;
  sh:targetClass rico:Record ;          # 検査対象は Record のインスタンス
  sh:property [
    sh:path      rico:name ;
    sh:minCount  1 ;                    # 名称は1つ以上なければならない
    sh:message   "記録には名称が要る" ] ;
  sh:property [
    sh:path      rico:hasCreator ;
    sh:class     rico:Agent ;          # 作成者はエージェントでなければならない
    sh:message   "作成者に指定できるのはエージェントだけである" ] .

```

RiC-O のクラス階層を添え、事前推論をせずにこれを検証すると (次の注意ボックス)、`ex:rec2`(名称の欠落) と `ex:rec3`(作成者が場所) の2件が違反として報告される。同じデータ、同じ RiC-O の語彙でありながら、推論器は何も言わず、SHACL は2件を指摘する。この差が、本書で繰り返してきた「OWL は意味を広げ、SHACL は形を締める」の中身である。

語彙を読み込ませないと、正しいデータまで違反になる

上の `sh:class` は、値の型と `rdfs:subClassOf` の経路を検査対象のグラフでたどる。RiC-O のクラス階層を渡さなければ、`rico:Person` が `rico:Agent` の下位クラスだとわからず、正しい `ex:rec1` まで違反になる。本例では RiC-O から抽出した名前付きクラスの階層を一緒に読み込み、事前の RDFS/OWL 推論を行わずに検査する。配布サンプルの `README.md` にその実行方法を載せた。ここで値域の推論まで先に行うと、場所がエージェントにも分類され、今度は `ex:rec3` の誤りが見えなくなる。クラス階層を参照することと、データに推論結果を加えることを区別し、検査に渡すグラフと推論の設定を決めておく必要がある。

同じデータに対する反応を、推論器と SHACL の3つの設定について表 8.4 にまとめる。正しい `ex:rec1` が適合し、誤りのある2件がともに違反になるのは、3行目の設定 (本例の設定) だけである。

表 8.4 同じデータに対する OWL の推論器と SHACL の反応。SHACL の結果は、検査に渡すグラフと推論の設定で変わる。

処理と設定	<code>ex:rec1</code> (正しい)	<code>ex:rec2</code> (名称なし)	<code>ex:rec3</code> (作成者が場所)
OWL の推論器	報告なし	報告なし	報告なし。横浜をエージェントと推論する
SHACL: クラス階層を渡さない	違反	違反	違反
SHACL: クラス階層を渡し、事前推論をしない	適合	違反	違反
SHACL: 値域の推論まで先に行う	適合	違反	適合 (誤りを見逃す)

なぜ SHACL は「無い」と言えるのか。鍵は `sh:targetClass` にある。この1行が「いま検査する

のはこの範囲だ」と対象を確定させ、その範囲の中では**手元のデータで数える**。つまり SHACL は、検査というその場かぎりの目的のために、閉世界仮説を局所的に持ち込む道具である。OWL の開世界仮説と矛盾するのではなく、層が違う。データそのものの意味は開世界のまま、「私の機関が受け入れてよいデータの形」だけを閉じた世界として判定する。第 6 章 6.9 節では、欠如の問い合わせを、論理の否定ではなく SPARQL の `FILTER NOT EXISTS` による検査として書くとした。SHACL は、同じ種類の検査を、機関の規則として宣言的に書いておく道具である。

SHACL でも一行では書けないこと

基本の語彙で書けるのは、1 つのノードとその近傍についての条件である。第 6 章 6.9 節で例に挙げた「同一フォンド内の整理番号は一意」のように、**複数のノードを見比べる**条件は、この範囲を越える。SHACL にはシェイプの中に SPARQL の問い合わせを埋め込む `sh:sparql` という逃げ道が用意されており、この種の制約はそちらで書く。「宣言的に書ける範囲と、問い合わせを書かなければならない範囲がある」という境界の引かれ方は、第 6 章 6.7 節で見た OWL の表現力の境界とよく似ている。道具の側の限界を見切って、越える手段を知っておくことが、どちらの技術でも同じように要る。

この節のシェイプと検査対象データは、本書のサイトで配布しているサンプルデータに `shapes.ttl`・`shacl-data.ttl` として入っている。上の 2 件の違反が実際に報告されること、および RiC-O を渡さないと正しい記述の `ex:rec1` まで違反になることは、手元で確かめられる (第 7 章 7.12 節)。

適用プロファイル (第 7 章 7.3 節) は、決めただけでは守られない。決めた内容をシェイプとして書き下し、変換のたびに検査を回してはじめて、方針が実際のデータの上で守られるようになる。SHACL はそのための道具であり、機関の適用プロファイルの**機械可読な部分**だと考えるのがいちばん近い。

8.5 学習ロードマップ

アーカイブズ学を学び始めた読者を想定した、無理のない順序を示す。

1. **概念の習得** (本書第 1~4 章 + RiC-CM 原文)。技術は一切不要。RiC-CM の実体とスコープノート、自分の知っている記録群に当てはめて読む。
2. **グラフに慣れる** (第 5 章 + 紙とペン)。身近な記録群を実体と関係の図に描く練習。この段階も技術不要で、しかし RiC 的な思考の核はここで身につく。
3. **Turtle と SPARQL の読み書き** (第 5~6 章 + RiC-O 公式サンプル)。テキストエディタと無償ツールがあればよい。第 7 章末尾の演習がこの段階に当たる。
4. **小さな実データで一巡** (第 7 章)。手元データ数十件の変換・格納・検索・推論。Python の初歩 (他分野でも汎用的に役立つ) を並行して学ぶと選択肢が広がる。
5. **必要に応じた掘り下げ**。実装の道に進むなら OWL・SHACL・システム運用へ。目録実務の道に進むなら、ここから先は概念の維持と、開発者と話せる語彙の維持で足りる。

最後に強調したい。RiC が要求する最も重要な素養は、プログラミングではなく**モデリング**—世界を実体と関係に切り分け、その切り分けの根拠を説明できる能力—である。そしてそれは、出所を見極め、編成を設計し、記述の判断を積み重ねてきたアーカイブズ学の伝統的な訓練と、本質的に同じ種類の能

力である。RiC はアーキビストに新しい種類の仕事を課すというより、アーキビストが暗黙に行ってきた知的作業を、明示的で機械と共有可能な形式に書き出すことを求めている。

確認問題 (ヒントは付録 A.6)

- 問 8.1** RDB の外部キーと RDF のオブジェクトプロパティの違いを 3 点挙げよ (参照整合性・向き・スキーマの扱いに着目せよ)。
- 問 8.2** 自分の役割に近いもの (目録担当/データ設計担当/開発者/管理職) を 1 つ選び、本章の技術地図から「次に学ぶ 2 つ」を選んで理由を述べよ。
- 問 8.3** データの形の検査という仕事において、SHACL と OWL の役割はどう違うか。開世界仮説という言葉を使って説明せよ。
- 問 8.4** 8.4 節のシェイプにならって、「記録集合は識別子を 1 つ以上持たなければならない」という制約を書け (`rico:RecordSet` と `rico:identifier` を使う)。

付録 A

付録

A.1 RiC-CM 1.0 実体一覧

ID	名称	定義 (要約)
E01	もの Thing	人間の経験の領域内のあらゆる観念・物質的な物・出来事
E02	記録資源 Record Resource	エージェントが生活・業務の中で生み出したまたは取得し保持した情報
E03	記録集合 Record Set	共有する属性・関係に基づきエージェントがまとめた 1 つ以上の記録
E04	記録 Record	担体に持続的・復元可能な形で記録された、ひとまとまりの情報内容
E05	記録部分 Record Part	記録の知的完結性に寄与する、独立の情報内容を持つ構成部分
E06	具現化 Instantiation	担体上への情報の記録。記録の物理的またはデジタルな現れ
E07	エージェント Agent	世界の中で活動を遂行するもの
E08	個人 Person	人間個人
E09	集団 Group	エージェントとして共に行動する 2 以上のエージェント
E10	家族 Family	出生・婚姻・養子縁組その他の社会的紐帯で結ばれた 2 人以上
E11	団体 Corporate Body	法的・社会的に認知された組織された集団
E12	職位 Position	集団の中での個人の機能的役割
E13	メカニズム Mechanism	個人・集団が作成した、活動を遂行するプロセスやシステム
E14	イベント Event	時間と空間の中で起こる出来事
E15	活動 Activity	エージェントが設計した目的のために何かを行うこと
E16	規則 Rule	エージェントの存在・責任・権限や活動の遂行を支配する条件
E17	マンデート Mandate	あるエージェントから別のエージェントへの責任・権限の委任
E18	日付 Date	実体の識別と文脈化に寄与する年代情報
E22	場所 Place	境界を持つ、名前のついた地理的領域

E19～E21 は欠番である (0.2 版の日付下位実体が 1.0 で廃止された)。

A.2 RiC-CM 1.0 属性一覧 — 原文との対照

RiC-CM §3.6 が定義する 42 の属性である。ID は RiC-A01 から RiC-A45 まで振られているが、A06・A14・A16 は欠番で、実際に定義されているのは 42 である (本書では接頭辞を省いて A21 のように書く)。並びは原典に合わせ、原文名のアルファベット順にした。ID が名前の順と揃っていないのは、版を重ねるあいだに属性が足し引きされたためである。RiC-CM §1.8 によれば、1.0 では日付種別 (A42) と、規則・マンドートを分類するための種別属性 (A44・A45) が追加され、確実性と日付標準の 2 属性が削除された。A06・A14・A16 の欠番はこうした削除の跡だが、どの属性が抜けたのかは 1.0 の本文からは分らない。

各欄の読み方を 3 つ。**定義域**は、その属性を持てる実体である。1 つの属性が複数の実体で使えることが多い。**値スキーマ**は §3.5 の 4 区分で、自由 (自由テキスト)、モデル準拠 (語形や構文を決めた文)、統制値 (統制語彙から選ぶ)、規則準拠 (ISO 8601 など機械処理を前提とした形式) を指す。原文が明言するとおり、これは指示的なものであって規範ではない (第 4 章 4.3 節)。**反復**は同じ属性を複数回使えるかで、○ は可、— は不可、△ は「記録集合の全メンバーを記述するときは不可、一部のメンバーを記述するときは可」のように文脈で変わるものである。△ は 5 つある。そのうち人口学的集団 (A15) を除く 4 つは、第 4 章 4.3 節で述べた「記録集合の属性には 2 種類の意味がある」という問題の現れ方の 1 つである。A15 だけは事情が違い、人口学的な区分の種類によって反復できるかどうかが変わる。

統制値をとる属性 (種別系のほとんど) は、実装では実体すなわちクラスとして扱われることが多い。RiC-O では `rico:ActivityType` や `rico:CarrierType` のように `rico:Type` の下位クラスになっている (第 5 章 5.5 節)。CM の属性一覧と RiC-O のクラス一覧を突き合わせるときは、この読み替えが要る。

ID	原文の属性名	本書の訳	定義域	値スキーマ	反復
A01	Accruals	追加受入	記録集合	モデル準拠/自由	—
A02	Activity Type	活動種別	活動	統制値	—
A03	Authenticity Note	真正性注記	記録資源 / 具現化	モデル準拠/自由	—
A04	Carrier Extent	担体の数量	具現化	モデル準拠/自由	—
A05	Carrier Type	担体種別	具現化	統制値	○
A07	Classification	分類	記録資源	統制値/モデル準拠	○
A08	Conditions of Access	アクセス条件	記録資源 / 具現化	モデル準拠/自由	—
A09	Conditions of Use	使用条件	記録資源 / 具現化	モデル準拠/自由	—
A10	Content Type	内容種別	記録資源	統制値	○
A11	Coordinates	座標	場所	規則準拠	○
A12	Corporate Body Type	団体種別	団体	統制値	○
A13	Date Qualifier	日付の限定	日付	統制値	—
A42	Date Type	日付種別	日付	統制値	—
A15	Demographic Group	人口学的集団	個人 / 集団	統制値 (推奨)	△
A17	Documentary Form Type	文書形式種別	記録集合 / 記録 / 記録部分	統制値	△
A18	Event Type	イベント種別	イベント	統制値	—
A19	Expressed Date	表記された日付	日付	モデル準拠/自由	○
A20	Family Type	家族種別	家族	統制値	—
A43	General Description	一般記述	もの	自由	—

(前ページからの続き)

ID	原文の属性名	本書の訳	定義域	値スキーマ	反復
A21	History	歴史	記録資源 / 具現化 / エージェント / イベント / 規則 / 場所	モデル準拠/自由	—
A22	Identifier	識別子	もの	規則準拠/モデル準拠/自由	○
A23	Instantiation Extent	具現化の数量	具現化	モデル準拠/自由	○
A24	Integrity Note	完全性注記	記録資源	モデル準拠/自由	△
A25	Language	言語	記録資源 / エージェント	統制値	○
A26	Legal Status	法的地位	記録資源 / エージェント	統制値	△
A27	Location	所在地	場所	自由	○
A44	Mandate Type	マンドート種別	マンドート	統制値	—
A28	Name	名称	もの	モデル準拠/自由	○
A29	Normalized Date	正規化日付	日付	規則準拠	○
A30	Occupation Type	職業種別	個人	統制値	○
A31	Physical Characteristics Note	物理的特徴注記	具現化	モデル準拠/自由	○
A32	Place Type	場所種別	場所	統制値	—
A33	Production Technique	制作技法	具現化	自由/統制値	○
A34	Quality of Representation Note	表現品質注記	具現化	モデル準拠/自由	○
A35	Record Resource Extent	記録資源の数量	記録資源	モデル準拠/自由	○
A36	Record Set Type	記録集合種別	記録集合	統制値	—
A37	Representation Type	表現種別	具現化	統制値	—
A45	Rule Type	規則種別	規則	統制値	—
A38	Scope and content	範囲と内容	記録資源	モデル準拠/自由	—
A39	State	状態	記録資源	統制値	△
A40	Structure	構造	記録資源 / 具現化	モデル準拠/自由	—
A41	Technical Characteristics	技術的特徴	メカニズム	自由	—

関係そのものに付く属性 (RA01～RA06) は、この表とは別に §5.5 で定義される。第 4 章 4.4 節を参照。

A.3 関係の 13 の型 (RiC-CM §5.2)

型の名称の原語と、各型でもっとも広い関係を挙げる。定義域・値域付きの表は第 4 章の表 4.4 にある。

型 (本書の訳)	原文	各型の最上位の関係
全体 – 部分	Whole-part relations	R002 has or had part
順序	Sequential relations	R008 precedes or preceded
主題	Subject relations	R019 has or had subject
記録資源間	Record Resource to Record	R022 is record resource
	Resource relations	associated with record resource
記録資源 – 具現化	Record Resource to Instantiation	R025 has or had instantiation
	relations	
出所	Provenance relations	R026 has provenance
		R033 documents
具現化間	Instantiation to Instantiation	R034 is instantiation associated
	relations	with instantiation
管理	Management relations	R036 has or had authority over
エージェント間	Agent to Agent relations	R044 is agent associated with
		agent
イベント	Event relations	R057 is event associated with
規則	Rule relations	R062 is rule associated with
日付	Date relations	R068 is date associated with
空間	Spatial relations	R074 is place associated with

すべての関係は最上位の R001 is related to(ものはものに関係する) の下位に位置づけられ、ポリヒエラルキーをなす。1 つの関係が複数の型に属することもある。関係自体の属性として、確実性 (RA01 Certainty of Relation)、日付 (RA02 Date of Relation)、記述 (RA03 Description of Relation)、識別子 (RA04 Identifier of Relation)、出典 (RA05 Source of Relation)、場所 (RA06 Place of Relation) が定義されている。

A.4 RiC-CM 1.0 関係一覧 — 原文との対照

本書の図表で用いた日本語の関係名と、RiC-CM 1.0 の原文名との対照表である。RiC-CM §5.4 の各関係の定義 (ID、Name、Domain/Range) から起こした。ID は原典では RiC-R001 から RiC-R086 まで振られている (本書では接頭辞を省いて R001 と書く)。R043 は欠番で、実際に定義されている関係は 85 である。各関係がどの型に属するかは A.3 節と第 4 章の表 4.4 にある。

読むときの注意は次の 3 つである。

- 原文の has or had、is or was は、「現在そうである」場合と「過去にそうだった」場合の両方を含むことを示す言い回しで、RiC が時間の中で変化する関係を扱うための工夫である。訳では煩雑になるため省いた。
- 各関係には逆向きの関係が定義されている。多くは ID の末尾に i を付けた別の関係 (R024i など) だが、R001 や R049 のような対称的な関係では逆関係がその関係自身であり、i 付きの ID はない。この表は順方向だけを挙げている。
- 定義域 (出発点になれる実体) と値域 (到達点になれる実体) は、A.1 節の実体一覧の名称で示した。

ID	原文の関係名	本書の訳	定義域 → 値域
R001	is related to	に関係する	もの → もの
R002	has or had part	部分を持つ	もの → もの
R003	has or had constituent	構成部分を持つ	記録・記録部分 → 記録・記録部分
R004	has or had component	構成要素を持つ	具現化 → 具現化
R005	has or had subdivision	下部組織を持つ	集団 → 集団
R006	has or had subevent	下位イベントを持つ	イベント → イベント
R007	contains or contained	含む (場所として)	場所 → 場所
R008	precedes or preceded	先行する	もの → もの
R009	precedes in time	時間的に先行する	もの → もの
R010	is original of	の原本である	記録・記録部分 → 記録・記録部分
R011	is draft of	の草稿である	記録 → 記録
R012	has copy	複本を持つ	記録資源 → 記録資源
R013	has reply	返信を持つ	記録資源 → 記録資源
R014	has or had derived instantiation	派生した具現化を持つ	具現化 → 具現化
R015	migrated into	へ移行した	具現化 → 具現化
R016	has successor	後継を持つ	エージェント → エージェント
R017	has descendant	子孫を持つ	個人 → 個人
R018	has child	子を持つ	個人 → 個人
R019	has or had subject	主題を持つ	記録資源 → もの
R020	has or had main subject	主要な主題を持つ	記録資源 → もの
R021	describes or described	を記述している	記録資源 → もの
R022	is record resource associated with record resource	記録資源として関連する	記録資源 → 記録資源
R023	has genetic link to record resource	記録資源と生成上のつながりを持つ	記録資源 → 記録資源
R024	includes or included	含む (記録集合として)	記録集合 → 記録・記録集合
R025	has or had instantiation	具現化を持つ	記録資源 → 具現化
R026	has provenance	出所を持つ	記録資源・具現化 → エージェント
R027	has creator	作成者を持つ	記録資源・具現化 → エージェント
R028	has accumulator	集積者を持つ	記録資源・具現化 → エージェント
R029	has receiver	受領者を持つ	記録資源・具現化 → エージェント
R030	has collector	収集者を持つ	記録資源・具現化 → エージェント
R031	has sender	送信者を持つ	記録資源・具現化 → エージェント
R032	has addressee	受信者を持つ	記録資源・具現化 → エージェント
R033	documents	文書化している	記録資源・具現化 → 活動
R034	is instantiation associated with instantiation	具現化として関連する	具現化 → 具現化
R035	is functionally equivalent to	と機能的に等価である	具現化 → 具現化
R036	has or had authority over	に権限を持つ	エージェント → もの
R037	is or was owner of	の所有者である	集団・個人・職位 → もの
R038	is or was manager of	の管理者である	エージェント → 記録資源・具現化
R039	is or was holder of	の所持者である	エージェント → 記録資源・具現化
R040	is or was holder of intellectual property rights of	の知的財産権者である	集団・個人・職位 → 記録資源・具現化
R041	is or was controller of	を統制している	エージェント → エージェント
R042	is or was leader of	の指導者である	個人 → 集団
R043	欠番 (<i>RiC-CM §5.4</i> に定義がない)		
R044	is agent associated with agent	エージェントとして関連する	エージェント → エージェント
R045	has or had subordinate	部下を持つ	エージェント → エージェント
R046	has or had work relation with	と業務上の関係を持つ	エージェント → エージェント
R047	has family association with	と家族的なつながりを持つ	個人 → 個人
R048	has sibling	きょうだいを持つ	個人 → 個人
R049	has or had spouse	配偶者を持つ	個人 → 個人
R050	knows of	の存在を知っている	個人 → 個人
R051	knows	と面識がある	個人 → 個人

(前ページからの続き)

ID	原文の関係名	本書の訳	定義域 → 値域
R052	has or had correspondent	文通相手を持つ	個人 → 個人
R053	has or had teacher	師を持つ	個人 → 個人
R054	occupies or occupied	に就いている	個人 → 職位
R055	has or had member	構成員を持つ	集団 → 個人
R056	exists or existed in	の中に置かれている	職位 → 集団
R057	is event associated with	イベントとして関連する	イベント → もの
R058	has or had participant	参加者を持つ	イベント → もの
R059	affects or affected	に影響を与えている	イベント → もの
R060	is or was performed by	によって遂行された	活動 → エージェント
R061	results or resulted in	をもたらした	イベント → もの
R062	is rule associated with	規則として関連する	規則 → もの
R063	regulates or regulated	を規律している	規則 → もの
R064	is or was expressed by	によって表現されている	規則 → 記録資源
R065	issued by	によって発行された	規則 → エージェント
R066	is or was enforced by	によって執行されている	規則 → エージェント
R067	authorizes	に権限を与える	マンドート → エージェント
R068	is date associated with	日付として関連する	日付 → もの
R069	is beginning date of	の開始日である	日付 → もの
R070	is birth date of	の生年月日である	日付 → 個人
R071	is end date of	の終了日である	日付 → もの
R072	is death date of	の没年月日である	日付 → 個人
R073	is modification date of	の改変日である	日付 → もの
R074	is place associated with	場所として関連する	場所 → もの
R075	is or was location of	の所在地である	場所 → もの
R076	is or was jurisdiction of	の管轄区域である	場所 → エージェント
R077	is or was adjacent to	と隣接している	場所 → 場所
R078	overlaps or overlapped	と重なっている	場所 → 場所
R079	has author	著者を持つ	記録 → 個人・集団・職位
R080	is creation date of	の作成日である	日付 → 記録資源・具現化
R081	is or was creation date of all members of	の全メンバーの作成日である	日付 → 記録集合
R082	is or was creation date of some members of	の一部メンバーの作成日である	日付 → 記録集合
R083	is or was creation date of most members of	の大半のメンバーの作成日である	日付 → 記録集合
R084	is date of occurrence of	の発生日である	日付 → イベント
R085	is within	の期間内にある	日付 → 日付
R086	intersects	と期間が重なる	日付 → 日付

A.5 よく使う RiC-O 語彙の抜粋

語彙	用途
<code>rico:RecordSet / rico:Record / rico:RecordPart</code>	記録資源の 3 区分
<code>rico:Instantiation</code>	具現化
<code>rico:Person / rico:Group / rico:CorporateBody / rico:Family / rico:Position / rico:Mechanism</code>	エージェント
<code>rico:Activity / rico:Event / rico:Rule / rico:Mandate / rico:Date / rico:Place</code>	文脈の実体
<code>rico:Proxy</code>	記録資源の「特定の集合の中での分身」(順序・文脈情報の担い手)
<code>rico:name / rico:title / rico:identifier</code>	基本のリテラル属性 (実体化しない簡便法)
<code>rico:generalDescription / rico:scopeAndContent / rico:history</code>	散文系の属性
<code>rico:includesOrIncluded / rico:isOrWasIncludedIn</code>	記録集合と記録 (集合) の所属
<code>rico:directlyIncludes / rico:includesTransitive</code>	直接の包含/推移閉包 (推論用)
<code>rico:hasCreator / rico:hasOrganicProvenance / rico:hasAccumulator</code>	出所・作成
<code>rico:performsOrPerformed / rico:documents</code>	活動の遂行/活動の文書化
<code>rico:hasSuccessor / rico:isSuccessorOf</code>	エージェントの変遷
<code>rico:hasOrHadInstantiation(analogue/digital 変種あり)</code>	記録資源と具現化
<code>rico:thingIsSourceOfRelation / rico:relationHasTarget</code>	実体化された関係と両端の実体を結ぶ (前者は始点の実体→関係、後者は関係→終点の実体。第 6 章 6.5 節のプロパティ連鎖で使う)
<code>rico:isAssociatedWithDate / rico:normalizedDateValue</code>	日付の関係づけと正規化値

A.6 確認問題のヒント

■第 1 章 問 1.1:1.2 節の図 1.1 と 4 つの項。1.3 節の表 1.3 も参考になる。問 1.2:CM は概念 (安定)、O は実装 (修正が入る)。1.2 節、1.4 節の表 1.4 と第 5 章 5.5 節。問 1.3:1.3 節の一次資料表と RiC-AG の構成 (表 1.2)。本書の章をどう読むかは序文の図 1 も参考になる。

■第 2 章 問 2.1:2.4.2 項。識別子を持つこと=参照の受け手になれること。問 2.2: 第 7 章 7.4 節の EAD と EAC-CPF の例が使える (`authfilenumber` と `relationEntry`)。問 2.3:2.4.4 項の第 4 期と、第 7 章 7.4 節の図 7.2。問 2.4:2.6 節「図書館」の段落と第 7 章 7.6 節。

■第 3 章 問 3.1: 表 3.4(自由と負担の対応)。問 3.2:3.6 節の (6)。同じ作成者関係を直接プロパティで書く場合と関係実体で書く場合など。問 3.3:3.1 節のコラムと表 3.1。

■第 4 章 問 4.1:4.2.2 項の注意ボックス「「集合」という名前の読み方」(文集のたとえ) と図 4.5。問 4.2: 使う関係は図 4.8(遂行・文書化) と図 4.9(日付) から選ぶ。第 2 章の図 2.3 右側が形の参考になる。問 4.3:4.5 節の末尾の段落。検証可能性と改訂履歴。問 4.4:4.5 節で図 4.12 の作例を説明する部分のうち、記述者の扱い (記述者は特別扱いされない) を述べたところ。

■第5章 問 5.1:5.1 節の文法コラムと 5.4 節。rico:isOrWasIncludedIn は「～に含まれる (た)」。
問 5.2: 文法コラムの 3 通りの等価な書き方。問 5.3:5.2.1 項の R^- の定義。問 5.4:5.2.1 項のコラム「ク
ラスが主語になる書き方」。

■第6章 問 6.1:6.5 節で rico:hasOrganicProvenance の連鎖を規則の形に書き直した式をなぞる。
記号の読み方は第 5 章 5.2.1 項の「連鎖」。問 6.2:6.8.3 項。問 6.3:6.8.2 項。削除・更新時の再計算に注
目。問 6.4:6.6 節の図 6.5 とリスティング。出所関係→団体→後継の順。

■第7章 問 7.1:7.3 節の工程 2。問 7.2:7.4.2 項の Turtle が手本。問 7.3:7.4.4 項の「境界を越える関係
はどこに置くか」(関係の 3 つの置き方と図 7.5)。多の側の定石。問 7.4:7.7 節の和暦の正規化と表 7.7。
年度は期間 1933-04/1934-03。問 7.5:7.8 節の 4 つの道具立て。

■第8章 問 8.1: 表 8.2 と、第 7 章 7.4 節の図 7.2(参照整合性の有無)。問 8.2: 表 8.1 で自分の役割に
近い列 (4 つの役割は序文の図 1 と同じ) を見て、図 8.1 の矢印 (おおよその学習順) に沿って選ぶ。問
8.3:8.4 節 (表 8.4) と第 6 章 6.9 節。OWL の公理は「制約」ではなく推論の根拠。問 8.4:8.4 節のシェ
イプの sh:minCount の部分をまねる。sh:targetClass を rico:RecordSet に、sh:path を rico:
identifier に変えればよい。

A.7 用語集 — 原語との対照、分野で意味がずれる言葉

前半はアーカイブズ学の基本用語で、分野の外から来た読者のために原語を添えた。後半は、隣接分
野のあいだで意味がずれるために注意が要る言葉である。

■アーカイブズ学の基本用語

出所 provenance 記録がどの主体の、どんな営みの産物であるかという**生成の文脈**。RiC-FAD §5 の言
い方では「ある個人・集団が生活や業務の中で作成・蓄積・使用した」という関係を指す。日常
語の「出どころ」に引かれて「どこを経てきたか」まで含めたいが、記録が作成者の手を離
れたあとの移り変わりは**伝来**として区別するのが標準の用法である。RiC は出所を単一の蓄積者
に限らず、作成・集積・送受信に関わった複数の主体と、記録を生んだ活動にまで広げた (第 3 章
3.3 節)。

伝来 記録が作成者の手を離れたあとの、所有・責任・管理の移り変わり。ISAD(G) の記述要素 3.2.3 が
これにあたり (国立公文書館訳の見出しが「伝来」)、編成の履歴や検索手段の作成、ソフトウェ
アのマイグレーションなど、現在の構造をかたちづかった行為も含む。英語名は標準によって異
なり、ISAD(G) は archival history、米国の DACS(5.1) は custodial history と呼ぶが、日本語で
はどちらも伝来である。目的は真正性・完全性・解釈にとって重要な履歴を示すこと。RiC では
散文としては History 属性 (RiC-A21) に収まり、構造化して書くなら管理関係 (所持者 R039、
管理者 R038) で表す。出所関係とは型が別である (第 3 章 3.3 節のコラム)。

出所原則 principle of provenance 出所を記述と管理の基礎に置く原則。2 つの下位原則からなる。**フォ
ンド尊重** (respect des fonds): ある主体が作成・蓄積した記録はひとまとまりに保ち、他の主体の
記録と混ぜない。**原秩序尊重** (respect for original order): 蓄積・使用の過程で形成された記録ど

うしの秩序を保存する。図書館の分類が主題によって資料を並べ替えるのに対し、アーカイブズは出所によるまとまりを崩さない。第2章参照。

フォンド fonds 1つの主体が生活または業務の中で作成・蓄積・使用した記録の全体。フランス語由来で単複同形。従来の記述はフォンドを頂点とする階層で組み立てられた。RiCでは記録集合の種類の1つになる(第3章3.4節)。

記述単位 unit of description ISAD(G)で、1件の記述の対象となるまとまり。フォンドでも1通の書簡でも、同じ26要素で記述できるという抽象。RiCはこれを記録・記録集合・記録部分の3実体に解体した。

多段階記述 multilevel description フォンドから下位のまとまりへと段階的に降りていくISAD(G)の記述方式。上位に書いたことを下位で繰り返さない(非反復の原則)などの規則を伴う。

検索手段 finding aid / **目録** 検索手段は、利用者が記録にたどり着くための道具の総称(finding aidの定着訳)。目録・ガイド・索引などはその種類で、日本の実務の代表格が目録。ISAD(G)は検索手段の中身となる記述の標準、EADはその符号化形式。第2章参照。

典拠レコード authority record 作成者などの情報を記録の記述から切り出して独立させたレコード(ISAAR(CPF))。RiCではAgent実体とその関係に対応する(第7章7.4節)。

アクセスポイント access point 記述の中に埋め込まれる、人物・団体・場所・主題の統制された名前(文字列)。それ自体は記述されない。

具現化 instantiation 記録という知的な内容が、担体(紙、フィルム、デジタルファイル)の上に記録された現れ。同じ記録が複数の具現化を持ちうる。RiCが新たに立てた実体である(第3章3.5節)。

GLAM 美術館・図書館・文書館・博物館(Galleries, Libraries, Archives, Museums)の総称。文化資源の収集・保存・記述という共通の仕事を持つ。第2章2.6節参照。

■分野で意味がずれる言葉

管理メタデータ administrative metadata 記述それ自体についてのメタデータ(誰がいつどの規則で作成・改訂したか)。RiCでは記述レコードという一段上の記録への記述メタデータとして、同じ実体・関係で表す。第4章4.5節参照。

参照 reference ある記述の中から別のものを名指すこと。名前の文字列→レコードID→IRIと強化されてきた(第2章)。矢印の向きの読み方は第5章5.1.2項。

実体 entity ERMで、関心対象の「もの」の種類(実体型)またはその実例。RiC-CMの19実体は実体型である。第4章4.1節参照。

スコープノート scope note RiC-CMが実体・属性・関係を記述するときに使う欄の1つで、定義だけでは伝わらない適用範囲や判断の目安を述べる(実体ならID・名称・定義・スコープノート・例・注釈(Comments)の6欄の1つ。属性では欄名がScopeになる)。本書が「スコープノートは～と言う」と書くときはこの欄を指す。シソーラスや件名標目表でも同じ語が使われるので、文脈に注意がいる。RiC-Oは、RiC-CMのスコープノートと同じか、それに基づく内容をskos:scopeNoteに載せている。

クラス class RDF/OWLにおける実体型に相当する概念。RiC-OのクラスはRiC-CMの実体＋実装用の追加概念。

個体 individual / **資源** resource RDF/OWL におけるインスタンス。IRI で識別される。

属性 attribute / **プロパティ** property ERM の属性は実体の特性。RDF ではもの-値を結ぶデータタイププロパティと、もの-ものを結ぶオブジェクトプロパティに分かれ、後者は ERM では「関係」に当たる。同じ語が層によって別物を指すので注意。

関係 relation ERM で実体間のつながり。RDF 実装ではオブジェクトプロパティまたは関係クラス (実体化) として現れる。

「一種である」と「属する」実体階層の「一種である」(記録は記録資源の一種、という分類) と、記録が記録集合に「属する」(通常の関係。記録集合から見れば R024「含む」、記録から見ればその逆の R024i「含まれる」) は別物である。記録と記録集合の間に「一種である」の関係はない。第 4 章 4.2 節の注意ボックス「「集合」という名前の読み方」参照。

定義域 domain / **値域** range 関係 (プロパティ) の出発点/到達点になれる実体 (クラス) の指定。

トリプル triple RDF のデータ単位。主語・述語・目的語の 3 つ組。

a(Turtle の述語) 「～というクラスのインスタンスである」を表す述語 `rdf:type` の略記。第 5 章の文法コラム参照。

IRI 国際化資源識別子。世界規模で対象を識別するための名前。自分が管理するドメインの下で発行すれば、他機関の名前と区別できる。同じ IRI を別の対象に使い回さない責任は発行者にある。ページが開けることは条件ではない。第 5 章 5.1 節参照。

名前空間 namespace IRI の共通する前半部分。「どの機関の、どの語彙集の名前か」を表す。Turtle では PREFIX 宣言で短い接頭辞 (`rico:` など) を与えて略記する。

リテラル literal 文字列・数値・日付などの生の値。

オントロジー ontology ある領域のクラスとプロパティを機械可読に定義した語彙体系。哲学の存在論とは別物 (ただし語源は同じ)。

公理 axiom オントロジーに書かれた宣言の 1 つ 1 つ。「`rico:Person` は `rico:Agent` の下位クラス」のように、個々のデータについての事実ではなく、その語彙を使うかぎりつねに成り立つと定めた約束事を指す (本書での用法。OWL の正式な用語法では、個体についての言明も公理に含まれる)。第 5 章 5.2.1 項参照。

推論 inference 明示されたデータと公理から、書かれていないトリプルを機械的に導くこと。

実体化 reification 関係や属性値を、それ自体 1 個のノード (個体) に昇格させる技法。関係に属性を付けたいときに使う。

グラフ graph 点 (ノード) と、点をつなぐ線 (アーク) からなる構造。棒グラフや折れ線グラフとは別物である。路線図や家系図と同じ形で、木 (階層) はその特殊な場合。第 2 章 2.3 節参照。

推移閉包 transitive closure 直接の関係からたどれるかぎりたどって得られる関係の全体。「直接含む」の推移閉包が「(間接も含めて) 含む」である。第 6 章 6.2 節参照。

プロパティ連鎖 property chain 「A から B へ、B から C へたどれたら、A から C への近道のプロパティも成り立つ」と語彙の側で宣言しておく公理。長い経路で書いた記述から近道が推論で得られる。RiC-O の関係実体の場合は、関係ノードを自分自身に結ぶ `_role` の 1 行もデータに書いておく (OWL 2 RL の推論器で近道を得るため)。第 6 章 6.5 節参照。

ロールアップ roll-up 細かい下位プロパティで書いた言明が、推論によって上位のプロパティへ吸い上げられること。詳しく書いても粗い検索から漏れない、という性質を支える。第 6 章 6.4 節参照。

開世界仮説 open world assumption 「書かれていないことは偽ではなく不明」とする解釈の立場。OWL はこの立場をとる。

閉世界仮説 closed world assumption 「書かれていないことは偽」とする立場。リレーショナルデータベースや Prolog の「失敗による否定」は、この考え方と関係が深い。新しい情報を得たときに以前の結論を取り消す非単調推論とは別の概念である。データが完全でない領域では誤った否定を導くため、OWL は開世界仮説を採った。第 3 章 3.1 節のコラム参照。

セマンティックウェブ Semantic Web ウェブ上の情報を人間だけでなく機械にも意味の分かる形で書き、リンクをたどって推論させようという構想。1990 年代末に構想され、2001 年の雑誌記事で広く知られるようになった。RDF・OWL・SPARQL はいずれもこの系譜の技術である。第 3 章 3.1 節のコラム参照。

LOD (Linked Open Data) IRI で名指され相互リンクされた、公開 RDF データの総称ないし公開の作法。

owl:sameAs 2 つの IRI が同じものを指すと宣言する OWL のプロパティ。自館の Agent を VIAF などの外部典拠に結びつけるときに使う。第 7 章 7.6 節参照。

適用プロファイル application profile RiC-O の語彙のうち自機関で何を使い、どれを必須とするかを決めて文書化したもの。RiC は「どちらでもよい」と言う場面が多いため、機関としての方針をここに集約する。第 7 章 7.3 節参照。

SPARQL RDF グラフの検索言語。

SHACL RDF データの形を検査する制約記述言語。

SKOS 統制語彙 (シソーラス・件名標目表・分類表) を RDF で表す W3C 標準。概念 (skos:Concept) を個体として立て、prefLabel/altLabel と broader/narrower で結ぶ。broader は subClassOf ではない。第 7 章 7.5 節のコラム参照。

名前付きグラフ named graph トリプルに「所属グラフ」のラベルを付けて格納・検索する仕組み (SPARQL の RDF データセット。2008 年の SPARQL 1.0 から)。典拠系と目録系を分割管理したまま論理的に統合できる。第 7 章 7.4.4 項参照。

トリプルストア triplestore RDF を格納し SPARQL 検索 (しばしば推論も) を提供するデータベース。

A.8 一次資料と参考リソース

本書が依拠した一次資料は RiC-FAD 1.0・RiC-CM 1.0・RiC-O 1.1・RiC-AG 0.1 の 4 つで、いずれも以下の所在から無償で入手できる。本書が参照したのは、2026 年 7～8 月の時点で公開されていた版である。本書の作例は、本書のサイトで配布しているサンプルデータに実行できる形で収めてあり、RiC-O 1.1 の本体 (RDF と構成要素の CSV 一覧) も同梱している。

- RiC-FAD 1.0 (2023) — github.com/ICA-EGAD/RiC-FAD
- RiC-CM 1.0 (2023) — github.com/ICA-EGAD/RiC-CM (ICA サイトからも入手可)

- RiC-O 1.1 (2025) — github.com/ICA-EGAD/RiC-O
- RiC-AG 0.1 草案 (2025 年 10 月 29 日、ICA バルセロナ大会で公開) — github.com/ICA-EGAD/RiC-AG(本文、既存 4 標準→ RiC-CM および EAD 2002 → RiC-O 1.1 の公式クロスウォーク、ダイアグラム)
- RiC-O 解説サイト — ica-egad.github.io/RiC-O/(About、Why use RiC-O?、Diagrams、Examples ほか)
- RiC-O Converter — フランス国立公文書館と Sparna 社による EAD/EAC-CPF 変換ツール (2020 年初版)— github.com/ArchivesNationalesFR/rico-converter
- RiC-ResourceList — ica-egad.github.io/RiC-ResourceList(RiC の利用者コミュニティが共同で作り EGAD が管理する、実装・ツール・文献・データセットの一覧)
- AnF Sparnatural デモ (公証人記録の知識グラフ検索)— sparna-git.github.io/sparnatural-demonstrateur-an
- Records in Contexts users — 利用者コミュニティ (Google Group)
- 従来標準:ISAD(G) 2nd ed. (2000)、ISAAR(CPF) 2nd ed. (2004)、ISDF (2007)、ISDIAH (2008)、いずれも ICA サイトで公開
- 関連標準:ISO 23081(記録管理メタデータ)、PREMIS(保存メタデータ)、EAD/EAC-CPF(符号化)、W3C RDF/OWL/SPARQL/SKOS/SHACL 各仕様

索引

A

ABox, **70**, 94
authority record, → 典拠レコード

B

BIBFRAME, **18**

C

CC0, 119
CIDOC CRM, **18**

D

Datalog, **71**

E

EAC-CPF, **8**, 102
EAD, **8**, 79, 102
EDTF, **121**
entity, → 実体
ERM, **35**

F

finding aid, → 検索手段
FranceArchives, 99

G

GLAM, **18**
GraphDB, **92**

I

identifier, → 識別子
IIIF, **18**, 119
inference, → 推論
instantiation, → 具現化
IRI, 17, **56**, 93, 117
ISAAR(CPF), **8**
ISAD(G), **8**
ISDF, **8**
ISDIAH, **8**
ISNI, 118
ISO 23081, **18**

J

JSON, **61**
JSON-LD, **61**

L

LOD, **13**, 138

M

materialization, → マテリアライゼーション
Memobase, 99

N

N-Triples, **61**

named graph, → 名前付きグラフ

O

OAI-PMH, **119**
OWL, **64**, 114
OWL 2 プロファイル, **95**

P

PIAAF, 99
PREMIS, **18**
provenance, → 出所
Proxy, **79**

R

RDF, **56**
RDF/XML, **61**
RDFS, **64**
reference, → 参照
reification, → 実体化
RiC-AG, **1**
RiC-CM, **1**
RiC-FAD, **1**
RiC-O, **1**
RiC-O Converter, **101**

S

SHACL, 74, 101, **141**
SKOS, 73, **112**
SNAC, 118
SPARQL, 74, **82**, 92

T

TBox, **71**, 94
TriG, 109
Turtle, **61**, 83

V

VIAF, **117**
VoID, 110

W

Wikidata, **117**

X

XML, **11**, 62

あ

アーク (arc), **12**
アクセス制限, 123
アクセスポイント (access point), **9**
値スキーマ (value schema), **43**, 146
異体字, **121**
一階述語論理 (first-order logic), **66**
エージェント (agent), **37**, 69, 118
オントロジー (ontology), 3, **64**

か

解決 (resolution), **16**
 開世界仮説 (open world assumption), **22**, 96, 141, 142
 外部キー (foreign key), **139**
 関係 (relation), 2, 14, 28, **47**, 77, 86, 134, 139
 管理メタデータ (administrative metadata), **52**
 記述の記述, **52**
 記述論理 (description logic), **65**
 逆プロパティ (inverse property), **62**, 86
 記録 (record), **37**
 記録資源 (record resource), 28, **39**, 79, 104
 記録集合 (record set), 28, **37**, 45, 140
 記録部分 (record part), 28, **39**, 140
 空白ノード (blank node), **58**
 具現化 (instantiation), 18, 19, **29**, 41, 141
 クラス (class), 3, 44, 45, **64**, 142
 グラフ (graph), **12**, 54, 123
 クロスウォーク (crosswalk), **101**
 結合表, **13**
 決定可能性 (decidability), **67**
 検索手段 (finding aid), **8**, 21, 64, 103
 語彙 (vocabulary), 4, **73**, 90, 116, 142
 公文書管理法, **123**
 公理 (axiom), **65**
 国立公文書館デジタルアーカイブ, **119**
 個人情報, **123**
 コンテントネゴシエーション (content negotiation), **120**

さ

参照 (reference), **13**, 62
 サンプルデータ, **135**
 シェイプ (shape), **141**
 識別子 (identifier), **13**, 117
 実体 (entity), 2, 18, 32, **35**
 実体化 (reification), 50, **77**, 110, 139
 弱実体 (weak entity), **140**
 ジャパンサーチ, **119**
 出所 (provenance), **25**, 51, 89, 110
 出所原則 (principle of provenance), **9**
 シリーズシステム (series system), **111**
 推移性 (transitivity), **83**
 推移閉包 (transitive closure), **84**
 推論 (inference), 22, 64, **82**, 140–142
 スコープノート (scope note), **24**
 正規化 (normalization), **13**, 122
 整形形式 (well-formed), **12**
 セマンティックウェブ (Semantic Web), **22**
 全体像の図 (RiC-CM 図 3), **38**
 層構造, **73**
 属性 (attribute), 24, **43**, 45, 77, 108, 146
 属性 (attribute、XML), **11**
 存在従属 (existence dependency), **140**

た

対称プロパティ (symmetric property), **62**
 多言語ラベル, **121**
 多次元記述 (multidimensional description), **24**
 多段階記述 (multilevel description), **9**
 縦持ち・横持ち, 57
 妥当 (valid), **12**
 値域 (range), **51**
 積み上げ型 (stacked), **57**
 データ独立性 (data independence), **139**
 定義域 (domain), 45, **51**, 141, 146
 適用プロファイル (application profile), **101**, 141
 典拠レコード (authority record), **13**, 102

伝来, **25**

ドメイン知識, **112**
 トリプル (triple), 13, **56**, 109
 トリプルストア (triplestore), 57, **92**

な

名前空間 (namespace), **58**, 90
 名前付きグラフ (named graph), **109**
 ノード (node), **12**

は

廃棄, **123**
 パニング (punning), **71**
 半決定可能 (semi-decidable), **67**
 非単調推論 (non-monotonic reasoning), **22**
 評価選別 (appraisal), **123**
 表計算での記述, 133
 表現の一意性, **30**
 フォンド (fonds), **9**, 28, 84, 111
 フラグメント (fragment), **59**
 プロパティ (property), 3, 32, 51, **64**, 117
 プロパティ連鎖 (property chain), **87**
 閉世界仮説 (closed world assumption), **22**, 142
 保存期間, **123**

ま

マッピング (mapping), **101**
 マテリアライゼーション (materialization), **92**
 名称統制, **13**
 目録, **8**

や

矢印の向き, **62**
 要素 (element、XML), **11**
 読み (ヨミ), 121

ら

リテラル (literal), 31, **56**, 101
 量化 (quantification), **66**
 リンクセット (linkset), **110**
 レファレンスコード (reference code), **13**
 ロールアップ (roll-up), **86**

わ

和暦, **121**

Records in Contexts 入門

—アーカイブズ記述の新しい国際標準を理解する—

著者 久保山哲二

(学習院大学大学院人文科学研究科アーカイブズ学専攻/計算機センター)

初版発行 2026 年 7 月

発行 2026 年 9 月 (第 1.5.0 版)

本書のサイト <https://tkub.github.io/edu/ric/>

連絡先 tkuboyama@tk.cc.gakushuin.ac.jp

本書では、主に調査や作図などに大規模言語モデル Claude Fable 5 と Claude Opus 5(Anthropic) を活用した。本書の構成と内容に関する文責は久保山哲二にある。本文組版は LuaLaTeX(luatexja、原ノ味フォント) による。

本書はクリエイティブ・コモンズ表示 4.0 国際ライセンス (CC BY 4.0) で提供する。出典を表示すれば、複製・再配布・改変・翻訳・授業や出版での利用を自由に行ってよい。一次資料のうち、ICA EGAD の RiC-FAD・RiC-CM・RiC-O も同じ CC BY 4.0 で公開されている。
